

ШАМИМ БХУЯН | ТИМУР ИСАЧЕНКО

ГЕНЕРАТИВНЫЙ ИИ

С ОБУЧЕНИЕМ БОЛЬШИХ
ЯЗЫКОВЫХ МОДЕЛЕЙ (LLM) ДЛЯ ДЖУНОВ

ДОРОЖНАЯ КАРТА ДЛЯ СОЗДАНИЯ ИИ-ПРИЛОЖЕНИЙ

ПРОМПТ-
ИНЖИНИРИНГ

AI-АГЕНТЫ

ЛОКАЛЬНЫЕ
LLM

ПУТЕВОДИТЕЛЬ ПО GPT И AI

SHAMIM BHUIYAN | TIMUR ISACHENKO

GENERATIVE AI

WITH LOCAL LLM

ШАМИМ БХУЯН | ТИМУР ИСАЧЕНКО

ГЕНЕРАТИВНЫЙ ИИ

С ОБУЧЕНИЕМ БОЛЬШИХ
ЯЗЫКОВЫХ МОДЕЛЕЙ (LLM)

ДЛЯ ДЖУНОВ

УДК 004.8
ББК 32.813
Б94

Shamim Bhuiyan and Timur Isachenko
Generative AI with local LLM
A comprehensive roadmap for building AI-Driven applications with local LLMs
© 2024 Shamim Bhuiyan

Бхуян, Шамим.
Б94 Генеративный ИИ с обучением больших языковых моделей (LLM) для джунов / Шамим Бхуян, Тимур Исаченко ; [перевод О. И. Перфильева]. — Москва : Эксмо, 2025. — 320 с. — (Путеводитель по GPT и AI).

ISBN 978-5-04-220583-5

Это практическое руководство по созданию приложений на основе генеративного искусственного интеллекта и больших языковых моделей (LLM). Особое внимание уделяется прикладным аспектам: промпт-инжинирингу, работе с локальными LLM, тонкой настройке моделей на частных данных и созданию автономных AI-агентов. Приводятся примеры реальных решений, таких как интеллектуальная обработка SQL-запросов и автоматизация работы с изображениями.

Подходит для разработчиков и аналитиков данных с базовыми знаниями Python, желающих освоить генеративный ИИ.

УДК 004.8
ББК 32.813

ISBN 978-5-04-220583-5

© Перфильев О.И., перевод на русский язык, 2025
© Оформление. ООО «Издательство «Эксмо», 2025

ОГЛАВЛЕНИЕ

Предисловие	9
О чем эта книга	13
Примеры кода	15
Целевая аудитория	15
Форматирование и условные обозначения	16
Отзывы читателей	17
Об авторах	18
Благодарности	19
Глава 1. Начало работы с локальными LLM	20
Инструменты и фреймворки, используемые в книге	22
Установка и настройка LLM с локальным инференсом	25
Полезные команды и интерфейсы	30
Дополнительные настройки	32
Удаление локальной платформы LLM	33
Установка клиента с графическим интерфейсом пользователя (GUI) для работы с локальным LLM	34
Настройка виртуальной среды Python для разработки ИИ	35
Установка Python 3	36
Установка менеджера пакетов Python pip3	37
Установка и настройка Miniconda	39
Установка IDE (интегрированной среды разработки) JupyterLab	42
Установка и настройка базы данных SQLite	44
Дополнительные настройки	45
Создание первого приложения с локальной LLM	46
Устранение ошибок	50
Аппаратное ускорение	50

Рабочая станция с GPU	52
Включение AVX/AVX2 для ускорения работы процессора	53
Использование сторонней платформы ASIC или VPS с поддержкой GPU	54
Использование сервиса Google Colab или Kaggle	55
Заключение	55
Глава 2. Погружение в теорию генеративного ИИ	57
Искусственный интеллект (ИИ)	58
Машинное обучение (ML)	59
Глубокое обучение (DL)	63
Обработка естественного языка (NLP)	66
Трансформер	68
Механизм самовнимания	69
Архитектура кодировщика-декодировщика	71
Генеративный ИИ	74
Что относится к сфере генеративного ИИ, а что не относится?	77
Категории генеративного ИИ	79
Большая языковая модель	84
Как устроена LLM?	84
Обучение LLM	98
RAG	103
ИИ-агенты	105
Промпт-инжиниринг	109
Ресурсы	112
Заклучение	113
Глава 3. RAG, обогащение моделей LLM с помощью частных наборов данных	114
RAG или файн-тюнинг LLM?	115
Ключевые концепции RAG	117
Эмбединги	118
Векторная база данных	118
Семантический поиск	120
Чем семантический поиск отличается от полнотекстового?	122
Реальные примеры использования RAG	124
Внедрение RAG в частной компании	125

Пошаговый пример. Загрузка, извлечение и обработка пользовательских документов с помощью LLM	129
Заключение	141
Глава 4. Преобразование текста в SQL, улучшение ответов LLM за счет интеграции с базой данных	142
Что такое преобразование текста в SQL (Text-to-SQL)?	143
Проблемы преобразования текста в SQL	145
LLM для преобразования текста в SQL	147
Шаблоны проектирования систем преобразования текста в SQL с примерами	148
Шаблон проектирования 1. Генерация и выполнение SQL-запросов.	150
Шаблон проектирования 2. Использование агента для обработки ошибок и обеспечения корректности.	159
Шаблон проектирования 3. Преобразование текста в SQL-запросы с помощью RAG.	167
Заключение	179
Глава 5. Файн-тюнинг (дообучение) LLM	180
Шаги по файн-тюнингу предварительно обученной модели	183
Техники файн-тюнинга	187
Полный файн-тюнинг	188
Параметрически эффективный файн-тюнинг (PEFT)	189
Дистилляция знаний (KD)	193
Популярные фреймворки файн-тюнинга LLM	195
Пошаговый пример файн-тюнинга LLM	198
Обязательные требования	199
Часть 1. Анализ бизнес-требований, выбор базовой модели и настройка окружения.	199
Часть 2. Обзор и исследование обучающего набора данных.	205
Часть 3. Предварительная обработка набора данных и настройка адаптера.	214
Часть 4. Обучение модели.	224
Часть 5. Оценка модели.	230
Часть 6. Сохранение и развертывание готовой модели.	238
Заключение	239

Глава 6. Обработка и генерация изображений с помощью LLM	241
Распознавание изображений (Image visioning)	242
Возможности и функциональные особенности LLaVA-v1.6.	242
Архитектура LLaVa	243
Пошаговый пример. Использование LLaVA-v1.6 для распознавания изображений.	244
Внедрение LLaVA в приложение для анализа изображений.	255
Обработка изображений	259
Советы по улучшению процесса обработки изображений.	267
Ссылки	268
Заключение	268
 Глава 7. Разработка и использование ИИ-агентов	 270
Будущее ИИ-агентов	271
Отличие ИИ-агентов от ИИ-инструментов	273
Примеры использования ИИ-агентов в сфере генеративного ИИ.	274
Примеры использования с точки зрения разработчика.	275
Примеры использования с точки зрения менеджера продукта.	278
Классификация ИИ-агентов в сфере генеративного ИИ	281
Архитектура ИИ-агентов	284
Фреймворки для разработки ИИ-агентов	287
Пошаговое руководство по созданию ИИ-агента на практике.	292
Заключение	314
 Заключительные слова	 316

*Моему любящему сыну Мишелю,
чьи любопытные вопросы о том,
почему я так много времени
провожу за компьютером, ежедневно
вдохновляют меня.*

ПРЕДИСЛОВИЕ

В последнее время тема искусственного интеллекта (ИИ) обрела невероятную популярность и освещается повсюду — от школьных газет до дебатов в Сенате США. Эта быстро развивающаяся область привлекает внимание как экспертов в сфере машинного обучения, так и обычных людей. Кое-кто с беспокойством рассуждает о том, в каком направлении движутся современные технологии, а кто-то предлагает такие экстремальные меры, как уничтожение центров обработки данных. Свое мнение о будущем ИИ высказывают даже представители Администрации президента.

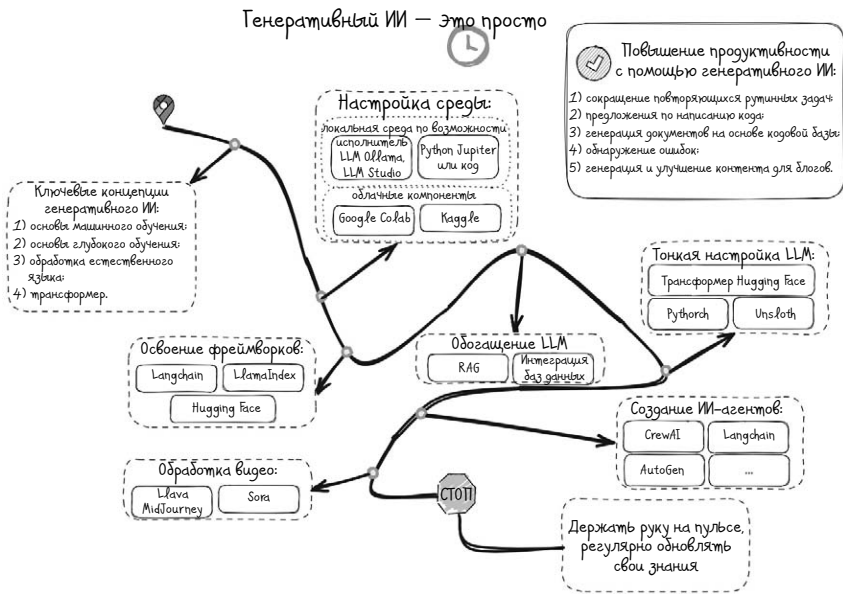
Но мы считаем, что гораздо продуктивнее не жить в страхе перед возможными злоупотреблениями в данной сфере, а сосредоточиться на полезных сторонах ИИ. То есть на том, что он может улучшить нашу повседневную жизнь — с помощью Siri или других ИИ-ассистентов.

Искусственному интеллекту, особенно генеративному ИИ и большим языковым моделям, посвящены бесчисленные статьи и публикации, рассчитанные на читателей с самым разным уровнем подготовки. К сожалению, для полного понимания большинства из них требуется определенный уровень знаний.

Вместе с тем не существует какого-то общепринятого пособия, с помощью которого можно было бы легко овладеть этой темой, поэтому мы и написали эту книгу. По нашему замыслу, она должна стать четкой и понятной дорожной картой, которая провела бы но-

вичков через все терминологические и концептуальные сложности. Чтобы с ее помощью разобраться в некоторых принципах машинного обучения, вовсе не обязательно быть экспертом в области компьютерных технологий, поэтому не бойтесь приступать к обучению!

За несколько недель, проведенных в исследованиях и составлении планов, мы разработали надежное практическое пособие по генеративному искусственному интеллекту. И рады поделиться им с вами.



В целом полный путь освоения темы генеративного ИИ состоит из семи взаимосвязанных этапов.

1. Знакомство с ключевыми понятиями в сфере генеративного ИИ

- Искусственный интеллект. Основные понятия и принципы его работы, включая то, что он собой представляет и как применяется.
- Машинное обучение. Основные концепции в данной области, включая обучение с учителем и без учителя, алгоритмы и обучение моделей.
- Глубокое обучение. Основы глубокого обучения, нейронные сети и принципы их работы, включая такие популярные архи-

тектуры, как CNN и RNN (сверточные и рекуррентные нейронные сети).

- Обработка естественного языка (NLP, Natural Language Processing). Знакомство с обработкой естественного языка, подразумевающей взаимодействие между компьютерами и людьми, дающими команды и инструкции на естественном языке.

Эти ключевые понятия помогут вам перейти к более сложным темам, связанным с ИИ.

2. Настройка среды для локальных LLM

- Запуск локальных моделей. Эксперименты с их запуском с помощью таких инструментов, как *Ollama* или других программ с открытым исходным кодом. Знания о том, как настраивать и использовать модели на локальной машине.
- Настройка локальной среды Python с инструментами Miniconda и JupyterLab, которые помогут вам начать путешествие в мир генеративного ИИ.
- Платформы для обмена моделями. Знакомство с такими платформами, как *Hugging Face*, предлагающими доступ к предварительно обученным моделям и наборам данных. Они позволят вам экспериментировать с широким спектром моделей генеративного ИИ.

3. Освоение фреймворков

LangChain. Создание приложений, использующих языковые модели. Так вы узнаете, как можно внедрить генеративный ИИ в реальные приложения.

- LlmIndex. Индексирование и поиск по большим языковым моделям.
- Python. Повышение навыков программирования на Python, самом распространенном языке для разработки ИИ. Основное внимание уделяется таким библиотекам, как TensorFlow, PyTorch и Transformers от *Hugging Face*.

4. Тонкая настройка/обогащение LLM

- Методы тонкой настройки. Как настраивать предварительно обученные модели для лучшего решения конкретных задач или для конкретных наборов данных.
- Методы «обогащения». Метод RAG (Retrieval-Augmented Generation, «расширенная генерация данных»), позволяющий улучшать качество выходных данных моделей.

5. Создание проектов и агентов

- Небольшие проекты. Создание небольших проектов по выполнению таких задач, как генерация текста, изображений или других медиафайлов, с целью закрепления полученных знаний.
- Агенты. Разработка агентов, которые смогут выполнять некоторые повседневные задачи, — например, искать информацию в Интернете и генерировать качественный текст для блога на определенную тему.

6. Продолжение обучения

- Интерес к области ИИ. Регулярное знакомство с последними достижениями в области ИИ, просмотр соответствующих блогов, каналов YouTube и онлайн-курсов.
- Сотрудничество и обмен информацией. Взаимодействие с онлайн-сообществами, участие в решении задач в области ИИ и обмен проектами на таких платформах, как GitHub.

7. Продвинутые темы

- Овладев основами, вы сможете погрузиться в такие продвинутые темы, как обучение с подкреплением, продвинутые архитектуры моделей и передовые исследования в области ИИ.

Описанный выше курс заложит прочный фундамент знаний в области генеративного ИИ и позволит вам постепенно наращивать свой опыт с применением его в реальных сценариях.

О чем эта книга

Данная книга представляет собой развернутое руководство по использованию генеративного искусственного интеллекта и больших языковых моделей (LLM) в локальной среде разработки. Она знакомит читателей с основами генеративного ИИ и такими продвинутыми методами, как тонкая настройка моделей, обогащение их частными наборами данных и использование в практических сценариях. Например, для обработки SQL-запросов и изображений, а также разработки агентов. Если ваша продукция в какой-то степени связана с ИИ, если вы разработчик приложений, специалист по анализу данных или просто энтузиаст своего дела, увлекающийся данной темой, то эта книга вооружит вас знаниями и инструментами, необходимыми для эффективного использования ИИ в своих проектах.

Глава 1. Начало работы с локальными LLM

Прежде чем приступить к реализации ИИ-моделей, нужно создать надежную локальную среду их разработки. В этой главе мы расскажем вам о том, как настроить компьютер для разработки ИИ-приложений, в том числе установить необходимое программное обеспечение, настроить среду Python и выбрать подходящее оборудование (например, графический процессор, или GPU) для оптимальной производительности. Глава также познакомит вас с такими инструментами, как блокноты Jupyter, которые помогут упростить рабочий процесс разработки ИИ.

Глава 2. Погружение в теорию генеративного ИИ

Наше путешествие в мир генеративного ИИ начинается с основ, то есть с объяснения того, как и почему работает генеративный ИИ, что это такое, и какую революцию он совершил в различных областях. В главе рассматриваются фундаментальные понятия ИИ, машинного обучения и глубокого обучения. В ней же закладывается основа для дальнейшего понимания того, как LLM генерируют контент — текст, код и изображения. Вы познакомитесь с различными типами генеративных моделей, в том числе с механизмом самовнимания,

с архитектурой «кодировщик-декодировщик» и трансформерами, а также с их применением в различных областях.

Глава 3. RAG, обогащение моделей LLM с помощью частных наборов данных

В этой главе рассматривается передовая концепция Retrieval-Augmented Generation (RAG), подразумевающая улучшение LLM моделей за счет использования частных наборов данных. Вы узнаете о том, как можно обогащать модели ИИ специфичными для конкретной области знаниями, благодаря чему они будут генерировать более точные и релевантные результаты. В главе описаны технические шаги по настройке RAG, включая создание векторных баз данных, индексацию частных данных и настройку модели для получения и генерации ответов на основе этой обогащенной информации.

Глава 4. Преобразование текста в SQL, улучшение ответов LLM за счет интеграции с базой данных

Большую ценность в последнее время приобретает автоматизация взаимодействия с базами данных на основе ИИ, и эта глава посвящена тому, как LLM помогают преобразовывать текст в SQL-запросы (так называемая задача «Text-to-SQL»). Вы узнаете, как пользоваться моделями, способными конвертировать запросы на естественном языке в команды SQL, специально адаптированные для взаимодействия с системой BigQuery. Глава содержит практические примеры и фрагменты кода, показывающие, как создавать системы для взаимодействия с базами данных без знания языка запросов SQL, что делает поиск данных более доступным и эффективным.

Глава 5. Файн-тюнинг (дообучение) LLM

В этой главе мы подробно рассмотрим процесс тонкой настройки LLM или файн-тюнинга (fine-tuning) — важнейший этап в создании LLM-моделей, предназначенных для решений конкретных задач. Вы узнаете, как адаптировать предварительно обученные модели к вашим данным и улучшать их производительность для задач, относящихся к нужной области. В главе рассматриваются технические аспекты тонкой настройки, включая подготовку данных, настройку гиперпараметров и оценку эффективности модели. К концу главы

вы сможете совершенствовать модели, повышая их точность и релевантность для своих приложений.

Глава 6. Обработка и генерация изображений с помощью LLM

Сфера генеративного ИИ не ограничивается текстом, он также играет важную роль в обработке и генерации изображений. В данной главе рассматривается использование LLM для создания и обработки изображений. В главе приведены практические примеры и описаны полезные инструменты, в том числе модели LLaVa и OpenJourney, которые помогут интегрировать генерацию изображений в ваши проекты.

Глава 7. Разработка и использование ИИ-агентов

Последняя глава посвящена одному из самых интересных применений LLM: разработке ИИ-агентов. Вы узнаете о назначении и преимуществах использования ИИ-агентов искусственного интеллекта для автоматизации повседневных задач. Мы рассмотрим архитектуру ИИ-агентов и их ключевые компоненты, а также опишем мультиагентный фреймворк, позволяющий пользователям без специальных технических знаний управлять ИИ-агентами и задачами с помощью интуитивно понятных высокоуровневых абстракций.

Примеры кода

Все примеры кода, скрипты и более подробные примеры можно найти в репозитории GitHub.

Целевая аудитория

Целевая аудитория данной книги — владельцы и разработчики цифровых продуктов, специалисты по анализу данных и энтузиасты в области искусственного интеллекта с минимальными навыками в области программирования. Никаких особых знаний для освоения материала книги не требуется, хотя неплохо иметь хотя бы поверхностное знакомство с Python, Java и такими инструментами, как Conda.

Форматирование и условные обозначения

1. Курсивом и жирным шрифтом выделяются новые термины, важные слова, ссылки на интернет-страницы (URL), имена файлов и расширения файлов.

2. Код блока задается следующим образом.

```
def process_image(image_file_path, prompt):
    print(f"\nProcessing {image_file_path} file \n")
    image = Image.open(image_file_path)
    display(image)

    with image as img:
        with BytesIO() as buffer:
            img.save(buffer, format='PNG')
            image_bytes = buffer.getvalue()

    # Генерация описания изображения
    for response in generate(model='llava:34b',
                             prompt=prompt,
                             images=[image_bytes],
                             stream=True):
        # Вывести ответ на консоль и добавить к полному ответу
        print(response['response'], end='', flush=True)
```

3. Любая команда или вывод в командной строке печатаются следующим образом.

```
!pip install ollama
export OLLAMA_HOST="192.168.1.124"
Processing ./Text2SQL.png file
```

Совет

Значок указывает на совет или предложение.

Внимание

Значок указывает на предупреждение или предостережение.

Инфо

Значок указывает на общие сведения.

Отзывы читателей

Мы хотели бы услышать ваши комментарии — например, что понравилось или не понравилось в содержании книги. Эти отзывы помогут нам в следующий раз написать книгу получше, а другим — разобратся в освещаемых концепциях. Чтобы оставить свой отзыв, воспользуйтесь ссылкой для обратной связи.

ОБ АВТОРАХ

Шамим Бхуян в настоящее время работает корпоративным архитектором (Enterprise architect), отвечающим за проектирование и реализацию высокомасштабируемого программного обеспечения с большой нагрузкой. В 2007 году он получил в России степень кандидата наук по информационным технологиям во Владимирском университете. Работает в сфере ИТ уже более двадцати двух лет и специализируется в области аналитики данных и промежуточных решений. Ранее также занимался SOA-решениями и выступал с лекциями на тему обработки больших данных. Принимает активное участие в разработке и проектировании высокопроизводительного программного обеспечения для ИТ, телекоммуникаций и банковской сферы. В свободное время ведет блог [frommyworkshop](#) и делится своими идеями с другими увлеченными людьми.

Тимур Исаченко — опытный технический специалист и архитектор решений, обладающий экспертными знаниями в области разработки бэкенда и архитектуры микросервисов. Более четырнадцати лет работает в ИТ-индустрии, имеет богатый опыт разработки и внедрения высоконагруженных систем в сфере финансов, здравоохранения и онлайн-банкинга. Тимур окончил Кубанский государственный университет по специальности «Вычислительная техника и прикладная математика».

Он руководил множеством проектов, включая разработку образовательных платформ, биллинговых систем для букмекерских и игорных платформ, а также интеграционных сервисов для крупных предприятий. Один из авторов книги «High Performance In-Memory Data Grid with Ignite» («Высокопроизводительная сетевая память на платформе Ignite»), выступал с докладами на различных конференциях.

В личной жизни Тимур — любящий отец трех дочерей и заядлый бегун, в течение целого года ежедневно выходивший на пробежку.

БЛАГОДАРНОСТИ

Эта книга не вышла бы в свет без поддержки и вклада нескольких человек.

Прежде всего, я хотел бы выразить глубочайшую благодарность моему соавтору Тимуру Исаченко. Твоя интуиция, твой опыт и преданность делу были бесценными на протяжении всего этого путешествия. Благодарю тебя за сотрудничество и за искреннее стремление воплотить проект в реальность.

Спасибо моему сыну Мишелю за неизменную поддержку и за понимание, пока я долгие часы работал над данной книгой. Твои терпение и вера в меня служили постоянным источником мотивации.

Также я хотел бы выразить особую благодарность блогерам и влогерам на YouTube, щедро делившимся своими знаниями и опытом. Ваша ценная информация и поддержка сыграли решающую роль в оформлении этой книги, и я благодарен за тот контент, который вы предоставляете сообществу.

Наконец, хочу выразить сердечную благодарность нашим читателям, которые приобрели и прочитали книгу на ранней стадии, оставив полезные комментарии и отзывы. Ваши любопытство и страсть к обучению — вот истинные движущие силы данной работы. Я надеюсь, что она вдохновит вас на дальнейшее изучение сферы ИИ и языкового моделирования с их захватывающими возможностями. А информация из книги и приведенные на ее страницах примеры пригодятся в своем собственном путешествии.

Мы очень ценим ваши отзывы и участие, которые будут вдохновлять нас на дальнейшие начинания.

ГЛАВА 1

НАЧАЛО РАБОТЫ С ЛОКАЛЬНЫМИ LLM

Я до сих пор помню свои летние каникулы много лет назад в египетском Шарм-эль-Шейхе — красивом курортном городе, расположенном на берегу Красного моря. На второй день поездки я решил искупаться и был совершенно очарован подводным миром, изобилующим кораллами, рыбами и другими морскими обитателями. С того момента я влюбился в море. Мы с другом загорелись идеей попробовать погрузиться с аквалангом. Готовясь к этому, мы не находили места от возбуждения, хотя немного волновались.

Инструктор заметил наш страх, улыбнулся и сказал слова, которые я запомнил на всю жизнь: «Под водой находится другой прекрасный мир. Возможно, он поначалу покажется вам не очень комфортным. Но нужно попробовать прикоснуться к нему, чтобы понять, подходит он вам или нет». Благодаря поддержке я все-таки собрался с духом и погрузился (в буквальном смысле) в новый мир, с того дня полюбив дайвинг. К сожалению, у моего друга из-за давления возникла боль в ушах, с которой не удалось справиться. С тех пор он больше не погружался. Но для меня данное переживание послужило ценным уроком: когда сталкиваешься с чем-то новым и пугающим, нужно сначала самому прикоснуться к нему, чтобы понять, подходит тебе это или нет.

Почему я поведал вам эту историю? В начале знакомства с генеративным искусственным интеллектом было чувство, что на меня одновременно навалилось слишком много разрозненных обрывков информации. Вдобавок к этому люди постоянно говорили, что тема очень сложная, а для запуска нейросетей требуется дорогое

высококачественное оборудование. Но как только я составил четкую дорожную карту обучения и попробовал воспользоваться ею на практике, то понял, что этот мир тоже мой, потому что я сделал первый шаг и сделал его таким.

Поэтому вот вам мой совет: просто попробуйте сделать свое первое приложение или первый проект в области генеративного ИИ и посмотрите, понравится ли вам это. Если нет, ничего страшного, вы всегда сможете найти другое занятие себе по душе. Но если в вас вспыхнет искра интереса — замечательно! С этого момента вы станете заниматься увлекательным делом, приносящим настоящее удовольствие.

В любом случае лучший способ научиться чему-то новому — сразу приступить к работе, начав с простых примеров и экспериментов. Так вы получите необходимые знания и познакомитесь с самыми элементарными концепциями конкретной технологии. Позже, получив базовое представление о том, что можно делать с помощью этой данной технологии, вы всегда сможете научиться чему-то более сложному.

В качестве отправной точки в данной главе приведены основные инструкции по установке **сервера LLM — Ollama**. Позже мы обсудим, как настроить локальное окружение для Python, чтобы начать разработку приложений с помощью локальной LLM с открытым исходным кодом.

Вкратце в этой главе рассматриваются следующие темы.

1. Инструменты и фреймворки, используемые в данной книге.
2. Установка и настройка инструмента для инференса LLM: Ollama.
3. Установка клиента с графическим интерфейсом пользователя (GUI) для работы с Ollama.
4. Настройка среды Python для разработки ИИ.

5. Аппаратное ускорение.

Прежде, чем мы приступим к делу

Для начала нужно объяснить, почему мы решили пользоваться локальной большой языковой моделью (LLM), а не удаленной системой вроде OpenAI. Такое решение обусловлено четырьмя основными причинами.

1. Файн-тюнинг. Для ваших конкретных целей могут понадобиться сложные сценарии использования, требующие настройки LLM, а делать это легче локально.
2. Конфиденциальность и безопасность. В вашей компании могут действовать строгие правила, запрещающие передавать собственный код или конфиденциальную информацию сторонним поставщикам из-за потенциального риска раскрытия или неправомерного использования данных.
3. Глобальная доступность. Во многих странах проприетарные или принадлежащие сторонним поставщикам LLM могут быть недоступны, из-за чего возникает потребность в локальном решении.
4. Экономическая эффективность. Для оказания крупномасштабных услуг, ориентированных на пользователя, вам может понадобиться экономически эффективное решение, основанное на вашей собственной финансовой модели.

Хотя некоторые сторонники открытого ПО могут рассматривать локальную установку LLM всего лишь как забавный и любопытный любительский эксперимент, наш опыт показывает, что такой вариант подходит и для бизнеса. Выбирая LLM с локальным размещением, мы не только удовлетворяем свои потребности, но и сохраняем контроль над конфиденциальной информацией.

Инструменты и фреймворки, используемые в книге

Мы рассмотрим различные инструменты и фреймворки, необходимые для работы с LLM и ИИ-приложениями. Эти инструменты были тщательно отобраны с учетом их широкого распространения,

надежности и способности справляться с высокими вычислительными требованиями в сфере разработки ИИ-приложений. К концу книги вы получите хорошее представление о том, как с помощью данных инструментов создавать ИИ-модели, внедрять их на практике и осуществлять файн-тюнинг.

Основной язык программирования в данной книге — *Python*, пользующийся широкой популярностью в сообществе энтузиастов и специалистов в области ИИ, а также имеющий обширную экосистему доступных библиотек и фреймворков. Простота и легкость чтения *Python* делают его идеальным выбором как для начинающих исследователей, так и для опытных разработчиков ИИ-приложений.

Для построения, обучения и обогащения моделей LLM мы будем опираться на такие мощные фреймворки, как *Langchain*, *TensorFlow* и *PyTorch*. Они представляют собой комплексные инструменты создания нейронных сетей, выполнения вычислений на GPU для ускорения производительности и настройки индивидуальных решений в области ИИ. Отличаются гибкостью и пользуются большой поддержкой со стороны сообщества, что делает их незаменимыми для современных исследований и разработок в области ИИ. В частности, средства построения динамических графов вычислений в *PyTorch* и готовые к промышленной эксплуатации возможности *TensorFlow* делают их подходящими для широкого спектра практического применения — от академических исследований до промышленного внедрения.

Для решения задач обработки естественного языка (NLP) мы будем использовать библиотеку трансформеров **Hugging Face**. Она предоставляет доступ к предварительно обученным моделям и инструментам дообучения (файн-тюнинга) современных моделей трансформеров — BERT, GPT и T5. Интуитивно понятный интерфейс Hugging Face и обширная документация делают ее основным ресурсом для работы с LLM, помогающим разработчикам создавать сложные NLP-приложения с минимальными усилиями.

В дополнение к этим основным инструментам мы рассмотрим такие фреймворки, как *Ollama*, позволяющие настраивать локальные LLM

и управлять ими. Ollama упрощает развертывание LLM на локальных машинах и предлагает удобный интерфейс для запуска моделей, управления зависимостями и оптимизации производительности. С ее помощью разработчики могут осуществлять контроль над своими данными и моделями, сохраняя конфиденциальность, поддерживая безопасность и одновременно используя широкие возможности искусственного интеллекта.

Сведем все это в таблицу.

Функции	Инструменты и фреймворки
Исполнитель LLM	Ollama, Groq
LLM	Llama 3.1, codestral, Llava, Openjourney
Язык программирования	Python
Разработка приложений	Langchain, Vanna, CrewAI
Платформа машинного обучения (ML)	Hugging Face
Файн-тюнинг	Pythorch
Обработка изображений	Llava, Openjourney

Вместе эти инструменты и фреймворки представляют собой универсальный набор средств для разработки, файн-тюнинга и развертывания моделей ИИ. В книге мы шаг за шагом расскажем о том, как интегрировать данные инструменты в ваши проекты и раскрыть с их помощью весь потенциал ИИ и LLM.

Для локального запуска LLM требуются сложные программные и аппаратные компоненты, включая многоядерный процессор, достаточный объем оперативной памяти и, возможно, графический процессор. Необходимую конфигурацию можно реализовать на различных платформах: на рабочей станции, на сервере домашней лаборатории или на арендованном выделенном частном сервере, предоставленном каким-нибудь провайдером вроде Amazon. При локальной установке LLM следует принять во внимание несколько соображений.

Компоненты	Описание
Центральный процессор (CPU)	Восьмиядерный (минимум) процессор Intel/AMD
Оперативная память (RAM)	Минимум 16 ГБ
Операционная система (OS)	Linux, MacOS
Сеть	Подключение к Интернету

Установка и настройка LLM с локальным инференсом

Как говорилось выше, наши приложения, связанные с ИИ, будут строиться на базе Ollama — инструмента с открытым исходным кодом, позволяющего осуществлять эффективное управление различными большими языковыми моделями (LLM). Этот универсальный инструмент можно использовать для локального запуска широкого спектра LLM. Ключевое преимущество Ollama перед другими инструментами, такими как *LM Studio*, — исключительная производительность: эта платформа быстра и занимает мало памяти. Кроме того, Ollama предоставляет необходимый API для взаимодействия с ней, что упрощает ее интеграцию в конкретную систему.

Для локального выполнения («инференса») мы будем использовать LLM с открытым исходным кодом в сочетании с Ollama. Такая комбинация позволит воспользоваться всем потенциалом Ollama и организовать бесперебойную и эффективную работу моделей.

Платформу Ollama можно устанавливать и запускать в различных вариантах — в зависимости от уровня вовлеченности и интереса. Для быстрого старта можно воспользоваться бинарным дистрибутивом, или же, если вы предпочитаете бóльшую степень контроля, можете собрать Ollama из ее исходного кода. В данном разделе мы покажем процесс установки с помощью бинарного дистрибутива.

Шаг 1. Скачайте и установите Ollama.

- Посетите веб-сайт Ollama, чтобы загрузить последнюю версию установщика для вашей операционной системы. Ищите прямую ссылку на загрузку или руководство по установке.
- Для **macOS**.

› откройте загруженный файл и следуйте инструкциям на экране, чтобы установить Ollama на вашу систему. Обычно для этого нужно перетащить приложение в папку Applications на macOS.

- Для **Linux**.

› выполните команду `curl -fsSL https://ollama.com/install.sh | sh`

› после успешной установки вы должны увидеть сообщение, аналогичное указанному ниже:

```
>>> Making ollama accessible in the PATH in /usr/local/bin
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
WARNING: No NVIDIA/AMD GPU detected. Ollama will run in
CPU-only mode.
```

- Для **Windows**.

› на момент написания книги специально для операционной системы Windows Ollama доступна в виде предварительной версии.

- Для **Docker**:

- › убедитесь, что ваш экземпляр Docker запущен и работает. Выполните следующую команду, чтобы запустить Ollama внутри контейнера Docker.

```
docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
```

После установки вы можете проверить установку, открыв терминал и выполнив следующую команду.

```
ollama --version
```

Следующая команда отобразит текущую версию Ollama, установленную на вашем компьютере, и подтвердит, что установка прошла успешно.

```
Warning: could not connect to a running Ollama instance
Warning: client version is 0.3.6
```

Такие сообщения говорят о том, что экземпляр Ollama не запущен, а версия клиента — 0.3.6.

Шаг 2. Запустите Ollama.

Запуск экземпляра Ollama состоит из двух простых шагов.

- Чтобы запустить Ollama без LLM, выполните следующую команду:

```
ollama serve
```

Она инициализирует сервер Ollama, после чего он сможет локально запускать LLM и управлять им. Выводимые при этом сообщения должны походить на следующие:

```

2024/08/27 15:17:09 routes.go:1125: INFO server config
env="map[OLLAMA_DEBUG: false OLLAMA_FLASH_ATTENTION:
false OLLAMA_HOST: http://127.0.0.1:11434 OLLAMA_KEEP_
ALIVE:5m0s OLLAMA_LLM_LIBRARY: OLLAMA_MAX_LOADED_MODEL S:0
OLLAMA_MAX_QUEUE:512 OLLAMA_MODELS:/Users/shamim/.ollama/
models OLLAMA_NOHISTORY: false OLLAMA_NOPRUNE: false
OLLAMA_NUM_PARALLEL:0 OLLAMA_ORIGINS:[http://localhost
https://localhost http://localhost:* https://localhost:*
http://127.0.0.1 https://127.0.0.1 http://127.0.0.1:*
https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0
http://0.0.0.0:* https://0.0.0.0:* app://* file://*
tauri://*] OLLAMA_RUNNERS_DIR: OLLAMA_SCHED_SPREAD: false
OLLAMA_TMPDIR:]"
time=2024-08-27T15:17:09.034+03:00 level=INFO
source=images.go:782 msg=\
"total blobs: 10"
time=2024-08-27T15:17:09.036+03:00 level=INFO
source=images.go:790 msg=\
"total unused blobs removed: 0"
time=2024-08-27T15:17:09.037+03:00 level=INFO
source=routes.go:1172 msg\
="Listening on 127.0.0.1:11434 (version 0.3.6)"
time=2024-08-27T15:17:09.038+03:00 level=INFO
source=payload.go:30 msg=\
"extracting embedded files" dir=/var/folders/lj/
qs6zzt3j7jvb5yrh4x1x39x
00000gn/T/ollama883440228/runners
time=2024-08-27T15:17:09.087+03:00 level=INFO
source=payload.go:44 msg=\
"Dynamic LLM libraries [cpu cpu_avx cpu_avx2]"
time=2024-08-27T15:17:09.087+03:00 level=INFO
source=types.go:105 msg="inference compute" id=""
library=cpu compute="" driver=0.0 name="" total=16.0
GiB available=6.3 GiB"

```

- Для запуска экземпляра Ollama с моделью LLM (в данном случае Llama 3.1) воспользуйтесь следующей командой:

```
ollama run llama3.1
```

Она скачает модель llama3.1 8b из репозитория, запустит сервер Ollama и загрузит модель llama3.1 для обработки запросов или задач локально. Следующее сообщение говорит о том, что Ollama успешно скачала и запустила LLM:

```
ollama run llama3.1buj pulling manifest
pulling 8eeb52dfb3bb... 100% 4.7 GB/4.7 GB 2.7 MB/s    \
0s
pulling 11ce4ee3e170... 100% 1.7 KB
pulling 0ba8f0e314b4... 100% 12 KB
pulling 56bb8bd477a5... 100% 96 B
pulling 1a4c3c319823... 100% 485 B
verifying sha256 digest
writing manifest
removing any unused layers success
>>> Send a message (!? for help)
```

Полный список моделей, поддерживаемых Ollama, доступен [здесь](#).

Инфо

Учтите, что для локального запуска LLM требуется достаточно большой объем оперативной памяти. В частности, для моделей 7B требуется не менее 8 ГБ свободной оперативной памяти, для моделей 13B рекомендуется 16 ГБ, а для моделей 33B требуются 32 ГБ.

Ollama будет загружать модель только во время ее первоначальной настройки. В последующем команда `ollama run llama3.1` будет запускать локально кэшированную LLM, а не скачивать ее из репозитория.

Теперь, когда LLM настроена, можно взаимодействовать с ней с помощью ввода команд в чате. Для начала попробуйте набрать что-нибудь вроде `tell me a joke` («Расскажи мне анекдот»).

В данный момент команда `ollama ps` в другом окне терминала вызовет сообщение вроде следующего:

```
NAME ID    SIZE PROCESSOR UNTIL
llama3.1: latest 91ab477bec9d    6.2 GB 100% CPU 4 minutes
from now
```

Если у вас система с поддержкой GPU, сообщение должно выглядеть примерно так:

```
NAME ID    SIZE PROCESSOR UNTIL
llama3.1: latest 62757c860e01 6.7 GB 100% GPU 4 minutes
from now
```

Чтобы выйти из чата, просто введите в командной строке `/bye`. Эта команда не только завершит разговор, но и остановит запущенную в данный момент LLM.

Подытожим разницу между двумя командами: `ollama serve` и `ollama run llama3.1`.

Команда `ollama serve` запускает экземпляр Ollama, но не инициализирует какую-либо конкретную LLM. Она настраивает сервер, делая его готовым к обработке запросов, но выбрать конкретные модели можно с помощью последующих команд.

Команда `ollama run llama3.1` запускает сервер Ollama и сразу же инициализирует модель llama3.1. Она настраивает сервер и загружает указанную модель для обработки запросов или задач локально.

Полезные команды и интерфейсы

Далее приведены некоторые полезные команды Ollama для базового контроля над этой системой, для проверки ее состояния или для выключения после использования.

- `ollama list-models` Перечисляет все доступные модели, которыми можно пользоваться в Ollama. Так можно просмотреть список всех установленных и готовых к использованию моделей.

- `ollama pull <имя_модели>` Скачивает указанную модель (например, llama3.1) из репозитория в вашу локальную систему. С помощью этой команды можно загрузить еще не установленные модели.
- `ollama rm llama3.1` Удаляет LLM из локального репозитория.
- `ollama create <имя_модели>` Создает новую модель из указанного файла GGUF Modelfile. Используйте эту команду для инициализации и настройки модели на основе конфигурации и параметров, определенных в файле Modelfile.

Следующие данные полезны для создания пользовательских моделей, предназначенных для конкретных задач или наборов данных.

- Загрузите файл предварительно обученной большой языковой модели (LLM) в формате GGUF из Hugging Face или другого источника, например, `llama-3-sqlcoder-8b-Q6_K.gguf`. Сохраните этот файл в отдельном каталоге на своем компьютере.
- Далее в том же каталоге создайте новый текстовый файл с названием `Modelfile`. Добавьте в него следующую строку кода:
- `FROM ./llama-3-sqlcoder-8b-Q6_K.gguf.`
- Под конец вернитесь в терминал и выполните команду `ollama create llama-sqlcoderq6 -f Modelfile` внутри того же каталога.
- Через некоторое время Ollama создаст и настроит новую модель LLM, с которой можно будет работать.

Для взаимодействия с моделями и управления ими Ollama предоставляет полезный REST API. Чтобы воспользоваться REST API, нужно либо установить в терминале CURL или WGET, либо воспользоваться любым REST-клиентом, например Postman.

- Чтобы загрузить модель, выполните следующий запрос:

```
curl http://localhost_OR_IP_ADDRESS:11434/api/generate -d
'{
  «model»: «gemma2"
}'
```


- Для получения списка моделей, в настоящее время загруженных в память, введите следующий запрос:

```
curl http://localhost_OR_IP_ADDRESS:11434/api/ps
```

- Чтобы сгенерировать потоковый запрос:

```
curl http://localhost_OR_IP_ADDRESS:11434/api/generate -d
'{
  "model": "llama3",
  "prompt": "Tell me a joke"
}'
```

Ответ будет выдаваться потоком. Полную документацию REST API можно найти [здесь](#).

Дополнительные настройки

Если вы используете Ollama в системе Linux — например, в Ubuntu, полезно будет знать следующие важные команды.

- **systemctl stop ollama.service** Останавливает службу ollama.service с помощью команды systemctl, являющейся частью менеджера систем и служб в системах на базе Linux. Так можно завершить службу Ollama, запущенную в данный момент. Чаще всего эту команду используют в тех средах, где Ollama настроена как служба systemd.
- **systemctl disable ollama.service** Отключает службу ollama.service и предотвращает ее автоматический запуск во время загрузки системы. Отключить службу таким образом бывает полезно, если вы хотите, чтобы Ollama не запускалась при загрузке, а работу данной службы можно было контролировать вручную.
- **systemctl status ollama.service** Проверяет и отображает текущее состояние службы ollama.service: является ли служба активной, неактивной, запущенной или остановленной.
- **sudo journalctl -u ollama.service > ollama_logs.txt** Позволяет с помощью утилиты journalctl получить журналы

службы `ollama.service` и перенаправляет их вывод в файл с именем `ollama_logs.txt`.

Данные команды нужны для эффективного управления службой Ollama и позволяют пользователям запускать, останавливать, отключать и проверять состояние службы по мере необходимости. Они предоставляют структурированный способ управления поведением Ollama в среде эксплуатации или в среде разработки.

Удаление локальной платформы LLM

Если вы хотите удалить Ollama из вашей системы Linux или установить ее заново, выполните следующие действия:

- Остановите сервер Ollama: `systemctl stop ollama.service`.
- Отключите службу Ollama: `systemctl disable ollama.service`.
- Удалите файл службы: `sudo rm /etc/systemd/system/ollama.service`.
- Удалите бинарный дистрибутив Ollama: `sudo rm $(which ollama)`.
- Удалите загруженные модели: `sudo rm -r /usr/share/ollama`.
- По желанию также можно удалить пользователя и группу Ollama: `sudouserdelollama sudo groupdel ollama`.

Приведенные выше команды должны успешно удалить Ollama из системы Linux. Если в дальнейшем вы решите установить платформу Ollama заново, обратитесь к разделу «Установка и настройка LLM с локальным инференсом» и следуйте пошаговым инструкциям.

Чтобы полностью удалить Ollama из системы macOS, нужно выполнить вручную два следующих действия:

- Удалить приложение Ollama из папки Applications.
- Удалить папку `ollama` из каталога пользователя, чтобы стереть все загруженные LLM.

Установка клиента с графическим интерфейсом пользователя (GUI) для работы с локальным LLM

До сих пор мы настраивали локальную установку LLM, чтобы пользоваться моделью LLM через командную строку или REST API. Но такой метод для многих бывает неудобным. К счастью, сообщество LLM и энтузиасты разработали множество клиентских приложений с веб-интерфейсом в виде отдельных программ или инструментов для командной строки, упрощающих процесс взаимодействия с моделью. Чтобы узнать больше о различных инструментах, доступных для упрощения взаимодействия с LLM, обязательно загляните в раздел «Интеграция с сообществом» (*Community Integration*) страницы Ollama на GitHub. Там вы найдете исчерпывающий список клиентских инструментов, созданных сообществом LLM.

Для примера попробуем установить самый первый инструмент (Open WebUI), указанный в разделе «Интеграция с сообществом» на странице Ollama на GitHub. Этот пример покажет, как интегрировать в свой рабочий процесс созданные сообществом инструменты, и поможет получить представление об их возможностях.

На той же странице перечислены основные возможности Open WebUI. Этот инструмент предоставляет богатый веб-интерфейс, похожий на интерфейс ChatGPT. С помощью Open WebUI можно подключаться к любым сервисам LLM — как к локальным, установленным на платформе Ollama, так и к сторонним — таким как OpenAI.

Чтобы установить Open WebUI, выполните следующие шаги:

- Убедитесь, что в вашей системе установлен и настроен Docker.
- Если вы хотите установить Open WebUI на ту же машину, где запущена Ollama, воспользуйтесь следующей командой:

```
docker run -d -p 3000:8080 --add-host=host.docker.internal:
host-gateway\
```

```
> v open-webui:/app/backend/data —name open-webui —restart  
always ghc
```

```
r.io/open-webui/open-webui: main
```

- Если вы хотите установить Open WebUI на другой машине и использовать Ollama в качестве службы, выполните следующую команду:

```
docker run -d -p 3000:8080 -e OLLAMA_BASE_URL=https://IP_  
ADDRESS_OF_OLLAMA_SERVER -v open-webui:/app/backend/data -  
name open-webui -restart  
always ghcr.io/open-webui/open-webui: main
```

После установки веб-интерфейс Open WebUI будет доступен по адресу `http://IP_ - ADDRESS_OPENWEB:3000`.

Теперь, когда вы выполнили данный шаг, вы можете создать новый чат в веб-интерфейсе и начать работать с LLM, запущенной на Ollama.

Инфо

В данной книге для работы с LLM мы не используем никаких инструментов типа *OpenWebUI*. В будущих изданиях книги этот подраздел может быть удален.

Настройка виртуальной среды Python для разработки ИИ

До сих пор мы настраивали локальную среду исполнения и вывода («инференса»), которая позволяет взаимодействовать с LLM напрямую. С помощью этой среды можно задавать вопросы, резюмировать документы, обрабатывать изображения и многое другое. Но этого недостаточно для разработки приложений на основе LLM. Для того чтобы подняться на новый уровень в данной сфере, вам понадобится определенный набор инструментов — подобно тому,

как для изучения языка программирования требуется компилятор или интерпретатор. В данном случае необходимо следующее.

- Компилятор или интерпретатор для выбранного нами языка программирования (которым, как мы решили, будет Python).
- Интегрированная среда разработки (IDE) для написания и компиляции кода.

Волноваться не следует, так как установка Python относительно проста. В системах Linux и macOS он часто бывает предустановлен изначально, так что, скорее всего, уже у вас есть. Однако мы не рекомендуем пользоваться для разработки приложений системным языком Python. Это связано с тем, что изменение или обновление системных библиотек может сделать вашу операционную систему уязвимой для взломов, из-за чего, возможно, потребуется ее полная переустановка. Чтобы избежать данного риска, мы воспользуемся отдельной установкой Python и настроим среду специально для разработки. Процесс установки будет описан далее.

Установка Python 3

Для установки Python 3 на macOS или Linux выполните описанные ниже шаги.

Установка Python 3 на macOS.

- Проверьте, не установлен ли уже у вас Python. Откройте терминал и напечатайте:

```
python3 --version
```

Если Python 3 установлен, то отобразится номер его версии. Если нет, продолжайте.

- Установите Python 3 с помощью Homebrew. Homebrew — это популярный менеджер пакетов для macOS. Выполните следующую команду Homebrew:

```
brew install python
```

- После установки удостоверьтесь в том, что Python установлен, вновь проверив его версию:

```
python3 --version
```

Установка Python 3 на Linux.

- Обновите список пакетов. Для этого откройте терминал и наберите команду:

```
sudo apt update
```

- Установите Python 3. Для систем на основе Debian, например, Ubuntu, выполните команду:

```
sudo apt install python3
```

- Проверьте установку Python 3 следующей командой:

```
python3 --version
```

Эти шаги позволят установить Python 3 как на macOS, так и на Linux. После установки можно будет пользоваться python3 для запуска скриптов Python.

Установка менеджера пакетов Python pip3

Python pip3 — это менеджер пакетов для языка Python, который используется для установки, обновления и настройки сторонних пакетов (также называемых «библиотеками» или «модулями»), написанных на Python. Чтобы установить менеджер пакетов Python pip3 на macOS и Linux, выполните следующие действия.

Установка pip3 на macOS.

- Проверьте, не установлен ли уже pip3. Откройте терминал и введите:

```
pip3 --version
```

Если он уже установлен, то вы увидите номер его версии.

- Установите `pip3` с помощью Homebrew. Если `pip3` не установлен, но у вас есть Homebrew, то установить `pip3` можно вместе с Python 3 (поскольку `pip3` поставляется в комплекте с Python 3):

```
pip3 --install python
```

Установка `pip3` на Linux.

- Обновите список пакетов. Для этого откройте терминал и наберите команду:

```
sudo apt update
```

- Установите `pip3`. Для систем на основе Debian, например, Ubuntu, выполните команду:

```
sudo apt install python3
```

- Проверьте установку `pip3` следующей командой:

```
pip3 --version
```

Альтернативный метод установки: с помощью команды `get-pip.py`

Если в вашем менеджере пакетов `pip3` недоступен, можно установить его самостоятельно, воспользовавшись скриптом `get-pip.py`.

- Загрузите его:

```
curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
```

- Выполните скрипт в Python 3:

```
python3 get-pip.py
```

- Проверьте установку:

```
pip3 --version
```

Следуя этим шагам, вы сможете установить `pip3` как на macOS, так и на Linux, что упростит управление пакетами Python в ваших проектах.

Установка и настройка Miniconda

Conda — это мощный менеджер пакетов и система управления окружением для языка Python и других языков программирования. Ниже приведено пошаговое руководство по установке и настройке Conda на macOS. Установить этот инструмент на операционные системы на базе Debian, таких как Ubuntu, можно аналогичным способом, воспользовавшись командой `sudo apt-get install` в терминале.

Шаг 1. Загрузите установщик Miniconda.

Выберите подходящий установщик Miniconda.

- Зайдите на страницу загрузки Miniconda и выберите установщик для macOS и Python.
- Загрузите установщик.pkg для macOS.

Шаг 2. Запустите установщик.

1. Запустите установщик.

- Найдите загруженный файл.pkg (обычно он находится в каталоге Downloads («Загрузки») и дважды щелкните по нему, чтобы запустить программу установки.
- Следуйте указаниям по установке Miniconda.

2. Завершите установку.

- После завершения установки вы увидите окно подтверждения. Нажмите Close, чтобы выйти из программы установки.

Шаг 3. Проверьте установку.

1. Проверьте установку Conda.

- Напечатайте в терминале следующую программу и нажмите Enter:

```
conda - version
```

- Должна появиться надпись с версией Conda, говорящая о том, что установка прошла успешно.

2. Дополнительный шаг.

Можно настроить работу с Conda, указав для переменной окружения CONDA_HOME путь к установке Miniconda.

```
export CONDA_HOME=/opt/miniconda3  
export PATH="$CONDA_HOME/bin:$PATH"
```

По умолчанию Miniconda устанавливается в каталог /opt/miniconda3.

Шаг 4. Инициализируйте Conda.

1. Инициализируйте Conda в оболочке shell.

- Для инициализации Conda выполните в терминале следующую команду:

```
conda init
```

- Эта команда настроит вашу оболочку на использование Conda.

2. Перезагрузите терминал.

- Закройте и вновь откройте терминал, чтобы применить внесенные изменения.

Шаг 5. Создайте среду Conda и управляйте ею.

1. Создайте новую среду.

- Чтобы создать новое окружение Conda, выполните следующую команду:

```
conda create -name ai_book_env
```

- Нашей средой для разработки приложений будет ai_book_env.

2. Активируйте среду.

- Для активации среды выполните следующую команду:

```
conda activate ai_book_env
```

3. Деактивируйте среду.

Для деактивации среды выполните следующую команду:

```
conda deactivate
```

Шаг 6. Дополнительная настройка (необязательный).

1. Настройте каналы Conda.

- Для поиска пакетов Conda использует каналы. При необходимости можно добавить дополнительные. Например, чтобы добавить канал conda-forge, выполните следующую команду:

```
conda config --add channels conda-forge
```

2. Обновите Conda.

- Хорошей идеей будет поддерживать Conda в актуальном состоянии. Обновить ее можно с помощью следующей команды:

```
conda update conda
```

Выполнив данные шаги, вы установите и настроите Conda на вашем компьютере с macOS, тем самым подготовившись к эффективному управлению окружениями и пакетами.

Установка IDE (интегрированной среды разработки) JupyterLab

JupyterLab (ранее известная как iPython Notebook) — интерактивная среда программирования с блокнотами, позволяющая создавать документы с так называемым «живым кодом» (live code), формулами, визуализацией и комментариями. Этот мощный инструмент широко используется специалистами по машинному обучению для разработки приложений, связанных с искусственным интеллектом (ИИ).

Установить JupyterLab можно напрямую через менеджер пакетов pip3, но мы воспользуемся для ее установки инструментом Miniconda.

Для этого нужно выполнить следующие шаги.

1. Создайте новую среду conda (необязательно, но рекомендуется).

Для каждого отдельного проекта лучше всего создавать новую среду (окружение). Для JupyterLab можно создать новую среду с определенной версией Python, как показано ниже:

```
conda create -n jupyterlab-env python=3.11
```

Замените 3.11 на версию Python, которую вы хотите использовать. Когда появится запрос, введите y, чтобы продолжить установку.

2. Активируйте новую среду.

После создания среды активируйте ее следующей командой bash:

```
conda activate jupyterlab-env
```

Замените `jupyterlab-env` на имя вашей среды, если вы использовали другое имя.

3. Установите JupyterLab.

Теперь установите в активированную среду JupyterLab:

```
conda install -c conda-forge jupyterlab
```

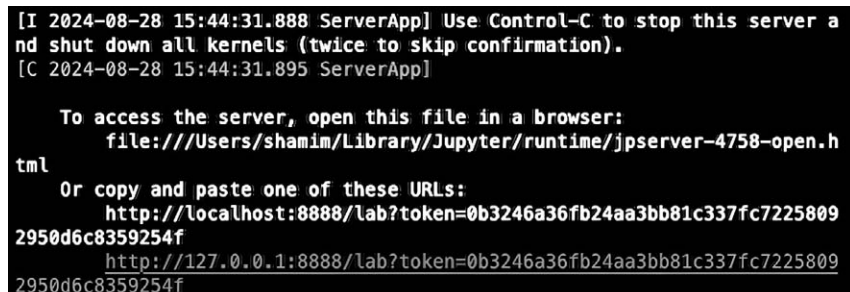
Эта команда устанавливает JupyterLab и все зависимости с канала `conda-forge`, который широко используется в сообществе для распространения пакетов `conda`.

4. Запустите JupyterLab.

После завершения установки можно запустить JupyterLab с помощью следующей команды:

```
jupyter lab
```

После этого Jupyter Lab автоматически откроется в вашем веб-браузере по умолчанию. Если этого не произошло, не волнуйтесь. Можно вручную открыть Jupyter Lab, скопировав и вставив URL-адрес из консоли Miniconda (см. иллюстрацию 1.1) в предпочитаемый вами веб-браузер.



```
[I 2024-08-28 15:44:31.888 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2024-08-28 15:44:31.895 ServerApp]

To access the server, open this file in a browser:
file:///Users/shamim/Library/Jupyter/runtime/jpserver-4758-open.html
Or copy and paste one of these URLs:
http://localhost:8888/lab?token=0b3246a36fb24aa3bb81c337fc72258092950d6c8359254f
http://127.0.0.1:8888/lab?token=0b3246a36fb24aa3bb81c337fc72258092950d6c8359254f
```

Иллюстрация 1.1

Следуя данным шагам, вы настроите JupyterLab в среде conda, что позволит эффективно работать с блокнотами Python и другими инструментами обработки данных.

Теперь, после завершения настройки, создайте новый блокнот Jupyter и напишите какой-нибудь фрагмент кода на Python-код с функцией `print`, чтобы протестировать среду, как показано ниже:

```
print("Hello world")
```

Установка и настройка базы данных SQLite

На протяжении всей книги для примеров мы несколько раз воспользуемся базой данных SQLite. Вы можете воспользоваться любой другой системой баз данных, хотя мы настоятельно советуем выбрать именно SQLite, так как это значительно упростит чтение и анализ программного кода.

Чтобы установить SQLite3 на macOS или ОС на основе Debian, выполните следующие шаги.

Установка SQLite3 на macOS.

- Установите SQLite3 с помощью Homebrew, выполнив команду:

```
brew install sqlite
```

- Установите переменные среды. Следуйте инструкциям консоли и добавьте все необходимые переменные окружения в файл `~/bash_profile`.

```
export LDFLAGS="-L/opt/homebrew/opt/sqlite/lib"
export CPPFLAGS="-I/opt/homebrew/opt/sqlite/include"
export PYTHON_CONFIGURE_OPTS="--enable-loadable-sqlite-extensions"
```

Проверьте, установлена ли SQLite3, напечатав:

```
sqlite3 --version
```

Установка SQLite на ОС на основе Debian.

- Обновите список пакетов, открыв окно терминала, чтобы у вас имелась последняя информация о доступных пакетах:

```
sudo apt update
```

- Установите SQLite3 с помощью следующей команды:

```
sudo apt install sqlite3
```

Если вам нужен графический пользовательский интерфейс для управления базами данных SQLite, можно воспользоваться DBeaver или DB Browser для SQLite.

Дополнительные настройки

Итак, все почти готово! Осталось всего несколько шагов до завершения настройки. Как уже говорилось, на протяжении всей книги для доступа к моделям и API мы будем пользоваться сторонними сервисами вроде Hugging Face и Groq. Для этого потребуются токены для аутентификации и авторизации на них. Для завершения подготовительного этапа создадим эти токены. Если они у вас уже есть, можете перейти к следующему этапу.

Нам понадобятся токены доступа и API-ключи от следующих сервисов.

1. Hugging face (токены доступа).
2. Groq (API-ключи).
3. LangSmith (API-ключи).

Процесс создания токенов доступа или API-ключей для различных сервисов прост и для большинства платформ одинаков. Ниже приведена общая инструкция по их созданию.

- Зарегистрируйтесь на сервисе с помощью электронной почты или учетной записи Google/Apple.

- Зайдите в настройки учетной записи.
- Перейдите в раздел «Токены доступа» или «API-ключи».
- Создайте или сгенерируйте новый токен доступа или API-ключ.

После создания токенов доступа или API-ключей надежно сохраните их в файловой системе своего компьютера для дальнейшего использования. Кроме того, эти учетные данные можно сохранить в качестве переменных окружения, чтобы не вводить их каждый раз вручную:

```
export HF_TOKEN=ВАШ_HUGGINGFACE_TOKEN
export LANGCHAIN_API_KEY=ВАШ_LANGCHAIN_API_КЛЮЧ
export GROQ_API_KEY=ВАШ_GROQ_API_КЛЮЧ
```

Вот и все! Теперь вы готовы сделать следующий шаг в захватывающий мир программирования искусственного интеллекта (ИИ).

Создание первого приложения с локальной LLM

Теперь, установив большую языковую модель (LLM) и настроив окружение, мы готовы перейти к разработке простой программы на Python с использованием локальной LLM. Основная цель этого упражнения — убедиться, что вся наша среда, включая LLM с локальным инференсом, работает правильно.

Для простоты и понятности воспользуемся нативным Python API Ollama.

Шаг 1. Перед тем как продолжить, убедитесь, что LLM Ollama Runner запущен и работает с загруженной LLM-моделью. Если нет, наберите в терминале следующую команду:

```
ollama run llama3.1: latest
```

Шаг 2. Активируйте среду Miniconda с именем jupyterlab-env с помощью следующей команды:

```
conda activate jupyterlab-env
```

Шаг 3. Запустите JupyterLab, введя следующую команду:

```
jupyter lab
```

Шаг 4. Создайте новый блокнот Jupyter.

Шаг 5. Переименуйте файл HelloWorld.

Шаг 6. В блокноте Jupyter добавьте следующий блок кода на языке Python:

```
!pip install ollama
import ollama
from ollama import chat
messages = [
    {
        'role': 'user',
        'content': 'Write a python program to calculate
factorial',
    },
]
from ollama import Client
client = Client(host='http://IP_ADDRESS:11434')
response = client.chat(model='llama3.1',
messages=messages)
print(response['message']['content'])
```

В данном блоке библиотека ollama используется для взаимодействия с ИИ-моделью под названием llama3.1 посредством чата. Ниже приведено пошаговое объяснение того, что делает каждая часть кода.

- Команда `!pip install ollama` используется для установки пакета Python `ollama`. Восклицательный знак `!` в начале указывает на то, что это команда оболочки, которая обычно используется в блокнотах Jupyter. Пакет `ollama` должен предоставлять функционал для взаимодействия с ИИ-моделями.
- `import ollama` Эта строка импортирует модуль `ollama`, делая все его функции и классы доступными для использования в скрипте.
- `from ollama import chat` Эта строка импортирует из модуля `ollama` функцию чата, благодаря чему ее можно использовать напрямую, без префикса `ollama`.
- `messages` Определяется список с именем `messages`, внутри которого находится один словарь. Этот словарь представляет собой сообщение, которое будет отправлено ИИ-модели. Ключ `role` указывает на роль отправителя сообщения, которым в данном случае является пользователь `user`. Содержание `content` ключа содержит собственно текстовое сообщение: «Напиши программу на языке python для вычисления факториала». Это так называемый «промт», с помощью которого мы просим модель искусственного интеллекта сгенерировать Python-код для вычисления факториала.
- `from ollama import Client` Эта строка импортирует класс `Client` из модуля `ollama`.
- `client = Client(host='http://IP_ADDRESS:11434')` Здесь мы создаем экземпляр класса `Client`, указывая URL хоста LLM-сервера. Если `Ollama` запускается локально на том же хосте, что и ваша среда Python, можно использовать `localhost`.

Инфо

В этом примере мы используем настраиваемый экземпляр `Client` с явным указанием хоста для подключения к (удаленному) LLM-серверу. Этот код будет работать независимо от топологии `Ollama`: локальной или удаленной.

`print(response['message']['content'])`: Эта строка выводит содержимое ответа ИИ-модели в консоль. В данном случае должна

появиться рабочая реализация функции вычисления факториала, напечатанная в виде кода Python.

Запустите блокнот, нажав кнопку **Restart the kernel and run all cells** («Перезапустить ядро и запустить все ячейки»).

После того как модель llama3.1 немного поработает, должно появиться примерно такое сообщение:

```
def factorial(n):
    """
    Calculate the factorial of a number.
    Args:
        n (int): The input number.
    Returns:
        int: The factorial of the input number.
    """
    if not isinstance(n, int):
        raise TypeError("Input must be an integer.")
    if n < 0:
        raise ValueError("Input must be non-negative.")
    elif n == 0 or n == 1:
        return 1
    else:
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result
# Example usage
print(factorial(5)) # Output: 120
```

Можно скопировать и вставить полученный код в другой блокнот Jupyter и попытаться его запустить. Программа должна вернуть `Output:120`. Если же она не сработает, не волнуйтесь. Главное — добиться ответа от модели llama3.1 и получить от нее пример кода. Если код получен, то все работает нормально, и ваша установка прошла базовый тест. Исходный код блокнота доступен [здесь](#).

Устранение ошибок

При использовании библиотеки `ollama` в Python-коде (как было показано выше) могут возникнуть некоторые известные проблемы. Вот возможные ошибки и их решения.

`ConnectError: [Errno 61] Connection refused`

- › причина: ошибка возникает, если сервер по указанному адресу не работает или отключен, или если указан неверный IP-адрес/порт;
- › решение: убедитесь, что по указанному IP-адресу и порту сервер работает, а также, что вы указали правильный адрес хоста и правильный порт.

`ResponseError: model "XYZ" not found, try pulling it first`

- › причина: указанная модель (XYZ) недоступна на сервере;
- › решение: проверьте правильность названия модели. Убедитесь, что модель загружена и доступна на сервере.

Эти ошибки могут возникать из-за проблем с конфигурацией, неправильного использования библиотеки `ollama`, проблем с подключением к сети или неожиданных ответов сервера. Во избежание описанных ошибок убедитесь в правильности конфигурации.

Аппаратное ускорение

При работе с искусственным интеллектом неизбежно возникает вопрос производительности, особенно при загрузке больших моделей или во время дообучения на огромном наборе данных. Кроме того, традиционные процессоры сами по себе с трудом справляются с вычислительными требованиями современных ИИ-приложений. На это есть несколько причин.

1. **Сложность моделей.** ИИ-модели, особенно модели глубокого обучения, подразумевают выполнение сложных математических

операций, требующих большого количества вычислений. Центральные процессоры компьютеров (CPU), даже высокопроизводительные, с трудом справляются с данными требованиями.

2. **Масштабируемость.** По мере увеличения размера и сложности ИИ-моделей требования к вычислениям растут в геометрической прогрессии. Центральные процессоры становятся при этом слабым звеном, ограничивая производительность и масштабируемость модели.
3. **Матричные операции.** Многие ИИ-алгоритмы включают в себя матричные операции (например, умножение матриц), не оптимизированные для выполнения на центральных процессорах. С такими операциями могут эффективно справляться специализированные аппаратные ускорители, значительно сокращающие время вычислений.
4. **Пропускная способность памяти.** ИИ-модели часто требуют большого объема памяти для хранения весов, смещений и промежуточных результатов. Процессоры обычно имеют ограниченную пропускную способность памяти, что приводит к медленной передаче данных и увеличению времени обработки.
5. **Параллелизация.** Аппаратные ускорители, такие как GPU (*Graphics Processing Units*, графические процессоры) или TPU (*Tensor Processing Units*, тензорные процессоры), могут выполнять тысячи вычислений параллельно, в то время как центральные процессоры ограничены выполнением нескольких десятков инструкций одновременно.

Примите к сведению, что для изучения и разработки базовых концепций ИИ вам вовсе не требуются специальный центральный процессор или высокопроизводительное оборудование (за исключением процесса дообучения LLM). Восьмиядерного процессора Intel или AMD с 16 ГБ оперативной памяти вполне будет достаточно для тестирования и реализации таких ИИ-функций, как эмбединг, обработка изображений и простых приложений в виде агентов. Хотя без аппаратного ускорения процесс может идти медленнее, для образовательных целей его скорость вполне приемлема.

Для преодоления этих ограничений можно воспользоваться следующими специальными аппаратными ускорителями.

- GPU (графические процессоры). Предназначены для матричных операций и параллельной обработки.
- TPU. Оптимизированы для вычислений, применяемых в сетях глубокого обучения, таких как сверточные нейронные сети (CNN).
- ASIC (Application-Specific Integrated Circuits, специализированные интегральные схемы). Микросхемы, разработанные для решения конкретных ИИ-задач, — например, распознавания изображений или речи. В частности, такие микросхемы используются в платформе инференса Groq.
- AVX/AVX2. Передовая технология векторных вычислений от Intel и AMD.

С помощью этих специализированных аппаратных ускорителей можно:

- Сократить время вычислений. Ускорить время обучения и инференса ИИ-моделей.
- Увеличить размер модели. Обучать более крупные и сложные модели без ущерба для производительности.
- Улучшить масштабируемость. Быстрее и лучше разворачивать крупномасштабные ИИ-модели.

Если вам нужно срочно повысить производительность при работе с ИИ, можно рассмотреть различные варианты, некоторые из которых мы опишем в заключительном разделе данной главы.

Рабочая станция с GPU

Как мы уже говорили, графические процессоры предназначены для эффективного выполнения параллельных вычислений, что делает их крайне полезными для таких ИИ-задач, как подготовка моделей глубокого обучения. Благодаря своей способности выполнять тысячи вычислений одновременно они значительно ускоряют процесс по сравнению с традиционными процессорами (CPU).

Если позволяет бюджет, то инвестиции в высокопроизводительную рабочую станцию с отдельным графическим процессором (например, NVIDIA RTX или Tesla) могут оказаться весьма оправданными. При этом нужно убедиться, что графический процессор имеет достаточно видеопамяти (VRAM) для работы с большими наборами данных и моделями. Этот вариант экономически достаточно эффективен, но для общих применений ИИ мы рекомендуем рассмотреть и другие варианты.

Если вы используете старую рабочую станцию с видеокартой Nvidia, которая поддерживает технологию CUDA (например, модель MacBook Pro 2014 года), можно попробовать включить ускорение с использованием GPU в таких ИИ-фреймворках как TensorFlow или PyTorch, установив необходимые драйвера и библиотеки (включая CUDA и cuDNN).

Стоит отметить, что компьютеры MacBook на базе процессоров Apple Silicon (включая модели M1/M2/M3) для выполнения операций LLM задействуют возможности встроенных GPU. Вот пример результатов работы Ollama на MacBook Pro с процессором M1:

```
NAME ID    SIZE PROCESSOR UNTIL
llama3.1: latest 62757c860e01 6.7 GB 100% GPU 4 minutes
from now
```

Включение AVX/AVX2 для ускорения работы процессора

AVX (Advanced Vector Extensions) и его преемник, AVX2 — это наборы инструкций для процессора, повышающие производительность требовательных приложений, включая ИИ-модели и процессы машинного обучения. Они позволяют процессору обрабатывать сразу несколько элементов данных за одну инструкцию, что оптимизирует ресурсоемкие вычисления.

Большинство современных процессоров Intel и AMD оснащены поддержкой AVX/AVX2 уже «из коробки». Чтобы проверить, поддерживает ли их ваш процессор, в Linux можно воспользоваться следующей командой:

```
grep -o 'avx[^]*' /proc/cpuinfo
```

Включив для своего процессора AVX/AVX2, вы получите значительный прирост производительности, особенно для LLM в Ollama. Ollama по умолчанию использует данную опцию для более эффективного использования ресурсов процессора. Ниже приведен фрагмент журнала Ollama, из которого видно, что при работе LLM действительно применяются расширения AVX/AVX2:

```
time=2024-08-27T15:17:09.087+03:00 level=INFO  
source=payload.go:44  
msg=\ "Dynamic LLM libraries [cpu cpu_avx cpu_avx2]"
```

Включение AVX/AVX2 — наиболее экономичный вариант повышения общей производительности системы без необходимости использования дополнительного оборудования.

Использование сторонней платформы ASIC или VPS с поддержкой GPU

Прикладные специализированные интегральные схемы (ASIC, Application-Specific Integrated Circuits) — это специализированное оборудование, оптимизированное под конкретные задачи, связанные, в частности, с искусственным интеллектом и глубоким обучением. Они обеспечивают превосходную производительность и энергоэффективность для больших рабочих нагрузок. Пионер технологии ASIC — компания Groq, использующая ее для своей платформы ИИ-инференса. Groq предоставляет разработчикам бесплатный API для использования своих языковых моделей, обеспечивающий большую гибкость без предварительных затрат. Несмотря на существующие в данный момент некоторые ограничения на использование данной технологии, включая тарифы на время (в минуту или в день) и на количество токенов, возможности пока что довольно щедры, и предоставляемых услуг достаточно для ежедневных потребностей отдельных пользователей.

Кроме того, доступ к мощным вычислительным ресурсам без необходимости приобретения и обслуживания собственного оборудова-

ния предоставляют виртуальные частные серверы (VPS) с поддержкой GPU. В качестве примера можно привести сервис TPU (Tensor Processing Unit) от Google, AWS с графическими процессорами NVIDIA или других облачных провайдеров, таких как Azure и IBM Cloud. Настройка своей ИИ-среды на данных платформах позволит эффективно использовать доступные вам ресурсы.

Использование сервиса Google Colab или Kaggle

Google Colab и Kaggle — это бесплатные облачные платформы, предоставляющие доступ к мощным вычислительным ресурсам, включая GPU и TPU. Они идеально подходят для частных лиц и небольших команд, которым требуется высокая производительность без вложений в дорогостоящее оборудование.

Для этого достаточно просто зарегистрироваться в Google Colab или Kaggle, после чего можно писать и выполнять код в блокнотах Jupyter прямо в браузере. Обе платформы предлагают предустановленные библиотеки и простую интеграцию с популярными фреймворками глубокого обучения — такими как TensorFlow, PyTorch и Keras. Также с их помощью можно воспользоваться ускорением GPU и TPU с помощью нескольких щелчков мыши.

Но у этих сторонних сервисов есть свои ограничения, и их использование с личными наборами данных может привести к утечке информации.

С помощью указанных стратегий вы сможете значительно повысить производительность и эффективность ИИ-процессов с одновременным сокращением времени для обучения и развертывания ИИ-моделей.

Заключение

В данной главе мы рассмотрели основные шаги по настройке и оптимизации локальных LLM для ИИ-программирования. Эти подготовительные задачи — от установки необходимых инструментов

и настройки окружения до генерации токенов доступа, помогут вам плавно перейти непосредственно к разработке ИИ. Практический пример разработки приложения на Python с использованием локальной LLM не только закрепит полученные знания, но и продемонстрирует потенциал данных моделей в реальных сценариях.

Мы также рассмотрели способы устранения самых распространенных ошибок, подчеркнув важность правильной настройки и управления зависимостями и конфигурациями. По мере усложнения ИИ-моделей ключевую роль начинает играть аппаратное ускорение. Специализированное оборудование, такое как GPU (графические процессоры) и TPU (тензорные процессоры), или оптимизация процессоров с помощью инструкций AVX/AVX2 могут значительно повысить производительность и масштабируемость ИИ-задач.

Наконец, мы обсудили различные решения для повышения производительности — от использования выделенных рабочих станций до облачных сервисов, благодаря которым можно преодолеть возможные недостатки своих моделей. Вооружившись данными знаниями, вы можете приступить к более сложным ИИ-проектам, пользуясь возможностями локальных LLM и аппаратных ускорителей для достижения эффективных результатов.

ГЛАВА 2

ПОГРУЖЕНИЕ В ТЕОРИЮ ГЕНЕРАТИВНОГО ИИ

Генеративный ИИ — это революционная область в сфере искусственного интеллекта, ориентированная на создание нового контента, от текста и программного кода до изображений, аудио и даже видео. В отличие от традиционных ИИ-систем, анализирующих и интерпретирующих существующие данные, генеративные модели изучают схемы (паттерны) и структуры огромных массивов данных, на основе которых выдают оригинальные и часто неожиданные результаты. Работая на основе алгоритмов глубокого обучения, такие модели могут имитировать творческие способности человека и генерировать контент, неотличимый от его работ. Это открывает целый мир возможностей: от автоматизации творческих задач и персонализации пользовательского опыта до ускорения научных открытий и расширения границ художественного самовыражения.

Что же собственно такое искусственный интеллект и генеративный ИИ? Искусственный интеллект — это, скорее, не какая-то революционная технология, а результат эволюции алгоритмов и технологий, развивавшихся в последнее десятилетие.

Одно из главных препятствий в изучении генеративного ИИ — это, на наш взгляд, непонимание базовой терминологии и принятых в данной сфере обозначений. Поэтому при исследовании генеративного искусственного интеллекта для начала нужно погрузиться в контекст.

Искусственный интеллект (ИИ)

Искусственный интеллект, или ИИ, — предмет отдельной сферы компьютерных технологий, занимающейся разработкой методов создания приложений или компьютерных систем, способных имитировать поведение человека. В этом смысле «искусственный интеллект» можно назвать отдельной дисциплиной, подобно тому, как физика — это отдельная дисциплина в рамках естественных наук.

Ключевые особенности ИИ следующие:

1. **Анализ.** ИИ-системы должны уметь обрабатывать информацию и делать логические выводы, подобно тому, как это делает человек.
2. **Обучение.** Цель разработок в области ИИ — дать машинам возможность обучаться на основе предлагаемых данных, выявлять закономерности в них и улучшать свою работу с течением времени без явного программирования для каждого отдельного сценария.
3. **Действие.** ИИ-системы должны на основе обработанной информации и исходя из поставленных целей предпринимать какие-то действия в реальном мире.

Существует несколько различных типов искусственного интеллекта.

Узкий или слабый ИИ. Предназначен для выполнения конкретных задач — например, игры в шахматы или выдачи рекомендаций по поводу товаров.

Общий или сильный ИИ. Гипотетический ИИ, обладающий интеллектом на уровне человека и способностью выполнять любые интеллектуальные задачи, которые тому под силу человеку

Искусственный сверхинтеллект. Гипотетический ИИ, превосходящий человеческий интеллект по всем параметрам.

Сфера ИИ быстро развивается и оказывает влияние на различные сферы человеческой деятельности, включая здравоохранение, финансовые технологии, транспорт и развлечения.

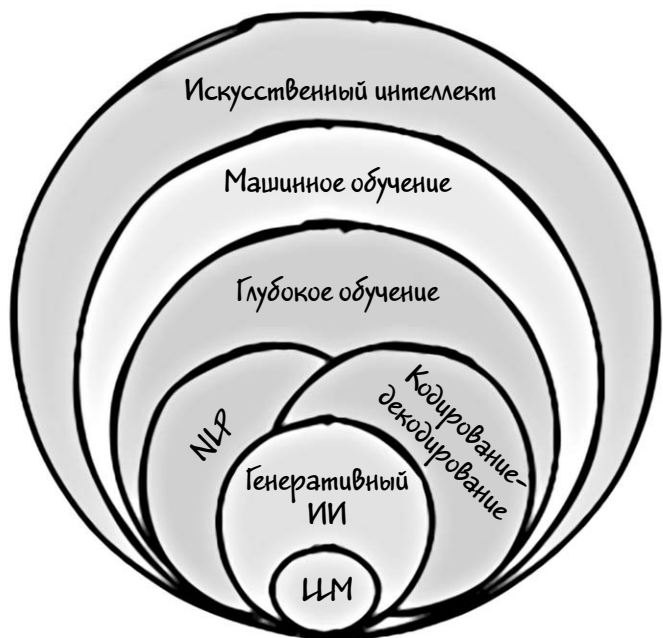


Иллюстрация 2.1

Как говорилось выше, сфера ИИ — это, скорее, дисциплина, нежели естественная наука, и она состоит из нескольких подобластей, как показано на иллюстрации 2.1. Вероятно, вы уже встречали подобные схемы, так как они широко распространены в Интернете и хорошо объясняют соотношение различных направлений исследований в области ИИ и их практических применений. Первое направление, с чего, собственно, и начинается ИИ, — это так называемое «машинное обучение».

Машинное обучение (ML)

Машинное обучение (Machine Learning) — направление в области создания и использования искусственного интеллекта (ИИ), иссле-

дующее методы и модели обучения компьютеров на основе данных без предварительного явного программирования.

Если вы когда-нибудь покупали что-нибудь на Amazon или eBay, то, скорее всего, уже взаимодействовали с моделями машинного обучения. Обычно на платформах электронной коммерции, таких как Amazon, используются системы рекомендаций, представляющие собой один из видов моделей машинного обучения. Они предлагают пользователям книги или другие товары, основываясь на их прошлом поведении или предпочтениях. Для более четкого понимания концепции машинного обучения рассмотрим следующий пример.

Предположим, на нашем сайте электронной коммерции имеется следующий набор данных.

ID пользо- вателя	Название книги	Жанр	Автор	Рей- тинг (1–5)
1	«Apache Ignite»	Технологии	Шамим Ахмед	5
1	«Генератив- ный ИИ. С чего начать»	Технологии	Шамим Ахмед	5
2	«Высокопроиз- водительные вычисления в памяти»	Технологии	Шамим Ахмед	4
3	«Марсианин»	НФ	Энди Вейер	4
3	«Дюна»	НФ	Фрэнк Герберт	5

Допустим, мы хотим предсказать предпочтения какого-то определенного пользователя и порекомендовать книгу на основе таких факторов, как его предпочтения в жанре (например, «техника», «нон-фикшн», «научная фантастика»), предыдущие оценки (книги, которые пользователь оценил высоко), популярность книги (средняя

оценка других пользователей) и история чтения (типы книг, которые пользователь часто читает).

Для выявления закономерностей в поведении различных пользователей и для предоставления им рекомендаций используются модели машинного обучения. Например, если пользователь А и пользователь В высоко оценили определенные книги, система может предложить книги, которые понравились пользователю А, но которые пользователь В еще не читал, и наоборот.

Инфо

Столбцы «**Жанр**» и «**Рейтинг (1–5)**» из набора данных содержат данные, которые в терминах машинного обучения называются «**признаками**» или «**фичами**». Эти признаки используются алгоритмами для **прогнозирования** или **классификации**.

Основная идея машинного обучения такова.

1. **Данные.** Алгоритмы машинного обучения требуют для обучения большое количество данных, которые могут быть любыми: текст, числа, показания датчиков и т.д.
2. **Алгоритм.** Это рецепт обучения, согласно которому компьютер анализирует данные и ищет в них закономерности. Существует множество различных типов алгоритмов, каждый из которых подходит для решения разных задач.
3. **Обучение.** Алгоритму передаются данные, и он настраивает свои внутренние параметры так, чтобы повысить эффективность выполнения конкретной задачи. Представьте, как ребенок учится узнавать кошек на картинках, со временем у него это получается все лучше и лучше.
4. **Прогнозирование.** После обучения алгоритм может делать предсказания на основе новых данных, которых он до этого не рассматривал. Например, он может порекомендовать книгу, которую пользователь еще не читал.

Эту концепцию можно проиллюстрировать в виде следующей схемы процесса машинного обучения, показанной на иллюстрации 2.2.

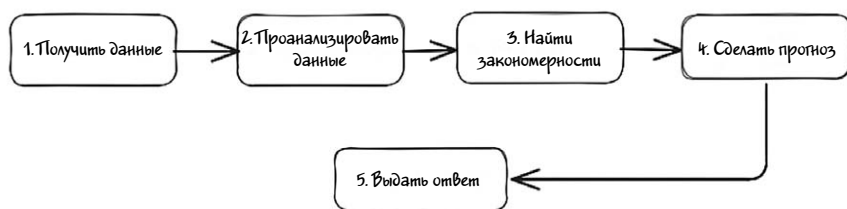


Иллюстрация 2.2

После того как инженер машинного обучения завершит обучение и оптимизацию алгоритма, тот будет выполнять стандартный процесс.

1. Получить новый входной сигнал в виде запроса пользователя.
2. Проанализировать данные.
3. Найти закономерность.
4. Сделать прогноз, выдать рекомендацию или осуществить классификацию.
5. Выдать результат пользователю.

Существует несколько типов алгоритмов машинного обучения.

Обучение с учителем (Supervised Learning). Алгоритм обучается на маркированных (помеченных) данных (например, взятых из таблиц базы данных). Он учится правильно сопоставлять входные данные с уже проставленными метками.

Обучение без учителя (Unsupervised Learning). Алгоритм обучается на немаркированных данных (например, изображениях собак, кошек и т.д.) и должен самостоятельно выявлять закономерности и взаимосвязи.

Обучение с подкреплением (Reinforcement Learning). Алгоритм обучается методом проб и ошибок, получая вознаграждение за правильные действия и наказание за неправильные.

Практические примеры машинного обучения.

Рекомендательные системы. Netflix предлагает пользователям посмотреть фильмы, которые могут им понравиться, Amazon рекомендует покупателям обратить внимание на конкретные товары.

Спам-фильтры. Идентификация и блокировка нежелательных писем.

Выявление мошенничества. Отметка подозрительных транзакций в банковской системе.

Машинное обучение совершает революцию во многих отраслях и сферах нашей жизни, и потенциал его продолжает расти. Если вы заинтересовались машинным обучением и хотите узнать больше, настоятельно рекомендуем прочитать несколько книг по данной теме и поэкспериментировать с некоторыми фреймворками, чтобы получить практический опыт.

Глубокое обучение (DL)

Глубокое обучение (Deep Learning) — это подобласть машинного обучения или, можно сказать, его особый вид. Технически процесс глубокого обучения тот же, что и в машинном обучении, но с использованием других методов и возможностей. Для глубокого изучения используют искусственные нейронные сети, с помощью которых выявляются закономерности в данных и связи между данными.

Ключевое различие заключается в том, что алгоритмы машинного обучения учатся на данных, выявляя *простые закономерности и взаимосвязи*, в то время как для глубокого обучения применяются искусственные нейронные сети с несколькими слоями (отсюда и слово «**глубокое**» в названии) взаимосвязанных узлов для изучения *сложных закономерностей и представлений* (признаков). Такие слои позволяют алгоритму постепенно все глубже анализировать данные и формировать некое понимание — подобно тому, как наш мозг обрабатывает информацию.

Искусственная нейронная сеть (Artificial Neural Network (ANN)) простыми словами.

Представьте, что ваш мозг состоит из миллиардов крошечных специализированных вычислительных блоков, называемых *нейронами* (или *узлами*), которые общаются друг с другом посредством связей (например, крошечных синапсов). Нейронная сеть — это упрощенная модель такой сложной системы, состоящая из искусственных узлов и их слоев и позволяющая классифицировать данные или составлять прогнозы на их основе. Каждый слой определенным образом анализирует и преобразует входные данные, благодаря чему сеть выявляет все более сложные закономерности и взаимосвязи.

На иллюстрации 2.3 показана схема искусственной нейронной сети из нескольких слоев, называемых **входным**, **выходным** и **скрытым слоями**. В каждом слое каждый **узел (нейрон)** связан со всеми узлами (нейронами) следующего слоя, и этим связям присваиваются определенные параметры, называемые **весами**.

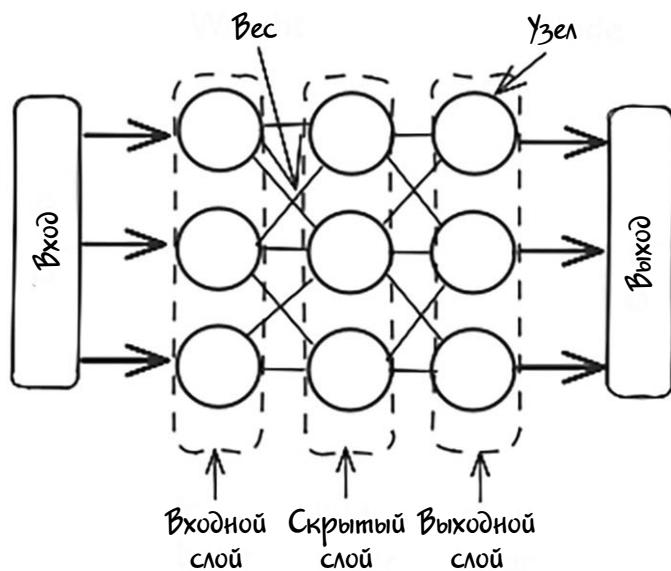


Иллюстрация 2.3

Рассмотрим пример: допустим, вам нужно распознать на картинке **велосипед**. Какая-нибудь простая компьютерная программа будет искать основные признаки — такие как **руль** или **колеса**. Но нейронная сеть глубокого обучения будет рассматривать несколько слоев деталей.

Слой 1: основные формы (например, круги, линии)

Слой 2: части объекта (например, руль, колеса)

Слой 3: объект в целом (например, велосипед)

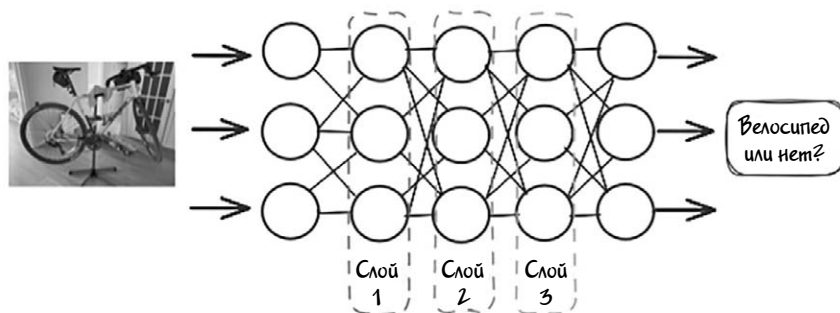


Иллюстрация 2.4

Каждый слой опирается на предыдущий, что позволяет сети изучать более сложные и тонкие характеристики изображения. Процесс определения связей и признаков называется обучением, в ходе которого в сеть подается множество примеров изображений велосипедов, а связи между узлами корректируются в зависимости от допущенных ошибок и неверных результатов классификации.

i Инфо

Обратите внимание, что в контексте глубокого обучения в данной книге будут широко использоваться такие термины, как **узлы**, **слои**, **веса**. Это основополагающие термины, необходимые для понимания различных концепций, включая тонкую настройку больших языковых моделей и обработку изображений.

Глубокое обучение включает в себя следующие процессы.

Извлечение признаков. Алгоритмы глубокого обучения могут автоматически извлекать из исходных данных релевантные признаки, благодаря чему устраняется необходимость в их ручном поиске.

Обработка сложных данных. Глубокие сети могут обрабатывать огромные объемы сложных данных, что делает их полезными для таких задач, как распознавание изображения или речи и обработка естественного языка.

Обобщение. Модели глубокого обучения, как правило, хорошо применяют полученные обобщения к новым данным, а это означает, что они могут удачно обрабатывать данные, которые ранее были им недоступны.

Примеры применения глубокого обучения в реальных сценариях.

- **Распознавание изображений.** Идентификация объектов, лиц и сцен на изображениях.
- **Распознавание речи.** Преобразование произнесенных слов в текст.
- **Самоуправляемые автомобили.** Анализ окружающей обстановки и принятие решений по управлению автомобилем.

Модели глубокого обучения можно первоначально обучать на огромных объемах данных, а затем настраивать для выполнения конкретных задач или работы в определенной сфере. Именно способность обрабатывать огромные массивы данных и обучаться на основе необработанного текста сделала модели глубокого обучения важным компонентом обработки естественного языка (NLP).

Обработка естественного языка (NLP)

Обработка естественного языка (Natural Language Processing) — одно из направлений ИИ (здесь и начинается путаница), посвященное компьютерному анализу (пониманию и интерпретации) и синтезу (генерации) человеческих высказываний и осмысленных текстов на естественных языках. Это направление находится на стыке лин-

гвистики, информатики и науки о данных, преодолевая тем самым разрыв между человеческим общением и машинным пониманием.

Сферы NLP и глубокого обучения тесно связаны между собой, причем модели глубокого обучения представляют собой мощные инструменты, значительно расширяющие возможности NLP. Взаимосвязь их следующая.

- **Нейронные сети.** Для глубокого обучения используются нейронные сети с несколькими слоями (так называемые глубокие нейронные сети), моделирующие сложные закономерности в данных. В NLP эти модели используются для обработки и понимания текста посредством обучения на основе больших массивов языковых данных.
- **Рекуррентные нейронные сети (RNN, Recurrent Neural Networks).** Поначалу для решения задач NLP методами глубокого обучения использовались RNN, обрабатывающие последовательные данные и выявляющие зависимости в тексте. Для устранения ограничений в работе с зависимостями между далеко расположенными фрагментами текста были введены такие элементы, как «долгая краткосрочная память» (LSTM, Long Short-Term Memory) и «управляемый рекуррентный блок» (GRU, Gated Recurrent Unit).
- **Сверточные нейронные сети (CNN, Convolutional Neural Networks).** Традиционно CNN использовались для обработки изображений, но их стали применять и для решения таких NLP-задач, как классификация предложений и распознавание именованных сущностей, так как сверточные сети позволяют хорошо выявлять локальные закономерности в тексте.

Инфо

NLP — это не отдельная часть «глубокого обучения». Нейронные сети глубокого обучения — в частности, модели глубокого обучения, используются для расширения возможностей NLP, так как позволяют выявлять сложные закономерности и представления на основе больших наборов данных.

NLP можно условно разделить на две области — **понимание естественного языка (NLU, Natural Language Understanding)** и **генерация естественного языка (NLG, Natural Language Generation)**. NLU фокусируется на том, чтобы помочь машинам понимать и анализировать человеческий язык, выявлять в нем понятия, сущности, эмоции и ключевые слова. NLG же предполагает создание на основе внутреннего представления данных связного и осмысленного текста, такого, как фразы, предложения и абзацы.

Модели, предназначенные для понимания и генерации текста на естественном языке, принято называть **языковыми моделями (LM, Language Models)**. Как правило, они основаны на рекуррентных нейронных сетях (RNN), которые доказали свою эффективность в выявлении дальних зависимостей в таких последовательных данных, как текст. Однако обучение LM на основе RNN требует больших вычислительных затрат, и в этих сетях часто наблюдается так называемое «переобучение», особенно при работе с длинными последовательностями слов.

В 2016 году появились модели глубокого обучения (особенно так называемые **«трансформеры»**), которые произвели настоящую революцию как в области понимания, так и в области генерации естественного языка. Благодаря этим моделям точность и эффективность NLP-предложений значительно повысились.

Перейдем теперь к изучению модели трансформера и рассмотрим ее структуру и принципы работы. Более подробно ознакомиться с концепцией NLP можно здесь.

Трансформер

Модель трансформера впервые была представлена в статье «Attention Is All You Need» («Все, что вам нужно, — это внимание»), описывавшей революционно новый метод построения сетей глубокого обучения и оказавшей огромное влияние на всю область обработки естественного языка (NLP). В отличие от традиционных моделей, таких как рекуррентные нейронные сети (RNN) и сверточ-

ные нейронные сети (CNN), в модели трансформера для выявления сложных связей между словами в предложении используются механизмы **самовнимания**. Способность обрабатывать дальние зависимости вне расстояния между элементами в тексте сделала трансформеры весьма полезным инструментом для решения широкого спектра задач NLP.

Основные компоненты (в оригинальной статье их гораздо больше) трансформеров следующие.

- Механизм самовнимания (Self-attention Mechanism). Этот механизм позволяет модели оценивать важность различных слов в предложении относительно друг друга и выявлять сложные взаимосвязи.
- Архитектура кодировщика-декодировщика. Необходима для таких задач, как машинный перевод. При этом кодировщик (кодер) обрабатывает входной текст, а декодировщик (декодер) генерирует выходной текст.

В следующих разделах мы подробно рассмотрим каждый из этих компонентов.

Механизм самовнимания

Механизм самовнимания (Self-Attention Mechanism) — ключевой компонент общего процесса, благодаря которому модель трансформера понимает естественный язык. Он помогает модели определять степень важности слов в предложении и их соотносённость друг с другом.

Допустим, вы читаете предложение типа **The dog barked loudly because it saw a stranger** («Собака громко залаяла, потому что увидела незнакомца»). Благодаря контексту вы автоматически понимаете, что слово **it** относится к собаке. Нечто подобное делает и механизм самовнимания — он рассматривает все слова в предложении и фокусируется на тех, которые важны для понимания друг друга.

Попробуем понять, как это работает, на примере приведенного выше предложения: *The dog barked loudly because it saw a stranger.*

- **Просмотр каждого слова.** Механизм самовнимания просматривает каждое слово в предложении и, дойдя до слова *it*, должен понять, к чему оно относится. К собаке? К лаю? К незнакомцу?
- **Фокусировка на важных словах.** Механизм рассматривает окружающие слова, такие как *dog* и *saw*, определяя, к какому из них относится слово *it*. При этом он не обращает внимания на менее важные слова, такие как *loudly* («громко») или *because* («потому что»), которые не помогают понять основную мысль.
- **Присвоение важности.** Модель присваивает каждому слову определенный вес (балл), уделяя больше внимания значимым словам. В данном случае слово *dog* получит более высокий балл, потому что имеет большее отношение к смыслу слова *it*.
- **Просмотр связей.** На основе связей модель понимает предложение в целом. Теперь она знает — собака лает, так как увидела незнакомца, что делает предложение понятным.

До разработки механизма самовнимания старые модели рассматривали слова по одному, часто упуская из виду важные связи между ними. Механизм самовнимания позволяет модели понимать значение слова на основе всех остальных слов в предложении, что делает ее намного умнее при выполнении таких задач, как перевод, резюмирование или даже завершение предложений.

Механизм самовнимания широко используется в повседневных приложениях, таких как Gmail или поисковые функции Google. Рассмотрим для примера сценарий, когда вы вводите в поиск Google запрос *I need a hotel that is close to the airport* («Мне нужен отель, который находится недалеко от аэропорта»). Механизм самовнимания распознает, что слово *close* («близко») относится к расположению отеля, а слово *airport* («аэропорт») указывает на расположение пункта назначения. Понимая весь контекст, а не только отдельные слова, модель выдает более точные результаты поиска.

Архитектура кодировщика-декодировщика

Архитектура кодировщика-декодировщика — особая структура сетей, используемая во многих языковых моделях, особенно тех, которые применяются для задач вроде перевода текста. Ее рабочий процесс состоит из двух этапов. Сначала одна часть сети — кодировщик («энкодер») — принимает входную информацию, а затем другая часть — декодировщик («декодер») — на основе этой информации генерирует выходную информацию. Данный процесс можно сравнить с работой переводчика, который сначала читает и понимает предложение на одном языке (энкодер), а затем переводит его на другой язык (декодер).

Механизм кодирования-декодирования — важнейшая часть любых моделей, позволяющая им справляться с такими сложными задачами, как перевод с одного языка на другой, резюмирование и даже поддержка чат-ботов. Кодировщик помогает модели понять входной сигнал (будь то предложение, абзац или изображение), а декодировщик генерирует соответствующий выходной сигнал, обеспечивая точность передачи смысла.

Чтобы полностью понять, как модели трансформеров обрабатывают информацию, разделим их архитектуру на два основных компонента: кодировщик и декодировщик. Рассмотрим каждый из этих важнейших элементов подробнее, с описанием их роли и функционала.

- Кодировщик. Понимание входного сигнала.
 - › Задача кодировщика состоит в том, чтобы принять входной сигнал — например, предложение, и понять его смысл. Если, допустим, на вход поступает предложение *The cat is sleeping on the sofa* («Кот спит на диване»), кодер обрабатывает каждое слово, понимает связи между ними и преобразует все предложение в некий код или резюме (так называемое «скрытое представление»).
 - › Это резюме отражает суть предложения.

- Декодировщик. Создание выходных данных.
 - › Задача декодера заключается в том, чтобы получить резюме от кодировщика и использовать его для создания желаемого результата. Например, если мы хотим перевести предложение The cat is sleeping on the sofa («Кот спит на диване») на французский язык, декодировщик на основе закодированного резюме сгенерирует французское предложение: Le chat dort sur le canapé.
 - › Декодировщик начинает с кодированного резюме и предсказывает по одному слову за раз, сверяясь с кодировщиком, чтобы убедиться, что процесс идет правильно.

Этот процесс можно проиллюстрировать схемой, показанной на иллюстрации 2.6.

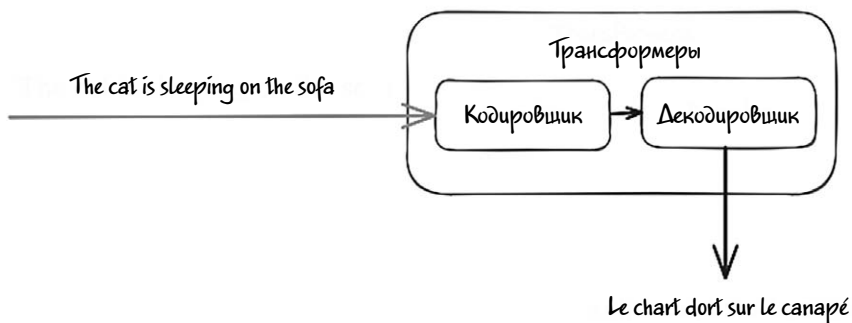


Иллюстрация 2.6

Теперь воспользуемся другой аналогией и представим механизм кодировщика-декодировщика в виде гида, который помогает вам переводить указатели в аэропорту.

- Кодировщик. Гид читает указатель на французском языке, который гласит: «Sortie de secours» (что означает «Аварийный выход»). Понимает, что эта надпись означает по-французски.
- Декодировщик. Поняв смысл, гид говорит вам, что этот указатель означает по-английски «Emergency Exit».

Кодировщик понял французское предложение, а декодировщик на основе этого понимания создал английское предложение.

Такая краткая схема позволяет понять основные принципы работы архитектуры кодировщика-декодировщика. В последующих разделах данной главы мы рассмотрим архитектуру трансформера более подробно, после того как получим более четкое представление о токенах и векторах.

Архитектура трансформера служит основой для разработки нового поколения языковых моделей (LM, Language Models), обучаемых на определенном корпусе данных. В качестве примера нового поколения LM можно привести такие модели, как BERT и CamemBERT. Эти LM часто называют LM Gen1 («Поколение 1») или «языковыми моделями после трансформеров». Ключевая характеристика данных моделей — относительно небольшое количество параметров, что делает их хорошо подходящими для решения базовых языковых задач. Например, BERT, разработанная компанией Google в 2018 году, представляет собой LLM с одним лишь кодировщиком, 12 слоями и 12 модулями внимания («голов внимания», от англ. «attention heads»), что позволяет ей обрабатывать около 110 миллионов параметров.

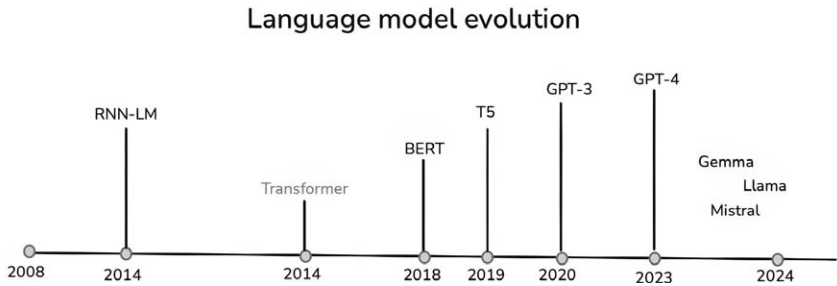


Иллюстрация 2.7

Позже на основе моделей трансформеров были созданы более сложные предварительно обученные модели — например, GPT (Generative Pre-trained Transformer) или T5. Эти передовые модели обучаются на огромных объемах текстовых данных, их можно адаптировать для решения конкретных NLP-задач с хорошей производительностью на современном уровне. Возникновение данных передовых моделей ознаменовало собой начало захватывающей

и многообещающей эпохи генеративного искусственного интеллекта.

Генеративный ИИ

Генеративный ИИ (Generative AI) — это тип искусственного интеллекта, разработанный в результате исследований в различных областях ИИ, проводимых на протяжении последних десятилетий. Он сочетает в себе возможности машинного обучения, глубокого обучения и создания креативного контента различного типа. В отличие от систем, которые просто распознают закономерности или делают прогнозы, системы генеративного ИИ могут генерировать (то есть создавать) совершенно новый контент — текст, изображения, музыку или даже программный код — посредством синтеза информации, полученной из существующих данных.

Ярким примером генеративного ИИ служат такие большие языковые модели (LLM, Large Language Models), как GPT и Llama, специализирующиеся на генерации и понимании текста. На основе методов и механизмов глубокого обучения такие модели выявляют закономерности и представления признаков из огромного количества существующих данных. Как правило, системы генеративного ИИ часто рассматриваются как подмножество ИИ-систем глубокого обучения.



Инфо

Генеративный ИИ — широкий термин, охватывающий любые ИИ-модели, способные генерировать новый контент, будь то изображения, музыка, программный код или текст. С другой стороны, LLM — это подмножество моделей генеративного ИИ, ориентированное специально на распознавание и генерацию естественного языка. Все LLM можно назвать генеративными ИИ-моделями, но не все генеративные ИИ модели — это LLM.

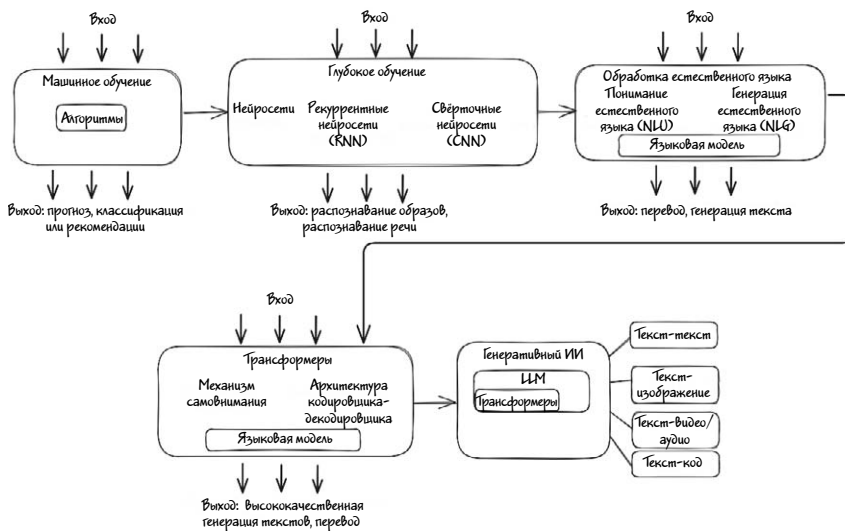


Иллюстрация 2.8

Краткое описание основных этапов на иллюстрации 2.8.

- Машинное обучение открыло путь к распознаванию образов.
- Глубокое обучение и нейронные сети позволили моделям обрабатывать сложные данные.
- Архитектура трансформеров произвела революцию в NLP и генерации текстов.
- Масштабное обучение сделало генеративные модели ИИ — например, GPT или Llama, достаточно мощными, чтобы справляться с такими сложными задачами, как генерация текста и диалог.

Как уже говорилось, в 2017 году благодаря разработке **трансформеров** в области обработки естественного языка был совершен значительный прорыв. Новая схема с **механизмом самовнимания** позволила моделям гораздо эффективнее обрабатывать большие объемы текста с фокусировкой на наиболее значимых его частях. В результате трансформеры стали основой, на которой были созданы многие современные генеративные модели ИИ.

С появлением огромных массивов данных и увеличением вычислительной мощности исследователи стали обучать генеративные

модели на огромных объемах данных. Такие большие языковые модели, как **GPT** от **OpenAI** и **BERT** от **Google**, обучаются на огромных текстовых корпусах, благодаря чему генерируют осмысленный и связный текст или вступают в диалог с пользователями, как это происходит в чат-ботах, работающих на основе ИИ.

Генеративный ИИ обладает огромным количеством преимуществ, а сферы его применения обширны и разнообразны. Ниже перечислены лишь некоторые из наиболее значимых преимуществ.

- **Творчество и инновации.** Генеративный ИИ помогает создавать креативный контент — такой как художественные, музыкальные и литературные произведения. Он помогает художникам, музыкантам и писателям исследовать новые идеи и облегчать творческий процесс.
- **Повышение эффективности и автоматизация.** Генеративный ИИ автоматизирует рутинные задачи и процессы, что позволяет экономить время и сокращать количество человеческих ошибок. Например, он может составлять отчеты, создавать маркетинговые материалы и обрабатывать запросы в службу поддержки клиентов.
- **Персонализация.** Генеративный ИИ может адаптироваться к предпочтениям конкретных пользователей, повышая тем самым их удовлетворение от общения с ним. Так, например, он может выдавать персональные рекомендации на потоковых сервисах, показывать персонализированную рекламу и составлять планы индивидуального обучения.
- **Создание контента.** Генеративный ИИ позволяет быстро и в больших объемах создавать качественный контент. Например, с его помощью можно генерировать реалистичные изображения, писать статьи или создавать видео, что очень важно для отраслей, ориентированных на контент.
- **Аугментация данных.** Генеративный ИИ может создавать синтетические данные для дополнения реальных наборов данных, что бывает полезно для настройки моделей машинного обучения, особенно в тех случаях, когда реальных данных мало или они конфиденциальны.

- **Повышение доступности.** С помощью генеративного ИИ можно создавать инструменты и приложения, улучшающие доступность продукции и услуг — например, генерировать текстовые описания для изображений или озвучивать видео для людей с нарушениями зрения.
- **Исследование и моделирование.** Генеративные модели могут имитировать различные сценарии и среды, что бывает полезно для проведения научных исследований, виртуального обучения и планирования.

Что относится к сфере генеративного ИИ, а что не относится?

Область искусственного интеллекта развивается настолько стремительно, что можно легко запутаться в терминах, жаргоне и отличиях одного направления от другого, особенно когда речь заходит о генеративном ИИ. Что относится к его сфере, а что нет? В этом разделе мы постараемся подробнее очертить эти рамки.

На иллюстрации 2.9 показана простая схема, с помощью которой можно определить, какие системы и процессы относятся к сфере генеративного ИИ, а какие нет.

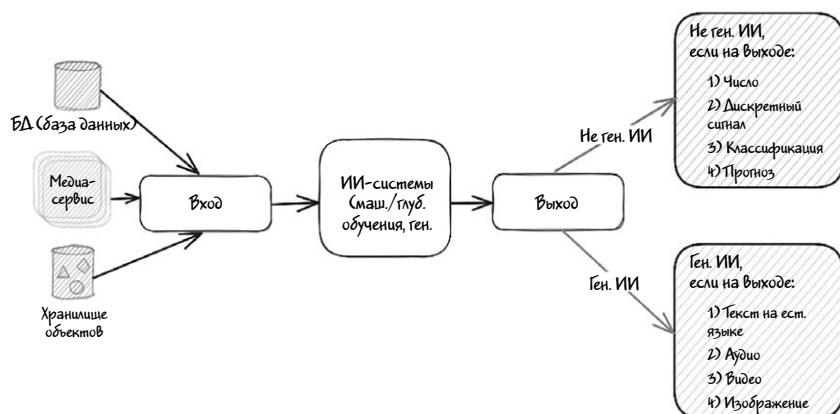


Иллюстрация 2.9

Если система — будь то модель машинного обучения, глубокого обучения или любая другая ИИ-модель — на выходе выдает числа,

классификацию (например, делит электронные письма на «спам» и «не спам») или вероятность (например, «вероятность покупки продукта = 90%»), то она не считается генеративной.

Однако если на выходе получается что-то вроде текста на естественном языке (например, статья или стихотворение) или изображения (вроде фотографии с кошкой или собакой), то такие системы попадают под определение генеративных.

Приведенные выше объяснения можно также выразить в математических терминах следующим образом:

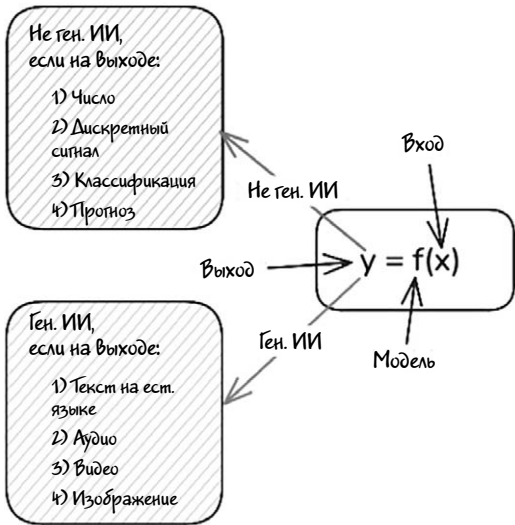


Иллюстрация 2.10

Уравнение $y = f(x)$ символически означает некий способ получения результата (Y) на основе различных данных (X). В данном случае Y — это результат работы ИИ-системы («выход»), F — это некая функция или модель, используемая для расчетов, а X может означать все, что угодно — от необработанного текста (например, статьи в Википедии) до CSV-файлов (например, годовых отчетов) или даже изображений. Выход модели определяется всеми входными данными. Если Y представляет собой число — например, прогнозируемую цену акций, то данная модель не считается генеративной. Однако если Y представляет собой **предложение** — например, *the share price*

(«цена акции...»), то модель уже можно причислить к генеративным, так как она генерирует ответ.

Категории генеративного ИИ

За последние несколько лет сфера генеративного ИИ претерпела значительные изменения. В 2024 году мы наблюдаем широкий спектр моделей генеративного ИИ, включая следующие.

1. Модели «текст-текст». Такие модели принимают текст на вход и генерируют текст на выходе. Тип текста может варьироваться — от статей на естественном языке и стихов до программного кода и даже документов HTML. Пример таких моделей — Gemini от Google, GPT-4, Claude Opus и LLaMA 3.1.
2. Модели преобразования «текст-изображение». Эти модели генерируют изображения на основе текстового описания (промпта). Например, можно предложить описание собаки или кошки с зелеными ушами — и будет сгенерировано соответствующее изображение. Пример такой модели — Midjourney.
3. Модели «изображение-изображение». Эти модели на основе предоставленного изображения выдают его измененную версию или комбинируют изображения.
4. Модели «изображение-текст». Эти модели генерируют текст на основе изображений, что бывает особенно полезно для таких задач, как перевод презентаций в текстовый вид. Пример модели такого типа — LLaVA.
5. Модели «речь-текст». Эти модели транскрибируют речь в текст, что бывает полезно для таких задач, как запись совещаний — например, при общении в Zoom. В качестве примера можно привести Whisper и DeepSpeech.
6. Модели преобразования «текст-аудио». Эти модели на основе текстовых промптов генерируют музыку или звуки. Особенно они

полезны для аудиотранскрипции. Пример модели такого типа — Aura от Deepgram.

7. Модели «текст-видео». Такие модели на основе текстовых промптов генерируют видео. В ближайшем будущем мы, возможно, увидим целые фильмы, созданные на основе текстовых сценариев. Примеры таких моделей — Sora, Dream Machine и Kling.

Приложения или модели генеративного ИИ в зависимости от их реализации можно разделить на две основные категории.

1. Фундаментальные модели — это большие, предварительно обученные модели *машинного обучения*, служащие основой для различных ИИ-приложений. Такие модели обучаются на обширных и разнообразных наборах данных (текст, изображения или аудио), и их можно настраивать для решения конкретных задач с минимальным дополнительным обучением. Благодаря своему масштабу и универсальности, фундаментальные модели адаптируются для решения широкого спектра задач — от перевода с одного языка на другой до распознавания изображений.

В качестве примера можно привести DALL-E и Gato от DeepMind — это фундаментальные модели, обученные на изображениях и текстах и генерирующие оригинальные изображения на основе текстовых описаний.

Ключевые характеристики.

- Фундаментальные модели обучаются на обширных наборах данных из различных источников, благодаря чему получают способность понимать язык, распознавать изображения и другие типы данных.
- Их можно адаптировать или дообучить для решения различных конкретных задач — таких как анализ настроения, генерация текста или распознавание объектов.
- Накопленные ими знания можно переносить на новые задачи с минимальным дополнительным обучением.

2. Большие языковые модели (LLM) — это разновидность фундаментальных моделей, которые используют механизмы *глубокого обучения* для анализа огромных объемов данных и обучения на них. В результате LLM достигают исключительной эффективности в обработке и генерации текста, почти неотличимого от составленного человеком. По сути, основываясь на распознанных на основе полученных данных закономерностях и структурах, они способны создавать совершенно новые текстовые комбинации, отражающие нюансы и сложности естественного языка.

Лучшие примеры LLM — GPT, LLaMA.

Ключевые характеристики.

- Разработаны специально для решения задач, связанных с естественным языком.
- Генерируют текст, обрабатывают язык и понимают языковые шаблоны.
- Обучаются на массивных наборах данных из различных источников.

Проще говоря, LLM — это особый тип фундаментальной модели. Как фундаментальные модели, так и LLM представляют собой ведущие системы генеративного ИИ, благодаря которым машины создают разнообразный инновационный контент.

Если вам кажется, что вы запутались, волноваться не стоит. Термин «фундаментальная модель» (*foundation model*) был предложен исследователями из Стэнфордского университета и Центра исследований фундаментальных моделей (CRFM), но оригинальная статья не совсем понятна. Далее мы попытаемся раскрыть это понятие в более простых терминах.

- Фундаментальная модель — это тип ИИ-системы с широкими возможностями, позволяющими адаптировать к различным задачам и приложениям. Представьте ее в виде базового слоя или исходной модели, служащей прочным фундаментом, на котором можно строить другие модели. Этим она отличается

от многих других ИИ-систем, которых специально обучают для какой-то одной цели, и которые нелегко адаптировать для других областей.

- Для большей ясности представьте себе здание. Фундаментальная модель будет похожа на бетонную плиту, служащую устойчивым основанием для всей конструкции. На ней можно возводить различные этажи, стены и элементы, каждый из которых служит для определенной цели. В отличие от этой модели многие другие ИИ-системы похожи на отдельные комнаты в здании — они предназначены для выполнения одной функции и их нельзя легко изменить или переназначить для другой цели.

Примерами фундаментальных моделей служат многие из систем, упомянутых выше (например, LLM). В качестве более конкретного примера рассмотрим ChatGPT. Изначально он был создан на основе модели LLM GPT-3.5. Чтобы адаптировать ее для чата, специалисты Open AI воспользовались дополнительными данными, специфичными для разговорного стиля или беседы, и в результате получили модифицированную версию GPT-3.5. Затем эта скорректированная модель была использована для разработки ChatGPT.

Инфо

Один из ярких примеров фундаментальной модели — Gato компании DeepMind, разработанная для решения широкого спектра задач, выходящих за рамки обработки языка. Помимо генерации текста, Gato может управлять роботами и играть в сложные видеоигры. Благодаря такой универсальности Gato смело можно назвать фундаментальной моделью, но вряд ли ее можно отнести к большим языковым моделям (LLM).

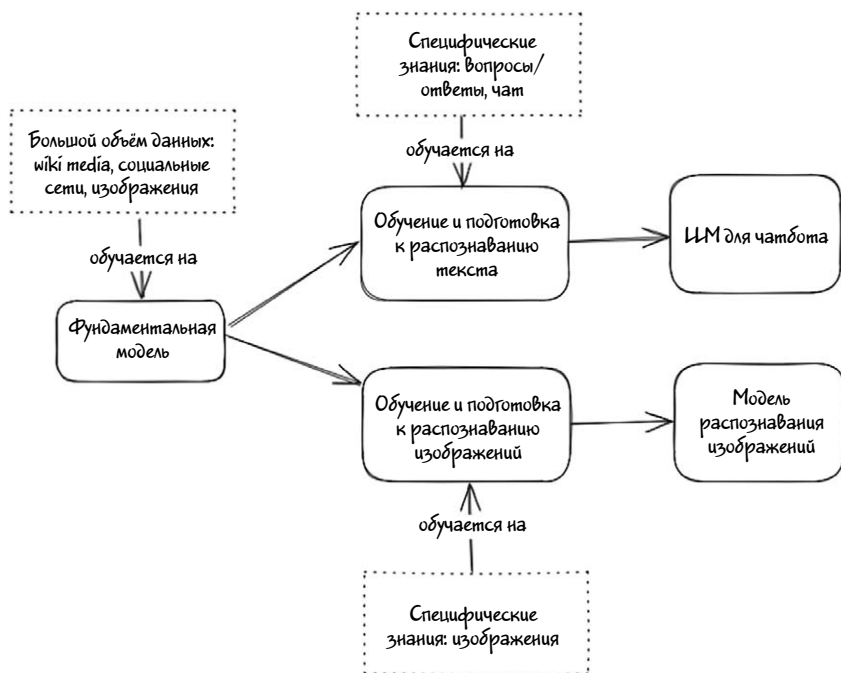


Иллюстрация 2.11

В настоящее время термин «фундаментальная модель» часто используется в значении «большая языковая модель», поскольку языковые модели — это самый яркий пример систем с широкими возможностями, которые можно адаптировать для различных целей. Ключевое различие между ними заключается в том, что термин «*большие языковые модели*» относится к системам, ориентированным именно на решение языковых задач, в то время как термин «*фундаментальная модель*» имеет более широкое значение и означает систему, на основе которой в будущем могут появиться новые типы систем.

В следующих разделах мы рассмотрим различные концепции, лежащие в основе систем LLM, и объясним, как они работают.

Большая языковая модель

Итак, большая языковая модель (LLM, Large Language Model) — это тип модели, предназначенный для выполнения задач, связанных с языком. Но почему она называется «большой»? Ответ прост: в данном случае термин «большая» (Large) относится к количеству параметров, обрабатываемых моделью (несколько миллиардов), и к объему обучающих данных (от миллиардов до триллионов слов).

Почему размер модели и объем данных имеют такое значение?

Когда большая языковая модель (LLM) обучается на огромных наборах данных, она для предсказания правильного вывода использует входные данные. Во время обучения модель сравнивает свое предсказание с реальным текстом, чтобы понять, оказалось это предсказание верным или нет. При ошибке она корректирует параметры. Этот процесс повторяется миллионы и даже миллиарды раз, благодаря чему модель учится делать более точные предсказания, а со временем начинает понимать сложную грамматику, генерировать связный текст и следовать логическим закономерностям.

Как устроена LLM?

Выше упоминалось о том, что LLM разработаны специально для задач обработки естественного языка (NLP). Но как они понимают грамматическую структуру (синтаксис) и смысл (семантику) входного текста? Как именно генерируют вывод с правильным синтаксисом и соответствующим смыслом?

В данном разделе мы рассмотрим различные методы и механизмы, используемые при обработке естественного языка, так что сосредоточьтесь!

Большие языковые модели (LLM) выявляют языковые закономерности («паттерны»), обрабатывая огромные объемы текстовых данных. Делают они это с помощью нейронных сетей, предназначенных для эффективной обработки последовательных данных.

Вот упрощенная схема их работы.

1. Нейронная сеть состоит из множества узлов в виде чисел («параметров»), связанных между собой. Каждой из этих связей также присваивается определенное число («вес») — *weight*.
2. Нейронная сеть обрабатывает только числа, поэтому, когда вы посылаете входной сигнал (промпт или вопрос), она преобразует его именно в них. Результат своей работы, зависящий от настройки параметров, сеть также выдает в числовом виде.
3. Любой тип контента (текст, изображения и прочее) можно представить в виде чисел и передать в модель.

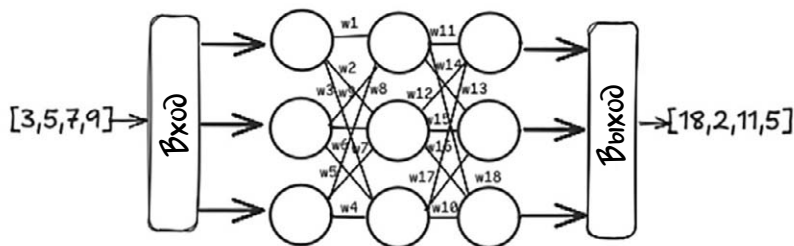


Иллюстрация 2.12

Допустим, мы вводим в LLM начало предложения The sky is («Небо...»). Модель преобразует слова в числа, обрабатывает их с помощью нейронной сети и выдает результат типа The sky is blue («Небо голубое»). Это базовая демонстрация того, как LLM предсказывает следующее слово, и это общая задача машинного обучения.

🔑 Совет

Когда говорят, что LLM — это набор предварительно обученных и файн-тюнинговых текстовых моделей с размером от 8 до 405 миллиардов параметров, имеется в виду количество весов (связей между узлами) в нейронной сети модели. Исходя из этого можно сказать, что у изображенной на иллюстрации 2.12 сети 18 параметров.

И вот тут-то все становится по-настоящему интересным: когда вы объединяете входные и выходные данные и возвращаете их обратно в LLM, она может продолжать генерировать новый контент, предсказывая следующее слово. Данный процесс может повторяться бесконечно, благодаря чему создается непрерывный поток генерируемого текста — так же, как это происходит при взаимодействии с ChatGPT или LLaMA.

```
>>> finish the sentence "The sky is"
...painted with colors of sapphire, amethyst, and gold!

Or was that what you were thinking?

Here are a few other options:

* ...always changing, yet always the same.
* ...the perfect backdrop for a beautiful sunset.
* ...a reminder of how small we truly are.

But I have to say, my favorite is:
* ...no limit to its beauty and wonder!
```

На первый взгляд схема работы кажется простой и понятной, не так ли? Но в глубине скрывается множество увлекательных действий. Разберемся в этом шаг за шагом.

1. Токенизация. Модель разбивает текст на более мелкие единицы, называемые токенами (словами или «подсловами»), которые служат в качестве входных данных.

2. Эмбединг. Под этим термином подразумевается перевод токенов в числовое представление (вектор), отражающее его значение в разрезе текста.

3. Архитектура трансформера. Трансформер состоит из слоев *механизмов внимания*, помогающих модели понять отношения между токенами независимо от их места в предложении. Благодаря этому при генерации или анализе текста модель сосредоточивается на релевантных словах.

Начнем с токенизации.

Токенизация

Этот процесс подразумевает преобразование фрагментов текста на естественном языке (например, предложений) в биты данных, с которыми может работать программа. Проще говоря, разбивает предложения на отдельные слова или части, называемые токенами, которые служат строительными блоками для семантического анализа.

Таким образом, токен — это единица текста, выделенная для того, чтобы большая языковая модель эффективно обрабатывала данный текст. LLM пытается понять статистические отношения между токенами и, исходя из их отношений, создать следующий токен в последовательности.

Токенами могут быть целые слова, буквы, комбинации слов или даже знаки препинания. Такую сегментацию текста выполняет алгоритм или функция под названием «токенизатор».

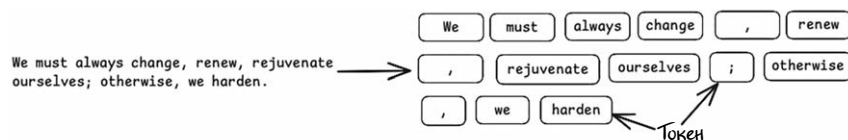


Иллюстрация 2.13

Токенизаторы бывают разные, но каждый из них обладает своими особенностями, преимуществами и недостатками. Среди известных стоит упомянуть NLTK (Natural Language Toolkit), SpaCy, токенизатор BERT и Keras.

Простой токенизатор на основе слов рассматривает в качестве токена каждое слово или знак препинания, то есть количество токенов будет соответствовать количеству слов в предложении. Например, в приведенном выше примере «We must always change, renew, rejuvenate ourselves; otherwise, we harden» («Мы должны всегда меняться, обновляться, омолаживаться, иначе затвердеем») будет 15 токенов.

Ниже показан пример разбиения предложения на токены токенизатора SpaCy:

```
import spacy
nlp = spacy.blank("en")
tokens = nlp ("We must always change, renew, rejuvenate
ourselves; otherwise, we harden.")
len (tokens)
for token in tokens: print (token)
```

Выполнив этот код, получим следующий результат:

```
we
must
always
change
,
renew
,
rejuvenate ourselves
;
Otherwise
,
we
harden
.
```

При токенизации символы часто считаются отдельно, включая пробелы между словами. Однако таким образом работают не все токенизаторы — некоторые учитывают контекст, в котором используется текст, что приводит к различным результатам.

В системе BERT, разработанной Google для LLM, используется контекстно-зависимый токенизатор, расширяющий возможности языкового анализа. Сравним предыдущий пример с примером работы токенизатора, используемого для GPT-4 (и доступный онлайн на странице бесплатного онлайн счетчика токенов OpenAI). В данном случае то же предложение разбивается на 17 отдельных токенов.

Tokens	Characters
17	73
We must always change, renew, rejuvenate ourselves; otherwise, we harden.	

Иллюстрация 2.14

Токены также служат полезной единицей измерения. В них измеряется количество текста, которое может обработать или сгенерировать LLM. Кроме того, стоимость работы LLM напрямую зависит от количества обрабатываемых токенов: чем их меньше, тем дешевле, и наоборот.

Размер LLM определяется количеством токенов, которые она может принять на вход, а каждая модель при этом имеет свои ограничения. Токены хранятся и обрабатываются в памяти, поэтому данные ограничения помогают поддерживать эффективность модели и оптимизировать использование ресурсов. Ниже приведены ограничения на количество токенов для различных LLM.

Модель	Лимит токенов
Llama 2	4096
GPT 4	8192

После разбиения входного текста на токены токенизатор кодирует их по определенной схеме и генерирует специализированные векторы, которые может понимать LLM. В следующем разделе мы объясним, что такое векторы, рассмотрим их и разберем, как токены могут быть представлены в векторном формате.

Вектор

Даже после деления входных данных LLM не сможет сразу же понять смысл текста. Поэтому перед подачей текста в модель первым делом нужно каким-то образом преобразовать строчные значения в числовые (иными словами, выполнить *векторизацию* текста).

Векторы выполняют важную функциональную роль как в сфере LLM, так и в области генеративного ИИ в целом. Это связано с тем, что LLM умеют обрабатывать числа только в определенном формате. Чтобы в полной мере оценить роль и значение векторов в LLM,

нужно понять, что же это такое, как они генерируются и как используются в данных моделях.

В математике вектор — объект, характеризующийся как величиной, так и направлением в разных измерениях. Рассмотрим следующий пример, в котором вектор представляет собой массив из пяти элементов:

Создание вектора в виде массива

```
!pip install numpy
import numpy as np
vector = np.array([1, 2, 3, 4, 5])
print("Vector of 5 elements:", vector)
```

Выход:

```
Vector of 5 elements: [1 2 3 4 5]
```

В контексте LLM векторы используются для представления текста или данных в числовом формате, благодаря чему модель может их понимать и обрабатывать. Переводить данные в числовой формат необходимо, поскольку машины воспринимают только числа, а не собственно текст или изображения. Эти данные, представленные в векторном виде, нейронные сети — в частности, на основе архитектуры трансформеров — обрабатывают уже очень хорошо. Более того, операции над векторами позволяют определить степень их похожести. По сути, именно благодаря операциям над векторами нейросети выявляют сходство или различие между данными в векторных базах данных или в памяти.

Совет

Здесь мы встретились с новой, набирающей популярность концепцией — векторными базами данных. Они хранят данные в виде высокоразмерных векторов, называемых «эмбеддингами» и отражающими семантический смысл различных элементов и их взаимосвязи между собой. В них используются специализированные

методы индексирования, такие как хэширование, квантование, а также методы на основе графов, позволяющие быстро обрабатывать запросы на поиск по сходству или семантике. В LLM векторные базы непосредственно не используются, однако они необходимы, если вы планируете расширить LLM с помощью технологии Retrieval-Augmented Generation (RAG), о которой говорится в главе 3.

Итак, токенизатор разбивает и кодирует входные данные по определенной схеме и генерирует специальные векторы, понятные для LLM. Следует иметь в виду, что схема кодирования очень сильно зависит от конкретной LLM, поэтому в разных случаях могут получаться разные векторы, которым соответствует каждое слово или даже части слов. И наоборот, проходя через *декодировщик* («декодер»), вектор легко преобразуется обратно в текст.

Ниже приведен пример псевдокода, преобразующий упомянутое выше мотивационное предложение в токены с помощью модели Phi-2. Здесь для кодирования текста в векторы и декодирования его обратно в текст мы воспользовались классом `AutoTokenizer` от Hugging Face:

```
!pip install transformers
from transformers import AutoTokenizer
from huggingface_hub import interpreter_login

# Воспользуйтесь своим API-КЛЮЧОМ от Hugging Face и нажмите
Enter

interpreter_login()
model_name='microsoft/phi-2'
tokenizer = AutoTokenizer.from_pretrained(model_name,
token="HF_TOKEN")
txt = "We must always change, renew, rejuvenate ourselves;
otherwise, w\ e harden."
token = tokenizer.encode(txt) print(token)
decoded_text = tokenizer.decode(token) print(decoded_text)
Output:
```

```
[1135, 1276, 1464, 1487, 11, 6931, 11, 46834, 378, 6731, 26, 4306, 11, \ 356, 1327, 268, 13]
```

```
We must always change, renew, rejuvenate ourselves;  
otherwise, we harde\ n.
```

На практике токенизация включает в себя различные приемы, такие как дополнение («падинг»), маркеры конца строки (end-of-line markers) и другие. Подробности токенизации рассматриваются в главе 5 «Файн-тюнинг LLM».

Самих по себе векторов для LLM недостаточно, поскольку они представляют только основные числовые характеристики токена, но не передают ее богатое семантическое значение. Векторы — это просто математическое представление, которое можно ввести в модель. Чтобы передать семантические отношения между токенами, нам нужно нечто большее, а именно эмбединги.

Эмбединг

Эмбединг — это более сложная версия вектора, которая обычно создается в процессе обучения на больших наборах данных. В отличие от необработанных векторов, эмбединги отражают семантические отношения между токенами. Это значит, что токены с похожими значениями будут иметь похожие эмбединги, даже если встречаются в разных контекстах. Именно эмбединги позволяют большим LLM улавливать тонкости языка, включая контекст, нюансы и значения слов и фраз. Они строятся в процессе обучения модели, когда она поглощает огромные объемы текстовых данных и кодирует не только отдельные индивидуальные токены, но и их связи с другими токенами.

Благодаря эмбедингам LLM получают своего рода представление о языке, что позволяет им выполнять такие задачи, как анализ настроения, резюмирование текста и ответы на вопросы с нужным уровнем точности. Эмбединги служат точкой входа для модели, позволяя ей получать доступ к огромным объемам текстовых данных и обрабатывать их. Кроме того, они используются и независимо — для преобразования текста в векторы с сохранением его семантического контекста.

Когда текст проходит через необходимую модель, на выходе создается вектор, содержащий эмбединги. При этом сохраняются основные смыслы и связи между словами и фразами, что позволяет языковой модели анализировать тексты, обобщать их смысл и отвечать на вопросы исходного текста.

Обычно эмбединги генерируются с помощью таких методов, как *Word2Vec*, *GloVe*, или с помощью современных нейронных сетей-трансформеров. Вот пример того, как OpenAI Embeddings генерирует эмбединги следующих входных текстовых данных: Lion, Tiger и iPhone.

```
from langchain_openai import OpenAIEmbeddings
embeddings_model = OpenAIEmbeddings(api_key="ВАШ КЛЮЧ OPEN
API")
txt = "Lion"
embedded_query = embeddings_model.embed_query(txt)
print (len(embedded_query))
embedded_query[:5]
```

Выход:

```
Embedding: Lion      Embedding: Tiger      Embedding: Iphone
[-0.0015009930357336998[-0.013500549830496311,
[-0.0076760281808674335,      ,
      -0.010024921968579292, -0.009651594795286655, -
0.021282033994793892,
      -0.015631644055247307, -
0.008970077149569988, 0.0060024564154446125,
      -0.023150335997343063, -0.0019277227111160755-
0.02365402691066265,      ,
      -0.01021870318800211]-0.018589219078421593]-
0.016182253137230873]
```

Обратите внимание, что эмбединги для слов Lion («лев») и Tiger («тигр») более похожи друг на друга по сравнению с эмбедингом слова iPhone. Это объясняется тем, что они отражают такие отношения, как принадлежность к полу или королевскому семейству,

а также категории объектов, и представляют эти концепции в непрерывном векторном пространстве.

Прежде чем погружаться в более технические детали устройства LLM, такие как архитектура трансформеров, повторим вкратце то, что мы уже изучили.

1. **Токенизация.** Процесс преобразования необработанного текста в токены (слова или «подслова»). Пример: «Machine leaning» => [«Machine», «learning»].

2. **Вектор и векторизация.** Каждый токен преобразуется в числовой вектор. Пример: «Machine» => [0.5, 0.1, 0.7, ...].

3. **Эмбеддинг.** Числовое представление, созданное в процессе обучения сети и передающее семантический смысл. Пример: слова Machine и Learning будут иметь схожие эмбеддинги, если модель понимает, что это связанные термины.

Трансформеры

Как было сказано ранее, в основе LLM лежит архитектура трансформеров, и, в частности, механизм внимания, позволяющий моделям справляться с такими задачами, как перевод и резюмирование. При предсказании или интерпретации значения он помогает моделям сфокусироваться на важных частях входных данных, таких как специфические слова в предложении.

Попробуем описать работу трансформеров в простых терминах.

Представьте, что вы читаете длинное предложение. Мы, будучи людьми, естественным образом понимаем, какие части предложения самые важные, и сосредотачиваемся на них в зависимости от контекста. Возьмем для примера следующее предложение:

The ocean is blue because it has no color itself.

«Океан синий, потому что сам по себе он не имеет цвета».

Читая это предложение, вы естественным образом понимаете, что слово *it* («это», «он») относится к океану. Ваш мозг знает, что оно соотносится с океаном, а не со словами *blue* («синий») или *color* («цвет»). Но для машины это не так очевидно. Именно здесь на помощь приходит механизм внимания. До появления механизма внимания модели с трудом устанавливали степень важности слов в длинных последовательностях, что часто приводило к плохим предсказаниям в таких задачах, как перевод или ответы на вопросы.

Механизм внимания — это своего рода прожектор, который помогает машине определить, какие слова в предложении самые важные для понимания смысла. Вместо того, чтобы рассматривать каждое слово по отдельности и без контекста, механизм внимания выделяет определенные части предложения, основываясь на их взаимосвязи друг с другом.

Инфо

Проще говоря, механизм внимания помогает модели просмотреть весь входной материал (например, предложение), а затем решить, на каких словах сосредоточиться при создании выходных данных (например, перевода или резюме).

Разберем теперь пошагово, как же работает механизм внимания.

Шаг 1. Понимание каждого слова в предложении.

Представьте, что компьютер пытается понять слово *it* в предложении: *The ocean is blue because it has no color itself* («Океан синий, потому что сам по себе он не имеет цвета»).

- Слово *it* («это», «он») довольно сложное, потому вне контекста оно не имеет особого значения. Нам нужно знать, к чему оно относится в предложении.
- Компьютер смотрит на другие слова в предложении, такие как *ocean*, *blue*, *color* («океан», «синий», «цвет») и пытается понять, как слово *it* к ним относится.

Шаг 2. Присвоение баллов внимания.

Механизм внимания присваивает каждому слову баллы внимания в зависимости от того, насколько *it* с ним связано.

Например:

Ocean («океан») получает **высокий балл внимания (очень большую степень важности)**, потому что *it* относится к ocean.

Blue получает **низкий балл (малую степень важности)**, потому что *it* не относится к blue.

Color может получить **средний балл (небольшую степень важности)**, потому что это слово связано со словом ocean, но *it* относится не к нему.

Механизм внимания вычисляет эти баллы, сравнивая связи между всеми словами в предложении.

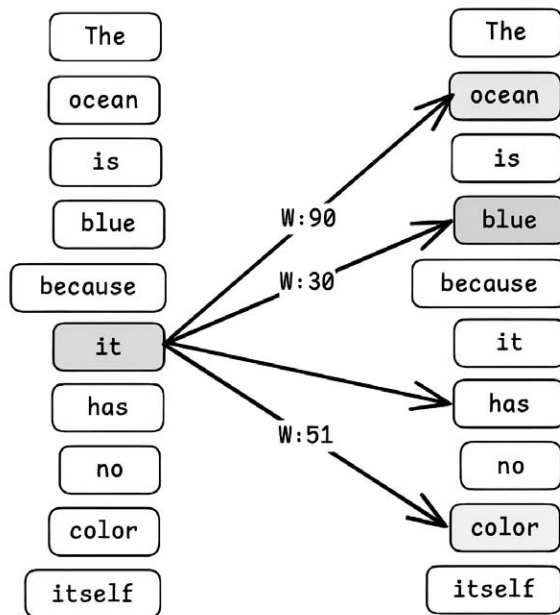


Иллюстрация 2.15

Шаг 3. Взвешивание слов.

Присвоив словам баллы внимания, модель использует их для корректировки влияния (или «веса») каждого из них. Слова с *более высокими баллами внимания* вносят больший вклад в выявление смысла предложения. Слова с более низкими баллами вносят меньший вклад.

Так компьютер узнает, что слово *ocean* («океан») — самое важное, на котором следует сосредоточиться, пытаясь понять смысл слова *it* («это», «оно»). Механизм внимания направляет модель на то, чтобы она уделяла больше внимания слову *ocean* и меньше другим словам, таким как *blue* или *color*.

Этот процесс помогает компьютеру правильно понять предложение: *The ocean is blue because it has no color itself* («Океан синий, потому что сам по себе он не имеет цвета»).

Шаг 4. Генерация вывода.

Определив, на каких словах следует сосредоточиться, модель использует данную информацию для создания вывода. Это может быть:

- Перевод предложения на другой язык.
- Резюмирование длинного текста.
- Ответ на вопрос на основе входных данных.

Допустим, вы используете ИИ-инструмент перевода, чтобы перевести это предложение с английского на французский. Механизм внимания поможет ИИ понять, что слово относится к океану, и убедиться, что перевод правильно отражает данную связь. Вместо того чтобы считать каждое слово одинаково важным, ИИ будет уделять больше внимания таким основным словам, как *ocean* и *it*, чтобы создать правильный перевод: *L'océan est bleu parce qu'il n'a pas de couleur en lui-même*.

В этом разделе мы описываем внутреннюю работу LLM при выполнении задач NLP (обработки естественного языка) на самом отвлеченном уровне. За кулисами скрывается множество сложных процессов, включая декодирование, матричные вычисления и взвешивание (присвоение и регулирование весов). Чтобы полностью разобраться в этих тонкостях, требуется серьезная подготовка в области машинного обучения, что выходит за рамки данной книги. Однако если вам интересно узнать подробности, я настоятельно рекомендую начать со статьи в блоге Джея Аламмара *The Illustrated Transformer* («Иллюстрированный трансформер»).

Обучение LLM

Обучение — важнейший этап разработки больших языковых моделей. Оно позволяет создать мощные и универсальные модели, способные решать самые разные задачи — от генерации общего текста до выполнения специализированных функций.

Процесс обучения делится на две основных стадии: предобучение (предварительное обучение) и фajn-тюнинг (дообучение). Каждая стадия играет свою роль в подготовке модели к использованию по назначению.

Предварительное обучение закладывает основу. На этой стадии модели предлагают широкий набор данных и общие задачи, что помогает освоить основы работы с языком.

Далее, на стадии тонкой настройки, на основе более специализированного набора данных модель адаптируют для решения частных задач или работы в более узкой сфере. Модель совершенствуют, чтобы она эффективнее показывала себя в конкретных приложениях.

Вот более подробный обзор этих этапов и их различий.

Предварительное обучение

Предварительное обучение — это начальная стадия, когда большая языковая модель обучается на широком и разнообразном наборе

данных, благодаря чему выявляет общие языковые шаблоны, структуры и семантику. На данной стадии модель усваивает основные правила обработки языка, которые можно применять при решении различных задач.

Рассмотрим пример. Допустим, модель обучается на таком наборе данных, как статьи из Википедии. В результате она научилась предсказывать замаскированные слова в таких предложениях, как `The quick brown [MASK] jumps over the lazy dog` («Быстрая бурая [МАСКА] перепрыгивает через ленивую собаку»), и для данного случая правильно выдает слово `fox` («лиса»), сгенерированное на основе понимания контекста, представленного окружающими маску словами.

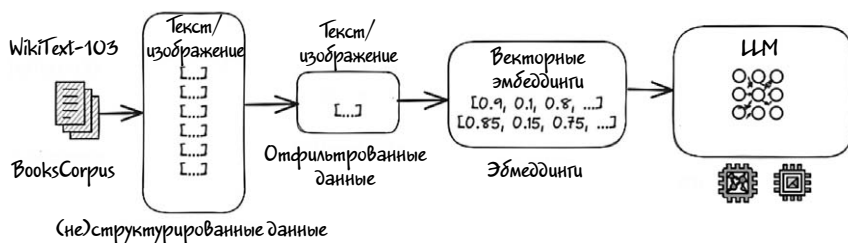
Предварительное обучение LLM включает в себя обучение модели на большом наборе данных общего назначения для изучения широкого спектра языковых моделей. Обычно на этой стадии выполняются следующие задачи.

Моделирование языка с маскировкой. Случайное маскирование некоторых входных токенов и предсказание их контекста. Очень похоже на наш предыдущий пример.

Предсказание следующего предложения. Попытки предсказать, являются ли два заданных предложения соседними в тексте.

Благодаря предварительному обучению модель получает обширные общие знания о языке, включая грамматику, синтаксис, семантику, прагматику и структуру дискурса. Эту стадию часто выполняют на больших наборах данных, таких как:

1. BooksCorpus. Коллекция из примерно 800 миллионов слов.
2. WikiText-103. Набор данных, содержащий статьи Википедии.



Ключевые аспекты стадии предварительного обучения.

- Цель. Сформировать общее понимание языка за счет представления модели больших объемов текстовых данных.
- Данные. Используется обширный и разнообразный контент, часто включающий книги, статьи, веб-сайты и другие текстовые источники.
- Задачи обучения. Общие задачи предварительного обучения — например, моделирование языка с маскировкой или переупорядочивание предложений.
- Языковое моделирование. Предсказание следующего слова в последовательности (авторегрессионные модели) или заполнение пропущенных слов (языковые модели с маскированием).
- Контекстуальное понимание. Обучение пониманию и генерации контекстно-адекватного текста.

Для предварительного обучения характерна чрезвычайно **высокая стоимость**, к тому же оно требует огромных объемов данных, затраты на него порой исчисляются миллионами долларов. Это одна из причин, по которой модели с открытым исходным кодом зачастую будут превосходить частные и коммерческие, поскольку большинство LLM с открытым исходным кодом изначально разрабатывались корпорациями, которые вкладывали значительные средства в предварительное обучение. Новичкам в данной области важно понимать, что они вряд ли будут заниматься предварительным обучением самостоятельно.

Файн-тюнинг

Файн-тюнинг — это последующая стадия, на которой предварительно обученная модель проходит дальнейшее обучение на более конкретном и часто меньшем наборе данных, предназначенном для специфической задачи или области. На данной стадии совершенствуются возможности модели, а ее общие знания адаптируются к специализированным приложениям.

Например, если ваша цель — разработать модель, отвечающую за медицинские темы, то нужно будет дообучить предварительно обученную модель на наборе данных, связанных с темой медицины и с ответами на подобные вопросы. Такая настройка поможет модели лучше понимать и генерировать специальный текст.

Процесс файн-тюнинга включает в себя переобучение модели с использованием следующих элементов.

- **Данные, связанные с конкретной задачей.** Набор общих данных заменяется на меньший, более специализированный и содержащий релевантную информацию.
- **Различные выходные слои.** Выходной слой настраивается в соответствии с форматом конкретной задачи (например, ответы на вопросы с множественным выбором для анализа настроения).

Ключевые аспекты файн-тюнинга.

- **Цель.** Адаптировать общую модель понимания языка к конкретным задачам или областям.
- **Данные.** Используется целевой набор данных, относящихся к конкретному приложению — например, отзывы покупателей для анализа настроения или медицинские тексты для задач, связанных со здравоохранением.
- **Задачи обучения.** Конкретные задачи — например, ответы на вопросы.
- **Классификация.** Присвоение тексту меток (например, классификация настроений).
- **Резюмирование.** Создание резюме длинных документов.

- Перевод. Перевод текста с одного языка на другой.

Как правило, при адаптации LLM под конкретную область в компании или для личного использования применяют файн-тюнинг, а не обучают модель с нуля. Объяснению процесса тонкой настройки с примерами мы посвятили целую главу книги (глава 5).

Рассмотрим теперь ключевые различия между этими двумя стадиями обучения.

Область применения	Предобучение	Файн-тюнинг
Цель	Дать модели широкое и общее представление о языке. Основополагающий шаг, готовящий модель к выполнению широкого круга задач.	Адаптация модели для успешного выполнения конкретных задач или работы в конкретной сфере. Улучшает возможности модели на основе специализированных данных.
Данные	Для построения общей языковой модели используется большой и разнообразный набор данных, охватывающий множество тем и стилей.	Используется меньший набор данных, специфичный для конкретной сферы и имеющий отношение к конкретному приложению или задаче.
Цели обучения	Включают в себя такие задачи, как предсказание следующего слова или заполнение пропущенных слов с целью улучшения общего понимания языка.	Задачи классификации или обобщения, позволяющие адаптировать работу модели к конкретным приложениям.
Размер данных	Для охвата широкого спектра языковых моделей требуются масштабные наборы данных.	Используются меньшие, целевые наборы данных, специфичных для конкретного приложения.

Адап- тация модели	Создается широкая база знаний и лингвистических способностей.	Адаптация и уточнение знаний с целью повышения эффективности выполнения конкретных задач или работы в конкретных сферах.
--------------------------	---	--

Вместе предварительное обучение и файн-тюнинг позволяют создавать мощные и универсальные языковые модели для решения широкого спектра задач — от генерации текстов общего назначения до приложений, специфичных для конкретной сферы. В следующих разделах этой главы мы рассмотрим другие связанные с LLM аспекты.

RAG

Наряду с файн-тюнингом LLM для расширения возможностей языковых моделей используется также метод RAG (Retrieval-Augmented Generation, «расширенная генерация данных»), но они служат разным целям и работают принципиально по-разному.

RAG — это гибридный подход, сочетающий в себе сильные стороны как методов поиска информации, так и методов генерации языковых моделей. Он включает в себя два основных компонента.

- **Модуль поиска** («извлечения», от англ. «retrieval»). Этот компонент выполняет поиск в большой внешней базе знаний (например, в частном Confluence, частном облачном хранилище документов или любой другой базе данных), извлекая наиболее релевантные фрагменты информации на основе входного запроса.
- **Генеративный модуль**. Обычно это предварительно обученная языковая модель, такая как LLaMA или GPT, использующая полученную информацию вместе с исходным запросом для создания связного и информативного ответа.

Вместо настройки модели можно воспользоваться моделью RAG для улучшения LLM на основе частного набора данных. Процесс работы с RAG довольно прост.

1. Когда вы предоставляете входные данные для модели RAG, она сначала извлекает релевантную информацию из большой базы данных или графа знаний с помощью модели поиска. Затем полученная информация используется для дополнения («аугментации») входных данных.

2. На основе дополненной информации модель-генератор создает текст исходя из ваших требований. Благодаря такому процессу модели RAG используют актуальные знания и предоставляют более точные и информативные ответы.

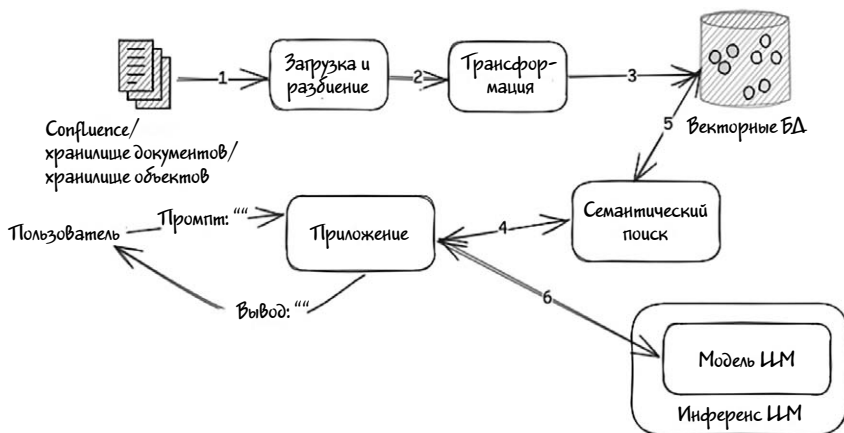


Иллюстрация 2.17

Ключевые аспекты RAG.

- **Знания по конкретной сфере.** Модели RAG разработаны так, чтобы охватывать широкий круг тем и предоставлять более точные и актуальные ответы.
- **Интеграция графов знаний.** Эти модели часто используют в своей работе обширные графы знаний из различных источников данных, обеспечивающие более структурированное и комплексное понимание информации.

- **Экономическая эффективность.** RAG — это высокоэффективный способ расширения LLM с точки зрения затрат на разработку и аппаратное обеспечение.

Для выбора оптимального подхода между RAG и файн-тюнингом LLM очень важно знать ключевые различия между этими двумя методами. Рассмотрим их ниже.

Характеристика	RAG	Файн-тюнинг LLM
Механизм	Сочетание получения информации из внешнего источника с генерацией.	Изменение весов (параметров) модели на основе специфичных данных.
Источник данных	Внешняя база знаний во время инференса.	Опора на обучающие данные, используемые для тонкой настройки.
Адаптивность	Динамическая интеграция новой информации без повторного обучения.	Требуется повторное дообучение для адаптации к новой информации.
Примеры использования	Контроль качества (QA) на основе фактов, динамическая интеграция знаний.	Специфические приложения, такие как анализ настроения, перевод.
Стоимость	Низкая стоимость разработки.	Высокая стоимость обучения LLM.

В целом RAG идеально подходит для приложений, где модель должна получать доступ к потенциально динамической внешней информации и интегрировать ее в режиме реального времени. Обсуждению RAG мы посвятили целую главу, где приводятся пошаговые инструкции по разработке системы RAG для частной компании.

ИИ-агенты

Под широким термином «ИИ-агенты» подразумеваются автономные или полуавтономные системы, предназначенные для выполнения

конкретных задач или решения проблем посредством анализа окружения, обработки информации, принятия решений и выполнения определенных действий. Такие агенты используются, например, в автомобилестроении или для управления ядерными реакторами.

В контексте генеративного ИИ и LLM ИИ-агенты — это сложные системы, использующие генеративные модели, такие как LLaMA, Mistral и GPT-4, для выполнения сложных задач, связанных с обработкой естественного языка (его пониманием, генерацией и взаимодействием с пользователем). Эти агенты предназначены для имитации взаимодействия с человеком, автоматизации процессов принятия решений и облегчения работы различных приложений — от создания контента до персонализации пользовательского опыта. Основное внимание при этом уделяется использованию генеративных ИИ-агентов для упрощения повседневных задач, хотя в скором времени они могут найти применение и в других сферах. Представьте, что вы планируете одиночный велосипедный тур по Сейшельским островам в Индийском океане. Для организации всей поездки можно нанять туроператора, но у вас ограничены время и бюджет. Поэтому вы решили попросить свою любимую LLM составить план самостоятельного путешествия. Основываясь на имеющейся у нее информации, LLM может предложить общий план поездки по дням.

Но вам нужен более подробный план, включающий прогноз погоды и рекомендации по ночлегу. Если поискать в Google и самостоятельно просмотреть все найденные страницы, это займет много времени и будет утомительно.

Но для создания подробного плана велосипедного тура по Сейшельским островам можно воспользоваться ИИ-агентом. Этого агента можно настроить так, чтобы он пользовался поисковыми системами для получения информации о регионе, сезонных погодных условиях, доступных вариантах перелета, а также впечатлений и отзывов других велосипедистов. Затем агент искусственного интеллекта обобщит данную информацию и с помощью LLM составит комплексный план тура. Такой вариант может быть не идеальным, но станет надежной основой для последующих доработок.

Возможно, приведенный выше пример и не идеален, но он дает хорошее представление о способностях ИИ-агентов. Такая интеграция LLM общего назначения со специализированными ИИ-агентами (то, что мы называем LLM-агентом) позволяет сочетать различные возможности для предоставления более полных и релевантных ответов на запросы пользователей.

Основные цели ИИ-агентов ИИ в области генеративного ИИ следующие.

- **Улучшение взаимодействия с пользователями.** ИИ-агенты могут поддерживать диалог и выступать в роли виртуальных помощников, обеспечивая тем самым более естественное и увлекательное взаимодействие с пользователями. Например, ИИ-ассистент, настроенный на медицинские данные, может давать советы по вопросам здравоохранения.
- **Автоматизация сложных процессов.** Эти агенты могут автоматизировать сложные рабочие процессы, понимая инструкции пользователя, последовательно выполняя задания и сообщая о результатах. В качестве примера можно представить ИИ-инструмент, на основе промпта генерирующий рекламный текст, публикации в блогах или контент для социальных сетей.
- **Динамическая интеграция знаний.** ИИ-агенты могут получать доступ к внешним источникам данных в режиме реального времени, что позволяет им предоставлять актуальную и точную информацию, выходящую за рамки статических знаний, закодированных в их обучающих данных. Пример — агент-фреймворк, в котором разные модули отдельно обрабатывают генерацию текста, вызовы API и управление пользовательским интерфейсом.
- **Персонализация.** ИИ-агенты могут со временем изучать предпочтения и поведение пользователей, чтобы предлагать персонализированные рекомендации или предоставлять помощь с учетом индивидуальных потребностей. Пример — бот для обслуживания клиентов, улучшающий свои ответы на основе взаимодействия с пользователем и отзывов.
- **Поддержка принятия решений и решение проблем.** ИИ-агенты могут анализировать большие объемы данных, выявлять

закономерности и предоставлять выводы, рекомендации или резюме, облегчая тем самым процесс принятия решений. Пример — агент, отвечающий на технические запросы, извлекающий данные из научных баз данных и излагающий информацию на естественном языке.

Архитектура LLM-агента состоит из четырех основных компонентов.

1. Мозг. Ядро агента, обычно LLM, действующая как центр принятия решений.
2. Планировщик. Этот компонент разбивает сложные задачи на управляемые шаги.
3. Память. Агент сохраняет контекст и информацию для последующего использования, помогая поддерживать непрерывность.
4. Инструменты. Агент использует различные инструменты, осуществляющие вызовы API, запросы к базе данных и даже создающие программный код для выполнения указанных пользователем задач.

На иллюстрации 2.18 показана общая схема работы LLM-агента.

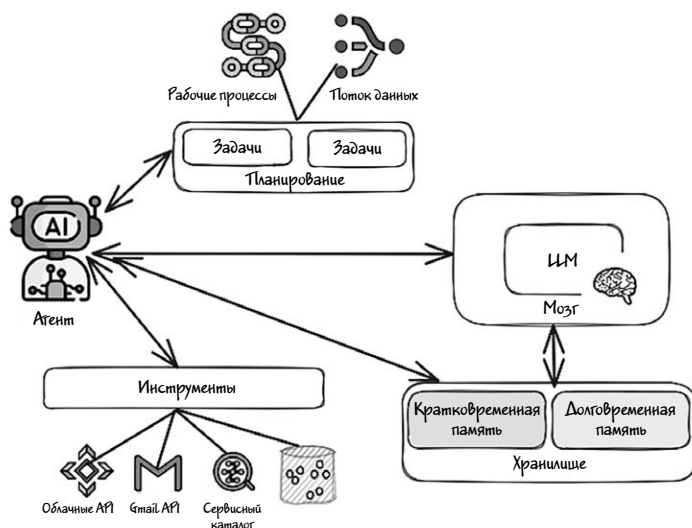


Иллюстрация 2.18

Учтите, что в этом разделе мы рассмотрели не все аспекты LLM-агента, поскольку этой теме посвящена целая глава. LLM-агенты имеют широкую сферу применения и, как ожидается, в ближайшем будущем станут ключевыми инструментами для управления рутинными задачами, подобно тому, как сегодня используются Alexa, Siri и другие помощники домашней автоматизации.

Лично я в своем ежедневном рабочем процессе часто использую агентов для таких задач, как резюмирование технической документации, проверка грамматики и улучшение читаемости электронных писем, создание контента для социальных сетей и даже генерация шаблонного кода для MVP-проектов. Эти агенты полезны не только для решения личных задач, но и для совместной работы в команде. Они повышают производительность, создавая высококачественную документацию по кодовой базе, улучшая программный код, выявляя ошибки и выполняя другие задачи. Потенциал таких агентов огромен, поэтому очень важно следить за их развитием.

Промпт-инжиниринг

Эта глава была бы неполной без обсуждения одного из важнейших аспектов максимального использования потенциала генеративного ИИ, а именно так называемого «промпт-инжиниринга». Промпт-инжиниринг («промпт-инженерия», «составление запросов») — это практическая дисциплина, занимающаяся разработкой и совершенствованием запросов («промптов») или инструкций, которые передаются языковым моделям вроде Llama и GPT для получения ответа на вопрос пользователя или решения интересующей его задачи. Так как эти модели работают на основе вводимого пользователями и интерпретируемого этими моделями текста, на результат работы модели существенно влияют его качество и структура.

Почему промпт-инжиниринг имеет такое большое значение? Потому что он предоставляет следующие преимущества.

- **Точность вывода.** Хорошо составленный запрос — это гарантия того, что модель поймет контекст и выдаст точные и релевантные ответы.
- **Эффективность.** Минимизация проб и ошибок позволяет сэкономить время и вычислительные ресурсы.
- **Контроль поведения.** Пользователи получают возможность контролировать тон, стиль и формат ответов, что особенно полезно в таких приложениях, как создание контента, поддержка клиентов и генерация кода.
- **Сокращение количества ошибок.** Четкая формулировка запросов уменьшает в ответах количество ошибок, связанных с непониманием или предвзятостью.

Приведем несколько примеров.

1. Генерация контента.

Сценарий. Предположим, вы хотите создать запись в блоге на тему «Преимущества возобновляемых источников энергии».

Базовый запрос: «Напиши о возобновляемых источниках энергии».

Уточненный запрос: «Напиши подробную статью в блоге объемом в 500 слов, в которой говорится об экологических, экономических и социальных преимуществах возобновляемых источников энергии, включая такие примеры, как солнечная энергия и энергия ветра».

Результат. Переформулированный запрос задает модели более четкие рамки, в результате чего генерируется более структурированная и информативная статья, охватывающая все необходимые аспекты.

2. Генерация кода.

Сценарий. Написать функцию для вычисления факториала числа на языке Python.

Базовый запрос: «Напиши на Python функцию для вычисления факториала».

Уточненный запрос: «Напиши на языке Python функцию с именем `factorial`, которая принимает на вход целое число `n` и возвращает факториал числа `n`. Включи обработку ошибок для отрицательных входных данных и добавь строку документации с объяснением функции».

Результат. Благодаря переформулированному запросу создается хорошо документированный код функции с обработкой ошибок, что делает его более надежным и удобным для пользователя.

Существуют несколько методов достижения желаемого результата с помощью промпт-инжиниринга.

1. Назначение роли. Задавать контекст модели можно с помощью назначения ей определенной роли. Например: «Ты — туристический гид», или «Ты — редактор».

2. Четкие инструкции. Качество ответов повышает четкая формулировка формата, стиля или необходимых для выполнения задачи шагов.

Пример. «Ты — редактор, которому нужно написать публикацию для размещения в блоге. Твоя цель — написать текст, соответствующий лучшим практическим образцам журналистики».

3. Определение последовательности. Разбиение сложной подсказки на последовательность более простых подсказок, что позволяет модели систематически выполнять каждый шаг.

Пример. Вычитка на предмет грамматических ошибок и соответствия фирменному стилю бренда.

4. Обучение с небольшим количеством примеров (*few-shot learning*). Включение в запрос примеров, на основе которых модель должна построить свой ответ.

Пример. «Результат должен быть в формате markdown».

Подводя итог, можно сказать, что промпт-инжиниринг очень важен тем, что позволяет преодолеть разрыв между намерением пользователя и ответом модели. Он повышает надежность языковых моделей и их полезность в различных практических приложениях.

Ресурсы

При написании этих глав мы обращались к многочисленным учебникам, блогам и видеороликам на YouTube. Ниже перечислены наиболее запомнившиеся ресурсы. Если какой-либо пропущен, то это произошло чисто случайно и по человеческой ошибке.

1. Attention Is All You Need <https://arxiv.org/html/1706.03762v7>
2. Natural Language Processing with Transformers, Book: <https://transformersbook.com>
3. What is NLP <https://www.analyticsvidhya.com/blog/2020/05/what-is-tokenization-nlp/>
4. Word embedding:
https://en.wikipedia.org/wiki/Word_embedding
5. Text similarity search in Elasticsearch using vector fields | Elastic Blog:
<https://www.elastic.co/blog/text-similarity-search-with-vectors-in-elasticsearch>
6. Comparison Between BagofWords and Word2Vec — PyImageSearch:
<https://pyimagesearch.com/2022/07/18/comparison-between-bagofwordsand-word2vec/>
7. Word, Subword, and Character-Based Tokenization: Know the Difference:
<https://towardsdatascience.com/word-subword-and-character-based-tokenization-know-the-difference-ea0976b64e17>
8. Word embeddings:
https://www.tensorflow.org/text/guide/word_embeddings
9. The amazing power of word vectors:
<https://blog.acolyer.org/2016/04/21/theamazing-power-of-word-vectors/>

10. Understanding NLP Word Embeddings — Text Vectorization:
<https://towardsdatascience.com/understanding-nlp-word-embeddings-textvectorization-1a23744f7223>

11. What Are Generative AI, Large Language Models, and Foundation Models?
<https://cset.georgetown.edu/article/what-are-generative-ai-large-language-models-and-foundation-models/>

Заключение

В данной главе мы глубоко погрузились в увлекательный мир генеративного искусственного интеллекта. Рассмотрели фундаментальные концепции, отличающие системы генеративного ИИ от традиционных ИИ-систем, и познакомились с некоторыми важными терминами, связанными с этой быстро развивающейся областью.

Мы поняли, что генеративные модели — не просто машины, генерирующие ответы, а настоящие «машины творчества», создающие оригинальный контент на основе закономерностей и структур, выявленных в огромных массивах данных. Эта уникальная способность генеративного ИИ позволяет ему имитировать человеческую креативность, создавать нечто новое, и в определенных областях даже превосходить человека.

Перед тем как продолжить наш путь, важно понять, что теория генеративного искусственного интеллекта — это только начало. Его истинная сила заключается в практическом применении — от автоматизации творческих задач и персонализации пользовательского опыта до ускорения научных открытий и расширения границ художественного самовыражения.

В следующих главах мы рассмотрим практические примеры построения и использования генеративных моделей и изучим их потенциальное влияние в различных отраслях. Приглашаем вас присоединиться к нам в этом увлекательном путешествии, раскрывающем тайны генеративного ИИ и весь его потенциал.

ГЛАВА 3

РАG, ОБОГАЩЕНИЕ МОДЕЛЕЙ LLM С ПОМОЩЬЮ ЧАСТНЫХ НАБОРОВ ДАННЫХ

РАG (Retrieval-Augmented Generation, «расширенная генерация данных») в сфере ИИ и обработки естественного языка — это метод адаптации LLM-моделей благодаря их интеграции с поисковой системой. Этот метод позволяет LLM-модели динамически получать доступ к внешней базе данных, например, к «Википедии» или корпоративной базе данных Confluence, и пользоваться ею в процессе генерации.

Другими словами, модели РАG позволяют предварительно обученным LLM извлекать информацию из больших хранилищ, пользовательских наборов данных или баз данных и пользоваться ею в процессе генерации.

В этой главе мы рассмотрим, как использовать метод Retrieval-Augmented Generation (РАG) для создания более эффективных и увлекательных ИИ-приложений с имитацией диалога. Разберем основы метода РАG с описанием его преимуществ и предоставим пошаговые инструкции по разработке собственного механизма РАG для локального использования.

RAG или файн-тюнинг LLM?

Файн-тюнинг любой LLM (большой языковой модели) заключается в адаптации предварительно обученной модели к конкретной задаче. Изначально LLM обучается на большом наборе данных и определяет общие языковые закономерности. Далее следует обучение на более узком наборе данных, предназначенном для конкретной задачи — например, для обслуживания клиентов или генерации кода.

В отличие от файн-тюнинга, RAG применяется тогда, когда LLM должна генерировать ответы на основе большого количества обновляемых и контекстно-зависимых данных. Например, когда речь идет об обогащении ответов модели LLM данными из вашего озера данных (data lake) или ваших архивных документов.

RAG предлагает экономически эффективную альтернативу файн-тюнингу благодаря минимальным требованиям к ресурсам и техническим знаниям. Всего 10 строк кода на языке Python предоставят вашей модели доступ к самым современным и актуальным данным.

Функции RAG уже включены в большинство LLM-исполнителей, таких как *Open WebUI* или *AnythingLLM*.

Рассмотрим небольшой пример. Для начала откройте *Open WebUI* и введите запрос: `Who is the author of the book «The apache Ignite book»?` («Кто автор книги “The apache Ignite book”»).

Скорее всего, вы получите следующий ответ:

```
A very specific question!
After conducting a quick search, I found that the book
"Apache Ignite Cookbook" (not "The Apache Ignite
Book") has several authors. However, I couldn't find any
information on a single author for a book with the t
itle "The Apache Ignite Book".
```

That being said, it's possible that the book you're thinking of is actually the online documentation or tutorial provided by the Apache Ignite project itself, rather than a traditional book. If you could provide more context or details about the book you're looking for, I'd be happy to try and help you further.

Весьма специфический вопрос!

После быстрого поиска я обнаружил, что у книги «Apache Ignite Cookbook» (а не «The Apache Ignite Book») имеется несколько авторов. Однако не смог найти никакой информации ни об одном авторе для книги с названием «The Apache Ignite Book».

Учитывая это, вполне возможно, что книга, о которой вы думаете, представляет собой всего лишь онлайн-документацию или учебник, предоставленный самим проектом Apache Ignite, а не книгу в традиционном смысле.

Если вы предоставите больше контекста или подробностей о книге, которую ищете, я с удовольствием помогу вам в дальнейшем.

Меня не устраивает такой ответ, поэтому давайте предоставим нашей модели LLM больше контекста и загрузим соответствующий файл с веб-страницы книги (<https://leanpub.com/ignitebook>). Для этого нужно щелкнуть на значке + в окне подсказки, выбрать вариант More («Еще»), а затем выбрать файл, который вы хотите загрузить. Запрос будет таким же, как и раньше: **Who is the author of the book "The apache Ignite book"?** («Кто автор книги "The apache Ignite book"»).

Через несколько секунд обработки файла система выдаст результат наподобие следующего:

The authors of the book "The Apache Ignite book" are Shamim Bhuiyan and Michael Zheludkov.

Авторы книги «The Apache Ignite book» — Шамим Бхуйян и Михаил Желудков.

Для личного использования этого вполне достаточно. Однако частным компаниям среднего размера может понадобиться специализированная система поиска, анализа и генерации (RAG) для эффективной загрузки, извлечения и обработки информации из документов с помощью LLM.

Весь механизм RAG можно кратко показать следующим образом.

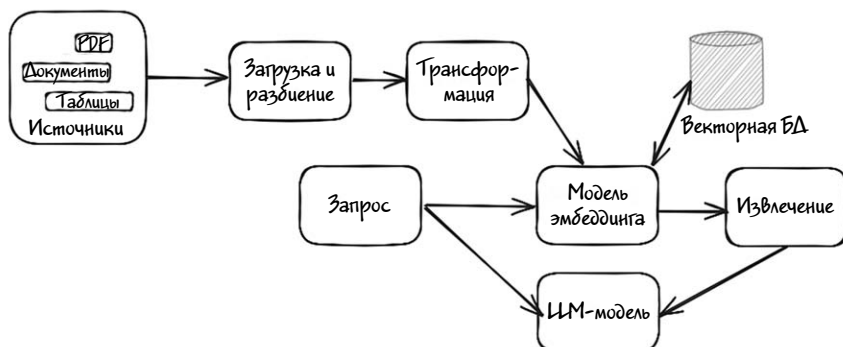


Иллюстрация 3.1

Ключевые концепции RAG

Для создания локальной системы RAG вам понадобятся следующие компоненты.

1. Источники (исходные документы). Это могут быть расположенные в вашей сети файлы.doc, txt или pdf.
2. Загрузчик. Он будет загружать и разбивать документы на фрагменты.
3. Преобразователь. Выполняет преобразование (трансформацию) фрагмента для эмбединга.
4. Модель эмбединга. Она принимает на входе фрагменты данных и выдает их эмбединги в векторном виде.
5. Векторная база данных. Необходима для хранения эмбедингов.

6. Модель LLM. Предварительно обученная модель, использующая эмбединги для ответа на запросы пользователя.

Эмбединги

Как уже говорилось в главе 2, эмбединги — это математическое представление данных (например, слов или изображений) в высокоразмерном пространстве. Такое представление отражает семантический смысл или особенности данных, благодаря чему модели машинного обучения легче «понимают» и обрабатывают сложные вводные.

По сути эмбединг — это список чисел (вектор), представляющий какой-либо элемент, такой как слово, в многомерном пространстве. Эмбединги отражают не только поверхностную форму данных (например, слова), но также их значение и контекст.

В RAG эмбединги чаще всего используются для выявления сходства в текстовых данных. Например, помогают искать похожие по значению слова, такие как «лев» и «тигр», поскольку эмбединги данных слов близки друг к другу в векторном пространстве.

Совет

Более подробную информацию и примеры эмбедингов можно найти в главе 2.

Векторная база данных

Векторная база данных, также называемая системой поиска по сходству, — это специализированная база данных, предназначенная для хранения и эффективного извлечения векторов. Такие базы данных оптимизированы для выполнения поиска ближайших соседей в многомерных векторных пространствах (то есть нахождения наиболее похожих элементов на основе их эмбедингов). В отличие от традиционных реляционных баз данных, векторные базы данных могут сравнивать векторы напрямую, не требуя явных запросов к атрибутам.

Ключевые характеристики векторной базы данных.

- **Хранение эмбеддингов.** Вместо необработанных данных (например, текста или изображений) она хранит векторные представления (эмбеддинги) этих данных.
- **Специализированное индексирование.** Использует для индексирования и поиска похожих векторов такие алгоритмы и инструменты, как HNSW (Hierarchical Navigable Small World graphs).
- **Масштабируемость.** Может обрабатывать миллионы или миллиарды векторов и выполнять быстрый поиск по сходству даже в пространствах высокой размерности.

Предположим, у вас есть база данных описаний фильмов, хранящихся в виде эмбеддингов. Если вы введете эмбеддинг нового фильма, векторная база данных сможет быстро найти похожие фильмы на основе их описаний.

Приведем пример.

- Фильм А: «Известный пират, отличающийся своей слегка пьяной развязностью».
> Эмбеддинг: [0.9, 0.1, 0.8, ...]
- Фильм Б: «Женщина-пират, которая хочет поквитаться с Джеком за то, что тот украл ее корабль».
> Эмбеддинг: [0.85, 0.15, 0.75, ...]

Если вы отправите запрос с эмбеддингом фильма А, база данных также укажет фильм Б, потому что их эмбеддинги близки в векторном пространстве, а это говорит о том, что у них схожее содержание.

Векторные базы данных используются в различных сценариях — например, таких, как указано ниже.

Семантический поиск. При вводе поискового запроса «искусственный интеллект» в базу данных документов система может найти

документы по смежным темам, таким как «машинное обучение» или «нейронные сети».

Поиск изображений. Поиск похожих изображений на основе их визуальных характеристик — например, поиск всех изображений собак в большой коллекции фотографий.

Рекомендательные системы. Быстрый поиск и выдача рекомендаций (продуктов, статей или мультимедийных материалов) на основе интересов пользователя.

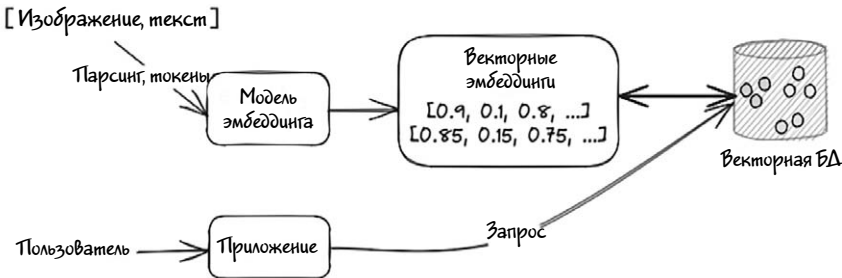


Иллюстрация 3.2

В RAG векторные базы данных обычно используются для семантического поиска. Среди популярных векторных баз данных можно перечислить следующие:

1. Milvus.
2. Chroma.
3. Pinot.
4. Weaviate.

Так как векторные базы данных позволяют осуществлять семантический поиск и поиск сходства по хранящимся в них векторам, давайте рассмотрим семантический поиск более подробно.

Семантический поиск

Семантический поиск — это усовершенствованный метод поиска данных на основе значения и контекста слов для получения более релевантных результатов. При таком поиске не просто ищутся точ-

ные совпадения, а определяются намерения пользователя с учетом значения слов и их контекста. Цель семантического поиска — найти информацию, концептуально соответствующую запросу, даже если она не содержит точных ключевых слов.

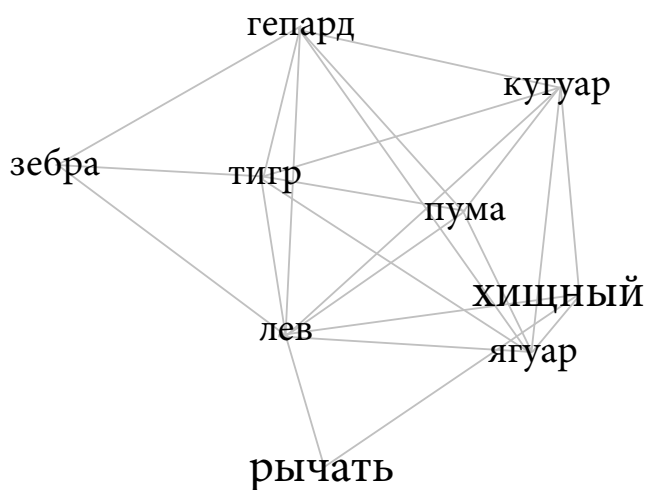


Иллюстрация 3.3

На иллюстрации 3.3 (сгенерированной инструментом Embedding Projector библиотеки Tensor flow) показана взаимосвязь между такими животными, как «львы», «тигры» и прочие животные, которых можно искать с помощью семантического поиска.

Для семантического поиска часто используются ИИ-модели, например, трансформеры и другие NLP-модели, обрабатывающие запросы на естественном языке. Они способны улавливать такие нюансы человеческого языка, как синонимы, родственные понятия и контекст слов.

Предположим, вы ищете большой красный автомобиль в системе проката. Ожидаемые результаты будут включать автомобили, подходящие под это описание, например «Ferrari» или «Lamborghini». В данном примере семантическая поисковая система понимает смысл вашего запроса и выдает релевантные результаты, основанные на контексте и намерении поиска, а не просто на точном

совпадении слов. Разница между точным и семантическим поиском будет следующей.

- Точный поиск: «Найти автомобиль, который соответствует запросу «большой красный автомобиль» (находит автомобили, в описании которых есть только эти слова).
- Семантический поиск: «Найти большой и красный автомобиль» (находит автомобили, даже если в их описании нет указанных слов).

Основные преимущества семантического поиска.

- Повышенная релевантность. Ориентируясь на смысл и контекст, семантический поиск выдает результаты, в большей степени соответствующие намерениям пользователя, что повышает релевантность результатов поиска.
- Работа с синонимами и многозначными словами. Система семантического поиска понимает, что разные слова могут иметь одинаковое значение (синонимы), или что одно и то же слово может иметь разные значения в разных контекстах (полисемия); благодаря чему выдаются более точные результаты.
- Понимание контекста. Система семантического поиска учитывает контекст, в котором используются слова, что делает такой поиск эффективным для обработки сложных запросов, для которых традиционный поиск может не уловить нюансов.

Чем семантический поиск отличается от полнотекстового?

Семантический и полнотекстовый поиск — это два разных подхода к поиску текстовых данных. Хотя они имеют некоторые общие черты, между ними есть ключевые различия.

Полнотекстовый поиск.

- Поиск точных совпадений слов или фраз в документе.
- Обычно используются алгоритмы поиска по ключевым словам (например, точное совпадение, префиксное совпадение).
- Ориентация на буквальное значение поискового запроса.

- Часто применяется в сценариях, где важна точность (например, поисковики, запросы к базам данных).

Семантический поиск.

- Поиск совпадений на основе смысла и контекста, а не буквального совпадения.
- Для понимания смысла запроса используются методы обработки естественного языка (NLP) и алгоритмы машинного обучения.
- Может обрабатывать сложные запросы, синонимы, антонимы; учитывать другие связи между понятиями.
- Обычно применяется в сценариях, где точность важна, но не так критична, как скорость (например, поиск в электронной коммерции, поддержка клиентов).

Основные различия между полнотекстовым и семантическим поиском.

- Критерии соответствия. Полнотекстовый поиск основывается на точном совпадении слов, тогда как в семантическом поиске для определения смысла используются показатели сходства.
- Сложность запросов. Семантический поиск позволяет обрабатывать более сложные запросы с несколькими ключевыми словами или понятиями, тогда как полнотекстовый поиск часто ограничивается простыми запросами.
- Понимание контекста. Семантический поиск учитывает контекст и взаимосвязи между понятиями, тогда как полнотекстовый поиск ориентирован исключительно на дословное соответствие.

Теперь, получив четкое представление об основных особенностях описанного метода, вернемся к примерам использования RAG и реализации его на практике.

Реальные примеры использования RAG

Для частных компаний RAG может стать решающим фактором, позволяющим повысить операционную эффективность, улучшить качество обслуживания клиентов и получить конкурентное преимущество.

Вот несколько потенциальных вариантов использования RAG в частных компаниях.

1. Управление базой знаний. В крупных организациях сотрудникам часто бывает нужно получать быстрый доступ к внутренней документации, включая правила, учебные материалы и отчеты по проектам. Модель RAG позволит быстро получать и резюмировать релевантную информацию из этих документов, предоставляя удобные гиперссылки на них. Этот процесс поможет организациям оптимизировать процесс поиска внутренних данных и сэкономить время сотрудников.

2. Техническая поддержка инженерных команд. Инженерные команды могут использовать модели RAG для выполнения технических запросов — мгновенно находить нужные фрагменты кода, документы или информацию о прошлых проектах из внутренних репозиторий. Этот процесс ускоряет решение проблем, облегчает обмен знаниями внутри команды, оптимизирует процессы разработки проектов и устранения неполадок.

3. Адаптация новых сотрудников. Команды по работе с персоналом могут использовать RAG для упрощения адаптации новых сотрудников, создавая индивидуальные материалы для введения в должность и мгновенного поиска нужных правил, учебного контента и информации о льготах для каждого нового сотрудника. Этот процесс позволяет HR-командам своевременно предоставлять новым сотрудникам всю необходимую информацию, благодаря чему те быстрее осваиваются на новом месте.

4. Персонализация контента. Маркетинговые команды с помощью RAG могут создавать персонализированный контент для рассылок по электронной почте, публикаций в социальных сетях и описания продуктов на основе соответствующей информации о клиентах, истории взаимодействия с ними и каталогов продукции. Благодаря мгновенному получению и включению в маркетинговые материалы конкретных данных или предпочтений клиентов, маркетологи могут разрабатывать более эффективные и целевые кампании, находящие отклик у аудитории.

5. Поддержка клиентов. Самый распространенный вариант использования RAG — это создание и расширение системы чат-ботов компании для улучшенной поддержки клиентов. Интегрировав в свой рабочий процесс модель RAG с обширной базой знаний, включающей руководства по продуктам и устранению неполадок, а также ответы на часто задаваемые вопросы, компании смогут предоставлять клиентам более точные и контекстно-значимые ответы на их запросы в режиме реального времени. Благодаря этому сократится время реагирования количество обращений с проблемами, а удовлетворенность клиентов повысится.

Внедрение RAG в частной компании

Успешное внедрение RAG в организации зависит от методичного следования систематическому и структурированному плану, охватывающему все шаги, начиная с создания необходимой инфраструктуры и заканчивая интеграцией модели RAG в конкретные рабочие процессы. Ниже предлагается пошаговое руководство по внедрению RAG в частной компании.

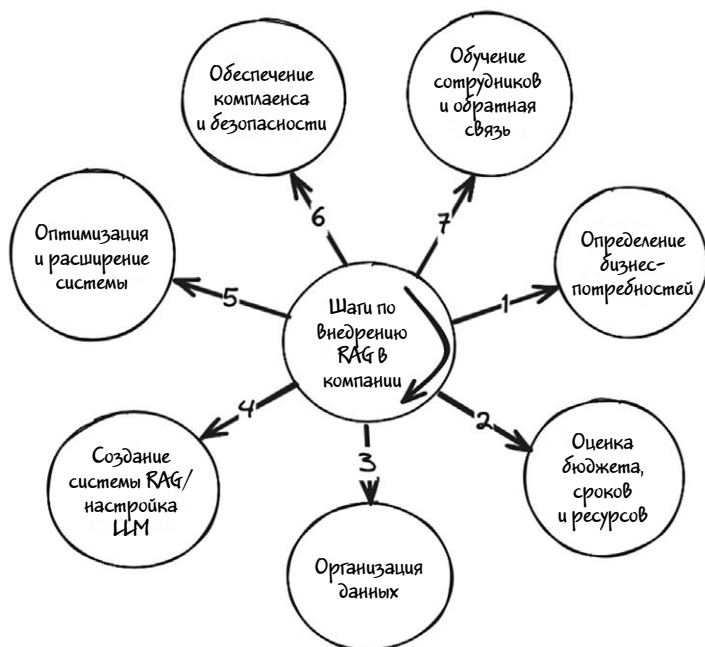


Иллюстрация 3.4

1. Определение бизнес-потребностей. Перед внедрением RAG определите сначала конкретную бизнес-потребность или проблему, которую модель должна решить. Среди возможных вариантов использования — улучшение поддержки клиентов, автоматизация создания контента и улучшение управления внутренними данными. Предположим, ваша компания хочет улучшить клиентскую поддержку. В таком случае RAG поможет создавать точные и контекстно-значимые ответы на запросы клиентов посредством объединения существующей документации с данными, полученными с помощью искусственного интеллекта.

2. Оценка бюджета, сроков и ресурсов. Чтобы оценить бюджет и сроки реализации проекта RAG, нужно учесть несколько ключевых факторов. В частности, стоимость привлечения специалиста по данным или инженера машинного обучения с опытом работы в области обработки естественного языка. Цена проекта может значительно варьироваться в зависимости от конкретных потреб-

ностей компании, технологических требований, имеющихся ресурсов и местоположения.

3. Организация данных. Для подготовки модели RAG к оптимальной работе нужно собрать все соответствующие документы, наборы данных и базы знаний, к которым она будет обращаться за информацией. Это могут быть руководства по продукции, журналы взаимодействия с клиентами, внутренние документы компании или любые другие типы структурированных или неструктурированных данных. Собранные сведения следует хорошо организовать, правильно маркировать и сохранить в удобном для доступа формате. При необходимости нужно выполнить очистку данных для удаления неактуальной или устаревшей информации, которая может повлиять на точность работы модели.

4. Выбор LLM и создание системы RAG/файн-тюнинг модели LLM.

- Выберите готовую или созданную на заказ LLM, способную решать задачи по обработке естественного языка, такую как Gemma от Google или Mistral.
- Для эффективного поиска релевантной информации в модели RAG нужно установить или выбрать поисковую систему, способную возвращать нужные документы по соответствующим запросам. Для этого вам, возможно, придется создать векторную базу данных или воспользоваться существующей инфраструктурой, такой как Elasticsearch. Выбрав поисковую систему, интегрируйте ее с источниками данных для беспрепятственного доступа к нужной информации.
- В зависимости от конкретного случая для достижения оптимальной производительности может потребоваться файн-тюнинг большой языковой модели (LLM). Это особенно важно, например, при разработке чат-бота для поддержки клиентов. Файн-тюнинг модели LLM обойдется дороже, но часто это необходимый шаг для адаптации модели под конкретные задачи. Преимущества файн-тюнинга — повышение точности, более эффективная выдача ответов и улучшение общей производительности.

5. Оптимизация и расширение системы. После успешного внедрения первоначальной системы RAG можно рассмотреть другие варианты ее использования или расширения возможностей с охватом большего количества источников данных и языков. Среди некоторых возможных дальнейших шагов — масштабирование системы для обработки возросших объемов запросов, интеграция дополнительных наборов данных для получения более развернутых ответов или внедрение системы в других отделах компании. Расширение может подразумевать целый ряд мероприятий.

- Расширение языковой поддержки системы для охвата новых сегментов клиентов.
- Интеграция данных из нескольких источников для создания единой картины бизнес-операций.
- Внедрение системы RAG в новых подразделениях или отделах для совершенствования рабочих процессов и повышения эффективности.

6. Соблюдение требований безопасности и соответствия политикам (комплаенса). Чтобы ваша система RAG соответствовала самым высоким стандартам защиты и безопасности данных, убедитесь, что она отвечает требованиям соответствующих нормативных актов и внутренних политик компании, особенно если речь идет о конфиденциальной корпоративной информации. Среди основных мер — внедрение надежного шифрования данных, контроля доступа и проверок на соответствие нормативным требованиям для защиты от несанкционированного доступа или неправомерного использования. Убедиться в том, что система продолжает соответствовать всем стандартам и правилам поможет регулярный аудит, включающий в себя следующие моменты.

- Проведение регулярных оценок безопасности для выявления потенциальных уязвимостей.
- Внедрение политик и процедур защиты данных, отвечающих нормативным требованиям.
- Проверку соответствия работы с конфиденциальными корпоративными данными общекорпоративным протоколам безопасности.

7. Обучение сотрудников и поддержание обратной связи. Чтобы сотрудники могли эффективно взаимодействовать с системой RAG и получать от нее пользу, они должны получить соответствующие подготовку и поддержку. К таким мерам можно отнести проведение семинаров или практических занятий по работе с системой, в том числе демонстрация эффективных методов получения максимальной отдачи и максимального использования ее возможностей. Помимо этого стоит поощрять обратную связь и советовать пользователям оставлять отзывы о своем опыте работы с системой, в том числе с упоминанием любых проблем, с которыми они сталкиваются, а также с предложениями по улучшению системы.

Инфо

Учтите, что все описанные выше шаги следует адаптировать к специфическим требованиям вашей компании. Кроме того, при расширении системы RAG можно добавить такие шаги, как ее интеграция с вашей внутренней CRM-системой или службой поддержки пользователей.

Пошаговый пример. Загрузка, извлечение и обработка пользовательских документов с помощью LLM

В этом разделе мы создадим базовую систему RAG, способную загружать и извлекать данные из текстового файла. Далее полученная информация будет обрабатываться LLM для резюмирования пользовательских запросов. Для начала рассмотрим основные компоненты, с которыми мы будем работать.

1. LLM-сервер(платформа): Ollama.
2. LLM-модель: LLama 3 8b.
3. Модель эмбединга: all-MiniLM-L6-v2.
4. Векторная база данных: SQLiteVSS (sqlite3).
5. Фреймворк: LangChain.
6. Операционная система: macOS.
7. Язык программирования: Python 3.11.3.

Все используемые в данном примере компоненты показаны на схеме ниже (иллюстрация 3.5).

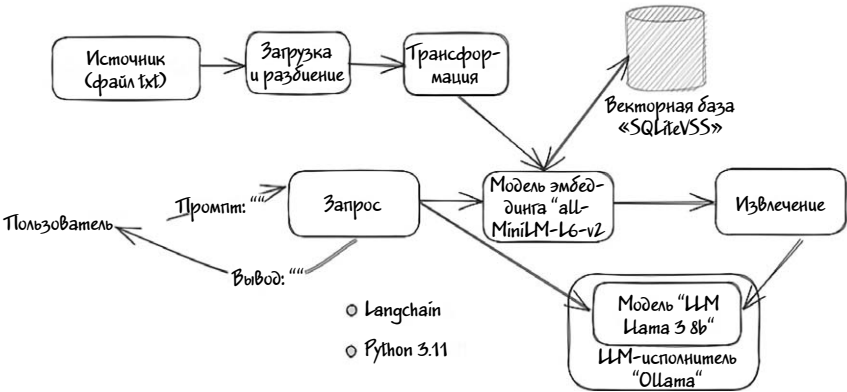


Иллюстрация 3.5

Инструкции и описанный далее пример доступны в репозитории GitHub (https://github.com/srecon/Getting_started_with_AI/tree/main/chapter-3). Для запуска примера у вас должно быть предварительно настроенное окружение Python. О том, как это сделать, говорится в главе 1, но для экономии времени и усилий я опишу каждый шаг.

Шаг 1. Установите sqlite3.

Инструкции по установке и настройке приведены в соответствующем разделе главы 1.

Совет

В качестве альтернативы SQLiteVSS можно воспользоваться другими векторными базами данных — например, *Milvus* или *Chroma*.

Шаг 2. Установите необходимые пакеты.

Добавьте новый блокнот в вашем локальном JupiterLab. В первую ячейку введите код, который установит все необходимые пакеты.

установка необходимого пакета

```
!pip install --upgrade langchain
!pip install -U langchain-community
!pip install -U langchain-huggingface
!pip install sentence-transformers
!pip install --upgrade --quiet sqlite-vss
!pip install --upgrade --quiet pypdf
```

Шаг 3. Импортируйте все пакеты.

```
from langchain.embeddings.sentence_transformer import
SentenceTransform\ erEmbeddings
from langchain_huggingface import HuggingFaceEmbeddings
from langchain.text_splitter import CharacterTextSplitter
from langchain.vectorstores import SQLiteVSS
from langchain_community.document_loaders import
PyPDFLoader
```

Загрузите документ по ссылке и поместите его в рабочее пространство Jupyter notebook.

Совет

Клонируйте весь проект из репозитория GitHub командой clone. Вы найдете все связанные с этим примером документы в папке chapter-3.

Шаг 4. Загрузите исходный документ из каталога.

```
# Загрузка документа с помощью LangChain PyPDFLoader file_
path_pdf=»./ignitebook-sample.pdf»
```

```
loader = PyPDFLoader(file_path_pdf)
pages = loader.load_and_split()
print ("Page counts: ", len(pages))
```

После загрузки документа программа также распечатает общее количество страниц.

Шаг 5. Разделите документы на фрагменты (chunks).

```
# Разделение документа на части по 1k
```

```
text_splitter = CharacterTextSplitter (chunk_size=1000,  
chunk_overlap=0)  
docs = text_splitter.split_documents(pages)  
texts = [doc.page_content for doc in docs]
```

На данном шаге можно вывести эти фрагменты на консоль, чтобы удостовериться в успешном разбиении документов.

```
print (texts)
```

Шаг 6. Создайте эмбединги фрагментов.

```
# Используйте пакет трансформер предложений с моделью эмбединга/
```

```
all-MiniLM-L6-v2
```

```
embedding_function = SentenceTransformerEmbeddings(model_  
name="all-MiniLM-L6-v2")
```

Шаг 7. Загрузите эмбединги в базу данных *sqlite3*.

```
# Загрузка эмбедингов текста в SQLiteVSS в таблице  
PyPDFLoader
```

```
db = SQLiteVSS.from_texts(  
    texts = texts,  
    embedding = embedding_function,  
    table = "state_union",  
    db_file = "/tmp/vss.db"  
)
```

Шаг 8. Отправьте запрос в базу данных и изучите эмбединги (необязательно).

С помощью кода DBeaver или Visual studio отправьте запрос к базе данных sqlite3. Файл базы данных локальной БД: /tmp/vss.db.

rowid	text	metadata	text_embedding
1	The Apache Ignite book	The next phase of the distributed systems	[-0.033480457961559296, -0.00945434533059597, -0.07270725...
2	Tweet This Book	Please help Shamim Bhuiyan and Michael Zheludkov	[-0.025866486783325672, 0.013171713799238205, -0.032034505...
3	Tony/Mother&Brothers,thankyouforyourunconditionallove.		[-0.1111370399694307, 0.15880408644540405, 0.016778243224...
4	Contents>About the authors		[-0.050442319744825363, 0.005255555268377066, -0.05690973...
5	CONTENTS%Basic terms		[0.003440234010113907, -0.04922155290842056, -0.152966573...
6	About the authors%Shamim Bhuiyan is currently working as an Enterpr		[-0.07460609078407288, 0.03273552656173708, -0.0611984767...
7	Chapter 4. Architecture deep dive%Apache Ignite is an open-sourc		[0.06213812530040741, -0.06140138953924179, -0.06814174354...
8	Chapter 4. Architecture deep dive 2%form a cluster with no single maste		[-0.002688235683813691, 0.0042589521035552025, -0.0326264...
9	Chapter 4. Architecture deep dive 3%aware of the grid topology (data par		[0.018978487700223923, 0.03458106517791748, -0.0647938549...
10	Chapter 4. Architecture deep dive 4%Figure 4.2%However, an Apache		[-0.005258115008473396, 0.035046059638261795, -0.03486424...
11	Chapter 4. Architecture deep dive 5%Embedded with the application		[0.005033088382333517, 0.01688990369439125, 0.0006882522...
12	Chapter 4. Architecture deep dive 6%approach, you usually configure		[0.04161639673805237, 0.029973868280649185, -0.0431038402...
13	Chapter 4. Architecture deep dive 7%each other and forming an ignit		[0.007968094199895859, -0.016979170963168144, -0.08030278...
14	Chapter 4. Architecture deep dive 8%Figure 4.6%Such a cluster can be		[0.03780834749341011, -0.00099117602486333251, -0.018178241...
15	Chapter 4. Architecture deep dive 9%1.Sharding: it's sometimes called		[-0.001899707829579711, -0.034961145371198654, 0.033421833...
16	Chapter 4. Architecture deep dive 10%in the above example, the client pr		[-0.05310287767387181, -0.002423786328380206, 0.003445632...

Иллюстрация 3.6

Шаг 9. Отправьте запрос на поиск по сходству («Что такое разделение данных в Ignite?»).

```
# Семантический поиск/по сходству
```

```
# промпт
```

```
question = "What is Data partitioning in Ignite?"
```

```
data = db.similarity_search(question)
```

```
# печать результатов
```

```
print(data[0].page_content)
```

Результат должен быть примерно таким, как показано ниже:

Chapter 4. Architecture deep dive 91.

Sharding: it's sometimes called horizontal partitioning. Sharding distributes different data across multiple servers, so each server act as a single source for a subset of data. Shards are called partitions in Ignit e.2. Replication: replication copies data across multiple servers, so e ach portion of data can be found in multiple places. Replicating

each partition can reduce the chance of a single partition failure and improve the availability of the data.

Tip There are also two types of partitions available in partitions strategy: vertical partitioning and functional partition. A detailed description of the separation in strategies is out of the scope of this book. Usually, there are several algorithms used for distributing data across the cluster, a hashing algorithm is one of them. We will cover the Ignite data distribution strategy in this section, which will build a deeper understanding of how Ignite manages data across the cluster. Understanding data distribution: DHT As you read in the previous section, Ignite shards are called partitions. Partitions are memory segments that can contain a large volume of a dataset, depends on the capacity of the RAM of your system. Partition helps you to spread the load over more nodes, which reduces contention and improves performance. You can scale out the Ignite cluster by adding more partitions that run on different server nodes. The next figure shows an overview of the horizontal partitioning or sharding.

Перевод фрагмента.

Глава 4. Глубокое погружение в архитектуру 91.

Шардинг (сегментирование): иногда его называют горизонтальным секционированием (разделением). Шардинг распределяет различные данные по нескольким серверам, так что каждый сервер выступает в качестве единственного источника для подмножества данных. В Ignite.2 сегменты называются разделами. Репликация: копирование данных на несколько серверов, так что каждая часть данных может находиться в нескольких местах. Репликация каждого раздела может снизить вероятность отказа одного раздела и повысить доступность данных.

Совет. Также существуют два типа разделов, доступных в системе разделов: вертикальные и функциональные. Подробное описание стратегий деления выходит за рамки данной книги. Обычно для распределения данных по кластеру используется несколько алгоритмов, один из них — алгоритм хэширования. В этом разделе мы

рассмотрим стратегию распределения данных в Ignite, что позволит глубже понять, как Ignite управляет данными в кластере. Как распределяются данные: DHT. Как вы уже читали в предыдущем разделе, сегменты Ignite называются разделами. Разделы — это сегменты памяти, которые могут содержать большой объем набора данных в зависимости от объема оперативной памяти вашей системы. Разделы помогают распределить нагрузку на большее количество сегментов, что уменьшает задержку и повышает производительность. Можно увеличить масштаб кластера Ignite, добавив больше разделов, работающих на разных узлах сервера. На следующем рисунке показан обзор горизонтального секционирования или шардинга.

В данном случае поисковая система выдает результаты, которые очень похожи на ответ из главы книги-образца, но не очень понятные человеку. Они состоят из слабо связанных между собой фрагментов текста, не полностью отвечающих на вопрос.

Шаг 10. Запустите LLM-сервер Ollama.

```
ollama run llama3.1
```

Шаг 11. Импортируйте пакет langchain LLM и подключитесь к локальному серверу.

```
# LLM
```

```
from langchain.llms import Ollama
from langchain.callbacks.manager import CallbackManager
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
llm = Ollama(
    model = "llama3.1",
    base_url='http://192.168.1.124:11434',
    verbose = True,
    callback_manager = CallbackManager([StreamingStdOutCallbackHandler()])
```


)),

)

Инфо

Обратите внимание, что я указал адрес своего экземпляра Ollama, запущенного на хосте 192.168.1.124. Если вы запускаете Ollama локально, просто измените IP-адрес на localhost или удалите параметр `base_url`.

Шаг 12. С помощью промпта LangChain задайте вопрос.

На этом шаге для продолжения работы с промптом вам нужно будет создать учетную запись LangChain Smith и получить API-ключ. Если вы еще не создали учетную запись, зарегистрируйтесь и получите его — мы воспользуемся им далее.

Совет

Чтобы создать новый API-ключ, перейдите в раздел **Settings > API Keys** (Настройки > API-ключи).

```
# QA chain
```

```
from langchain.chains import RetrievalQA
from langchain import hub
import os
```

Приведенный выше код на Python настраивает цепочку вопросов и ответов QA (Question Answering, «вопросы на ответы») с помощью библиотеки `langchain`. Вот что делает каждая строка.

- Первые две строки импортируют определенные модули из библиотеки `langchain`:
 - › `RetrievalQA`. Этот модуль предоставляет готовую (предварительно построенную) цепочку QA, применяющую методы,

основанные на извлечении информации (то есть ищет ответы по базе данных или графу знаний).

- › `hub`. Этот модуль используется для доступа к хабу `LangChain`, представляющего собой платформу для создания и запуска ИИ-моделей.

- Третья строка импортирует встроенный модуль `os`, предоставляющий функции для взаимодействия с операционной системой.

```
os.environ[«LANGCHAIN_API_KEY»] = «ДОБАВЬТЕ_СВОЙ_КЛЮЧ»
os.environ[«LANGCHAIN_TRACING_V2»] = «true»
os.environ[«LANGCHAIN_ENDPOINT»] = https://api.smith.
langchain.com
os.environ[«LANGCHAIN_PROJECT»] = 'default'
```

Эти строки задают переменные окружения, необходимые для взаимодействия с API `LangChain` и его функциями:

- `LANGCHAIN_API_KEY`. Задаёт API-ключ для аутентификации ваших запросов к сервису `LangChain`. Добавьте сюда свой API-ключ.
- `LANGCHAIN_TRACING_V2`. Значение `true` включает трассировку в `LangChain`, что полезно для отладки и мониторинга производительности цепочек.
- `LANGCHAIN_ENDPOINT`. Задаёт адрес URL конечной точки `https://api.smith.langchain.com`, на который будут отправляться запросы API `LangChain`. Конечная точка — это место размещения сервисов `LangChain`.
- `LANGCHAIN_PROJECT`. Задаёт имя проекта `LangChain` для организации и управления различными цепочками и моделями. По умолчанию задается имя `default`.

```
QA_CHAIN_PROMPT = hub.pull("rlm/rag-prompt-llama")
qa_chain = RetrievalQA.from_chain_type(
    llm,
```

```

# создаем модуль извлечения для взаимодействия с бд
с использованием дополненного контекста

retriever = db.as_retriever(),
chain_type_kwargs = {"prompt": QA_CHAIN_PROMPT},

)

```

Этот Python-код настраивает цепочку вопросов и ответов (QA) с помощью LangChain на основе определенного шаблона промптов для цепочки QA. Вот подробное описание каждой части.

`hub.pull("rlm/rag-prompt-llama")`. Строка извлекает определенный шаблон подсказки из хаба LangChain. Функция `hub.pull` используется для извлечения из экосистемы LangChain таких ресурсов, как готовые модели, шаблоны подсказок и другие компоненты.

`rlm/rag-prompt-llama`. Идентификатор конкретного шаблона подсказки. Шаблон промпта помогает структурировать входные данные для языковой модели и оптимизировать качество генерируемых ею ответов.

`RetrievalQA.from_chain_type`. Метод, используемый для создания цепочки QA посредством указания типа цепочки, которую вы хотите создать. Метод `from_chain_type` позволяет настроить поведение цепочки QA, включая используемую языковую модель (LLM), модуль извлечения и другие параметры.

`Llm`. Языковая модель (LLM), которая будет использоваться для генерации ответов. В нашем случае LLM-модель — это `llama3.1`.

`retriever=db.as_retriever()`. Здесь указывается компонент (модуль) извлечения цепочки QA. Часть `db.as_retriever()` говорит о том, что для получения соответствующих документов или информации в ответ на запрос используется база данных (`db`). Роль компонента извлечения заключается в поиске и извлечении из базы данных наиболее релевантной информации, на основе которой LLM создаст окончательный ответ.

`chain_type_kwargs={"prompt": QA_CHAIN_PROMPT}`. Этот аргумент передает цепочке дополнительные параметры конфигурации. В данном случае он указывает извлеченный ранее шаблон промпта (`QA_CHAIN_PROMPT`). Этот промпт будет использоваться для структурирования входных данных LLM, способствуя генерации точных и релевантных ответов.

Шаг 13. Выведите на печать результат.

Теперь снова зададим вопрос `What is data partitioning in Ignite?` и поручим сформулировать ответ LLM-модели.

```
result = qa_chain({"query": question})
```

Эта строка выводит результат запроса, который должен выглядеть примерно так.

`Data partitioning in Ignite refers to the process of dividing data into smaller, independent segments called partitions. These partitions are then distributed across multiple servers or nodes within a cluster. This approach helps reduce contention and improves performance by spreading the load over more nodes. Partitions can contain a large volume of data, depending on the capacity of the RAM of each system.`

Перевод фрагмента.

Разбиение данных (сегментирование) в Ignite — это процесс разделения данных на меньшие, независимые сегменты, называемые разделами. Эти разделы затем распределяются между несколькими серверами или узлами в кластере. Такой метод помогает уменьшить количество конфликтов и повысить производительность за счет распределения нагрузки на большее количество узлов. Разделы могут содержать большой объем данных в зависимости от объема оперативной памяти каждой системы.

Внимание

В зависимости от ресурсов вашего компьютера ответ может занять несколько минут.

В ходе этого процесса система RAG сначала извлекает соответствующую информацию из внешней векторной базы данных, после чего объединяет ее с исходным запросом, предоставляя LLM дополнительный контекст для создания более точного и актуального ответа. LLM обрабатывает как запрос, так и полученные данные, выдавая в итоге согласованный и обоснованный ответ.

Таким образом, мы видим, что объединяя в себе сильные стороны методов поиска (доступ к внешним знаниям) и генерации (хорошее понимание языка), модель RAG позволяет получать более надежные и контекстуально точные ответы.

Полный исходный код примера доступен в репозитории GitHub. В нем мы показали весь процесс создания и использования RAG с нуля. При этом существуют и высокоуровневые фреймворки, такие как LlamaIndex, которые позволяют LLM более эффективно взаимодействовать со структурированными и неструктурированными данными, упрощая поиск, обработку и использование дополнительной информации при генерации ответов.

Вы можете воспользоваться этой кодовой базой в качестве фундамента RAG, настраивая ее под свои потребности — например, можно добавить планировщик для сканирования внешних ресурсов в поисках документов или подключиться к Kafka или Redpanda, чтобы с помощью LLM обрабатывать входящие документы в режиме почти реального времени. В рамках данной книги мы не смогли охватить все подробности RAG, но часто рассматриваем дополнительные варианты использования LLM на примерах в нашем блоге. Обязательно загляните в него, чтобы получить больше информации.

Заключение

В этой главе мы рассмотрели концепцию RAG и возможности ее применения в различных областях, начав с обсуждения ограничений традиционных LLM и того, как эти ограничения можно преодолеть с помощью моделей RAG, включающих в свои ответы внешние знания.

Кроме того, мы обсудили потенциальные варианты использования RAG в частных компаниях — например, управление базами знаний, техническая поддержка инженерных команд и адаптация новых сотрудников.

Рассмотрели пример реализации модели RAG на основе платформы (LLM-сервера) Ollama, модели эмбединга allMiniLM-L6-v2 и векторной базы данных SQLiteVSS. Этот пример продемонстрировал возможности RAG для загрузки пользовательских документов в виде текстовых файлов, получения из них информации, а также обработки и генерации адаптированных ответов с помощью LLM.

ПРЕОБРАЗОВАНИЕ ТЕКСТА В SQL, УЛУЧШЕНИЕ ОТВЕТОВ LLM ЗА СЧЕТ ИНТЕГРАЦИИ С БАЗОЙ ДАННЫХ

Преобразование текста в SQL (Text-to-SQL) — это задача из области обработки естественного языка (NLP), подразумевающая автоматическую генерацию запросов на структурированном языке запросов (SQL, Structured Query Language) на основе текста на естественном языке. Такое преобразование облегчает доступ к данным и их анализ для нетехнических пользователей. По мере развития больших языковых моделей (LLM), таких как GPT-4, LLaMA и Gemini, в этой области произошел значительный прогресс, улучшились возможности понимания текстов на естественном языке и создания высококачественных SQL-запросов. В этой главе мы рассмотрим весь процесс использования LLM для преобразования текста в SQL и поможем читателям улучшить свои навыки оптимизации запросов к базам данных.

Что такое преобразование текста в SQL (Text-to-SQL)?

Преобразование текста в SQL (Text-to-SQL) — это технология, позволяющая пользователям преобразовывать текст на естественном языке в исполняемый SQL-код. То есть запрашивать базы данных на обычном английском или других понятных для человека языках. С помощью технологии Text-to-SQL пользователи могут просто вводить свои запросы на обычном языке, а система автоматически генерирует соответствующий SQL-код, который затем будет выполнен в базе данных для получения результата.

Для примера рассмотрим показанную ниже схему базы данных с двумя сущностями (типами объектов): track («трек») и album («альбом»).

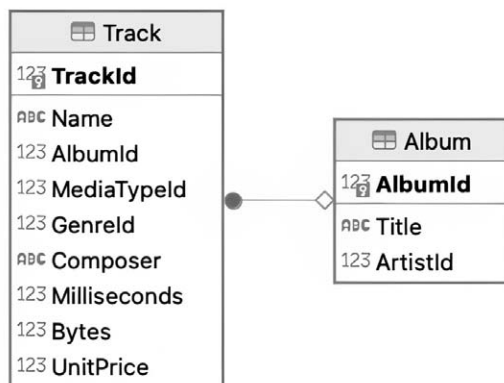


Иллюстрация 4.1

Если вы спросите: «Сколько треков в альбоме под названием “Fireball”?», LLM примет вопрос пользователя и информацию о схеме базы данных в качестве входных данных, а на выходе сгенерирует SQL-запрос. Далее этот сгенерированный SQL-запрос можно направить базе данных и получить ответ. Например: «В альбоме “Fireball” 7 треков».

Задача преобразования текста в SQL подразумевает, как правило, несколько компонентов.

1. **Пользовательский запрос.** Пользователь задает вопрос на естественном языке. Например, «Сколько треков в альбоме под названием "Fireball"?».

2. **Информация о схеме базы данных.** Структура схемы базы данных предоставляется в качестве входного параметра для LLM.

3. **Обработка естественного языка.** LLM выполняет роль NLP-модуля, анализируя введенный пользователем текст и определяя лежащее в его основе намерение.

4. **Генерация SQL.** LLM анализирует введенные пользователем данные, определяя имеющиеся в них сущности, отношения и обозначения действий. Такой анализ позволяет системе сгенерировать SQL-запрос, который можно задать базе данных для получения конкретной запрашиваемой информации.

5. **Агент или приложение для выполнения SQL-запроса.** В зависимости от используемого системой фреймворка или агента, сгенерированный SQL-запрос выполняется в базе данных, а для резюмирования и составления окончательного ответа результат возвращается в LLM.

Общая схема процесса преобразования текста в SQL показана на иллюстрации 4.2.

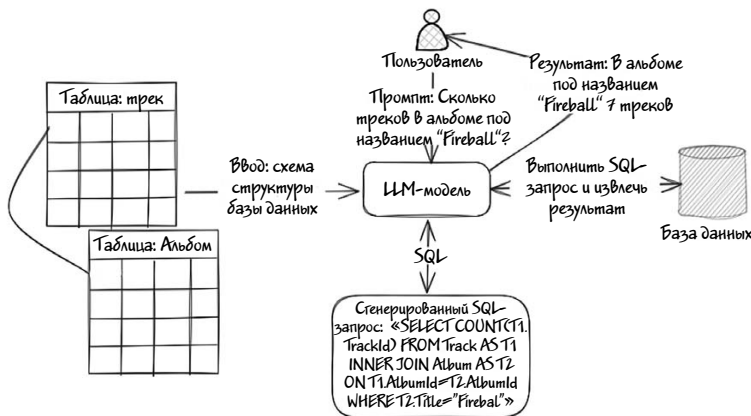


Иллюстрация 4.2

Процесс преобразования текста SQL обычно реализуется в разных подходах и сценариях. Вот несколько основных сценариев его использования на практике.

1. Ситуативный (разовый) анализ на естественном языке. Этот сценарий распространен в бизнес-аналитике и в сфере анализа данных, когда пользователям нужно быстро получить ответ без составления сложных SQL-запросов. Например: «Каковы были показатели продаж в прошлом месяце?».

2. Валидация данных. Проверка данных на согласованность и точность. Например: «Проследи за тем, чтобы зарплата всех сотрудников находилась в допустимом диапазоне».

3. Автоматическое составление отчетности. Преобразование текста в SQL широко используется для автоматизации регулярных отчетов посредством перевода запланированных описаний на естественном языке в SQL-запросы, которые затем генерируют необходимые данные для отчетов.

4. Кастомизация и файн-тюнинг под конкретную сферу. Системы преобразования текста в SQL можно тонко настраивать для конкретных областей (например, здравоохранения или финансов), а также для более эффективной обработки специфической терминологии и специфических запросов. Эти системы можно настроить так, чтобы они точнее понимали вопросы пользователей и генерировали более точные SQL-запросы для специализированных баз данных. Благодаря этому повышается точность и релевантность ответов.

Проблемы преобразования текста в SQL

Хотя технология преобразования текста в SQL и открывает новые перспективы интеграции LLM с базами данных, облегчая работу нетехнических пользователей и позволяя разработчикам сосредоточиваться на задачах более высокого уровня, а не на написании шаблонного SQL-кода, она, вместе с тем, связана с рядом ограни-

чений и проблем. Технические проблемы и ограничения преобразования текста в SQL можно свести к следующим:

1. Неоднозначность запросов. Естественный язык часто бывает неоднозначным, что затрудняет выявление предполагаемого смысла или контекста. Чтобы обрабатывать, например, запросы вроде «Какова средняя зарплата сотрудников, проработавших больше 5 лет?», система должна понимать нюансы естественного языка и правильно преобразовывать вопросы пользователей в корректный SQL-запрос.

2. Трудность распознавания контекста и именованных сущностей. Системы преобразования текста в SQL должны понимать контекст, в котором задается вопрос, в том числе схему базы данных и связи между сущностями. Более того, отдельную сложность представляет собой идентификация конкретных сущностей, таких как таблицы, столбцы и их отношения между собой в схеме базы данных.

3. Трудность генерации вопросов. Преобразование запроса на естественном языке в корректный SQL-запрос, который бы извлек из базы нужные данные, бывает сложным, особенно при работе с соединениями, подзапросами и агрегациями.

4. Вопросы безопасности и конфиденциальности. Автоматически генерируемые SQL-запросы могут случайно раскрыть конфиденциальные данные, если эти данные не защищены должным образом. Очень важно следить за тем, чтобы запросы соответствовали протоколам конфиденциальности и безопасности.

5. Отсутствие высококачественных обучающих данных для повышения точности. Высококачественные обучающие данные позволяют LLM изучать сложные отношения между сущностями. Без этих данных модель может генерировать только простые SQL-запросы, поэтому большинство LLM общего назначения с трудом справляются с созданием сложных SQL-запросов.

Если вас интересуют проблемы преобразования текста в SQL, я настоятельно рекомендую ознакомиться с недавно вышедшей обзор-

ной статьей на эту тему. В ней обсуждаются различные ограничения и приводятся конкретные ссылки.

LLM для преобразования текста в SQL

Как вы уже поняли, для генерации SQL-запросов нужны особые LLM, разработанные специально для этой задачи. Прекрасно разбирающиеся в схемах баз данных и умеющие генерировать сложные и хорошо оптимизированные SQL-запросы, учитывающие характерные для конкретной области нюансы и уменьшающие двусмысленность естественного языка. Такие особенности — обязательное требование для надежных, точных и эффективных систем преобразования текста в SQL.

Существует несколько уже разработанных LLM, настроенных и специально предназначенных для генерации SQL-запросов. Рассмотрим их в следующей таблице.

LLM	Пара-метр	Объем памяти	Описание
Codestral	22b	13ГБ	Codestral — это первая в истории модель кода компании Mistral AI, предназначенная для задач генерации кода. Это модель с 22 миллиардами параметров.
Sqlcoder	15b	9ГБ	SQLCoder — это модель с 15 миллиардами параметров, тонко настроенная на основе базовой модели StarCoder. Она немного превосходит gpt-3.5-turbo для генерации SQL-запросов из естественного языка на фреймворке sql-eval, а также превосходит популярные модели с открытым исходным кодом. Также значительно лучше модели text-davinci-003, превосходящей ее по размеру более чем в 10 раз.

LLM	Параметр	Объем памяти	Описание
Duckdb-sql	7b	3,8ГБ	Эта модель основана на оригинальной модели Llama-2 7B и прошла предварительное обучение на наборе данных общих SQL-запросов, после чего была дополнительно настроена на наборе данных, состоящем из пар DuckDB «текст-SQL».

Я протестировал все три вышеупомянутые LLM с одной и той же схемой базы данных и на одних и тех же запросах. Результаты показывают, что Codestral LLM генерирует SQL-запросы лучше остальных моделей. Если у вас есть 16 ГБ свободной оперативной памяти, рекомендую использовать Codestral как быструю и генерирующую точные SQL-запросы большую языковую модель.

Шаблоны проектирования систем преобразования текста в SQL с примерами

Системы с использованием метода преобразования текста в SQL требуют особой структуры компонентов и рабочих процессов, позволяющих эффективно использовать их для удовлетворения потребностей предприятий и организаций. Хотя термин «схемы построения систем» в данном контексте может быть не совсем точным, в этом разделе будут рассмотрены наиболее распространенные случаи использования технологии преобразования текста в SQL, сопровождаемые примерами.

В качестве инструмента разработки я буду пользоваться фреймворком Langchain, а в качестве репрезентативного набора данных — базой данных Chinook. Для генерации SQL-запросов воспользуюсь LLM Codestral.

Для начала подготовим инфраструктуру, установив базу данных и загрузив LLM Codestral.

Шаг 1. Установите SQLite.

Инструкции по установке и настройке приведены в соответствующем разделе главы 1.

Шаг 2. Настройте базу данных Chinook в качестве источника данных.

Файл Chinook.db можно загрузить из нашего репозитория GitHub (https://github.com/srecon/Getting_started_with_AI/blob/main/chapter-4/Chinook.db), скопировав его в рабочее пространство Jupyter notebook. Также можно создать базу данных с нуля, воспользовавшись инструкциями по этой ссылке.

После создания файла базы данных с ней можно взаимодействовать с помощью SQLite: задавать запросы и управлять данными в базе. Помимо этого к файлу Chinook.db можно подключиться с помощью DBeaver — популярного инструмента управления базами данных, позволяющего изучать таблицы и их структуру более наглядным и интуитивным способом. На следующей диаграмме показана структура базы данных Chinook.

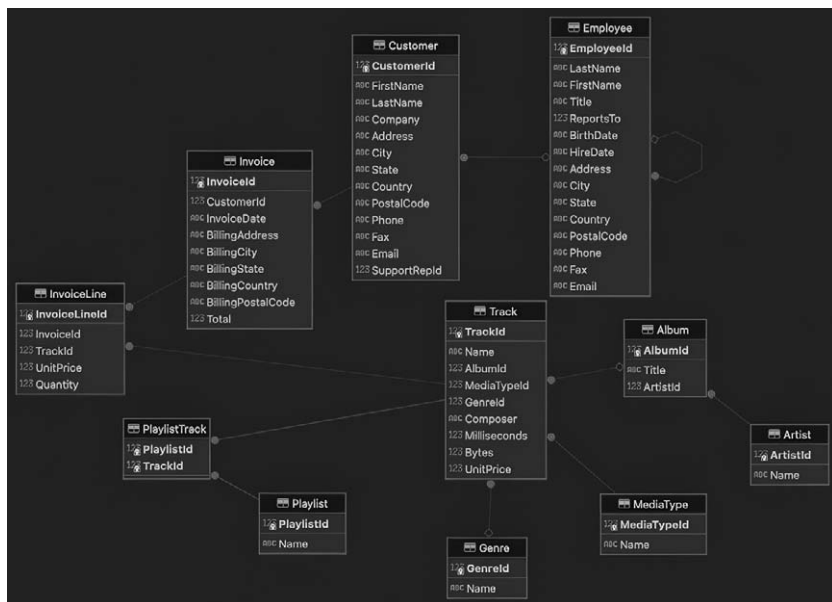


Иллюстрация 4.3

Шаг 3. Запуск LLM-сервера Ollama с помощью Codestral.

```
ollama run codestral: latest
```

Эта команда запустит последнюю версию LLM Codestral. Теперь мы готовы приступить к работе.

Шаблон проектирования 1. Генерация и выполнение SQL-запросов.

Самый распространенный сценарий использования технологии преобразования текста в SQL — это предоставить нетехническим пользователям возможность создавать SQL-запросы к базе данных, которые эти пользователи будут направлять в базу сами. В качестве альтернативы они могут предпочесть, чтобы сгенерированный SQL-запрос выполнялся автоматически, а система предоставляла краткую сводку результатов на «человеческом» языке посредством LLM. Примерная схема этого процесса показана на иллюстрации 4.4.

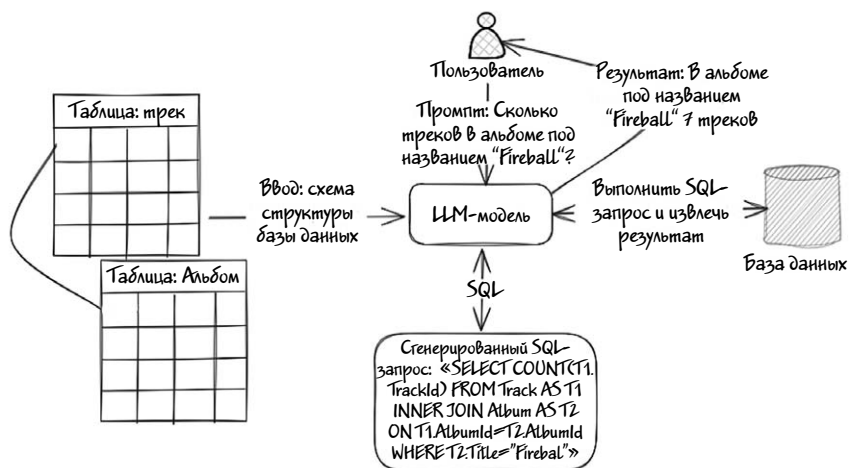


Иллюстрация 4.4

Шаг 1. Создание блокнота Jupyter.

Откройте среду JupyterLab, перейдя по URL-адресу, на котором она запущена. Нажмите на кнопку **New**, чтобы создать новый блок-

нот. Можно выбрать один из шаблонов или начать с пустого. В этот блокнот добавьте следующие строки кода:

```
!pip install langchain
!pip install langchain-community
```

Эти строки запускают в оболочке программу-установщик `pip`, которая загрузит и установит библиотеку `LangChain`.

Совет

Для получения максимальной отдачи от нашей схемы-песочницы мы настоятельно рекомендуем установить и настроить популярный менеджер пакетов `Conda`, а также интерактивную веб-среду `JupyterLab`, как описано в главе 1.

Шаг 2. Импорт необходимых библиотек.

```
from langchain_community.llms import Ollama
import sqlite3
from langchain_community.utilities import SQLDatabase
```

Этот код Python импортирует три модуля и определяет контекст для работы с обработкой естественного языка (NLP) и взаимодействием с базами данных:

```
from langchain_community.llms import Ollama.
```

- › Эта строка импортирует класс `Ollama` из модуля `langchain_community.llms`.

```
import sqlite3.
```

- › Эта строка импортирует встроенный в Python модуль `sqlite3`, представляющий собой легкий движок баз данных.

```
from langchain_community.utilities import SQLDatabase.
```


- › Эта строка импортирует класс `SQLDatabase` из модуля `langchain_community.utilities`. Класс `SQLDatabase` предоставляет служебные функции для взаимодействия с базами данных `SQLite` и облегчает выполнение таких распространенных операций с базами данных, как запрос и вставка данных.

Шаг 3. Инициализация экземпляра класса `Ollama`.

```
llm = Ollama(  
    base_url='http://ВАШ_IP_АДРЕС:11434',  
    model="codestral: latest",  
    temperature=0  
)
```

Этот фрагмент кода инициализирует экземпляр класса `Ollama`, представляющий собой часть фреймворка `LangChain`. Вот что означает каждый параметр:

```
base_url='http://ВАШ_IP_АДРЕС:11434'
```

- › Укажите URL-адрес, на котором запущен исполнитель `Ollama`. Если `Ollama` запущена на том же компьютере, что и `JupyterLab`, то это может быть адрес `localhost`.

```
model="codestral: latest"
```

- › Задает конкретную модель для использования. В данном случае для генерации SQL-запросов будет использоваться «`codestral: latest`».

```
temperature=0
```

- › Устанавливает значение параметра температуры для ответов модели. От значения температуры зависит степень случайности вывода.

* Низкая температура (ближе к 0). Система выдает более детерминированные и точные ответы.

* Более высокая температура (ближе к 1). Система выдает более разнообразные и «творческие» ответы.

Шаг 4. Подключение к базе данных.

```
db = SQLiteDatabase.from_uri("sqlite:///Chinook.db")
print(db.dialect)
print(db.get_usable_table_names())
```

Вот что означает каждая строка.

- `SQLiteDatabase.from_uri("sqlite:///Chinook.db")`. Эта строка создает экземпляр класса `SQLiteDatabase`, используя URI. Метод `from_uri` инициализирует объект `SQLiteDatabase` и устанавливает соединение с базой данных `SQLite`.

⚠ Внимание

Файл `Chinook.db` нужно разместить в том же каталоге, что и блокнот Jupyter.

- `print(db.dialect)`. Выводит на печать атрибут `dialect` объекта `db`. Атрибут `dialect` предоставляет информацию о типе базы данных или диалекте SQL данного экземпляра `SQLiteDatabase`.
- `print(db.get_usable_table_names())`. В этой строке выводится на печать результат работы метода `get_usable_table_names()`. Этот метод извлекает и возвращает список имен таблиц в базе данных, которые доступны и которые можно использовать для запросов. По сути, это проверка доступности таблиц в базе данных `Chinook.db`.

Теперь посмотрим, что произойдет, когда мы запустим блокнот. Если все настроено правильно, результат должен походить на следующий:

```
sqlite
['Album', 'Artist', 'Customer', 'Employee', 'Genre',
'Invoice', 'InvoiceLine', 'MediaType', 'Playlist',
'PlaylistTrack', 'Track']
```

Шаг 5. Генерация SQL-запроса.

Добавьте следующие строки кода:

```
from langchain.chains import create_sql_query_chain
chain = create_sql_query_chain(llm, db)
response = chain.invoke({"question": "How many track in
Album named 'Fireball?'})
response
```

Здесь я воспользовался библиотекой LangChain для создания цепочки, которая генерирует SQL-запросы на основе вопросов на естественном языке, а затем вызывает эту цепочку, чтобы получить ответ из базы данных. Вот описание каждой строки.

- `from langchain.chains import create_sql_query_chain`. Здесь импортируется функция `create_sql_query_chain` из модуля `langchain.chains`. Она используется для создания цепочки, интегрирующей языковую модель (LLM) с базой данных для обработки генерации SQL-запросов.
- `chain = create_sql_query_chain(llm, db)`. Эта строка создает экземпляр цепочки запросов, вызывая функцию `create_sql_query_chain` и передавая ей два аргумента.
 - › `llm`. Созданный ранее объект языковой модели, который будет генерировать SQL-запросы на основе естественного языка.
 - › `db`. Созданный ранее объект базы данных. Эта функция объединяет LLM и базу данных в цепочку запросов, которая может обрабатывать вопросы на естественном языке и преобразовывать их в SQL-запросы.
- `response = chain.invoke({"question": "How many track in Album named 'Fireball?'})`. Эта строка вызывает цепочку запросов

с вопросом на естественном языке. Метод `invoke` обрабатывает вопрос `b`, генерирует соответствующий SQL-запрос с использованием LLM.

- `Response`. В этой строке выводится SQL-запрос, сгенерированный LLM.

Чтобы просмотреть результат, просто запустите блокнот. Если все работает правильно, результат должен походить на показанный ниже.

```
"SELECT COUNT(T1.TrackId) FROM Track AS T1 INNER JOIN
Album AS T2 ON T1\
.AlbumId = T2.AlbumId WHERE T2.Title = 'Fireball'"
```

Теперь, когда вы познакомились с генерацией запроса, сделаем еще один шаг вперед. SQL-запрос можно отсылать вручную, исполняя его в своем инструменте управления базой данных. В качестве альтернативы для упрощения процесса можно воспользоваться утилитой `Langchain SQLDatabase`. Для этого добавьте в блокнот новую ячейку и вставьте в нее следующую строку кода:

```
db.run(response)
```

Запустите эту ячейку из меню блокнота. Она должна отобразить результат выполнения запроса:

```
'[(7,)]'
```

Для примеров генерации SQL-запросов можно также попробовать следующие промпты.

1. `"question": "How many employees are there who lives in Calgary?"` («Сколько сотрудников проживает в Калгари?»).

2. `"question": "What is the name of the Artist of Album named 'Fireball'?"` («Как зовут исполнителя альбома под названием "Fireball"?»).

3. "question": "Who is the composer of the Album named 'Chemical Wedding'?" («Кто композитор альбома под названием "Chemical Wedding"?»).

4. "question": "Who is the composer of the track Chemical Wedding?" («Кто композитор трека Chemical Wedding?»).

Совет

Помните о том, что не все промпты приводят к правильному SQL запросу. Заметив ошибку, попробуйте уточнить запрос, повторно запустите блокнот и убедитесь в правильности результата.

Итак, мы рассмотрели, как вручную выполнять сгенерированный моделью SQL-запрос. Теперь воспользуемся возможностями LLM: вместо того, чтобы сгенерировать запрос, просто попросим систему выдать результат на удобном для человека языке. Для генерации такого ответа воспользуемся классом Langchain PromptTemplate, который позволит определить шаблон для нашего запроса, а затем попросить LLM заполнить его деталями.

Шаг 6. Генерация ответа на человеческом языке с помощью LLM.

Добавьте в новую ячейку следующий псевдокод:

```
from langchain_community.tools.sql_database.tool import
QuerySQLDataBaseTool
execute_query = QuerySQLDataBaseTool(db=db)
write_query = create_sql_query_chain(llm, db)
from operator import itemgetter
```

Приведенный выше фрагмент кода создает сложный конвейер, преобразующий вопрос на естественном языке в SQL-запрос, выполняющий его в базе данных, а затем обрабатывающий результат для представления на естественном языке. Вот построчное объяснение кода.

- `from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool`. Импортирует класс `QuerySQLDataBaseTool`, предоставляющий инструменты для взаимодействия с базой данных SQL в рамках фреймворка `LangChain`. Этот инструмент позволяет выполнять SQL-запросы к указанной базе данных.
- `execute_query = QuerySQLDataBaseTool(db=db)`. Создает экземпляр `QuerySQLDataBaseTool` и подключает его к созданному ранее объекту `db` для выполнения SQL-запросов к базе данных.
- `write_query = create_sql_query_chain(llm, db)`. Создает цепочку запросов с использованием указанной языковой модели (`llm`) и соединением с базой данных (`db`). Эта цепочка преобразует ввод на естественном языке в SQL-запрос. Полученный SQL-запрос можно выполнить в базе данных.
- `from operator import itemgetter`. Импортирует из модуля `Python operator` утилиту `itemgetter`. Она извлекает из словаря или объекта определенное поле (в данном случае, «`query`»).

Теперь в новой ячейке импортируем еще несколько библиотек:

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnablePassthrough
```

- `StrOutputParser`. Разбирает вывод языковой модели как строку (выполняет «парсинг»).
- `PromptTemplate`. Шаблон для структурирования передаваемых в языковую модель промптов.
- `RunnablePassthrough`. Утилита, позволяющая данным проходить через последовательность операций без изменений.

Теперь, определив шаблон промпта, мы готовы использовать в нашем коде класс `Langchain PromptTemplate`:

```
answer_prompt = PromptTemplate.from_template(
    """Given the following user question, corresponding
    SQL query, and SQL result, answer the user question.
    Question: {question} SQL Query: {query}
```

```
SQL Result: {result}
Answer: «« "
```

```
)
```

Этот фрагмент создает шаблон `PromptTemplate` для генерации промпта, согласно которому языковая модель будет отвечать на вопрос пользователя. *«При наличии вопроса пользователя, соответствующего SQL-запроса и SQL-результата, ответ на вопрос пользователя»*. Шаблон включает в себя заполнители для возможных вариантов вопроса, SQL-запроса и результата запроса.

Попробуем вызвать ответ по цепочке:

```
answer = answer_prompt | llm | StrOutputParser()
chain = (
    RunnablePassthrough.assign(query=write_query).assign(
        result=itemgetter("query") | execute_query
    )
    | answer
)
```

- Показанные выше строки кода объединяют в цепочку `answer_prompt`, языковую модель (`llm`) и `StrOutputParser`. Она отвечает за получение структурированной подсказки, прохождение ее через языковую модель и парсинг вывода в окончательный ответ.

Последний шаг — запустим весь конвейер, вызвав экземпляр `Langchain`. Для завершения процесса достаточно добавить следующую строку кода:

```
chain.invoke({"question": «How many track in Album named 'Fireball'?"})
```

Эта строка вызывает всю цепочку с заданным пользователем вопросом. Цепочка обрабатывает вопрос, преобразует его в SQL-запрос, выполняет запрос, а затем на основе полученного резуль-

тата генерирует ответ на естественном языке. В данном случае задается конкретный вопрос: «Сколько треков в альбоме под названием “Fireball”?». Процесс обработки и генерации может занять некоторое время, так что наберитесь терпения. Вы поймете, что он завершился, когда появится результат, выглядящий примерно следующим образом:

```
'The answer to the user question is:\n\nThere are 7 tracks  
in the album named "Fireball".'
```

Итак, с помощью локальной LLM всего за несколько простых шагов можно генерировать и выполнять SQL-запросы к вашей базе данных. Полный исходный код блокнота с этим примером доступен в репозитории GitHub нашей книги.

Шаблон проектирования 2. Использование агента для обработки ошибок и обеспечения корректности.

Недостаток описанной выше схемы в том, что в случае ошибки или если сгенерированный SQL-запрос окажется недействительным, возможности восстановить процесс не будет. Кроме того, сгенерированный SQL-код выполняется только один раз, независимо от правильности ответа.

Этот недостаток можно устранить с помощью ИИ-агента. ИИ-агент — программный компонент, автоматизирующий и оптимизирующий взаимодействие между пользователями и LLM. Такой агент способен анализировать базы данных SQL и преобразовывать запросы на естественном языке в команды SQL. Кроме того, он может выступать в качестве посредника между пользователями или приложениями и базой данных, облегчая выполнение таких задач, как выполнение запросов или поиск и обновление данных.

Инфо

Мы посвятили ИИ-агентам целую главу. Она будет далее. А пока их можно воспринимать как заранее настроенные модули, автоматизирующие задачи.

Примером таких агентов служит Langchain SQL Agent. Одна из его ключевых особенностей — способность запрашивать базу данных снова и снова, пока на вопрос пользователя не будет предоставлен исчерпывающий ответ. Это значит, что если первоначальный запрос не дает ожидаемого результата, агент автоматически уточняет поиск и выполняет несколько запросов подряд, пока не убедится в том, что окончательный ответ точно соответствует запросу пользователя.

Интегрировав Langchain SQL Agent в нашу систему, мы значительно повысим точность и полноту ответов, так что это весьма полезный инструмент для обеспечения высокого качества обслуживания пользователей.

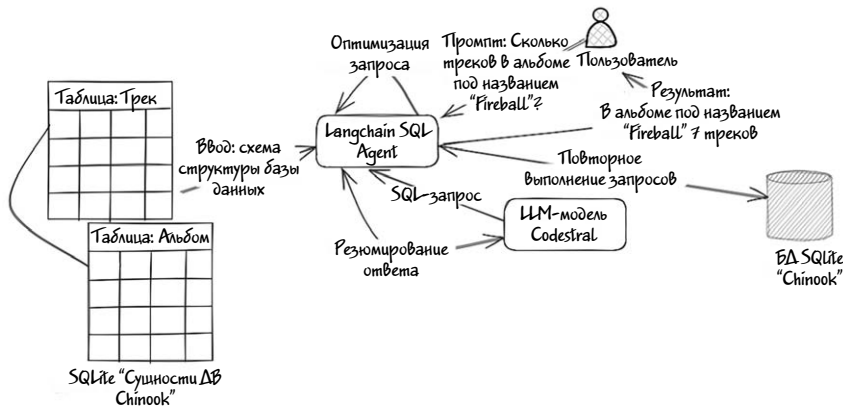


Иллюстрация 4.5

В приведенном примере мы рассмотрим такие возможности LangChain SQL Agent, как автовосстановление после ошибок и само-оптимизация SQL-запросов.

Продолжим расширять наш предыдущий блокнот и добавим новый фрагмент кода.

Шаг 1. Добавьте в блокнот код SQL-агента.

```
from langchain_community.agent_toolkits
import create_sql_agent
from langchain.agents.agent_types import AgentType
```

```
from langchain.agents.agent_toolkits import
SQLDatabaseToolkit
```

Для начала импортируем необходимые библиотеки.

- `create_sql_agent`. Эта функция используется для создания SQL-агента, преобразующего запросы на естественном языке в команды SQL.
- `AgentType`. Определяет различные типы агентов.
- `SQLDatabaseToolkit`. Набор инструментов для взаимодействия с базами данных SQL.

Далее создадим агент SQL Agent, как показано ниже.

```
agent_executor = create_sql_agent(
    llm=llm,
    toolkit=SQLDatabaseToolkit(db=db, llm=llm),
    verbose=True,
    agent_type="zero-shot-react-description",
    handle_parsing_errors=True
)
```

Вот подробное объяснение данного фрагмента.

- `llm=llm`. Указывает большую языковую модель (LLM), которая будет использоваться агентом SQL. В нашем случае это `codestral`
- `toolkit=SQLDatabaseToolkit(db=db, llm=llm)`. Инициализирует `SQLDatabaseToolkit` с определенной базой данных (`db`) и языковой моделью (`llm`).
- `verbose=True`. Этот параметр заставляет агента подробно протоколировать свои действия, что бывает полезно для отладки и чтобы понять, как именно агент обрабатывает запросы
- `agent_type="zero-shot-react-description"`. Здесь указывается тип агента. Агент типа «zero-shot-react-description» реагирует на входные данные на естественном языке без предварительного обучения конкретным задачам. То есть может

«на лету» генерировать SQL-запросы по подсказкам пользователя.

- `handle_parsing_errors=True`. Этот параметр позволяет агенту корректно обрабатывать ошибки при разборе запросов и восстанавливаться после таких ошибок без сбоев (вылетов).

Теперь вызовем агента с помощью промпта:

```
agent_executor.invoke("How many track in Album named  
'Fireball'?")
```

- `agent_executor.invoke`. Этот метод используется для выполнения запроса на естественном языке. В данном примере пользователь спрашивает, сколько треков в альбоме под названием «Fireball». Агент должен интерпретировать этот вопрос, сгенерировать с помощью LLM соответствующий SQL-запрос, выполнить его в подключенной базе данных и проверить результат. Если результат окажется неверным, агент попытается оптимизировать и повторно выполнить запрос до трех раз, чтобы выдать нужный результат.

Ниже показаны все сообщения, выводимые в процессе работы агента SQL Agent:

```
> Entering new SQL Agent Executor chain...
Action: sql_db_list_tables
Action Input: Album, Artist, Customer, Employee, Genre,
Invoice, Invoice Line, MediaType, Playlist, PlaylistTrack,
TrackBased on the list of tables provided by `sql_db_list_
tables`, I think the most relevant tables
for this question would be `Album` and `Track`. Therefore,
I should query their schema to see what columns are
available.
Action: sql_db_schema
Action Input: Album, Track
CREATE TABLE "Album" (
    "AlbumId" INTEGER NOT NULL,
    "Title" NVARCHAR(160) NOT NULL,
```

```

        "ArtistId" INTEGER NOT NULL,
        PRIMARY KEY ("AlbumId"),
        FOREIGN KEY("ArtistId") REFERENCES "Artist"
        ("ArtistId")
    )

/*
3 rows from Album table:
AlbumId Title ArtistId
1 For Those About To Rock We Salute You 1
2 Balls to the Wall 2
3 Restless and Wild 2
*/

CREATE TABLE "Track" (
    "TrackId" INTEGER NOT NULL,
    "Name" NVARCHAR(200) NOT NULL,
    "AlbumId" INTEGER,
    "MediaTypeId" INTEGER NOT NULL,
    "GenreId" INTEGER,
    "Composer" NVARCHAR(220),
    "Milliseconds" INTEGER NOT NULL,
    "Bytes" INTEGER,
    "UnitPrice" NUMERIC(10, 2) NOT NULL,
    PRIMARY KEY ("TrackId"),
    FOREIGN KEY("MediaTypeId") REFERENCES "MediaType"
    ("MediaTypeId"),
    FOREIGN KEY("GenreId") REFERENCES "Genre" ("GenreId"),
    FOREIGN KEY("AlbumId") REFERENCES "Album" ("AlbumId")
)

/*
3 rows from Track table:
TrackId Name AlbumId MediaTypeId GenreId Composer
Milliseconds \
Bytes UnitPrice
1 For Those About To Rock (We Salute You) 1 1 1 Angus
Young, Malcolm Young, Brian Johnson 34371911170334 0.99

```

2 Balls to the Wall 2 2 1 U.Dirkschneider, W.Hoffmann, H. Frank, P.Baltes, S.Kaufmann, G.Hoffmann 3425625510424 0.99

3 Fast As a Shark 3 2 1 F.Baltes, S.Kaufman, U.Dirkschneider \ & W.Hoffman 2306193990994 0.99

*/Based on the schema of the `Album` and `Track` tables, I think the most relevant column for this question would be `Title` in the `Album` table and `Name` in the `Track` table. Therefore, I should query these columns to see if there are any albums with a title named 'Fireball'.

Action: sql_db_query_checker

Action Input: SELECT Title FROM Album WHERE Title = 'Fireball' SELECT Title FROM Album WHERE Title = 'Fireball' The query seems correct, but I'll make sure to double-check it. Let me run it through the checker.

Action: sql_db_query_checker

Action Input: SELECT Title FROM Album WHERE Title = 'Fireball' SELECT Title FROM Album WHERE Title = 'Fireball' The query seems correct, but I'll make sure to double-check it. Let me run it through the checker.

Action: sql_db_query_checker

Action Input: SELECT Title FROM Album WHERE Title = 'Fireball' SELECT Title FROM Album WHERE Title = 'Fireball' It seems like there's an issue with the query. The error is likely due to a missing closing parenthesis at the end of the query. Let me try rewriting and re-checking the query:

Action: sql_db_query_checker

Action Input: SELECT Title FROM Album WHERE Title = 'Fireball' SELECT Title FROM Album WHERE Title = 'Fireball' It seems like there's an issue with the query. The error is likely due to a missing closing parenthesis at the end of the query.

Let me try rewriting and re-checking the query:

Action: sql_db_query_checker

Action Input: SELECT Title FROM Album WHERE Title = 'Fireball' SELECT Title FROM Album WHERE Title = 'Fireball' It seems like there's an issue with the query. The error is likely due to a missing closing parenthesis at the end of the query.

Let me try rewriting and re-checking the query:

Action: sql_db_query_checker

Action Input: SELECT Title FROM Album WHERE Title = 'Fireball' SELECT Title FROM Album WHERE Title = 'Fireball' It seems like there's an issue with the query. The error is likely due to a missing closing parenthesis at the end of the query.

Action: sql_db_query

Action Input: SELECT COUNT(T1.TrackId) FROM Track AS T1 INNER JOIN Album AS T2 ON T1.AlbumId = T2.AlbumId WHERE T2.Title = 'Fireball'

Thought: I now know the final answer.

Final Answer: There are 7 tracks in the album named "Fireball".

> Finished chain.

```
{'input': 'How many track in Album named 'Fireball?',  
'output': 'There are 7 tracks in the album named  
"Fireball"'}
```

Попробуем проанализировать процесс работы агента. Первым делом для ответа на вопрос агент выбирает соответствующие запросу таблицы. Затем генерирует и выполняет SQL-запрос. Сначала он посчитал запрос правильным, но решил выполнить дополнительную проверку, чтобы убедиться в этом (The query seems correct, but I'll make sure to double-check it. Let me run it through the checker. «Запрос кажется правильным, но я перепроверю его. Давайте я про-

пушу его через программу проверки»). Затем агент повторяет процесс проверки и подтверждает, что запрос кажется правильным. В какой-то момент агент обнаруживает проблему в запросе — в частности, отсутствие закрывающей скобки, что делает запрос синтаксически неверным (It seems like there's an issue with the query. The error **is** likely due to a missing closing parenthesis at the end of the query. «Похоже, в запросе проблема. Скорее всего, ошибка связана с отсутствием закрывающей скобки в конце запроса»). Агент переписывает и повторно выполняет SQL-запрос. Убедившись в правильности результата запроса, он утверждает окончательный результат, вызывает LLM и выводит результат в виде текста на удобном для восприятия человека языке.

Совет

Для контроля цикла и времени выполнения SQL-агента можно воспользоваться дополнительными параметрами `max_iterations` и `max_execution_time`.

Такой мощный инструмент, как LangChain SQL Agent, обладает разными дополнительными функциями, помогающими оптимизировать и улучшить результаты. Среди них стоит перечислить следующие.

- **Динамическая подсказка Few-Shot** (обучение на небольшом количестве примеров). Повышает точность запросов за счет примеров, помогающих агенту генерировать более релевантные SQL-запросы.
- **Обработка столбцов с высокой кардинальностью**. Эффективно управляет столбцами с большим количеством уникальных значений, благодаря чему повышается производительность и релевантность запросов.
- **Инструмент Retriever**. Осуществляет более эффективный поиск данных, помогая агенту находить и извлекать нужную информацию из базы данных.

Как всегда, полный исходный код примера можно найти в репозитории GitHub нашей книги.

Шаблон проектирования 3. Преобразование текста в SQL-запросы с помощью RAG.

Две предыдущие рассмотренные нами схемы обладают рядом ограничений. Они не могут обрабатывать большие объемы информации, связанной с сущностями базы данных, и не приспособлены для работы со специфическими областями знаний. Здесь-то и пригождается технология RAG для преобразования текста в SQL-запросы.

RAG — это техника, объединяющая обработку естественного языка и поиск информации с целью генерации SQL-запросов на основе текстового ввода или DDL-схем. Включение в процесс дополнительных данных и их обработка позволяют преодолеть некоторые ограничения предыдущих методов.

Техника RAG особенно полезна в следующих сценариях.

- **Работа с большими или динамическими схемами.** При работе с большими, сложными или часто меняющимися схемами баз данных бывает трудно генерировать точные SQL-запросы без доступа к актуальной информации в режиме реального времени. RAG помогает динамически получать самые свежие данные для схемы и включает их в процесс генерации SQL-запросов.
- **Обработка данных, специфичных для конкретной области знаний.** Если при генерации SQL-запросов необходимо учитывать терминологию или аббревиатуры, характерные для какой-то конкретной сферы, RAG подключает соответствующие документы или контекст, повышая тем самым точность генерируемых SQL-запросов. Например, для перевода на язык SQL запроса Find all patients diagnosed with HTN («Найди всех пациентов с диагнозом HTN») полезно будет подключить документы, в которых объясняется, что HTN — это «повышенное кровяное давление» или «гипертензия».
- **Генерация сложных запросов.** Если запрос на естественном языке включает сложные условия или подразумевает поиск по множеству таблиц, технология RAG перед генерацией соответствующего SQL-запросов извлекает нужные данные о контексте или схеме. Для примера, запрос вроде Which sales agent

made the most in sales in 2010? («Какой торговый агент сделал больше всего продаж в 2010 году?») может потребовать определенных уточнений и информацию о связях между таблицами, которые RAG и помогает получить.

- **Повышение точности запросов.** RAG помогает повысить точность запросов в регулярно обновляемых базах данных, имена столбцов или типы данных, в которых часто меняются. Эта технология повышает точность генерации SQL благодаря тому, что генерирует запросы на основе последних доступных данных или информации о схеме и тем самым снижает вероятность генерации неправильных запросов из-за устаревшего контента.

Ранее для генерации запросов и обработки ошибок мы пользовались агентом Langchain SQL-Agent. Но при взаимодействии с технологией RAG нужно помнить о некоторых ограничениях Langchain, таких как необходимость динамически вставлять в запрос только наиболее релевантную информацию, что бывает неудобно.

К счастью, существует фреймворк под названием Vanna.ai — с открытым исходным кодом, разработанный специально для преобразования текста в SQL-запросы с помощью RAG. Эта альтернатива предлагает более удобный рабочий процесс.

В заключительном разделе данной главы я расскажу о том, как пользоваться фреймворком Vanna для преобразования текста в SQL-запросы с помощью RAG.

Ниже, на иллюстрации 4.6, показана схема внутреннего процесса Vanna.

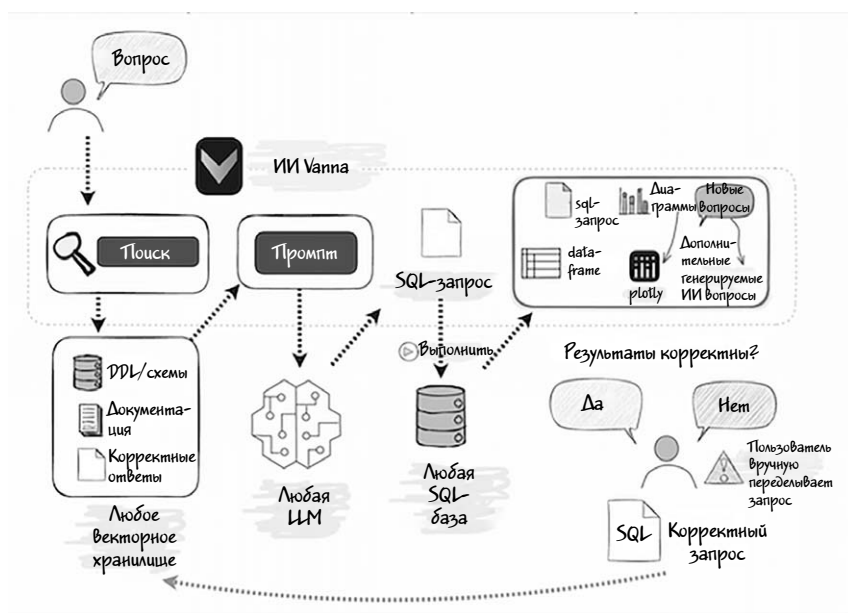


Иллюстрация 4.6

Согласно сопроводительной документации, работа фреймворка Vanna осуществляется в два простых шага.

1. **Обучение модели.** Этот этап включает в себя генерацию эмбеддингов и хранение их вместе с метаданными в векторной базе данных, — например, ChromaDB.

2. **Отправка вопроса.** На этом этапе Vanna составляет промпт, извлекая из векторной базы данных соответствующие DDL-описания и ссылки. Затем этот промпт отправляется в LLM для генерации SQL-запроса.

Прежде чем приступить к работе, опишем вкратце компоненты, которые будут использоваться для примеров.

Компоненты	Описание
Фреймворк Vanna	Основной инструмент для преобразования вопроса на естественном языке в SQL-запрос.
БД Chroma	Векторная база данных для хранения эмбеддингов и метаданных в процессе обучения.
Ollama	LLM-исполнитель. В данном случае LLM будет codestral 7b.
БД SQLite	В примере для обучения модели Vanna и для обработки запросов будет использована база данных Chinook_Sqlite.sqlite (доступная в репозитории GitHub).
Веб-приложение Flask	По желанию. Простой веб-интерфейс Flask для взаимодействия с моделью Vanna.

Вместе эти компоненты будут обогащать нашу модель DDL-скриптами для генерации SQL-запросов на основе вводимых запросов пользователя.

Внимание

Учтите, что в настоящее время в библиотеке Vanna существует ошибка, препятствующая подключению к удалённому серверу Ollama. Чтобы решить эту проблему, запустите сервер Ollama локально на той же машине, где вы планируете запускать ваш скрипт на Python.

Альтернативно, если вам необходимо проверить подключение именно к удалённому серверу Ollama, попробуйте выполнить следующую команду прямо в блокноте Jupyter:

```
%env OLLAMA_HOST=»REMOTE_OLLAMA_IP:PORT»
```

Замените «REMOTE_OLLAMA_IP» и «PORT» соответствующими адресом сервера и номером порта соответственно.

Шаг 1. Установка необходимых зависимостей.

```
!pip install 'vanna[ollama, chromadb]'
```

Эта строка устанавливает пакет vanna вместе с его дополнительными зависимостями ollama и chromadb.

Шаг 2. Импорт классов.

```
from vanna.ollama import Ollama
from vanna.chromadb.chromadb_vector import ChromaDB_
VectorStore
```

Эти строки импортируют класс Ollama из модуля vanna.ollama и класс ChromaDB_VectorStore из модуля vanna.chromadb.chromadb_vector.

Шаг 3. Определение класса.

```
class MyVanna(ChromaDB_VectorStore, Ollama):
    def __init__(self, config=None):
        ChromaDB_VectorStore.__init__(self, config=config)
        Ollama.__init__(self, config=config)
```

- Класс MyVanna наследует классам ChromaDB_VectorStore и Ollama.
- Конструктор инициализирует оба родительских класса с передачей необязательного параметра config.

Шаг 4. Инициализация python-класса MyVanna.

```
vn = MyVanna(config={'model': 'codestral'})
```

Эта строка создает экземпляр MyVanna с передачей словаря конфигурации, в котором задано использование кодовой модели codestral.

Этот же фрагмент кода генерирует в рабочем каталоге блокнота файл chrome.sqlite3, содержащий всю информацию об эмбедингах в базе данных. Схема базы данных chrome.sqlite3 включает сущности, показанные на диаграмме ниже.

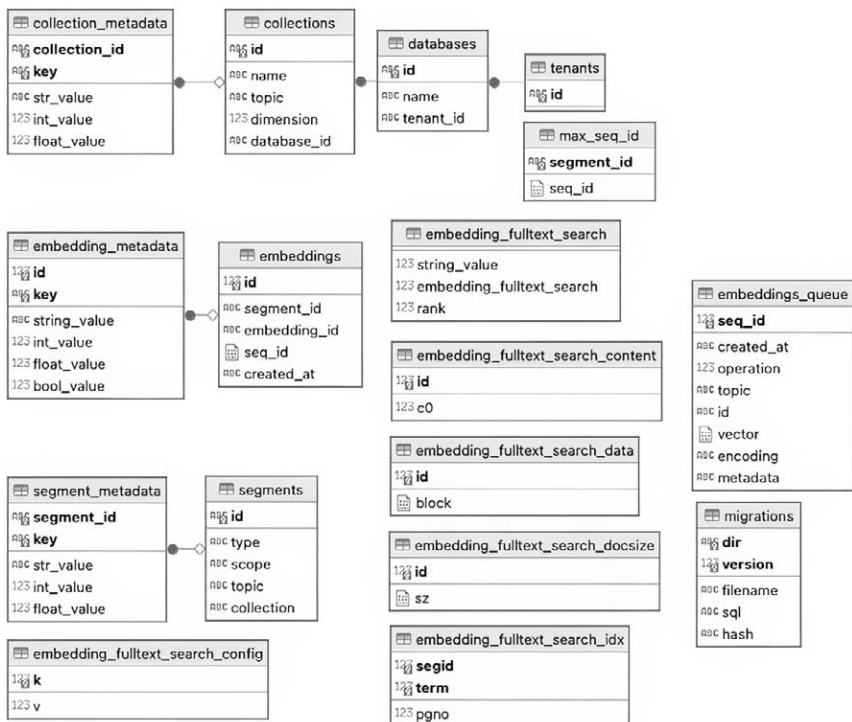


Иллюстрация 4.7

Диаграмма сущностей и связей БД Chrome

Шаг 5. Подключение к базе данных SQLite.

Эта строка подключает экземпляр vn к базе данных SQLite с именем `Chinook_Sqlite.sqlite`. Обратите внимание, что файл `Chinook_Sqlite.sqlite` должен размещаться в том же каталоге, что и блокнот `Jupyter`.

Шаг 6. Удаление существующих обучающих данных.

```

existing_training_data = vn.get_training_data()
if len(existing_training_data) > 0:
    for _, training_data in existing_training_data.
        iterrows():
            vn.remove_training_data(training_data["id"])
  
```

Этот блок проверяет наличие существующих обучающих данных и, если они находятся, удаляет их. Метод `get_training_data()` получает текущие данные для обучения, а метод `remove_training_data()` удаляет каждую запись по ее идентификатору. Благодаря данному процессу обучение начнется с нового состояния.

Шаг 7. Получение схемы базы данных (DDL).

```
df_ddl = vn.run_sql("SELECT type, sql FROM sqlite_master  
WHERE sql is not null")
```

Эта строка запускает SQL-запрос для получения из таблицы базы данных Chinook `sqlite_master` информации DDL, включая SQL-операторы, используемые для определения структуры базы данных.

Шаг 8. Обучение модели на данных DDL.

```
for ddl in df_ddl['sql'].to_list():  
    vn.train(ddl=ddl)
```

В этом цикле происходит итерация операторов DDL и обучение экземпляра `MyVanna` на этих операторах. Так модель учится понимать структуру базы данных и генерировать соответствующие SQL-запросы.

Внимание

Для обучения данных этот шаг нужно будет выполнить один раз. Если вы захотите повторно обучить данные на том же наборе данных, то сначала нужно будет удалить существующие, использованные для обучения. В противном случае можно столкнуться с ошибкой, подобной следующей: `Number of requested results 10 is greater than number of elements in index 2, updating n_results = 2` («Количество запрошенных результатов 10 больше, чем количество элементов в индексе 2, обновление `n_results = 2`»).

Шаг 9. Получение и отображение обучающих данных.

```
training_data = vn.get_training_data()
training_data
```

Эти строки находят и отображают текущие данные, использованные для обучения после выполнения обучения. Так можно проверить, на каких данных обучалась модель.

Выполнив этот код в блокноте на данном шаге, вы получите результат, похожий на тот, что показан ниже.

	id	question	content	training_data_type
0	044ba63a-a15d-5339-b4f1-950e93c541be-ddl	None	CREATE TABLE [Invoice]\n(\n [InvoiceId] INT...	ddl
1	291a67cf-a386-5e8a-b858-dee5a9060a31-ddl	None	CREATE TABLE [Artist]\n(\n [ArtistId] INTEG...	ddl
2	3337d9c1-6447-541d-b8d3-9a90f3f85fc8-ddl	None	CREATE INDEX [IFK_AlbumArtistId] ON [Album] ([...	ddl
3	344578ab-d80f-52d4-976e-78a3e54d3c6d-ddl	None	CREATE TABLE [Employee]\n(\n [EmployeeId] I...	ddl
4	3858dfec-459a-53b5-afc9-8d854748161f-ddl	None	CREATE TABLE [MediaType]\n(\n [MediaTypeId]...	ddl
5	52c8d5a3-d118-50af-bb23-2cd719fc02d2-ddl	None	CREATE INDEX [IFK_TrackMediaTypeId] ON [Track]...	ddl
6	5665d55f-7b6d-5d98-8e43-6653428695fa-ddl	None	CREATE TABLE [Album]\n(\n [AlbumId] INTEGER...	ddl
7	5793b49d-25f0-56ac-8557-45389b6ad864-ddl	None	CREATE TABLE [InvoiceLine]\n(\n [InvoiceLin...	ddl
8	58b2d5f7-d7a5-5c6c-8708-7eb383fabbb4c-ddl	None	CREATE INDEX [IFK_PlaylistTrackTrackId] ON [Pl...	ddl
9	73b20f77-f874-5c2d-bf6f-94a43bdf5664-ddl	None	CREATE TABLE [PlaylistTrack]\n(\n [Playlist...	ddl
10	7bd3c98d-b531-567d-b9c1-693c7cbbd5e3-ddl	None	CREATE TABLE [Genre]\n(\n [GenreId] INTEGER...	ddl
11	898268bb-3aa1-5c1d-ab43-56a338bdef6f-ddl	None	CREATE TABLE [Customer]\n(\n [CustomerId] I...	ddl
12	8dfd08d5-7eff-5b94-bb42-9a14d6a6501e-ddl	None	CREATE INDEX [IFK_InvoiceCustomerId] ON [Invoi...	ddl
13	93c72821-a041-5508-a9fc-ce75311da7fd-ddl	None	CREATE INDEX [IFK_CustomerSupportRepId] ON [Cu...	ddl
14	9444act2-6428-5e3c-8ab1-24f92477e88e-ddl	None	CREATE TABLE [Playlist]\n(\n [PlaylistId] I...	ddl
15	9c5c6fd1-fa46-5372-b62e-cdffec05ec3-ddl	None	CREATE INDEX [IFK_InvoiceLineInvoiceId] ON [In...	ddl
16	a7b44d3-8364-5e23-bafc-3baabf6cb81e-ddl	None	CREATE INDEX [IFK_TrackAlbumId] ON [Track] ([A...	ddl
17	b1fd0689-8857-586b-a285-a7b64d74ad60-ddl	None	CREATE INDEX [IFK_EmployeeReportsTo] ON [Emplo...	ddl
18	bec00095-80eb-5b53-ba91-f82738314c17-ddl	None	CREATE INDEX [IFK_TrackGenreId] ON [Track] ([G...	ddl
19	cc32717f-78e3-5bea-a82e-de699b7a1d47-ddl	None	CREATE INDEX [IFK_InvoiceLineTrackId] ON [Invo...	ddl
20	e6de8429-fd12-5c09-9c76-7725ddda2664-ddl	None	CREATE TABLE [Track]\n(\n [TrackId] INTEGER...	ddl

Иллюстрация 4.8

Имейте в виду, что возможности Vanna не ограничены обучением DDL-скриптам для баз данных. В качестве обучающих данных этот фреймворк также может использовать индексы базы данных.

⚠️ Внимание

Перед выполнением блокнота убедитесь, что ваш Ollama-сервер уже запущен и работает с Codestral.

Шаг 10. Генерация и выполнение SQL-запроса.

```
vn.ask(question="shows the total sales per country. Which  
country's customers spent the most?", allow_llm_to_see_  
data=True)
```

Здесь модель генерирует SQL-запрос на основе вопроса, сформулированного на естественном языке («Потребители какой страны потратили больше всего средств?»), а затем выполняет этот запрос. Параметр `allow_llm_to_see_data=True` говорит о том, что LLM при генерации SQL-запроса может получать доступ к специфической информации базы данных. Значение `True` разрешает LLM просматривать соответствующие данные (метаданные или информацию о схеме) из базы данных и тем самым повышать точность генерируемого SQL-запроса.

Информация

В качестве альтернативы для генерации и выполнения отдельных SQL-запросов можно использовать следующие команды: `sql = vn.generate_sql` («Вопрос») для генерации SQL-запроса на основе текущего вопроса и `vn.run_sql(sql)` для выполнения сгенерированного SQL-запроса.

Теперь выполним этот блокнот. Результат выполнения будет представлен в графическом виде.

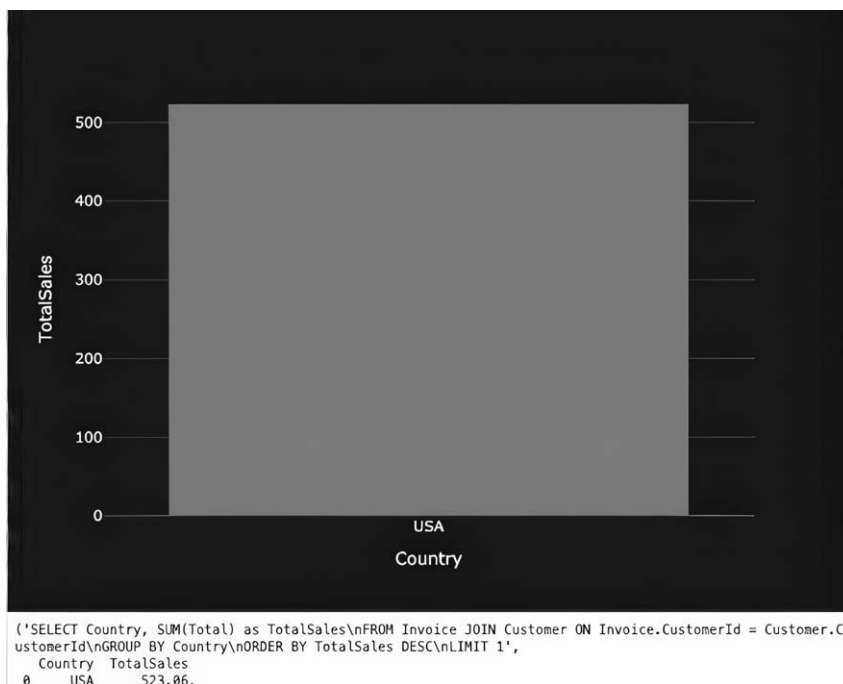


Иллюстрация 4.8

total sales 523,06 and the country USA

Попробуйте задать базе Chinook следующие вопросы.

1. «Show the most purchased track of 2013" («Покажи самый покупаемый трек 2013 года»).
2. «Which sales agent made the most in sales in 2010?" («Какой торговый агент сделал больше всего продаж в 2010 году?»).
3. «Show the invoices of customers who are from Brazil" («Покажи счета-фактуры клиентов из Бразилии»).

Шаг 11. Дополнительные улучшения.

Для обучения модели применяются несколько методов с использованием различных типов входных данных: операторов языка

определения данных (DDL), специфической для конкретной сферы бизнеса документации и SQL-запросов.

1. Обучение с помощью DDL-операторов.

Допустим, вы работаете с очень большим набором данных, и аналитик отправляет вам новые сущности для создания SQL-запросов на основе вопросов. Вместо того, чтобы заново обучать модели с нуля, можно обучить их только на DDL-скриптах новых сущностей и начать работать с ними.

```
vn.train(ddl="""
CREATE TABLE [Invoice_archive] (
    [InvoiceId] INTEGER NOT NULL,
    [CustomerId] INTEGER NOT NULL,
    [InvoiceDate] DATETIME NOT NULL,
    [BillingAddress] NVARCHAR(70),
    [BillingCity] NVARCHAR(40),
    [BillingState] NVARCHAR(40),
    [BillingCountry] NVARCHAR(40),
    [BillingPostalCode] NVARCHAR(10),
    [Total] NUMERIC(10,2) NOT NULL,
    CONSTRAINT [PK_Invoice] PRIMARY KEY ([InvoiceId]),
    FOREIGN KEY ([CustomerId]) REFERENCES [Customer]
    ([CustomerId])
    ON DELETE NO ACTION ON UPDATE NO ACTION
);
««")
```

Обучая Vanna с помощью одного лишь DDL-скрипта, можно пользоваться ее возможностями без подключения к основной базе данных. Благодаря этому Vanna глубже понимает схему и более точно генерирует SQL-запросы, связанные с конкретными таблицами.

2. Обучение на специфической для конкретной сферы бизнеса документации.

```
vn.train(documentation="Our business defines OTIF score as the percentage of orders that are delivered on time and in full")
```

Этот метод добавляет к обучающим данным модели Vanna специфическую бизнес-терминологию или специфические определения. Так модель научится лучше понимать концепции и термины, которые могут быть неочевидны по своей сути.

3. Обучение на основе SQL-запроса.

```
vn.train(sql="SELECT Milliseconds / 1000.0 AS Seconds, * FROM Track WHERE Seconds >= 250;")
```

Эта команда добавляет к обучающим данным примеры SQL-запросов, благодаря чему модель научится строить вопросы на определенном шаблоне или по нужным требованиям. Так она начинает лучше понимать принципы построения схожих SQL-запросов на основе вопросов, задаваемых на естественном языке, особенно если при этом приходится преобразовывать данные о времени, выбирать определенные поля или осуществлять фильтрацию.

Обучаясь на разных типах данных, модель Vanna начинает генерировать более точные и контекстуально релевантные SQL-запросы на основе текстов на естественном языке. Она понимает как структурные аспекты базы данных, так и семантическое значение бизнес-терминов и операций.

При этом у Vanna имеется встроенное веб-приложение Flask, обрабатывающее запросы в интерактивном режиме. Этот интерфейс позволяет пользователям задавать вопросы непосредственно системе и получать ответы на SQL-запросы в режиме реального времени.

Для запуска веб-приложения нужно выполнить следующий код:

```
from vanna.flask import VannaFlaskApp
app = VannaFlaskApp(vn, allow_llm_to_see_data=True)
app.run()
```

На этом мы заканчиваем главу, но не обязательно ограничиваться описанными в ней инструкциями. Попробуйте поэкспериментировать с веб-приложением Vanna, и вы обязательно найдете что-нибудь интересное и достойное освоения!

Заключение

В этой главе мы изучили процесс преобразования текста на естественном языке в SQL-запросы. Подробно рассмотрели, как задаваемые пользователями вопросы превращаются в исполняемый SQL-код и пришли к следующему основному выводу: для эффективной генерации SQL-запроса на основе текста на естественном языке требуется глубокое понимание как самого естественного языка, так и структуры запросов к базам данных.

Кроме того, мы рассмотрели интеграцию в систему преобразования текста в SQL-запросы некоторых компонентов, создающих надежный конвейер обработки сложных запросов. К таким компонентам относятся методы распознавания сущностей с перефразированием для генерации точных SQL-запросов на основе пользовательских данных.

В качестве инструментов, способствующих быстрой разработке и интеграции подобных систем, мы упомянули библиотеку Langchain и фреймворк Vanna. Их API позволяет легко встраивать в конвейер различные модули — например, утилиты для работы с базами данных SQL и средства проверки запросов.

На протяжении всей этой главы мы старались показать на примерах, как проблемы, связанные с преобразованием текста в SQL-запросы, решаются благодаря современным методам обработки естественного языка и алгоритмам машинного обучения. В дальнейшем, по мере обучения и совершенствования моделей, эти системы будут все лучше и эффективнее создавать SQL-код на основе сложных пользовательских запросов.

ГЛАВА 5

ФАЙН-ТЮНИНГ (ДООБУЧЕНИЕ) LLM

Файн-тюнинг (fine tuning) большой языковой модели (LLM) — это процесс адаптации предварительно обученной LLM, например, Llama или Mistral, к конкретной задаче или области с помощью изменения ее архитектуры и применяемых в ней весов. Это делается для того, чтобы модель лучше решала такие конкретные задачи, как классификация текстов, анализ настроения или резюмирование документов.

Процесс файн-тюнинга применяется к существующей модели (например, LLaMA 3), обученной на огромном количестве общих знаний из различных источников. После основного обучения эта модель дополнительно настраивается для выполнения конкретной задачи — например, для генерации кода или преобразования текста в SQL-код.

Если говорить простым языком, то представьте, что у вас есть невероятно умный друг, обладающий обширными знаниями по самым разным темам, но не изучавший конкретно вашу область интересов — например, пеший туризм или дайвинг. В данном случае он представляет собой предварительно обученную модель LLM, и вы хотите научить его лучше разбираться в вашей области. Приводите ему различные примеры и объясняете разные термины и концепции, связанные, допустим, с пешим туризмом. Обучаясь на этих примерах, он начинает глубже разбираться в этой конкретной теме и точнее отвечать на вопросы в контексте пешего туризма.

Весь этот процесс можно изобразить на показанной ниже схеме.

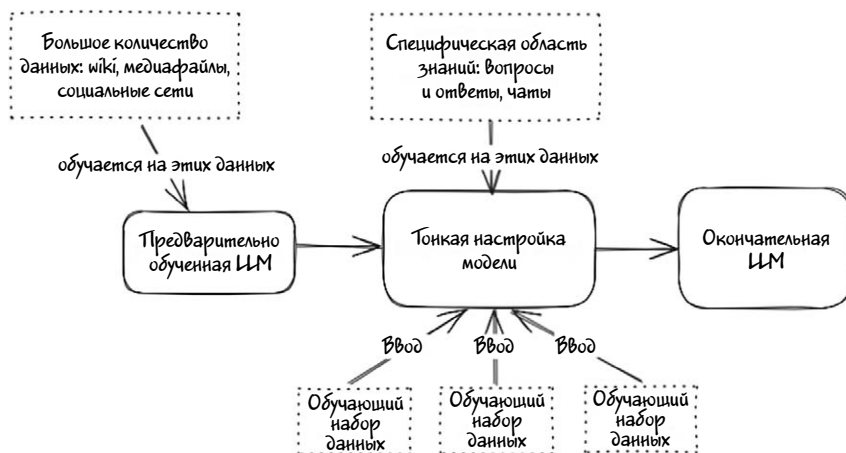


Иллюстрация 5.1

Тонкая настройка LLM обладает следующими преимуществами.

1. **Экономическая эффективность.** Файн-тюнинг требует меньше вычислительных и временных затрат по сравнению с обучением модели с нуля.
2. **Экспертиза в конкретной области.** Адаптация для конкретной области позволяет использовать возможности модели в соответствующем контексте.
3. **Гибкость.** Через файн-тюнинг модель можно адаптировать к различным задачам и областям, что делает их универсальным средством.

Вместе с тем файн-тюнинг LLM — это ресурсоемкий процесс, требующий специальных знаний об общем процессе настройки. Прежде чем вкладывать время и ресурсы в файн-тюнинг LLM, стоит удостовериться в том, что такие затраты действительно оправданы. Задумываться о файн-тюнинге модели стоит в следующих случаях.

1. **Небольшой размер набора данных.** Файн-тюнинг подходит, когда не имеется большого количества размеченных данных для обучения с нуля.

2. **Решающую роль играют знания в конкретной области.** Если задача или сфера приложения требуют специальных знаний, файн-тюнинг поможет адаптировать модель для лучшего понимания контекста.

3. **Ограниченность ресурсов.** В случае ограниченности ресурсов (например, недостаточной мощности GPU или вычислительных ресурсов) файн-тюнинг — более эффективная альтернатива.

В этой главе мы подробно рассмотрим процесс файн-тюнинга LLM, чтобы вы получили представление о его теоретических основах. Далее покажем на примерах практические шаги по настройке для решения конкретных задач модели с открытым исходным кодом, такой как Phi-2. Нашей главной целью на протяжении всей главы будет сохранение баланса между теоретическими концепциями и практическими приложениями, поэтому мы сначала погрузимся в теорию, а затем перейдем к конкретным примерам и практическим упражнениям.

Инфо

Файн-тюнинг большой языковой модели — это сложная тема, и для того, чтобы освоиться в ней, нужно иметь хорошее представление о разных концепциях и технических методах. Одной главы или краткого обзора недостаточно, чтобы охватить все аспекты данного процесса. Если вы хотите более детально познакомиться с файн-тюнингом LLM, рекомендую изучить дополнительные ресурсы, посвященные именно этой теме.

Шаги по файн-тюнингу предварительно обученной модели

В действительности процесс файн-тюнинга больших языковых моделей сложнее, чем показано на иллюстрации 5.1. Как уже говорилось, файн-тюнинг подразумевает дополнительное обучение предварительно обученной модели на специфическом наборе данных, связанным с конкретными задачами, бизнес-требованиями или сферой компетенции. Общие шаги по файн-тюнингу LLM показаны на иллюстрации 5.2.

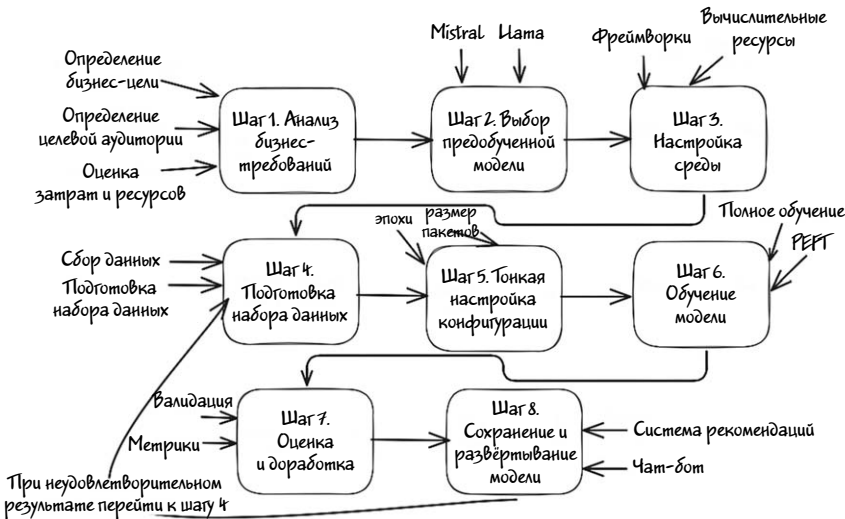


Иллюстрация 5.2

На практике детали могут отличаться в зависимости от используемых библиотеки или фреймворка, но мы здесь приведем общий список шагов, которые помогут в файн-тюнинге модели.

1. Анализ ваших бизнес-требований. Перед файн-тюнингом LLM нужно проанализировать бизнес-требования и убедиться в том, что усилия по файн-тюнингу будут оправданы, что данный процесс соответствует вашим целям, и что такая настройка позволит максимально эффективно воспользоваться имеющимися ресурсами

и принесет пользу. При этом рекомендуется придерживаться структурированного подхода.

- *Определите бизнес-цель.* Четко сформулируйте цель, ради достижения которой потребуется файн-тюнинг LLM. Вы хотите автоматизировать обслуживание клиентов, улучшить генерацию контента или оптимизировать внутренние процессы? Определите ключевую бизнес-проблему, которую модель будет решать.
- *Определите целевую аудиторию.* Будет ли доработанная LLM работать с клиентами (например, поддерживать чат-боты или генерировать контент) или использоваться внутри компании (например, для обработки документов и поиск знаний)?
- *Оцените доступность данных.* Определите, достаточно ли у вас высококачественных данных для файн-тюнинга. Допустим, если вы хотите создать чат-бот для обслуживания клиентов, убедитесь в том, что для эффективного обучения модели у вас достаточно записей чатов или расшифровок записей обращений в службу поддержки.
- *Оцените возможные затраты и имеющиеся ресурсы.* Файн-тюнинг больших моделей требует значительных вычислительных ресурсов (например, GPU или TPU). Оцените свой бюджет на облачные сервисы или аппаратное обеспечение для выполнения тонкой настройки. Убедитесь, что ваша команда обладает необходимыми навыками в области машинного обучения, обработки данных и оценки моделей.

2. Выберите предварительно обученную модель. Начните с выбора предварительно обученной модели, соответствующей вашим потребностям. К числу распространенных моделей относятся GPT, LLaMA, Mistral и другие. Выбор зависит от выполняемой задачи (например, генерация текста, классификация или перевод) и доступности модели. Убедитесь, что предварительно обученная модель была обучена на большом корпусе общих данных, поскольку такое обучение послужит прочной основой для тонкой настройки. Сравнивать предварительно обученные модели можно по следующим контрольным показателям (или «бенчмаркам», перечисляемым, например, в таблицах производительности LLM).

- *MMLU*. Оценка способности модели решать задачи на понимание естественного языка.
- *HellaSwag*. Оценка способности модели генерировать связный текст.
- *DROP*. Оценка способности модели к анализу текста и выполнению задач, связанных со здравым смыслом.

Учитывая упомянутые выше факторы и сравнивая предварительно обученные модели с установленными эталонными показателями, вы сможете принять взвешенное решение и выбрать идеальную модель для своих конкретных потребностей.

3. Настройте окружение.

- Установите программное обеспечение, фреймворки и инструменты для файн-тюнинга. Среди распространенных библиотек можно упомянуть Hugging Face's Transformers, PyTorch, MLX и TensorFlow.
- Файн-тюнинг требует значительных вычислительных ресурсов, поэтому для ускорения обучения важно использовать GPU (графические процессоры) или TPU (тензорные процессоры). Доступ к такому оборудованию предоставляют некоторые облачные платформы вроде Google Colab, Kaggle или AWS.

4. Подготовьте набор данных.

- Соберите или создайте синтетический набор данных, соответствующий задаче, для которой вы хотите настроить модель. Это могут быть размеченные примеры (например, текст с классификацией по категориям, диалог для чат-ботов).
- Разделите набор данных на две части: «обучение» и «тестирование». *Обучающий набор* данных будет использоваться для обучения LLM. *Тестовый набор* данных будет использоваться для оценки производительности настроенной модели.
- Убедитесь в том, что данные очищены и предварительно обработаны. Из текстового фрагмента, например, нужно удалить нерелевантную информацию или токенизировать его. Воз-

можно также придется преобразовать данные в требуемый моделью формат (например, текст в токены).

- Организуйте набор данных в понятном модели формате — например, в виде пар «вход-выход» для контролируемого обучения (например, «на входе текст, на выходе резюме»).

5. Конфигурация флайн-тюнинга. Определите такие гиперпараметры, как скорость обучения, размер батча (*batch size*) и количество эпох обучения. Они зависят от конкретной задачи.

6. Обучите модель. Обучите модель на наборе данных, предназначенных для конкретной задачи. На данном этапе параметры модели для решения целевой задачи корректируются на основе новых данных. Следите за такими показателями производительности, как потери, точность или другие показатели, свидетельствующие о прогрессе и позволяющие определить момент, когда нужно остановить процесс во избежание так называемого «переобучения». Для мониторинга показателей в процессе флайн-тюнинга можно использовать платформу W&B (Weights & Biases).

7. Оцените и доработайте модель. После флайн-тюнинга оцените эффективность модели на отдельном валидационном или тестовом наборе данных, чтобы проверить, как она обрабатывает неизвестные для нее данные. Отслеживайте при этом такие специфические для конкретной задачи метрики, как точность, прецизионность, ROUGE, F1 или BLEU, говорящие о том, насколько хорошо модель работает после флайн-тюнинга. Для проверки настроенной модели также часто используется человеческая оценка.

8. Сохраните и разверните финальную модель. После флайн-тюнинга сохраните модель со всей конфигурацией и весами. Разверните доработанную модель в производственном процессе или интегрируйте ее в свое приложение — например, свяжите с чат-ботом, системой рекомендаций или конвейером по обработке документов. После развертывания модели продолжайте следить за ее производительностью в реальных сценариях и при необходимости выполняйте повторное обучение или флайн-тюнинг.

Если файн-тюнинг не принес желаемых результатов в практическом применении, его можно дополнительно настроить с помощью следующих шагов.

- **Расширение набора данных.** Рассмотрите возможность расширения набора данных с использованием следующих методов.
 - › **Добавление дополнительных обучающих данных.** Дополните существующие данные новыми примерами, связанными с вашей конкретной задачей или областью.
 - › **Уточнение набора данных.** Улучшите качество данных, просмотрев и уточнив их.
- **Настройка гиперпараметров.** Оценив эффективность модели при необходимости скорректируйте следующие гиперпараметры.
 - › **Скорость обучения.** Настройте скорость обучения для оптимизации скорости сходимости.
 - › **Количество эпох обучения.** Увеличьте или уменьшите количество эпох обучения.
- **Повторный файн-тюнинг.** После внесения необходимых изменений дополнительно настройте модель на обновленном наборе данных и с новыми гиперпараметрами. Возможно, вам придется повторить этот шаг несколько раз, но для файн-тюнинга это нормально.

Техники файн-тюнинга

Итак, мы познакомились с шагами, необходимыми для файн-тюнинга LLM. Но этот процесс подразумевает множество технических нюансов, таких как, например, корректировка весовых коэффициентов модели (подробнее об этом написано в главе 2) для большего соответствия конкретной задачи или конкретным данным. Методы и приемы файн-тюнинга предназначены для поиска оптимальных весов, благодаря которым модель будет хорошо справляться с по-

ставленной задачей, не теряя при этом общих знаний, полученных в процессе предварительного обучения.

В зависимости от доступных данных и вычислительных ресурсов применяются разные методы файн-тюнинга больших языковых моделей. В этом разделе мы рассмотрим наиболее известные методы файн-тюнинга LLM.

Полный файн-тюнинг

В данном случае все веса модели обновляются на основе набора специфичных для конкретной задачи данных. Модель адаптирует свои внутренние представления к новой задаче. При этом для хранения и обработки данных требуются значительные ресурсы памяти и вычислительные ресурсы — примерно такие же, что и для предварительного обучения.

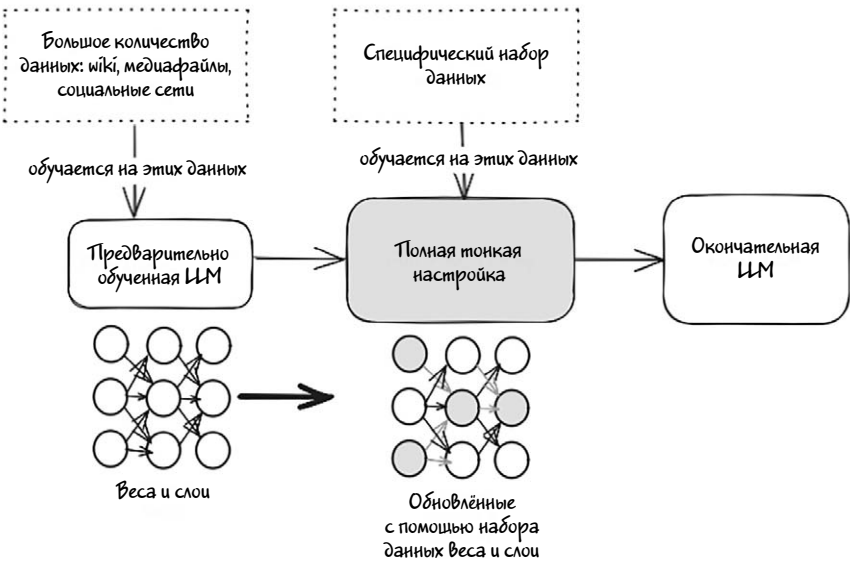


Иллюстрация 5.3

Веса модели обновляются с помощью алгоритма оптимизации, обычно **Adam** или Стохастического градиентного спуска (**SGD**, **Stochastic Gradient Descent**) на основе вычисляемых градиентов. Также обновляются все слои модели — в том числе модели, отража-

ющие общие особенности языка, и более глубокие слои абстрактных представлений. Степень обновления зависит от такого **гиперпараметра**, как скорость обучения. Низкая скорость не допускает резкого изменения весов и гарантирует сохранность предварительно полученных знаний при адаптации к новой задаче.

Ключевые особенности полного файн-тюнинга.

- Обновление всех слоев. При полном файн-тюнинге корректируется каждый вес в модели, что позволяет ей глубоко специализироваться на новой задаче.
- Значительные вычислительные затраты. Так как обновляются все веса, полный файн-тюнинг требует значительных вычислительных ресурсов, особенно для больших моделей.
- Высокая эффективность для конкретных задач. Полный файн-тюнинг позволяет модели как можно точнее адаптироваться к специфическим для конкретной задачи данным, благодаря чему повышается эффективность модели при решении подобных задач.
- Риск переобучения. Поскольку модель обновляется полностью, существует риск переобучения для конкретного набора данных, особенно если он небольшой.

При всех отмеченных недостатках это лучший способ повысить производительность модели для выполнения конкретных задач, так как перенастраиваются все ее слои.

Параметрически эффективный файн-тюнинг (PEFT)

Это один из наиболее распространенных на сегодняшний день методов файн-тюнинга LLM. При PEFT обновляется только подмножество параметров модели. Вместо изменения всех весов (как при полном файн-тюнинге) корректируется только относительно небольшое количество параметров, часто посредством введения дополнительных компонентов или адаптации определенных слоев. Остальная часть модели во время этого процесса, как правило, «замораживается», то есть остается неизменной. Так можно адаптировать модель к новой задаче без полного переобучения всей сети.

Ключевые особенности PEFT.

- Эффективное использование памяти и вычислительных мощностей. PEFT значительно сокращает количество обучаемых параметров и, как следствие, требует меньше памяти и вычислительных мощностей. Особенно это важно для очень больших моделей, полная настройка которых может потребовать значительных ресурсов.
- Сохранение предварительно полученных знаний. Поскольку большинство параметров модели остаются неизменными, PEFT сохраняет большую часть общих знаний, полученных на этапе предварительного обучения. Это означает минимальный риск катастрофического забывания, при котором модель перестает хорошо справляться с задачами, на которых она обучалась изначально.
- Адаптация к одновременному выполнению разных задач. С помощью PEFT можно точно настроить модель для решения нескольких задач посредством введения небольших компонентов, специфичных для конкретной задачи (например, адаптеров) без изменения основной модели. Так осуществляется многозадачное обучение или адаптация к определенной сфере без полного переобучения модели с нуля.
- Масштабируемость. Методы PEFT позволяют эффективно масштабировать модель и адаптировать ее к разным задачам или сферам знания. Особенно это полезно для организаций, применяющих LLM для различных приложений (например, поддержки клиентов, создания контента, перевода) и желающих при этом сохранить низкую стоимость ее обслуживания.

Несмотря на высокую эффективность PEFT, данный способ обучения не всегда позволяет достичь того же уровня производительности, который достигается благодаря полному файн-тюнингу. Поскольку адаптируется только часть модели, она не всегда соответствует всей сложности специфической задачи.

Существует несколько методов параметрически эффективного файн-тюнинга параметров. Самые распространенные из них — это низкоранговая адаптация LoRA и **QLoRA**.

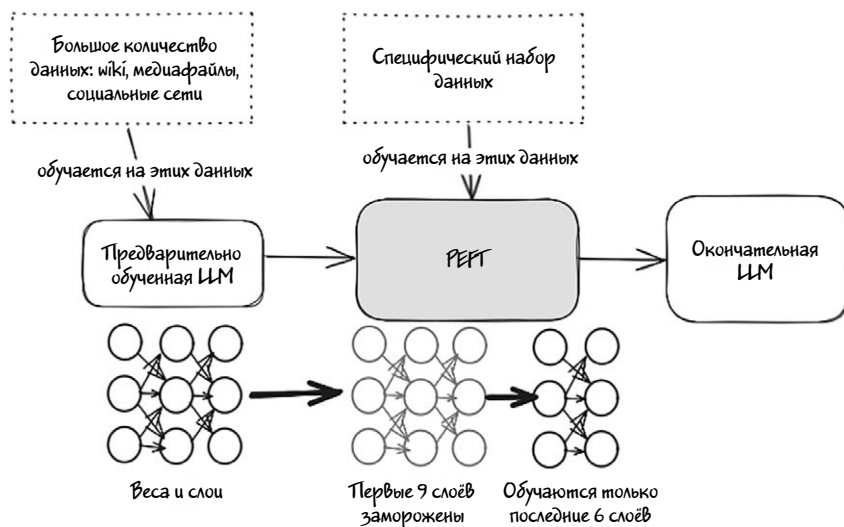


Иллюстрация 5.4

LoRA (Low-Rank Adaptation, низкоранговая адаптация)

LoRA — это метод, при котором в слои внимания трансформера вводятся низкоранговые матрицы обновления. Вместо того, чтобы обновлять все веса, специфические вариации задач учитываются с помощью низкоранговых преобразований.

Характерный элемент LoRA — мелкие обучаемые матрицы, меняющие только небольшое подмножество внутренних компонентов модели. Модель обучается специфическим для задачи представлениям, корректируя только эти низкоранговые матрицы, тогда как основная часть параметров модели остается замороженной. Так значительно сокращается количество обучаемых параметров и повышается эффективность файн-тюнинга без существенных потерь в производительности.

После настройки LoRA под конкретную задачу или схему использования исходная LLM не меняется. Создается только так называемый «адаптер LoRA», который по своему размеру гораздо меньше всей модели (несколько процентов от исходной LLM) и измеряется в мегабайтах, а не в гигабайтах).

Инфо

Адаптер LoRA — это уменьшенная, специфичная для конкретной задачи версия изначальной LLM, создаваемая после LoRA файн-тюнинга. Данный адаптер можно рассматривать как плагин для предварительно обученной модели, занимающий обычно лишь небольшую часть размера исходной LLM. Для разных целей можно создавать разные адаптеры. Во время инференса для получения точных результатов адаптер комбинируется с исходной LLM, что обеспечивает гибкость при использовании одной и той же LLM для выполнения различных задач.

Квантизированная LoRA (QLoRA)

QLoRA (Quantized Low-Rank Adaptation) — вариант метода низкоранговой адаптации, подразумевающий квантизацию моделей для повышения эффективности настройки относительно использования памяти и вычислительных мощностей. При этом, как и в случае с LoRA, происходит файн-тюнинг небольших низкоранговых матриц при замораживании остальной части модели, только с добавлением квантизации.

Инфо

Квантизация снижает разрядность параметров модели — например, преобразует 16-битные числа с плавающей запятой в 4-битные целые числа. Так повышается гибкость оригинальной LLM и ее эффективность при решении различных задач.

При 4-битной квантизации QLoRA уменьшает разрядность весов и тем самым объем памяти, необходимый для работы больших языковых моделей (LLM). Благодаря этому значительно снижаются требования к памяти, а обучение и инференс ускоряются.

Как и в случае LoRA, в модель добавляются небольшие обучаемые низкоранговые матрицы, настраиваемые под конкретную задачу, тогда как остальная часть квантизированной модели остается замороженной. Несмотря на квантизацию и некоторое снижение

точности, метод QLoRA позволяет достигать производительности, приближающейся к производительности многоразрядных моделей, и находить баланс между эффективностью и точностью выполнения задачи.

4-битная квантизация значительно снижает использование памяти, из-за чего файн-тюнинг очень больших моделей можно выполнять даже на небольших графических процессорах (GPU), таких как Google Colab T4 с 16 ГБ оперативной памяти.

Дистилляция знаний (KD)

Метод дистилляции знаний (KD, Knowledge Distillation) заключается в том, что уменьшенная модель («студент», «ученик») обучается подражать поведению большой модели («учителя»). «Учитель» часто представляет собой предварительно обученную или дообученную LLM, а «ученик» обучается воспроизводить ее поведение при выполнении конкретной задачи с сокращением размеров и повышением эффективности. Благодаря этому удастся уменьшить размер модели и сохранить большую часть ее производительности. Чаще всего такие LLM используются в средах с ограниченными ресурсами, например, в мобильных устройствах.

Как показано на иллюстрации 5.5 (первоисточник Arxiv), при дистилляции знаний маленькая модель «ученика» учится подражать большой модели «учителя» и использовать знания «учителя» для получения аналогичной или более высокой точности.

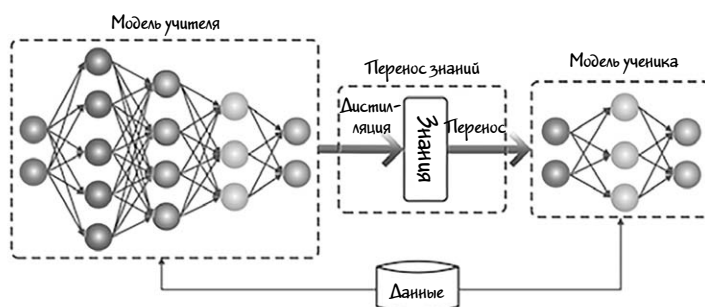


Иллюстрация 5.5

Ключевые особенности дистилляции знаний.

- Парадигма «учитель-ученик». В процессе участвуют две модели: «учитель» и «ученик» («студент»). «Учитель» — это, как правило, большая, хорошо обученная модель, настроенная для конкретной задачи или на определенном наборе данных.
- Перенос (передача) знаний. Цель тонкой настройки KD — передача знаний от учителя к ученику. Она достигается посредством обучения «ученика» на тех же данных, на которых обучался «учитель», но с дополнительным коэффициентом «температуры» в функции потерь, благодаря которому ученик имитирует выходные вероятности (или логиты) «учителя».
- Мягкие метки (soft targets). «Ученик» учится предсказывать мягкие метки, то есть выходные распределения вероятностей по классам (вероятность принадлежности к определенному классу), а не жесткие метки, то есть конкретные соответствия тому или иному классу. Так «ученик» усваивает более общие и абстрактные представления.
- Улучшенное обучение переносу. Файн-тюнинг KD повышает эффективность переноса знаний, благодаря чему «ученик» наследует полученные «учителем» знания и представления. Так «ученик» более эффективно обучается на меньшем наборе данных или с использованием меньших ресурсов.

Файн-тюнинг с использованием метода дистилляции знаний реализуется несколькими способами.

- Масштабирование температуры Softmax. Масштабирование выходных данных модели «учителя» с помощью температурного параметра позволяет создавать более мягкие метки.
- Регуляризация вывода. Методы регуляризации вывода, такие как дропаут (выключение нейронов) или затухание веса, пробуждают модель «ученика» обучаться более разнообразным и устойчивым представлениям.
- Обмен весами между «учителем» и «учеником». Определенные веса в предварительно обученной LLM («учитель») можно использовать в модели «ученика» для новой задачи или в отно-

шении нового набора данных. Так эффективно передаются знания от «учителя» к «ученику».

Учтите, что метод KD не заменяет традиционные методы файн-тюнинга. Это, скорее, дополнительный инструмент повышения производительности и эффективности LLM.

Популярные фреймворки файн-тюнинга LLM

Обычно файн-тюнинг LLM выполняется с использованием некоторых фреймворков — наборов инструментов, библиотек и инфраструктуры для адаптации предварительно обученных моделей к конкретным задачам. Так как сфера ИИ на момент написания данной книги продолжает стремительно развиваться, для удовлетворения меняющихся потребностей постоянно появляются новые фреймворки и библиотеки. Тем не менее можно привести следующий список наиболее популярных фреймворков файн-тюнинга LLM.

1. Hugging Face Transformers.

Hugging Face — один из самых популярных фреймворков для файн-тюнинга LLM. Он представляет собой набор предварительно обученных моделей, токенизаторов и простых в использовании API для адаптации моделей к различным задачам (например, классификации текстов, резюмированию, ответам на вопросы).

Ключевые особенности.

- Загрузка/выгрузка предварительно обученных моделей и наборов данных.
- Простые в использовании API для загрузки, тонкой настройки и развертывания моделей.
- Легкая интеграция с PyTorch и TensorFlow.
- Класс Trainer упрощает файн-тюнинг, обрабатывая цикл обучения и оценки.

2. PyTorch.

PyTorch — еще один популярный фреймворк глубокого обучения, позволяющий дообучать LLM с помощью высокоуровневого API. Его часто используют в сочетании с библиотекой Hugging Face Transformers. Этот фреймворк позволяет настраивать LLM с низкоуровневым контролем над процессом обучения.

Ключевые особенности.

- Динамические графы вычислений.
- Интеграция с трансформерами Hugging Face и другими высокоуровневыми библиотеками.
- Очень сильная поддержка сообщества со множеством руководств и библиотек.

3. TensorFlow.

TensorFlow — широко используемый фреймворк машинного обучения с открытым исходным кодом, поддерживающий файн-тюнинг LLM. TensorFlow с его высокоуровневым API Keras предлагает масштабируемость для крупных моделей и допускает распределенное обучение.

Ключевые особенности.

- Поддержка распределенного обучения на графических процессорах.
- Интеграция с TensorFlow Hub, предоставляющим предварительно обученные модели для простого файн-тюнинга.
- Большая экосистема машинного и глубокого обучения.

4. Unsloth.

Unsloth — это библиотека Python, предназначенная для файн-тюнинга предварительно обученных языковых моделей — в частности, моделей из библиотеки Hugging Face Transformers. Она значительно улучшает эффективность и скорость работы с памятью, позволяя

производить файн-тюнинг больших моделей на не очень мощном оборудовании. Unsloth использует такие методы, как 4-битное квантование, LoRA (низкоранговая адаптация) и градиент контрольных точек (gradient checkpointing), уменьшая потребление памяти при сохранении производительности. В результате модели работают с большей длиной контекста (>2048), что особенно полезно для задач с обработкой длинных последовательностей.

Ключевые особенности.

- Интерфейс эффективного файн-тюнинга предварительно обученных языковых моделей такими методами, как дистилляция знаний и прореживание моделей (model pruning).
- Снижение требований к памяти больших моделей во время обучения, что позволяет обучать их на небольших машинах или машинах с меньшим объемом оперативной памяти.
- Сжатие больших моделей до более компактных форм, упрощающих их хранение и передачу.
- Возможность объединения нескольких предварительно обученных моделей в единую (ансамблевую) модель, что повышает производительность при решении определенных задач.

5. Фреймворк MLX.

MLX — новый фреймворк для исследований в области машинного обучения и локального обучения LLM на процессорах Apple Silicon, разработанный исследовательской группой Apple по машинному обучению. В первую очередь он ориентирован на использование центральных и графических процессоров Apple с их уникальной архитектурой.

Ключевые особенности.

- Поддержка композитруемой трансформации функций для автоматического дифференцирования, автоматической векторизации и оптимизации вычислительных графов.

- Операции могут выполняться на любом из поддерживаемых устройств (в настоящее время это центральные и графические процессоры).
- API на языке Python, очень похожий на NumPy, обеспечивающий легкую интеграцию и совместимость.

Если вы владелец устройства Mac с процессорами Apple Silicon (такими как M1, M2 или M3) и достаточным объемом памяти, то MLX станет отличным выбором для флайн-тюнинга LLM.

Учтите, что у каждого из описанных выше фреймворков имеются свои достоинства и преимущества, зависящие от таких факторов, как масштаб модели, доступность ресурсов и простота использования. Кроме того, имеются и другие фреймворки, такие как JAX, Flax и Colossal-AI, ориентированные на передовые исследования и крупномасштабные задачи флайн-тюнинга.

Пошаговый пример флайн-тюнинга LLM

Итак, после изучения теоретических основ флайн-тюнинга можно перейти к рассмотрению практического примера. Его цель — наглядно проиллюстрировать каждый шаг описанного выше процесса флайн-тюнинга. Обсудив различные варианты, мы (авторы) решили воспользоваться фреймворком Hugging Face, потому что он прост в освоении и имеет отличную поддержку сообщества с обширной документацией и многочисленными учебными руководствами.

Вместе с тем, если вы имеете неплохие навыки программирования на языке Python или обладаете устройством Apple с процессором серии M, то можете воспользоваться такими фреймворками, как Unsloth или MLX. Эти альтернативы предлагают более широкие возможности и хорошо подходят для разработки среды флайн-тюнинга с нуля.

Обязательные требования

Перед тем как приступить к практическому примеру, рассмотрим обязательные требования.

Среда	Описание
Облако: Google Colab	16 ГБ GPU (T4), платная подписка не обязательна.
Облако: Kaggle	16 ГБ GPU (T4).
Hugging Face	Токены доступа.
Сеть	Подключение к Интернету.

Инфо

Если на вашей рабочей станции/ноутбуке есть графический процессор (GPU) с объемом оперативной памяти не менее 16 ГБ, вы можете попробовать запустить блокноты локально. На Python они доступны в нашем репозитории GitHub (глава 5).

В данном примере для запуска блокнотов воспользуемся облачной средой Google Colab с бесплатной подпиской. Для удобства мы взяли существующий пример файн-тюнинга LLM с сайта Kaggle, адаптировали его для нашего набора данных и модифицировали для запуска в Google Colab. Кроме того, чтобы избежать ошибок, связанных с нехваткой памяти, мы разбили весь исходный код на два отдельных блокнота Python. Большая часть кода осталась без изменений и доступна по этой ссылке.

Для наглядного пошагового объяснения всего процесса мы разделили его на удобные части, оставив прежней последовательность шагов.

Часть 1. Анализ бизнес-требований, выбор базовой модели и настройка окружения.

Предположим, что мы получили коммерческое предложение от небольшой частной компании, занимающейся исследованием рынка,

и эта компания хочет с помощью ИИ повысить эффективность своей повседневной деятельности.

Шаг 1. Анализ бизнес-требований.

В предложении говорится о разработке сложной системы, способной резюмировать технические статьи в удобном для чтения виде с сохранением их первоначального смысла. Система будет развернута в частной облачной инфраструктуре и построена с использованием технологий с открытым исходным кодом. Сгенерированные резюме станут использоваться в еженедельных информационных рассылках. Пользоваться системой будут всего три человека, поэтому нужно создать легко поддерживаемое и экономически эффективное решение.

Проведя небольшое исследование, мы пришли к следующим выводам.

1. Ядром системы станет локальный механизм инференса языковой модели.
2. Предлагается использовать LLM общего назначения с открытым исходным кодом.
3. Для достижения поставленных целей модель LLM нужно будет дообучить для понимания технических статей и отчетов.

Шаг 2. Выбор предварительно обученной модели.

Так как согласно условиям, можно использовать предварительно обученные модели с открытым исходным кодом, мы с помощью таблицы показателей Hugging Face Open LLM Leaderboard сравнили функциональные возможности различных LLM. Рассмотрев предложенные варианты, в качестве базовой модели выбрана Microsoft Phi-2. Этот выбор был обусловлен впечатляющим объемом обучающей базы данных (2,7 миллиарда параметров) при компактном размере (всего 5 ГБ) и широком распространении в приложениях для ответов на вопросы. При этом, отлично справляясь с ответами,

модель испытывает трудности с генерацией текстов, поэтому нам придется доработать ее в соответствии с нашими специфическими требованиями.

Внимание

Обратите внимание, что на практике эти шаги обычно разбиваются на несколько подэтапов, включая тщательные предварительные исследования и сравнение результатов всех рассматриваемых LLM. Кроме того, перед тем, как принять окончательное решение, полезно будет испытать LLM на тестовом наборе данных с локальным или облачным инференсом.

Шаг 3. Настройка окружения.

Так как у нас нет доступа к сложному оборудованию с мощными графическими процессорами, в качестве среды для обучения LLM воспользуемся сервисом Google Colab. Каждый день Google Colab предоставляет пользователям 16 ГБ оперативной памяти GPU на несколько часов, что должно быть достаточно для наших потребностей.

Инфо

Учтите, что в Google Colab можно использовать несколько учетных записей Google, что позволит вам еженедельно получать достаточно рабочего времени для выполнения рутинных задач.

Чтобы начать работу, просто войдите в Colab через любую учетную запись. Затем создайте новый блокнот и перейдите в раздел «Change runtime» («Сменить среду выполнения»), где выберите тип «Графический процессор T4».

В первую ячейку блокнота добавьте следующий фрагмент кода:

```
!pip install -U bitsandbytes transformers peft accelerate  
datasets scipy einops evaluate trl rouge_score
```

Эта команда устанавливает набор библиотек Python для обработки естественного языка.

Вот краткое описание каждой библиотеки.

- **Bitsandbytes.** Библиотека, которая помогает оптимизировать использование памяти для больших моделей за счет использования 8-битной разрядности при обучении, что снижает вычислительную нагрузку и потребление памяти без существенной потери точности модели.
- **Transformers.** Мощная библиотека, разработанная компанией Hugging Face и предлагающая предварительно обученные модели и инструменты для таких задач обработки естественного языка, как классификация текстов, перевод и ответы на вопросы. Включает в себя модели BERT, T5 и другие.
- **peft (Parameter-Efficient Fine-Tuning).** Библиотека Hugging Face для эффективного фاین-тюнинга больших моделей посредством замораживания большинства параметров и обучения только небольшого их подмножества.
- **Accelerate.** Библиотека, упрощающая распределенное обучение и обучение со смешанной разрядностью на нескольких устройствах (GPU/CPU) и позволяющая легко масштабировать обучение больших моделей.
- **Datasets.** Библиотека, также разработанная Hugging Face, предлагающая простой способ доступа к большим наборам данных и их обработки. Включает в себя тысячи наборов данных, широко используемых в сфере машинного обучения и обработке естественного языка.
- **Scipy.** Библиотека Python для научных вычислений, предоставляющая функции оптимизации, интегрирования, интерполяции, решения задач с собственными значениями и других сложных математических операций.
- **Einops.** Инструмент для операций с тензорами и их преобразований в моделях глубокого обучения. Позволяет с помощью простого синтаксиса выполнять сложные изменения формы многомерных массивов, что полезно для работы с изображениями и текстовыми данными.

- **evaluate**. Библиотека Hugging Face для упрощения оценки моделей машинного обучения. Предоставляет метрики и инструменты для оценки производительности моделей на различных задачах.
- **trl** (Transformers Reinforcement Learning). Библиотека с инструментами и моделями комбинированного обучения трансформеров с подкреплением, помогающая оптимизировать языковые модели с помощью методов обучения с подкреплением.
- **rouge_score**. Библиотека для вычисления метрик ROUGE, которые обычно используются для оценки качества задач генерации текста, таких как резюмирование. Метрика ROUGE сравнивает совпадения n-грамм (n-grams) между сгенерированным и эталонным текстами.

Инфо

Если появится сообщение об ошибке и о том, что преобразователь зависимостей `pip` не может учесть все установленные пакеты с предупреждением о возможном конфликте зависимостей, проигнорируйте это сообщение.

Теперь для начала работы импортируем все необходимые библиотеки.

```
from datasets import load_dataset

from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    HfArgumentParser,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    GenerationConfig
)
from tqdm import tqdm
from trl import SFTTrainer
```

```
import torch
import time
import pandas as pd
import numpy as np
from huggingface_hub import interpreter_login

interpreter_login()
```

Данные строки импортируют несколько необходимых компонентов и библиотек из Hugging Face, которые требуются для дальнейшей работы. Обратите внимание, что мы также импортируем torch — ядро модуля PyTorch, фреймворка глубокого обучения. Он предлагает функции для создания и обработки тензоров (многомерных массивов) и широко используется для построения и обучения нейронных сетей.

В последней строке кода вызывается функция `interpreter_login()`, предлагающая пользователю войти в систему Hugging Face с API-ключом. После такого входа пользователь получит доступ к ресурсам Hugging Face Hub.



```

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings t
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
warnings.warn(
  To login, `huggingface_hub` requires a token generated from https://hugg
Enter your token (input will not be visible): .....
Add token as git credential? (Y/n) Y
Token is valid (permission: fineGrained).
Your token has been saved in your configured git credential helpers (store).
Your token has been saved to /root/.cache/huggingface/token
Login successful

```

Иллюстрация 5.6

Для продолжения работы введите свой API-ключ Hugging Face в поле ввода (как показано на иллюстрации 5.6) и нажмите Enter.

Часть 2. Обзор и исследование обучающего набора данных.

На этом этапе ваше окружение настроено и готово к использованию. Теперь можно перейти к работе с набором данных. После краткого поиска в репозитории наборов данных Hugging Face мы нашли следующий подходящий для данного примера вариант: `yasminesarraj/texts_summary`. Этот набор данных содержит технологические статьи вместе с их резюме, что идеально соответствует нашим потребностям.

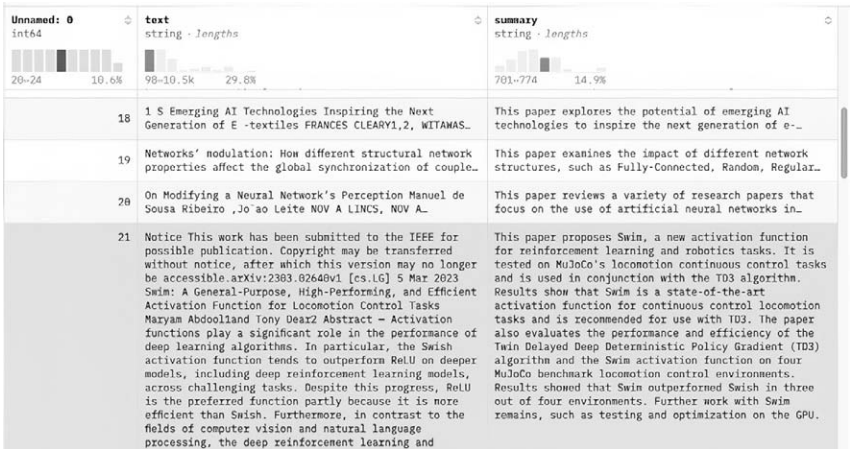


Иллюстрация 5.7

Набор данных разделен на две части — «сплиты» (splits): train («обучающий») и test («тестовый»). Каждый сплит содержит три столбца (Unnamed:0, text, summary) и 47 строк данных. Общий размер набора данных составляет около 2,48 МБ, что вполне подходит для нашего случая.

Шаг 1. Загрузка и изучение набора данных.

Загрузите набор данных с помощью следующего кода:

```
huggingface_dataset_name = "yasminesarraj/texts_summary"

dataset = load_dataset(huggingface_dataset_name)
print (dataset)
```

После выполнения этой ячейки блокнота будет выведен текст, похожий на показанный ниже:

```
DatasetDict({train: Dataset({features:
    'Unnamed: 0', 'text', 'summary'],
    num_rows: 47
}) test: Dataset({features: ['Unnamed: 0',
    'text', 'summary'], num_rows: 47
})
})
```

Сплит `train` мы будем использовать для обучения модели, а сплит `test` — для ее оценки.

Для дальнейшего изучения набора данных можно воспользоваться другими применимыми к нему функциями.

Функция	Описание
<code>dataset.features</code>	Возвращает тип данных столбцов {'Unnamed: 0': Value(dtype='int64', id=None), 'text': Value(dtype='string', id=None), 'summary': Value(dtype='string', id=None)}
<code>dataset['summary']</code>	Возвращает резюме столбца в строке 21.

Шаг 2. Конфигурация для флайн-тюнинга модели.

Сначала создадим конфигурацию `BitsAndBytes`, которая загрузит нашу модель в 4-битном формате. Такая квантизация позволит значительно сократить потребление памяти за счет незначительного снижения точности. Для этого добавьте в очередную ячейку следующий код и выполните ее.

```
compute_dtype = getattr(torch, "float16")
bnb_config = BitsAndBytesConfig(
```

```
load_in_4bit=True, bnb_4bit_quant_type='nf4',
bnb_4bit_compute_dtype=compute_dtype, bnb_4bit_use_
double_quant=False,
)
```

Краткое описание этих строк.

- `compute_dtype = getattr(torch, "float16")` использует функцию Python `getattr` для получения типа данных `float16` из библиотеки `torch` (PyTorch). Тип `float16` означает 16-битные числа половинной точности с плавающей точкой, которые занимают меньше памяти и обрабатываются быстрее по сравнению с более распространенными 32-битными числами (одинарными) с плавающей точкой.
- Параметр `load_in_4bit=True` в `BitsAndBytesConfig` означает 4-битную квантизацию — метод представления весов и активаций модели с использованием только 4 бит для отдельного значения, что значительно сокращает использование памяти по сравнению со стандартными 16-битными или 32-битными представлениями.
- `bnb_4bit_quant_type='nf4'` задает тип квантизации `nf4` (Normalized Float 4). Формат квантизации `nf4` разработан для повышения точности 4-битных квантизированных моделей по сравнению со стандартной 4-битной квантизацией.
- `bnb_4bit_use_double_quant=False` отключает двойную квантизацию, при которой применяются два ее уровня (обычно во время обучения и во время инференса). Двойная квантизация помогает дополнительно экономить память, но при ее отключении точность модели повышается.

Инфо

Если вы планируете обучать большую модель на большом наборе данных, попробуйте поэкспериментировать с параметром `bnb_4bit_use_double_quant` и задать ему значение `True` для экономии памяти.

Шаг 3. Загрузка предварительно обученной модели.

Как было сказано ранее, в качестве базовой предварительно обученной модели мы воспользуемся моделью Phi-2 компании Microsoft. Загрузим ее с помощью конфигурации BitsAndBytes.

```
model_name='microsoft/phi-2'

device_map = {"": 0}
original_model = AutoModelForCausalLM.from_pretrained(model_name,
    device_map=device_map,
    quantization_config=bnb_config,
    trust_remote_code=True,
    use_auth_token=True)
```

Краткое описание строк.

- `microsoft/phi-2` — это конкретная модель, доступная на хабе моделей Hugging Face в пространстве имен Microsoft.
- `device_map = {"": 0}` — выбор устройства (CPU или GPU), в которое будет загружаться модель. В данном случае мы попытаемся использовать оба устройства: CPU и GPU.

Шаг 4. Подготовка набора данных и токенизация.

Отдельная строка набора данных представлена в виде словаря с тремя парами «ключ-значение»: `Unnamed: 0`, `text` и `summary`. Рассмотрим отдельную строку с помощью следующего кода:

```
row_21 = dataset['train'][21]
print (row_21)
```

Результат его выполнения должен быть следующим:

```
{'Unnamed: 0': 21,
 'text': ' Notice \n This work has been submitted to
 the IEEE for possible publication.\n Copyright may be
```

```

transferred without notice, after which this version (...
TRUNCATED) ',
'summary': ' This paper proposes Swim, a new activation
function for reinforcement learning and robotics tasks.
It is tested on MuJoCo's locomotion continuous control
tasks (...TRUNCATED)'
}

```

Преобразуем теперь набор данных в токены. Для этого можно воспользоваться библиотекой Hugging Face Transformers, упрощающей этот процесс. Помимо токенизации текста функция выполняет несколько задач.

- Токенизирует входной текст.
- Преобразует выходные данные в тензоры PyTorch.
- Дополняет входные данные, чтобы привести их к одинаковой длине.

Добавьте в новую ячейку следующий фрагмент кода:

```

tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    trust_remote_code=True,
    padding_side="left",
    add_eos_token=True,
    add_bos_token=True,
    use_fast=False)

tokenizer.pad_token = tokenizer.eos_token

```

Этот код устанавливает токенизатор с помощью класса Hugging Face AutoTokenizer, автоматически выбирающего и загружающего токенизатор, соответствующий указанной предварительно обученной модели. Говоря вкратце, токенизатор обрабатывает вводимый текст и преобразует его в токены, которые затем загружаются в модель для решения таких задач, как генерация или классификация текста. Ключевые параметры следующие.

- `padding_side=»left«`. Указывает, что padding (дополнительные токены, добавляемые для того, чтобы сделать последовательность такой же длины, как и другие последовательности в пакете) будет добавляться к левой стороне входного текста. Добавление слева обычно используется в таких моделях, как GPT, Llama и других распространенных языковых моделях, где внимание сосредоточено на правой части последовательности (последние токены). Это также оптимизирует использование памяти во время обучения.
- `add_eos_token=True`. EOS означает *end of sequence* — конец последовательности. Этот параметр указывает токенизатору автоматически добавлять токен EOS (обычно обозначающий конец предложения или последовательности) к каждой входной последовательности. Это помогает модели определить, когда входные данные заканчиваются.

Чтобы посмотреть, как выглядит токенизированный текст, можно в качестве эксперимента выполнить следующий псевдокод, выводящий токены:

```
tokenizer.encode("Getting started with Generative AI!")
```

Результат выполнения кода должен быть аналогичен следующему:

```
[50256, 20570, 2067, 351, 2980, 876, 9552, 0]
```

Можно также задать максимальную длину токенов:

```
tokenizer.encode("Getting started with Generative AI!",  
padding='max_length', max_length=10)
```

Результат:

```
[50256, 20570, 2067, 351, 2980, 876, 9552, 0, 50256,  
50256]
```

Непосредственно из токенизатора можно также получать тензоры PyTorch:

```
tokenizer.encode("Getting started with Generative AI!",
                 padding='max_length', max_length=10,
                 return_tensors="pt")
```

Результат:

```
tensor([[50256, 50256, 50256, 20570, 2067, 351, 2980, 876,
        9552, 0]])
```

Показанный выше фрагмент кода токенизирует строку «*Getting started with Generative AI!*», преобразуя ее в тензор формата PyTorch, содержащий идентификаторы токенов с фиксированной длиной в 10 токенов. Если в токенизируемой последовательности меньше 10 токенов, то добавляются дополнительные значения. Если в последовательности больше 10 токенов, то она обрезается. В итоге получается тензор, готовый к использованию в модели PyTorch для таких задач, как классификация или генерация текста.

В новую ячейку добавьте еще один токенизатор для оценки набора данных:

```
eval_tokenizer = AutoTokenizer.from_pretrained
    (model_name,
     add_bos_token=True,
     trust_remote_code=True,
     use_fast=False
    )

eval_tokenizer.pad_token = eval_tokenizer.eos_token
```

Нам также понадобится функция для генерации текста из входного запроса с помощью предварительно обученной языковой модели. Она должна токенизировать входной запрос, генерировать новые токены из модели и декодировать результат в читаемый текст. В новую ячейку добавьте следующую функцию:

```
def gen(model, p, maxlen=100, sample=True):
    toks = eval_tokenizer(p,
```

```

return_tensors="pt"#,

#truncation=True,
#max_length=512

)
res = model.generate(**toks.to("cuda"),
    max_new_tokens=maxlen,
    pad_token_id=tokenizer.eos_token_id,
    do_sample=sample,
    num_return_sequences=1,
    temperature=0.1,
    num_beams=1,
    top_p=0.95,
    ).to('cpu')
return eval_tokenizer.batch_decode(res, skip_special_
tokens=True)

```

Функция **gen** принимает на входе промпт и на его основе генерирует текстовую последовательность с использованием предварительно обученной языковой модели (в данном случае это Phi-2). Функция выполняет следующие действия.

- Токенизирует входной промпт.
- Генерирует последовательность новых токенов длиной до maxlen с дополнительным семплированием для добавления элемента случайности.
- Декодирует сгенерированные токены обратно в удобный для восприятия человека текст.

Функция позволяет регулировать детерминированность или «креативность» ответов с помощью таких параметров, как «температура» и «семплинг», и для ускорения процесса генерации использует мощное аппаратное обеспечение (графический процессор GPU).

Шаг 5. Тестирование базовой модели генерации текста.

Теперь выполним простой текст для проверки базовой модели генерации текста. В новую ячейку добавьте следующий код и выполните его:

```
%%time
import textwrap
from transformers import set_seed
seed = 42
set_seed(seed)
index = 21
prompt = dataset['test'][index]['text']
summary = dataset['test'][index]['summary']
formatted_prompt = f"Instruct: Summarize the following
scientific reports .\n{prompt}\nOutput:\n"
res = gen(original_model, formatted_prompt, 100, )
output = res[0].split('Output:\n')[1]

prompt_to_print = textwrap.shorten(formatted_prompt, width=100,
placeholder="...")

dash_line = '-'.join(' ' for x in range(100))
print(dash_line)
print(f'INPUT PROMPT:\n{prompt_to_print}')
print(dash_line)
print(f'SUMMARY FROM FILE:\n{summary}\n')
print(dash_line)
print(f'MODEL GENERATION SUMMARY:\n{output}')
print(dash_line)
```

Код извлекает из набора данных сплит `test` и форматирует промпт о том, что нужно резюмировать текст. Предварительно обученная модель (Phi-2) на основе этого промпта генерирует резюме, и это резюме сравнивается с эталонным резюме из файла. Входной промпт (INPUT PROMPT), резюме из файла (SUMMARY FROM FILE) и созданное моделью резюме (MODEL GENERATION SUMMARY)

выводятся рядом, друг под другом, благодаря чему пользователь легко может их сравнить между собой.

INPUT PROMPT:

Instruct: Summarize the following scientific reports.

Notice This work\ has been submitted to the...

SUMMARY FROM FILE:

This paper proposes Swim, a new activation function for reinforcement learning and robotics tasks.

It is tested on MuJoCo's locomotion continuous control tasks and is used in conjunction with the TD3 algorithm ...

MODEL GENERATION SUMMARY:

$f(x) = x$

$2(kxp$

$1+k2x2+1)$; $f\theta(x) = 1 2(kxp$

$1+k2x2)$

$(p$

$1+k2x2)3+1)$ (1) where k is a constant that can be tuned or defined before training. We pick a k such that it can support our analysis that Swim outperforms

CPU times: user 9.67 s, sys: 417 ms, total: 10.1 s

Wall time: 19.9 s

При внимательном взгляде на выведенное сообщение можно заметить очевидное различие между резюме из файла и резюме, сгенерированном моделью. Такое различие говорит о том, что для адекватного выполнения поставленной задачи требуется дообучение.

Часть 3. Предварительная обработка набора данных и настройка адаптера.

Итак, до этого мы загрузили набор данных, токенизировали их и провели небольшое тестирование. Но для обучения или настройки модели нельзя использовать необработанный набор данных без

дополнительной их подготовки. Промпт должен строиться в соответствии с определенным форматом, благодаря которому модель будет его понимать.

Шаг 1. Предварительная обработка набора данных.

Существуют два варианта формата подготовки данных к тонкой настройке: их комбинация и разделение.

1. *Комбинированный формат.* В комбинированном формате входные и выходные последовательности объединяются в одно текстовое поле или в одну строку. При этом часто к входной (исходной) последовательности после особых разделителей или по определенным правилам (инструкциям) добавляется целевая (выходная) последовательность.

```
{Instruction: "Summarize the following text.", Input: "The full text document goes here.", Output: "The summary goes here."}
```

(Инструкция: «Резюмируй следующий текст», Вход: «Сюда подставляется полный текст документа», Выход: «Здесь выдается резюме».)

В этом формате модель обучается генерировать вывод (в данном случае резюме), предсказывая последовательность следующих токенов после инструкции Output. Весь промпт модель рассматривает как единое целое. У данного формата есть несколько преимуществ.

- Легок для обучения таких моделей, как Llama, Mistral и других языковых моделей общего назначения, где любой вход рассматривается как одна строка для генерации.
- Полезен при использовании предварительно обученных языковых моделей для задач генерации текста без подсказок (zero-shot) или с небольшим количеством подсказок (few-shot).

2. *Разделенный формат.* В таком формате входные (исходные) и выходные (целевые) последовательности хранятся в отдельных полях или столбцах. Этот формат типичен для случаев, когда важно четко

разграничивать входные и выходные данные — например, в моделях перевода или в архитектуре кодировщика-декодировщика.

Instruction: Summarize the following text.

Input: [The full text document goes here.]

Output: [The summary goes here.]

Инструкция: Резюмировать следующий текст.

Вход: [Полный текст документа]

Выход: [Резюме]

Такой формат явно указывает модели, какие данные должны подаваться на вход, и какие должны появляться на выходе. В моделях типа трансформеров с отдельными кодировщиком и декодировщиком входные данные обрабатываются кодировщиком, а выходные генерируются декодировщиком. Преимущества этого формата следующие.

- Четкое разграничение между входом и выходом, что важно для некоторых архитектур моделей, требующих явного разделения.
- Больше возможностей для контроля над предварительной обработкой и токенизацией входных и выходных последовательностей независимо друг от друга.

В нашем примере для подготовки набора данных мы воспользуемся комбинированным форматом и некоторыми дополнительными функциями. В отдельные ячейки блокнота добавьте три следующие функции:

```
def create_prompt_formats(sample):
```

```
    """
```

```
    Отформатируй разные поля образца
```

```
    (<instruction>, <output>) Затем объедини их, используя два символа новой строки param sample: Sample dictionary
```

```
    """
```

```

INTRO_BLURB = "Below is an instruction that describes
a task. Write a response that appropriately completes the
request."
INSTRUCTION_KEY = "### Instruct: Summarize the following
scientific reports."
RESPONSE_KEY = "### Output:"
END_KEY = "### End"
blurb = f"\n{INTRO_BLURB}"
instruction = f" {INSTRUCTION_KEY}"
input_context = f" {sample['text']}" if
sample["text"] else None response = f" {RESPONSE_KEY}\n
n{sample['summary']}"
end = f" {END_KEY}"
parts = [part for part in [blurb, instruction, input_
context, response, end] if part]
formatted_prompt = "\n\n".join(parts)
sample["text"] = formatted_prompt
return sample

```

В данном фрагменте кода определена функция `create_prompt_formats`, форматирующая разные поля из словаря-образца и создающая структурированный промпт для обучения модели генерации текста. Функция берет словарь-образец (`sample`) с полями `text` (input, Вход) и `summary` (output, Выход) и создает из них структурированный промпт (Below is an instruction that describes a task. Write a response that appropriately completes the request = «Ниже приведена инструкция, описывающая задачу. Напиши резюме, соответствующее запросу») с добавлением инструкций (Instruct: Summarize the following scientific reports = «Сделай резюме следующих научных докладов»), маркированием выходных данных и четкими границами для модели, чтобы она понимала задачу.

```

def get_max_length(model):
    conf = model.config
    max_length = None
    for length_setting in ["n_positions", "max_position_
embeddings", "seq_length"]:

```

```

        max_length = getattr(model.config, length_
        setting, None)
        if max_length:
            print(f"Found max length: {max_length}")
            break
    if not max_length:
        max_length = 1024
        print(f"Using default max length: {max_length}")
    return max_length
def preprocess_batch(batch, tokenizer, max_length):

« » »
Токенизация пакета (батча)
« » »

return tokenizer(
    batch["text"]
    max_length=max_length,
    truncation=True,
)

```

Этот фрагмент кода содержит две функции: `get_max_length` и `preprocess_batch`. Они определяют максимальную длину последовательности, которую может обрабатывать модель, и затем соответствующим образом токенизируют батч тестовых данных.

Главная задача функции `preprocess_batch` — проследить за тем, чтобы длина токенизированных последовательностей соответствовала максимальной длине моделей. Более длинные последовательности по мере необходимости обрезаются. Так текст подготавливается к вводу в модель.

```

from functools import partial

```

```

# SOURCE https://github.com/databricks/dolly/blob/master/training/trainer.py

```

```

def preprocess_dataset(tokenizer: AutoTokenizer, max_
length: int, seed, dataset):

    """Форматирование и токенизация для подготовки к обуче-
    нию
    : param tokenizer (AutoTokenizer): Модель токенизатора
    : param max_length (int): максимальное количество токе-
    нов, выдаваемых токенизатором
    """

    # Добавляет промпт к каждому образцу
    print("Preprocessing dataset...")
    dataset = dataset.map(create_prompt_formats)#,
batched=True)
    preprocessing_function = partial(preprocess_batch,
max_length=max_length, tokenizer=tokenizer)
    dataset = dataset.map(
        _preprocessing_function,
        batched=True,
        remove_columns=['Unnamed: 0', 'text', 'summary']
    )

    # Фильтрация образцов с input_ids больше max_length

    dataset = dataset.filter(lambda sample:
len(sample["input_ids"]) < max_length)

    # Перемешивание данных
    dataset = dataset.shuffle(seed=seed)
    return dataset

```

Функция `preprocess_dataset` задает последний этап подготовки набора данных к обучению LLM для задач обобщения. Она токенизирует и форматирует данные, выполняя необходимую фильтрацию. Функция также случайным образом перемешивает набор данных, чтобы модель не ориентировалась на какой-то определенный порядок. Параметр `seed` позволяет воспроизводить порядок перемеши-

вания. Если запускать функцию с одним и тем же значением `seed`, то порядок перемешивания будет одним и тем же.

Инфо

Вам не обязательно создавать новый блокнот Colab с нуля. Можно загрузить уже существующий блокнот `LLM_tuning_for_text_summary.ipynb` из папки `chapter-5` репозитория GitHub прямо в Colab через меню `File > Upload Notebook` («Файл» > «Загрузить блокнот») с указанием ссылки. Так вы сможете запускать кодовые ячейки сразу, а не копировать соответствующие фрагменты кода в новый блокнот.

Далее нужно подготовить к предварительной обработке набор обучающих данных (`train`) и тестовых данных для оценки (`eval`). Код для этого следующий:

```
max_length = get_max_length(original_model)
print(max_length)
train_dataset = preprocess_dataset(tokenizer, max_length,
seed, dataset['train'])
eval_dataset = preprocess_dataset(tokenizer, max_length,
seed, dataset['test'])
```

Для вывода формы (`shape`) набора данных можно воспользоваться следующими строками:

```
print(f"Shapes of the datasets:")
print(f"Training: {train_dataset.shape}")
print(f"Validation: {eval_dataset.shape}")
print(train_dataset)
```

Результат:

```
Shapes of the datasets:
Training: (9, 2)
Validation: (9, 2)
Dataset({features: ['input_ids', 'attention_mask'],
```

```

        num_rows: 9
    })

```

Напоследок стоит показать и разобрать еще одну важную функцию, выводящую количество обучаемых параметров модели.

```

def print_number_of_trainable_model_parameters(model):
    trainable_model_params = 0
    all_model_params = 0
    for _, param in model.named_parameters():
        all_model_params += param.numel()
        if param.requires_grad:
            trainable_model_params += param.numel()
    return f"trainable model parameters: {trainable_
model_params}\nall model parameters: {all_model_params}\
percentage of trainable model par ameters: {100 *
trainable_model_params / all_model_params:.2f}%"
print(print_number_of_trainable_model_parameters(original_
model))

```

Функция `print_number_of_trainable_model_parameters` вычисляет и выводит количество обучаемых параметров LLM, сравнивает их с общим количеством параметров модели и вычисляет процент обучаемых параметров. Это бывает полезно при работе с большими моделями, которые могут включать в себя помимо обучаемых параметров и замороженные (необучаемые) — например, при использовании метода PEFT.

Выполнив это, вы получите результат, похожий на показанный ниже:

```

trainable model parameters: 262364160
all model parameters: 1521392640
percentage of trainable model parameters: 17.24%

```

Иными словами, здесь говорится, что из всех параметров предварительно обученной модели понадобится переобучить примерно 17,24%.

Шаг 2. Настройка PEFT.

Итак, после завершения этапа подготовки набора данных можно перейти к подготовке модели для метода QLoRA. Далее показан код PEFT (параметрически эффективного фاین-тюнинга) с использованием метода QLoRA для донастройки предварительно обученной языковой модели с минимальными затратами памяти. Процесс включает в себя создание конфигурации QLoRA, подготовку модели к обучению с низкой разрядностью и применение к ней модификаций QLoRA.

```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

config = LoraConfig(
    r=32, #Ранк
    lora_alpha=32,
    target_modules=[
        'q_proj',
        'k_proj',
        'v_proj',
        'dense'
    ],
    bias="none",
    lora_dropout=0.05, # Типичный
    task_type="CAUSAL_LM",
)

# 1 – Включить градиент контрольных точек для сокращения
# памяти во время тонкой настройки

original_model.gradient_checkpointing_enable()

# 2 – Использование метода prepare_model_for_kbit_training
# из PEFT
original_model = prepare_model_for_kbit_training(original_model)
peft_model = get_peft_model(original_model, config)
```

Рассмотрим ключевые моменты показанного выше кода.

- `LoraConfig`, задает параметры LoRA:
 - › `r=32`. Ранг низкоранговых матриц в LoRA — размерность применяемой к весам модели факторизации. Более высокий показатель увеличивает производительность модели, но также требует больше памяти и вычислений.
 - › `lora_alpha=32`. Масштабный коэффициент для обновлений LoRA. Определяет вклад низкоуровневых матриц в общий результат модели. Этот гиперпараметр регулирует соотношение между весами исходной модели и адаптацией LoRA.
 - › `target_modules`. Список модулей (слоев) модели, к которым будет применяться LoRA:
 - * `q_proj`. Проекция запросов (query) в механизме внимания трансформера.
 - * `k_proj`. Проекция ключей (key).
 - * `v_proj`. Проекция значений (value).
 - * `dense`. Полносвязный (плотный, dense) слой трансформера. Это наиболее важные части конфигурации, и их тонкая настройка значительно улучшит работу модели.
 - › `bias="none"`. Указание на то, что LoRA не будет затрагивать смещения (biases). Другие варианты — это дообучение всех смещений (all) или смещений только в слоях, обрабатываемых LoRA (lora_only).
 - › `lora_dropout=0.05`. Коэффициент дропаута, применимый к слоям LoRA, что помогает предотвратить переобучение посредством случайного исключения частей модели во время обучения.
 - › `Task_type="CAUSAL_LM"`. Указание на определенный тип задачи для файн-тюнинга. В данном случае этот тип CAUSAL_LM («Общее моделирование языка»), обычно используемый для таких моделей, как Llama и Mistral.
- `original_model.gradient_checkpointing_enable()`. Включает градиент контрольных точек, что сокращает использование

памяти во время обучения за счет хранения только некоторых активаций (промежуточных результатов вычислений) и пересчета их по мере необходимости. Благодаря этому удастся сэкономить память для дополнительных вычислений в условиях ее недостатка, особенно для флайн-тюнинга больших моделей.

- `original_model = prepare_model_for_kbit_training(original_model)`. Подготовка модели к k-битному обучению — например, 4-битная или 8-битная квантизация, о которых мы говорили ранее.

После такой настройки можно воспользоваться функцией `print_number_of_trainable_model_parameters`, и узнать, сколько теперь параметров из общего числа предназначено для переобучения.

```
print(print_number_of_trainable_model_parameters(peft_
model))
```

Результат:

```
trainable model parameters: 20971520
all model parameters: 1542364160
percentage of trainable model parameters: 1.36%
```

Часть 4. Обучение модели.

Перед обучением модели нужно выполнить еще один последний шаг: определить аргументы для обучения и создать экземпляр Hugging Face Trainer.

Лучше всего задавать параметры с помощью инструмента `TrainingArgs`, позволяющего контролировать процесс оптимизации. Для этих целей в библиотеке Transformers содержится множество аргументов обучения с хорошей документацией.

Шаг 1. Обучение модели.

Сначала настроим аргументы для обучения следующим образом:

```

output_dir = './peft-scientific_reports-summary-training/
final-checkpoint'
max_steps=400

import transformers

peft_training_args = TrainingArguments(
    output_dir = output_dir,
    warmup_steps=1,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    max_steps=max_steps,
    learning_rate=2e-4,
    optim="paged_adamw_8bit",
    logging_steps=25,
    logging_dir="./logs",
    save_strategy="steps",
    save_steps=25,
    eval_strategy="steps",
    eval_steps=25,
    do_eval=True,
    gradient_checkpointing=True,
    report_to="none",
    overwrite_output_dir = 'True',
    group_by_length=True,
)
peft_model.config.use_cache = False

```

Некоторые пояснения.

- **output_dir.** Выходной каталог, в котором будут сохраняться финальные контрольные точки модели и результаты обучения в среде Colab.
- **max_steps=400.** Максимальное количество шагов обучения. В данном случае процесс обучения завершится через 400 шагов независимо от количества эпох. Этот показатель можно устанавливать по своему усмотрению, но лучше выше 100, так

как за меньшее число шагов адаптер не сможет обучиться как следует.

Рассмотрим теперь ключевые параметры оптимизации, указанные в этом экземпляре `TrainingArguments`.

- `per_device_train_batch_size=1`. Размер батча на 1 шаг для обучения на каждом из устройств (GPU). В условиях ограниченной памяти его следует устанавливать как можно меньше, особенно для больших моделей.
- `gradient_accumulation_steps=4`. Вместо того чтобы обновлять веса после обработки каждого батча, модель будет накапливать градиенты и выполнять обновление только после завершения обработки этих 4 батчей. Так можно успешно симулировать обработку батча более крупного размера без затрат памяти.
- `max_steps=max_steps`. Максимальное количество шагов обучения (в данном случае — 400).
- `optim="paged_adamw_8bit"`. Используемый оптимизатор. В нашем примере это AdamW, популярный оптимизатор для моделей трансформеров. Его версия `paged_adamw_8bit` сокращает потребление памяти благодаря 8-разрядности без снижения эффективности обучения.
- `save_strategy="steps"`. Указание на то, как часто следует сохранять контрольные точки модели. В данном случае модель сохраняется после определенного количества шагов.
- `save_steps=25`. Состояние модели фиксируется через каждые 25 шагов обучения. Вы можете увеличить этот параметр до 50.
- `gradient_checkpointing=True`. Включает градиент контрольных точек для уменьшения потребления памяти, которое сокращается за счет дополнительных вычислений во время обратного распространения. При этом происходит обмен памяти на вычисления, так как промежуточные активации пересчитываются во время обратного распространения.
- `report_to="none"`. Запрещает запись логов на любой внешней платформе (например, Hugging Face Hub). Если вы хотите воспользоваться внешней платформой для ведения логов (журналов), то можно задать значение `wand` или `tensorboard`.

Также мы отключили кэширование для модели строкой `peft_model.config.use_cache = False`. Кэширование обычно полезно при генерации последовательностей, но во время обучения его отключение снижает потребление памяти.

После завершения процесса настройки можно приступить непосредственно к обучению модели. Для этого добавьте новую ячейку с кодом и выполните ее.

```
peft_trainer = transformers.Trainer(
    model=peft_model,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    args=peft_training_args,
    data_collator=transformers.DataCollatorForLanguageModeling(
        tokenizer, mlm=False),
)

peft_trainer.train()
```

Если вы не изменили значение параметра `max_steps` (400), процесс обучения займет около часа. За это время можно немного отдохнуть и выпить чашку кофе.

Если процесс пройдет гладко, то после его завершения система выдаст окончательный результат.

✓ th  peft_trainer.train()



Step	Training Loss	Validation Loss
25	2.243100	1.841917
50	1.548500	1.454579
75	1.201600	1.076311
100	0.870700	0.700631
125	0.612600	0.412396
150	0.406600	0.226127
175	0.261900	0.122649
200	0.176000	0.069512
225	0.125000	0.040530
250	0.094100	0.026614
275	0.075100	0.019849
300	0.061500	0.015665
325	0.054700	0.013741
350	0.047600	0.012496
375	0.045700	0.011465
400	0.043000	0.011215

Иллюстрация 5.8

Поздравляем, вы дообучили свою модель на пользовательском наборе данных. Итоговая таблица отображает ход обучения адаптера QLoRA с течением времени. По ней видно, как по мере прохождения этапов обучения уменьшаются потери при обучении и валидации.

Обратите внимание, что потери при обучении (Training Loss) с 25 по 400 шаг неуклонно уменьшаются с 2,243100 до 0,043000. Это говорит о том, что модель последовательно улучшала свою работу с обучающими данными. Вместе с тем потери при валидации (Validation Loss) также уменьшались с показателя 1,841917, достигнув показателя 0,011215 на шаге 400. Потери при валидации немного выше потерь при обучении, что вполне нормально, поскольку модель оценивается на неразмеченных данных.

Стоит отметить, что во время настройки аргументов мы указали, чтобы Hugging Face Trainer создавал контрольную точку каждые

25 шагов. Каждая контрольная точка хранится в отдельной папке, где содержатся восемь файлов, связанных с адаптером QLoRA. В каждой папке с контрольными точками хранится полный адаптер QLoRA, который можно использовать с предварительно обученной моделью. Загрузите некоторые папки контрольных точек со всеми их файлами в свою локальную систему, чтобы воспользоваться ими на следующем этапе.

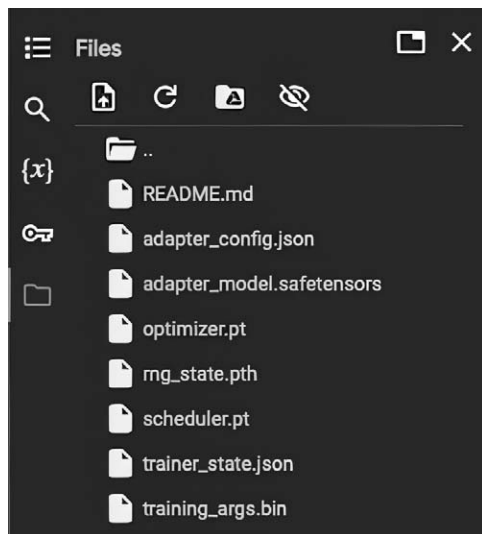


Иллюстрация 5.9

⚠ Внимание

Чтобы просмотреть все папки контрольных точек, созданных за время обучения, нажмите на значок папки в левой части интерфейса Colab. Они будут доступны только во время текущей сессии Colab, поэтому не забудьте скопировать несколько контрольных точек до завершения сессии.

Последним этапом станет оценка сгенерированного нами адаптера QLoRA и проверка того, насколько хорошо он справляется с поставленной нами задачей резюмирования. Этот этап мы и рассмотрим в конце данной главы.

Часть 5. Оценка модели.

Представьте, что вам нужно проверить, насколько хорошо кто-то выучил язык (допустим, английский). Для этого можно задавать этому человеку различные вопросы, проверять, насколько хорошо он понимает рассказы или попытаться поддержать с ним беседу. Точно так же и оценка LLM заключается в том, чтобы давать модели различные задания, на которые она должна выдавать удовлетворительные ответы.

- Ответы на вопросы. Способна ли модель давать точные и полезные ответы?
- Понимание текста. Может ли она улавливать смысл предложения или отрывка?
- Генерация языка. Может ли модель выдавать связный и релевантный текст?

В процессе оценки эксперты или автоматизированные инструменты измеряют производительность модели с помощью различных показателей — например, точность, релевантность, естественность (правильность языка) и соответствие фактам.

Простыми словами, процесс походит на то, как ученику ставят оценки за его умение читать, писать и понимать язык, но в данном случае «ученик» — это большая LLM.

Широко распространены два типа оценок LLM, в каждом из которых эффективность работы LLM измеряется по-своему.

1. Человеческая оценка. Работу языковой модели оценивают реальные люди на основе своих субъективных представлений.

При такой оценке эксперты (часто в области языка) читают сгенерированный моделью текст и оценивают его по различным критериям.

- Естественность. Выглядит и звучит ли текст так, словно его написал человек?

- Релевантность. Соответствует ли текст заданной теме?
- Правильность. Соответствует ли ответ имеющимся фактам и осмыслен ли он?

Следует учесть, что человеческая оценка бывает субъективной, медленной и дорогой, поскольку требует много ручной работы и в очень большой степени зависит от личности экспертов.

2. Оценка на основе метрик. Метрики оценки — это инструменты или методы, используемые для измерения того, насколько хорошо модель или система машинного обучения справляется со своими задачами. Они помогают оценить точность, качество и эффективность модели посредством сравнения ее выходных данных (или прогнозов) с правильными или ожидаемыми результатами.

Метрики оценки зависят от конкретной задачи — для разных задач требуются разные метрики. Например, для оценки модели перевода пользуются метрикой BLEU, а для оценки модели резюмирования (обобщения) — метрикой ROUGE или показателем «точность» (precision). При этом в индустрии ИИ широко распространены следующие метрики оценки.

- BLEU. Устанавливает степень естественности, грамматической правильности (fluency) и связности сгенерированного текста по сравнению с текстом, написанным человеком.
- ROUGE. Оценка, аналогичная BLEU, устанавливающая степень совпадения между сгенерированным моделью текстом и эталонным текстом (обычно используется при задачах резюмирования).
- METEOR. Измеряет степень сходства между сгенерированным текстом и текстом, написанным человеком, на основе поверхностных характеристик.

Попробуем применить эти два метода оценки LLM на практике. Для оценки нашего адаптера QLoRA с предварительно обученной моделью Ph-2 мы воспользуемся методом **человеческой оценки** и **показателем ROUGE**, так как они хорошо подходят именно для задачи резюмирования текста.

Чтобы избежать ошибок, связанных с нехваткой памяти, специально для оценки модели мы создадим новый блокнот Colab. Решать такого рода проблемы можно разными способами. Но они не всегда работают, если набор данных слишком велик и не помещается в память (особенно с учетом того, что в Colab нам выделяют всего лишь 16 ГБ памяти графического процессора GPU).

Большинство показанных ниже фрагментов кода уже использовались и подробно объяснялись в предыдущих шагах, поэтому далее мы пропустим объяснения. Скачайте блокнот из репозитория GitHub, загрузите его в Colab и выполните ячейки вплоть до восьмой.

Для переноса сгенерированного нами адаптера QLoRA в среду выполнения блокнота Colab воспользуемся сервисом Google Диск (Google Drive). Сначала загрузите файл checkpoint-400 (или аналогичный сохраненный на вашем диске файл) на Google Диск. Затем выполните следующую ячейку:

```
from google.colab import drive
drive.mount('/content/drive')
```

Google Colab запросит разрешение на доступ к вашему Google Диску. Дайте разрешение, установив соответствующий флажок, после чего файл загрузится.



Инфо

Вместо того, чтобы пользоваться Google Диском, можно загрузить файл непосредственно в раздел «Файлы» сервиса Colab. Но такой способ часто приводит к ошибкам из-за повреждений файла во время загрузки. Поэтому для работы с файлами в среде Colab мы настоятельно рекомендуем использовать Google Диск. Кроме того, адаптер QLoRA также доступен для загрузки из репозитория GitHub.

Как показано на иллюстрации 5.10, после успешного подключения к Google Диску в блокноте Colab станут доступными хранящиеся на этом диске файлы.

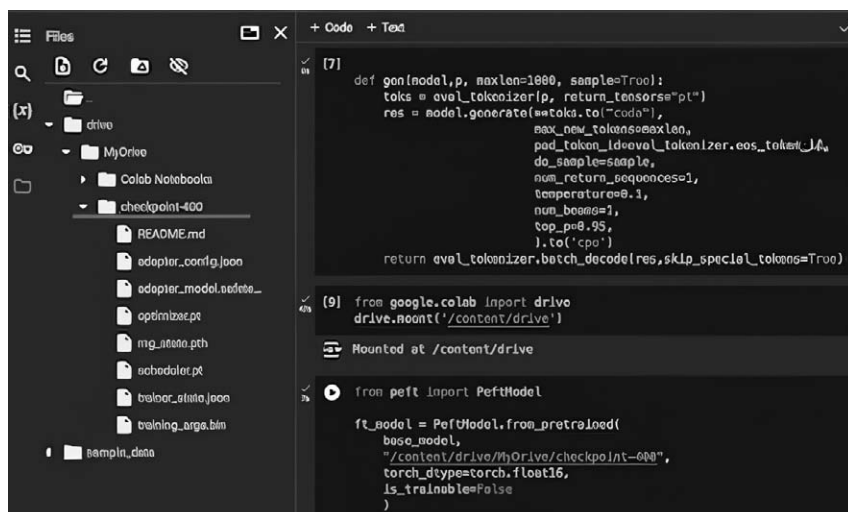


Иллюстрация 5.10

Обновите путь к своему файлу на Google Диске, указав его расположение (например, /content/drive/MyDrive/checkpoint-400), и выполните следующую ячейку:

```
from peft import PeftModel
```

```
ft_model = PeftModel.from_pretrained(
    base_model,
    "/content/drive/MyDrive/checkpoint-400",
    torch_dtype=torch.float16,
    is_trainable=False
)
```

Далее добавьте следующий фрагмент кода (или выполните следующую ячейку, если вы загрузили блокнот в Colab из репозитория GitHub).

```
%%time
import pandas as pd

txt_list = dataset['test'][21:23]['text']
summaries = dataset['test'][21:23]['summary']
```

```

original_model_summaries = []
instruct_model_summaries = []
peft_model_summaries = []

for idx, text in enumerate(txt_list):
    txt = txt_list[idx]
    summary = summaries[idx]

    prompt = f"Instruct: Summarize the following scientific reports .\n{txt}\nOutput:\n"
    peft_model_res = gen(ft_model, prompt, 100,)
    peft_model_output = peft_model_res[0].
    split('Output:\n')[1]
    peft_model_text_output, success, result = peft_model_output.partition ('#End')
    original_model_res = gen(base_model, prompt, 100,)
    original_model_text_output = original_model_res[0].
    split('Output:\n') [1]
    original_model_summaries.append(original_model_text_output)
    peft_model_summaries.append(peft_model_text_output)
    instruct_model_summaries.append(summary)
zipped_summaries = list(zip(instruct_model_summaries,
original_model_summaries, peft_model_summaries))

df = pd.DataFrame(zipped_summaries, columns = ['baseline_summaries', 'original_model_summaries', 'peft_model_summaries'])

df

```

Этот код импортирует библиотеку Python pandas для обработки и анализа данных. Затем из набора данных извлекается определенное подмножество фрагментов текста и соответствующих им резюме (в строках 21 и 22). Далее выполняется итерация по этому подмножеству, строится промпт для резюмирования текстов, а для генерации резюме используются две модели (ft_model

и `base_model`). Затем сгенерированные резюме извлекаются из результатов моделей и переносятся в объект `DataFrame` библиотеки `pandas`. Данный объект `DataFrame` содержит колонки для базовых (эталонных) резюме (`baseline_summaries`), резюме исходной модели (`original_model_summaries`) и резюме модели PEFT (`peft_model_summaries`), что помогает удобно их просматривать и сравнивать между собой.

Результат выполнения должен походить на показанный ниже:

CPU times: user 38.8 s, sys: 1.05 s, total: 39.9 s
Wall time: 42.6 s

1 to 2 of 2 entries Filter ⓘ ?

index	baseline_summaries	original_model_summaries	peft_model_summaries
0	This paper proposes Swim, a new activation function for reinforcement learning and robotics tasks. It is tested on MuJoCo's locomotion continuous control tasks and is used in conjunction with the TD3 algorithm. Results show that Swim is a state-of-the-art activation function for continuous control locomotion tasks and is recommended for use with TD3. The paper also evaluates the performance and efficiency of the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm and the Swim activation function on four MuJoCo benchmark locomotion control environments. Results showed that Swim outperformed Swish in three out of four environments. Further work with Swim remains, such as testing and optimization on the GPU.	This paper presents Swim, a new activation function that significantly improves the performance of deep reinforcement learning (DRL) on continuous control tasks (BC), based on the MuJoCo environment. It is shown to be superior to the previous state-of-the-art activation functions, including Swish, in terms of both reward-achievement and efficiency. The procedure to develop Swim, and its comparison	This paper presents Swim, a new activation function that excelled over Swish in benchmark tests on MuJoCo's continuous control tasks. It is shown to be more efficient and powerful than Swish, even when the best-performing Swish function was used as the baseline. The performance of Swish was improved by introducing a new scaling parameter to replace the exponential part of Swish with an additive function. We
1	This paper provides an overview of Ensemble Reinforcement Learning (ERL), a combination of reinforcement learning and ensemble learning used to handle complex tasks. It discusses strategies used in ERL, related applications, and open questions to guide future exploration. It also provides a brief introduction to reinforcement learning and ensemble learning, and outlines various combinations of models and training algorithms used in ERL from 2008 to 2022. Additionally, it discusses strategies for designing ERL methods, and provides examples of its application in the energy and environment, IoT and cloud computing, finance, and other areas. Lastly, it outlines three open questions in ERL-based research and suggests potential future research directions.	A simple agent-level method (Spade) which aims to minimize the variance of the output of the base learners is removed. There are several different strategies which implements, such as: [Machine Learning Segmentation [W Introduction [learning (EL) has become an important part of [[Introduction [[[Obstacles [[	A simple agent-based model of a mathematical function that transforms the (x) Introduction to This paper: A: Your name Figure 4: A similar approach using a binary separation system that can tolerate ambiguity and can be < [] ### [Your Name Isolation [< [Introduction to [Continuels [Goods: [[

Иллюстрация 5.11

Этот результат кажется удовлетворительным. По нему видно, что обученный адаптер QLoRA работает лучше исходной предварительно обученной модели. Сравним его резюме с резюме исходной предварительно обученной модели с нулевым обучением, не прошедшей дополнительную настройку, которое генерировалось на шаге 8 (и которое мы уже видели ранее).

Резюме исходной модели (НУЛЕВОЕ ОБУЧЕНИЕ)	Резюме модели PEFT
MODEL GENERATION SUMMARY:2(kxp;2(kxp(p(1) training. We pick a ksuch that it can support our analysis thatSwim outperforms.	This paper presents Swim, in benchmark tests on MuJoCo's continuous control tasks. Even when the best-performing Swish function was used as introducing a new scaling parameter to replace the We.

В первом случае получился почти непонятный набор слов, во втором же случае — довольно осмысленный фрагмент: «В данной работе представлены результаты бенчмарк-тестов на задачах непрерывного управления MuJoCo. Даже если в качестве нового параметра масштабирования использовалась функция Swish с новыми параметрами масштабирования вместо We...»

Дополнительно обученная модель и в самом деле демонстрирует более качественные результаты.

⚠ Внимание

Результаты могут отличаться в зависимости от количества шагов обучения. Мы использовали 400 шагов. Как говорилось выше, чем больше обучается модель, тем лучше она справляется с поставленной задачей.

Итак, это была человеческая оценка. Перейдем теперь к оценке по метрике ROUGE.

Выполните следующий фрагмент кода:

```
import evaluate

rouge = evaluate.load('rouge')

original_model_results = rouge.compute(
    predictions=original_model_summaries,
```

```

        references=original_model_summaries[0: len(original_
model_summaries
],
        use_aggregator=True,
        use_stemmer=True,
    )
peft_model_results = rouge.compute(
    predictions=peft_model_summaries,
    references=original_model_summaries[0: len(peft_model_
summaries)], use_aggregator=True, use_stemmer=True,
)

print('ORIGINAL MODEL:')
print(original_model_results)
print('PEFT MODEL:')
print(peft_model_results)
print("Absolute percentage improvement of PEFT MODEL over
ORIGINAL MODEL")

improvement = (np.array(list(peft_model_results.
values())) - np.array(list(original_model_results.
values())))/
for key, value in zip(peft_model_results.keys(),
improvement)
print(f' {key}: {value*100:.2f}%')

```

В приведенном выше коде библиотека Python evaluate используется для вычисления метрик ROUGE для двух наборов резюме: созданного с помощью исходной (оригинальной) модели и созданного с помощью модели PEFT. Затем вычисляются процентные показатели абсолютного улучшения модели PEFT по сравнению с исходной моделью, и результаты выводятся на экран.

Рассмотрим эти результаты:

```

ORIGINAL MODEL :
{'rouge1': 1.0, 'rouge2': 1.0, 'rougeL': 1.0, 'rougeLsum':
1.0}

```

PEFT MODEL :

```
{'rouge1': 1.0798425196850394, 'rouge2':  
1.042739726027397, 'rougeL': 1.00356955380578,  
'rougeLsum': 1.719685039370079}
```

Absolute percentage improvement of PEFT MODEL over
ORIGINAL MODEL

rouge1: 0.1%

rouge2: 0.03%

rougeL: 0.01%

rougeLsum: 1.60%

По сравнению с исходной моделью модель PEFT демонстрирует незначительные улучшения по всем метрикам ROUGE, причем наиболее заметное улучшение наблюдается у показателей ROUGE-Lsum. Несмотря на такие незначительные изменения, различия становятся заметнее при сравнении с резюме, выданных после нулевого обучения на шаге 8. Вы можете выполнить эту оценку самостоятельно в качестве домашнего задания.

Часть 6. Сохранение и развертывание готовой модели.

Допустим, вы удовлетворены работой своего адаптера QLoRA и хотите развернуть его для инференса в локальной или продуктивной среде. Для этого придется выполнить несколько шагов, зависящих от фреймворка, используемого для обучения модели. В *Unsloth* этот процесс прост и интегрирован во фреймворк, но для Hugging Face Trainer нужно выполнить следующее.

- Объединить адаптеры с базовой моделью, чтобы ею можно было пользоваться как обычной моделью трансформеров. Код для этого показан ниже (где `ft_model` — это определенный ранее наш адаптер QLoRA):

```
model = ft_model.merge_and_unload()  
model.save_pretrained("merged_adapters")
```

- После этого объединенная модель будет генерировать резюме текста так, как и стандартная модель (и как было описано выше). Учтите, что объединенная модель по объему будет гораздо больше обычной — около 2 ГБ.
- Загрузите итоговую модель из блокнота Google Colab на свой диск. Найдите папку с именем `merged_adapters`, содержащую три файла.

На этом этапе у вас будут два варианта выполнения инференса объединенной модели.

- Загрузить и использовать эту модель как обычную LLM. Можно загрузить объединенную с адаптером модель в Hugging Face Hub и использовать ее в качестве обычной LLM, как мы делали ранее.
- Конвертировать и запустить локально на Ollama. Если вы хотите запустить конечную модель на локальном Ollama-сервере, можно конвертировать конечную модель в формат GGUF. После успешной конвертации она запускается в Ollama так же, как и любая стандартная LLM.

Последние шаги подразумевают настройку конфигураций, специфичных для вашего окружения. Они, как правило, зависят от индивидуальных потребностей и актуальны для производственных или исследовательских целей, поэтому мы опустили детали. Более подробную информацию об этом процессе вы можете найти в нашем соответствующем блоге на сайте.

Заключение

В заключение следует отметить, что файн-тюнинг больших языковых моделей имеет решающее значение для адаптации предварительно обученных моделей к специализированным задачам. Используя такие передовые методы, как полный файн-тюнинг, параметрически эффективный файн-тюнинг (PEFT), LoRA и QLoRA, разработчики оптимизируют производительность базовых моделей и уменьшают их зависимость от ресурсов.

Файн-тюнинг не только расширяет возможности модели по решению конкретных задач, но и сохраняет уже существующие знания, что позволяет эффективно развертывать ее в различных приложениях. По мере неуклонного развития больших языковых моделей (LLM) передовые методы и инструменты файн-тюнинга позволяют еще больше повысить точность и получить более надежные результаты.

ГЛАВА 6

ОБРАБОТКА И ГЕНЕРАЦИЯ ИЗОБРАЖЕНИЙ С ПОМОЩЬЮ LLM

В последние годы в области искусственного интеллекта (ИИ) и машинного обучения произошел значительный прогресс. Были разработаны различные инструменты и технологии, радикальным образом изменившие наш подход к решению сложных задач. Некоторые мощные ИИ-модели завоевали популярность среди исследователей и разработчиков благодаря своим возможностям в области компьютерного зрения и обработки изображений. В этой главе мы дадим пошаговое руководство по использованию LLM для задач из области компьютерного зрения и обработки изображений.

Начнем с так называемого «компьютерного зрения» (*Image visioning*) или распознавания изображений. Распознавание изображений — одно из важнейших направлений ИИ, находящее практическое применение в самых различных областях, таких как здравоохранение, финансы и образование. Способность анализировать и распознавать визуальные данные предоставляет множество преимуществ, в том числе для улучшения процесса принятия решений, повышения качества обслуживания клиентов и эффективности работы. Но часто задачи в этой сфере кажутся довольно сложными, особенно при отсутствии опыта. Именно в таких случаях на помощь приходит LLaVA-v1.6 — мощный инструмент, упрощающий процесс распознавания изображений и позволяющий получать точные результаты.

Распознавание изображений (Image visioning)

LLaVA-v1.6 — это ИИ-модель глубокого обучения, разработанная группой исследователей и специализирующаяся на обработке естественного языка и распознавании изображений. Она предназначена для анализа и распознавания визуальных данных, включая изображения и видео. Архитектура модели представляет собой сочетание сверточных нейронных сетей (CNN) и рекуррентных нейронных сетей (RNN), извлекающих из изображений признаки и генерирующих текстовые описания.

Разработка LLaVA-v1.6 началась в 2019 году, и за время своего существования модель претерпела несколько обновлений. Последняя версия (v1.6) отличается повышенной производительностью и точностью. Среди ключевых особенностей LLaVA-v1.6 можно назвать следующие.

Анализ изображений. LLaVA-v1.6 способна анализировать изображения и извлекать из них признаки объектов, сцен и действий.

Генерация текста. Модель умеет генерировать текстовые описания на основе изображений, в том числе подписи и резюме.

Обнаружение объектов. LLaVA-v1.6 способна обнаруживать на изображениях различные объекты, включая людей, животных и транспортные средства.

Возможности и функциональные особенности LLaVA-v1.6.

LLaVA-v1.6 обладает многочисленными возможностями и функциями, что делает ее универсальным инструментом для различных приложений. Среди основных возможностей LLaVA-v1.6 можно выделить следующие.

Классификация изображений. LLaVA-v1.6 способна классифицировать изображения по различным категориям, таким как объекты, сцены и действия.

Обнаружение объектов. Модель способна обнаруживать объекты на изображениях, включая людей, животных и транспортные средства.

Создание подписей к изображениям. LLaVA-v1.6 способна генерировать текстовые подписи к изображениям, включая описания объектов, сцен и действий.

Архитектура LLaVa

По сути, LLaVa — это умный помощник, который может одновременно читать и просматривать изображения, выделять из них значимую для пользователя информацию и давать полезные ответы. С технической точки зрения архитектура LLaVa выглядит так, как показано на схеме ниже.

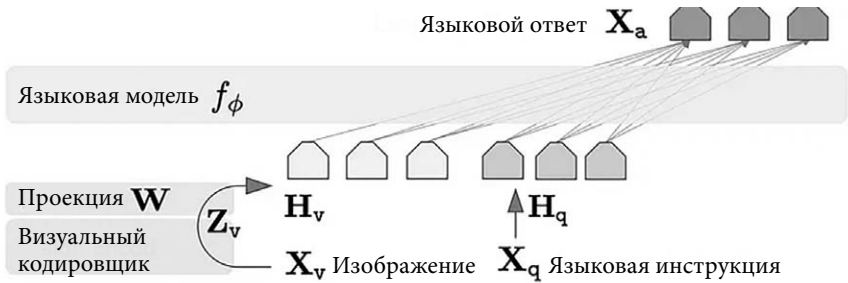


Иллюстрация 6.1

Эта позаимствованная из официальной документации LLaVA-v1.6 схема показывает процесс взаимодействия с моделью LLaVa, при котором пользователь вводит в систему изображение вместе с содержащим инструкции текстовым промптом. Текстовый промпт обрабатывается непосредственно токенизатором языковой модели, а изображение сначала преобразуется в токены с помощью визуального кодировщика. Затем эти токены отображаются в пространство языковой модели.

Пошаговый пример. Использование LLaVA-v1.6 для распознавания изображений.

В этом разделе мы приведем пошаговый пример использования LLaVa-v1.6 для таких задач, как распознавание изображений. Рассмотрим несколько простых примеров, связанных с составлением описаний изображений и извлечением из них программного кода или текста.

Обязательные условия.

Программное обеспечение. Для того чтобы пользоваться моделью LLaVav1.6, нужно установить и локально настроить Ollama. Инструкции по установке Ollama приведены в главе 1.

Оборудование. Для ускорения обработки рекомендуется использовать компьютер с выделенной видеокартой, но для образовательных целей достаточно будет 8-ядерного процессора.

Шаг 1. Установка и настройка LLaVA-v1.6.

В терминале выполните следующую команду:

```
ollama run llava:34b
```

Она загрузит и установит LLava 1.6 LLM в вашу локальную установку Ollama. Учтите, что для работы модели llava:34b требуется более 20 ГБ свободной памяти.

Шаг 2. Предварительная обработка и подготовка визуальных данных.

Создайте каталог для хранения изображений, выполнив команду:

```
mkdir images
```

Затем скопируйте в только что созданный каталог несколько любых изображений. Это можно сделать с помощью команды `cp`, например:

```
cp /path/to/your/images/* images/
```

(где «/path/to/your/images/» — это путь к файлам изображений).

Шаг 3. Анализ изображений с помощью LLaVA-v1.6.

Теперь можно загрузить изображения в модель LLaVa-v1.6 и начать задавать вопросы о них. Начнем с промпта `what is in the image` («что изображено на картинке»).

1. Описание изображений.



Иллюстрация 6.2

Промпт: `what is in the image /path_to_the_image` («что изображено на картинке /путь_к_изображению»).

В терминале, где запущена Ollama, выполните следующую команду:

```

shamim@ubuntu-22:~$ ollama run llava:34b
>>> hi
Hello! How can I help you today?
For example, I could give you some tips on how to improve your coding skills or assist you with a specific project. Just let me know what's on your mind and I'll do my best to assist you.
Best regards,

>>> what is in the image /home/shamim/images/IMG_2084.jpg
Added image '/home/shamim/images/IMG_2084.jpg'

```

Иллюстрация 6.3

Через непродолжительное время LLaVa выдает следующий ответ:

The image shows a piece of furniture with a metal frame, which appears to be the footboard or headboard of a bed. The metal is dark in color and there are visible bolts and other hardware components that suggest the bed frame is sturdy and securely assembled. There's no text present in this image. If you have any questions about the type of bed frame or need advice on furniture, feel free to ask!

«На картинке изображен предмет мебели с металлическим каркасом, скорее всего, изголовьем или спинкой кровати. Металл темного цвета, видны болты и другие элементы креплений, что говорит о том, что каркас кровати прочный и надежно собран. Текст на этом изображении отсутствует. Если у вас есть вопросы о типе каркаса кровати или вам нужен совет по мебели, не стесняйтесь спрашивать!»

Результат впечатляет!

2. Распознавание диаграммы или схемы.

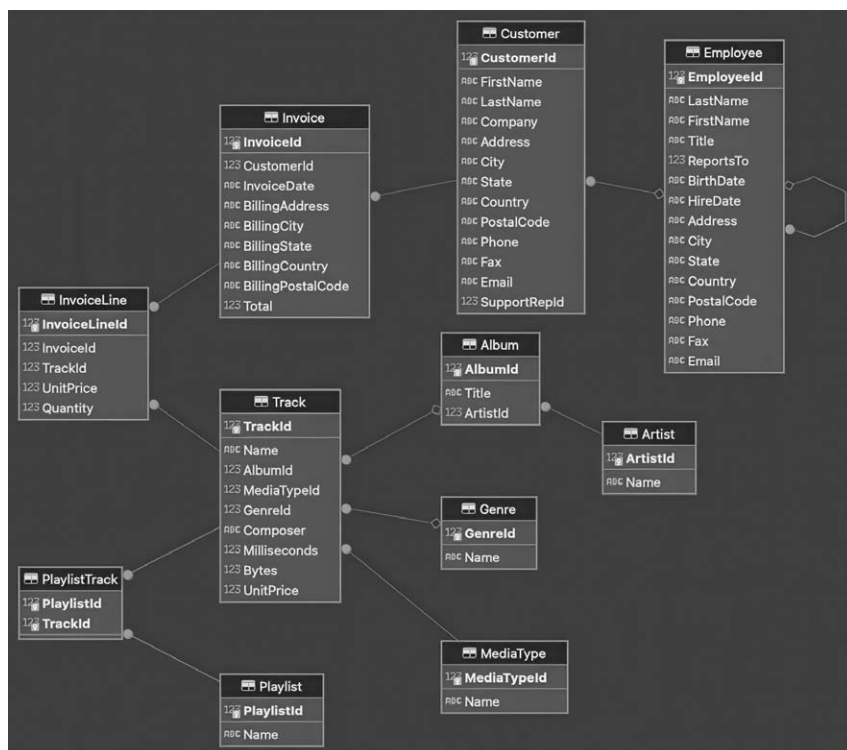


Иллюстрация 6.4

Промпт: describe the diagram /path_to_the_image («опиши диаграмму /путь_к_изображению»).

Результат:

The diagram appears to be a flowchart or an organizational chart, possibly representing a system or process with various components and data fields. It includes nodes connected by lines that suggest relationships or workflow between the elements. Each node is labeled with a title such as "Customer", "Employee", "Track", and others that seem to indicate different categories or entities within the system being represented. The diagram uses a dark background with white text and light-colored nodes, which creates a contrast for readability. The lines connecting the nodes are likely to represent the flow of information

or the sequence of steps in the process. Each node has fields on it that seem to correspond to specific data points, such as customer ID, name, email, and so forth, suggesting that this diagram might be related to a software application or database schema where different entities have their own attributes and relationships. The image does not provide enough context to determine the exact nature of the system or process being represented in the flowchart, but it is clear that it is designed to show the structure and interconnectivity of various elements within that system.

«Диаграмма выглядит как блок-схема или организационная схема, возможно, представляющая систему или процесс с различными компонентами и полями данных. Она включает узлы, соединенные линиями, указывающими на отношения или на рабочий процесс между элементами. Каждый узел имеет название, например «Клиент», «Сотрудник», «Трек» и другие, указывающие, по всей видимости, на различные категории или сущности в представленной системе. Для диаграммы используется темный фон с белым текстом и светлыми узлами, что создает контраст для удобства чтения. Линии, соединяющие узлы, скорее всего, представляют поток информации или последовательность шагов в процессе. В каждом узле есть поля, которые, по-видимому, соответствуют определенным точкам данных, таким как идентификатор клиента, имя, электронная почта и т.д., что позволяет предположить, что эта диаграмма может быть связана с программным обеспечением или схемой базы данных, в которой различные сущности имеют свои собственные атрибуты и отношения.

Изображение не предоставляет достаточного контекста, чтобы определить точную природу системы или процесса, представленного на блок-схеме, но ясно, что она призвана показать структуру и взаимосвязь различных элементов внутри этой системы».

Попробуем распознать еще одну диаграмму.

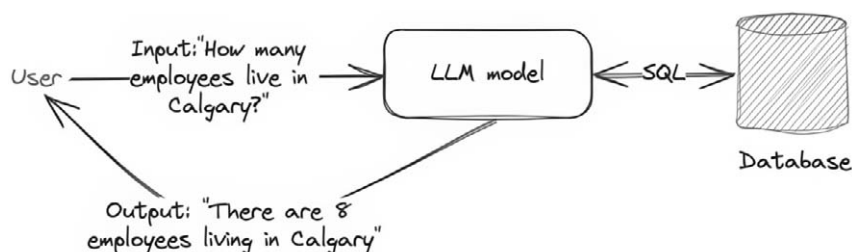


Иллюстрация 6.5

Промпт: Describe the meaning of the diagram /path_to_the_image («опиши смысл диаграммы /путь_к_изображению»).

Результат:

The image you've shared appears to be a flowchart or diagram that represents an example of how data processing works. It seems to illustrate a simple system where input data is processed and results are generated based on predefined conditions. Here's a breakdown of the elements in the diagram:

1. The input statement reads "How many employees live in Calgary?" This indicates that the system is designed to process information about employees who reside in Calgary.
2. There are two nodes labeled as 'Employee', which suggests that there might be multiple employee records or data points within the system.
3. Each 'Employee' node has fields such as 'ID', 'Name', and 'Location'. The 'Location' field is presumably used to store where each employee resides.
4. The diagram includes an 'Employee' node with the 'Location' field set to 'Calgary', which represents a specific data point within the system.
5. There are two decision points in the flowchart: 'Model 1' and 'Model 2'. These likely represent different algorithms or methods used for processing the input data.
6. The output statement at the bottom of the diagram reads "There are 3 employees living in Calgary". This suggests that after processing the input data, the system

has determined that there are three employees who live in Calgary. Overall, the meaning of this diagram is to illustrate a simple example of how an input question can be processed through a predefined model or algorithm to generate an output answer based on the available data within a system. It's a basic representation of data processing and does not include specific details about the inner workings of the models or algorithms involved in the process.

Изображение, которым вы поделились, по всей видимости, представляет собой блок-схему или диаграмму, отражающую пример того, как происходит обработка данных. Похоже, она иллюстрирует простую систему, в которой входные данные обрабатываются и результаты генерируются на основе заранее определенных условий. Вот разбивка элементов на диаграмме:

- 1. Вводные данные: «Сколько сотрудников живут в Калгари?». Это указывает на то, что система предназначена для обработки информации о сотрудниках, проживающих в Калгари.*
- 2. Есть два узла, помеченные как «Employee» («Сотрудник»), что говорит о том, что в системе может быть несколько записей или точек данных о сотрудниках.*
- 3. Каждый узел «Employee» имеет такие поля, как «ID», «Name» и «Location». Поле «Location», предположительно, используется для хранения данных о местонахождении каждого сотрудника.*
- 4. На диаграмме есть узел «Employee», в котором поле «Location» имеет значение «Calgary», что представляет собой конкретную точку данных в системе.*
- 5. На диаграмме есть две точки принятия решений: «Модель 1» и «Модель 2». Вероятно, они представляют собой различные алгоритмы или методы, используемые для обработки входных данных.*
- 6. В нижней части диаграммы приведен вывод: «В Калгари проживают 3 сотрудника». Это говорит о том, что после обработки входных данных система определила, что три сотрудника проживают в Калгари.*

В целом смысл этой диаграммы заключается в том, чтобы проиллюстрировать простой пример того, как входной вопрос можно обрабатывать с помощью заранее определенной модели или алгоритма

для создания выходного ответа на основе имеющихся в системе данных. Это базовая схема обработки данных, не отражающая подробностей о внутреннем устройстве моделей или участвующих в процессе алгоритмов.

Результат весьма впечатляющий и точный.

3. Извлечение кода из изображения.

```
while (true) {

    ByteBuffer msg = stream.readPending();

    if (msg == null) {
        TimeUnit.MILLISECONDS.sleep(10L);
        continue;
    }

    int offset = msg.arrayOffset();
    byte[] source = msg.array();
    int length = source.length - offset;
    logger.info(new String(source, offset, length));

    //feedback by LOG sequence Number
    stream.setAppliedLSN(stream.getLastReceiveLSN());
    stream.setFlushedLSN(stream.getLastReceiveLSN());

}
```

Иллюстрация 6.6

Промпт: extract code from the image /path_to_the_image («извлеки код из изображения /путь_к_изображению»).

Результат:

Python

```
while True:
    byte_buffer.msg = stream_read_pending()
    byte_buffer.stream.sleep(100)
    if byte_buffer.msg == None:
        continue
    time.sleep(1 / 60)
    for i in range(1,12):
        byte_buffer.message = message + " " + str(i)
```

```

source_sequence.set("seq_" + byte_buffer.msg +
"_" + str(len(byte_buffer.message)))
if source_sequence.length < 0:
    print("Error: Source Sequence Length is less
than zero")

```

Любопытно, что LLaVa распознала псевдокод, но отобразила его на языке Python.

4. Распознавание текста в изображении.

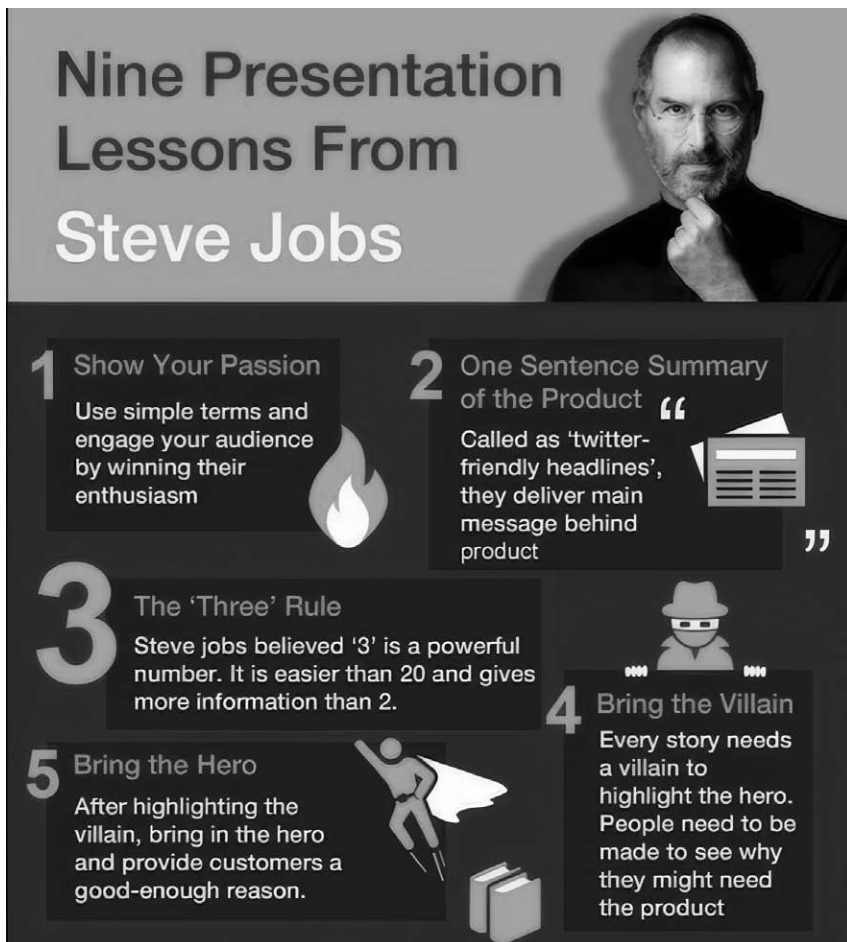


Иллюстрация 6.7

Промпт: extract text from the image path_to_the_image («извлеки текст из изображения /путь_к_изображению»).

Результат:

Nine Presentation Lessons **from Steve Jobs**

1. Show Your Passion
2. One Sentence Summary of Product
3. The Third Rule
4. Use Simplicity to Win the Battle
5. Don't Hide Behind Bullet Points
6. Make Everything Personal **and** Relatable
7. Create a Hero **for** Customers
8. Tell Them Stories About How Your Product Changes Lives
9. Keep the Data Clean

1. Bring the Hero After a Short Break
2. Every Story Needs an Ending
3. Don't Read from Notes – Prepare, Practice, and Preach
4. Be Yourself **and** Help You Give Your Best Performance
5. Prepare to Give a Good Presentation
6. Use a Visual Aid **for** Better Understanding of the Product
7. Tell Them About How Your Product Changes Lives
8. Bring the Hero After a Short Break
9. Keep the Data Clean
10. Don't Read from Notes – Prepare, Practice, and Preach
11. Be Yourself **and** Help You Give Your Best Performance
12. Use a Visual Aid **for** Better Understanding of the Product
13. Create a Hero **for** Customers
14. Bring the Hero After a Short Break
15. Every Story Needs an Ending
16. Don't Hide Behind Bullet Points
17. Make Everything Personal **and** Relatable
18. Use Simplicity to Win the Story
19. Show Your Passion
20. One Sentence Summary of Product

21. The Third Rule
22. Bring the Hero After a Short Break
23. Don't Hide Behind Bullet Points
24. Make Everything Personal and Relatable

Девять уроков презентаций от Стива Джобса

1. Покажите свою страсть
2. Краткое описание продукта в одном предложении
3. Третье правило
4. Выигрывайте битву простотой
5. Не прячьтесь за списками
6. Сделайте все личным и переживаемым
7. Создайте героя для клиентов
8. Расскажите им истории о том, как ваш продукт меняет жизнь.
9. Сохраняйте данные чистыми

1. После короткого перерыва представьте героя
2. Каждой истории нужен конец
3. Не читайте с листа, готовьтесь, практикуйтесь и проповедуйте
4. Будьте собой и помогайте себе выступить наилучшим образом
5. Подготовьтесь к хорошей презентации
6. Используйте наглядные пособия для лучшего понимания продукта
7. Расскажите о том, как ваш продукт меняет жизнь людей
8. После короткого перерыва представьте героя
9. Сохраняйте данные чистыми
10. Не читайте с листа, готовьтесь, практикуйтесь и проповедуйте
11. Будьте собой и помогайте себе выступить наилучшим образом
12. Используйте наглядные пособия для лучшего понимания продукта
13. Создайте героя для клиентов
14. После короткого перерыва представьте героя
15. Каждой истории нужен конец
16. Не прячьтесь за списками
17. Сделайте все личным и переживаемым
18. Выиграйте историю простотой
19. Покажите свою страсть
20. Краткое описание продукта в одном предложении

- 21. Третье правило
- 22. После короткого перерыва представьте героя
- 23. Не прячьтесь за списками
- 24. Сделайте все личным и переживаемым

Изображение имеет большой размер, поэтому LLaVa требуется несколько минут для его кодирования и декодирования. Однако результат впечатляет.

Внедрение LLaVA в приложение для анализа изображений.

Пользоваться возможностями модели LLaVA для анализа изображений и внедрять ее в свои проекты можно разными способами. Самый простой — воспользоваться Ollama Python API, но для этого подойдут и другие фреймворки, такие как LangChain. Ниже мы покажем программу на Python, определяющую функцию для анализа изображения и создания текстового описания на основе заданного промпта, с ее пошаговым описанием.

Для начала убедитесь в том, что в вашей среде разработки установлены и настроены нужные библиотеки. Как обычно, для разработки приложения мы воспользуемся средой Conda и инструментом JupyterLab.

Шаг 1. Установите нужные библиотеки.

С помощью показанных ниже команд установите нужные библиотеки Python. Перед их выполнением убедитесь, что вы находитесь в среде Conda.

```
!pip install —upgrade \ ollama \ pandas \ Pillow —quiet
```

Совет

Учтите, что если вы пользуетесь удаленным Ollama-сервером, то вам понадобится экспортировать следующую переменную окружения: `export OLLAMA_HOST=»YOUR_REMOTE_OLLAMA_IP_ADDRESS»`.

Замените фрагмент YOUR_REMOTE_OLLAMA_IP_ADDRESS на реальный IP-адрес своего сервера Ollama.

Шаг 2. Импорт и определение функций.

```
from PIL import Image
from io import BytesIO
import glob
import pandas as pd
import os
import BytesIO
```

обработка изображений

```
def process_image(image_file_path, prompt):
    print(f"\nProcessing {image_file_path} file \n")
```

Эти строки импортируют следующие модули: Image из библиотеки Python Imaging Library (PIL) и BytesIO из библиотеки io. Затем определяется функция process_image, принимающая два аргумента: image_file_path и prompt.

Шаг 3. Открытие файла и вывод изображения.

```
image = Image.open(image_file_path)
display(image)
```

Шаг 4. Преобразование изображения в байтовый формат.

```
with image as img:
    with BytesIO() as buffer:
        img.save(buffer, format='PNG')
        image_bytes = buffer.getvalue()
```

Этот фрагмент преобразует файл изображения в байтовый формат. Контекстный менеджер открывает изображение, сохраняет его в буфер в формате PNG и извлекает из буфера байты изображения.

Шаг 5. Генерация описания изображения.

```
# Генерация описания данного изображения

for response in generate(model='llava:34b',
    prompt=prompt,
    images=[image_bytes],
    stream=True):

# Вывести ответ на консоль

print(response['response'], end='', flush=True)
```

Этот фрагмент кода генерирует описание изображения с помощью модели LLaVA, передает ответ в режиме реального времени и выводит его по частям на консоль.

Шаг 6. Вызов функции.

```
file_name="./Text2SQL.png" prompt="Describe the diagram"
# invoke process image function process_image(file_name,
prompt)
```

Последний фрагмент кода указывает имя файла (file_name) для пути к файлу изображения и промпт для строки описания промпта. Затем с этими аргументами вызывается функция process_image.

Совет

Полный исходный код программы доступен в репозитории GitHub. Если запустить эту программу в блокноте Jupyter, то результат должен походить на показанный ниже.



Иллюстрация 6.8

Результат: Изображение, которым вы поделились, по всей видимости, представляет собой блок-схему или диаграмму, отражающую пример того, как происходит обработка данных. Похоже, она иллюстрирует простую систему, в которой входные данные обрабатываются и результаты генерируются на основе заранее определенных условий. Вот разбивка элементов на диаграмме:

1. Вводные данные: «Сколько сотрудников живут в Калгари?». Это указывает на то, что система предназначена для обработки информации о сотрудниках, проживающих в Калгари.
2. Есть два узла, помеченные как Employee («Сотрудник»), что говорит о том, что в системе может быть несколько записей или точек данных о сотрудниках.
3. Каждый узел «Employee» имеет такие поля, как «ID», «Name» и «Location». Поле «Location», предположительно, используется для хранения данных о местонахождении каждого сотрудника.
4. На диаграмме есть узел «Employee», в котором поле «Location» имеет значение «Calgary», что представляет собой конкретную точку данных в системе.
5. На диаграмме есть две точки принятия решений: «Модель 1» и «Модель 2». Вероятно, они представляют собой различные алгоритмы или методы, используемые для обработки входных данных. 6. В нижней части диаграммы приведен вывод: «В Калгари проживают 3 сотрудника». Это говорит о том, что после обработки входных данных система определила, что три сотрудника проживают в Калгари.

В целом, смысл этой диаграммы заключается в том, чтобы проиллюстрировать простой пример того, как входной вопрос можно обрабатывать с помощью заранее определенной модели или алгоритма для создания выходного ответа на основе имеющихся в системе данных. Это базовая схема обработки данных, не отражающая подробностей о внутреннем устройстве моделей или участвующих в процессе алгоритмов. Итак, LLaVA-v1.6 — это весьма мощная модель искусственного интеллекта, предлагающая множество возможностей и функций для решения задач, связанных с анализом и обработкой изображений. Благодаря своей способности анализировать и понимать визуальные данные LLaVA-v1.6 предусматривает множество применений в различных областях, включая здравоохранение, финансы и образование. Несмотря на некоторые ограничения, она представляет собой ценный инструмент для исследователей и разработчиков, которые стремятся освоить возможности ИИ для своих задач.

Обработка изображений

Последние достижения в области ИИ позволяют генерировать изображения на основе текстовых описаний, и один из наиболее ярких инструментов такой генерации — модель **OpenJourney**. В данном разделе мы расскажем вам о процессе создания изображений по текстовым описаниям (text-to-image) с помощью LLM OpenJourney и приведем примеры с пошаговыми объяснениями.

Что такое OpenJourney?

OpenJourney — это обученная модель MidJourney с открытым исходным кодом, которую можно использовать в качестве локальной хостинговой версии LLM. С технической точки зрения это стабильная модель Diffusion 1.5, дополнительно настроенная компанией PromptHero на наборе данных из более чем 124 000 изображений Midjourney v4. Основная задача модели — создавать высококачественные изображения на основе текстовых описаний. В настоящее время OpenJourney — одна из самых популярных моделей преобра-

зования текста в изображение (text-to-image), широко используемая исследователями, разработчиками и иллюстраторами.

OpenJourney получила широкое распространение в разных отраслях, включая искусство, дизайн, рекламу и образование. Эту модель использовали для создания различных приложений.

- Дизайнерских и оформительских. OpenJourney применялась в художественных и дизайнерских проектах, в том числе и при создании картин, скульптур и инсталляций.
- Рекламных. OpenJourney использовалась для рекламных кампаний, в том числе и при создании изображений продуктов и рекламных материалов.
- Образовательных. OpenJourney использовалась для создания образовательных материалов, в том числе учебников и онлайн-курсов.

Как устроен процесс преобразования текста в изображение.

Как уже говорилось, модель Openjourney основана на Stable Diffusion 1.5, представляющей собой модель генерации текста в изображение и создающей изображения на основе текстовых описаний (промптов). В своей основе этот процесс заключается в постепенном уменьшении шума изображения до тех пор, пока оно не будет соответствовать заданному описанию. Вот краткая схема этого процесса.

1. Кодирование текста. Поступающий на вход текст преобразуется в числовое представление с помощью предварительно обученной языковой модели.
2. Диффузия. Модель начинает со случайного шумового изображения и итеративно очищает его, ориентируясь на закодированный текст. На каждом шаге диффузии изображение приближается к желаемому результату с добавлением большего количества деталей, соответствующих текстовому промпту.

3. Генерация изображения. На заключительном этапе происходит постобработка изображения за множество ходов, в течение которых оно становится более четким. В итоге получается полностью сформированное изображение, соответствующее входному описанию.

Схема этого процесса генерации показана ниже.

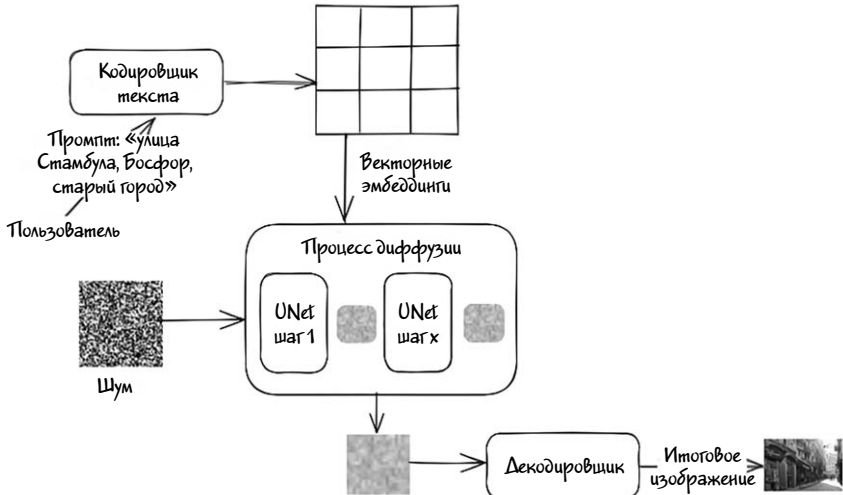


Иллюстрация 6.9

Материалы для более подробного изучения модели Stable Diffusion указаны в разделе ссылок.

Данные на входе и выходе модели.

Модель OpenJourney принимает на вход различные данные, включая текстовый промпт, необязательное начальное изображение, размеры изображения и дополнительные параметры для управления входными данными.

На основе этих входных данных она генерирует одно или несколько изображений.

- Вход.
 - › Промпт. Текстовый запрос, описание изображения, которое хочет получить пользователь.
 - › Изображение. Необязательное начальное изображение, на основе которого будут генерироваться варианты.
 - › Ширина/Высота. Желаемые размеры выходного изображения.
 - › «Сид» (Seed). Случайный параметр контроля над генерацией изображения.
 - › Планировщик (Scheduler). Планировщик удаления шума, который будет применяться при обработке изображения.
 - › Количество изображений (Num Outputs). Количество генерируемых изображений.
 - › Масштаб управления (Guidance Scale). Коэффициент для генерации без использования классификатора.
 - › Негативный промпт. Текстовое описание того, чего не должно быть в итоговом изображении.
 - › Сила промпта (Prompt Strength). Степень, с которой система должна придерживаться текстовой подсказки.
 - › Количество шагов инференса (Num Inference Steps). Количество шагов по уменьшению шума.
- Выход.
 - › Изображение (изображения). Одно или несколько сгенерированных изображений, возвращаемых в виде массива изображений.

Далее мы приведем пошаговый пример генерации изображений с помощью модели openjourney-v4 по текстовому описанию и постараемся более подробно рассмотреть процессы кодирования текста, генерации изображения и постобработки результата.

Внимание

Генерация изображения — это ресурсоемкий процесс. Если в вашей системе имеются графические процессоры (GPU), совместимые

с CUDA, рекомендуем установить последнюю версию драйвера с сайта Nvidia, чтобы в полной мере воспользоваться производительностью этих процессоров. Кроме того, можно воспользоваться возможностями сервиса Google Colab, задействовав для генерации изображений графический процессор T4, что ускорит процесс. Полный пример генерации изображений в Google Colab с помощью графического процессора T4 находится по этой ссылке.

Так как высокопроизводительное оборудование имеется не у всех, мы также приведем пример с использованием вычислительной мощности центрального процессора (CPU). К счастью, большая часть кода практически одинакова как для CPU, так и для GPU, что облегчает адаптацию к конкретной конфигурации.

Шаг 1. Установка необходимых библиотек.

Для того чтобы воспользоваться моделью OpenJourney, вам понадобится Python и несколько библиотек. Сначала убедитесь, что у вас установлен Python, а затем установите нужные пакеты:

```
! pip install --upgrade \\  
diffusers \\  
transformers \\  
safetensors \\  
sentencepiece \\  
accelerate \\  
bitsandbytes \\  
torch \\  
huggingface_hub --quiet  
!pip install ipywidgets
```

Этот псевдокод устанавливает необходимые для программы библиотеки. Знак ! в начале строки говорит о том, что эти команды выполняются в командной оболочке операционной системы, а не в Python.

Учтите, что локальная установка OpenJourney может сопровождаться трудностями из-за различий в операционных системах и кон-

фигурации. Поэтому на следующих шагах для доступа к модели OpenJourney мы воспользуемся API хаба Hugging Face.

Шаг 2. Авторизация на хабе Hugging Face.

Создайте новый Python-скрипт или блокнот и импортируйте нужные библиотеки:

```
import os
from huggingface_hub import login

login(token=os.environ.get("HF_TOKEN"))
```

Эти строки выполняют следующее.

- Импортируют необходимые модули.
- Осуществляют авторизацию на хабе Hugging Face Hub с помощью токена HF_TOKEN, хранящегося в переменной окружения.

Шаг 3. Импорт нужных модулей и проверка доступности CUDA.

```
from diffusers import DiffusionPipeline,
DPMolverMultistepScheduler, AutoencoderKL
import torch

# Проверка доступности CUDA
torch.cuda.is_available()
```

Эти строки выполняют следующее.

- Импортируют модули DiffusionPipeline, DPMolverMultistepScheduler и AutoencoderKL из библиотеки diffusers, а также библиотеку torch.
- Проверяют наличие CUDA (поддержка GPU).

Шаг 4. Настройка конвейера диффузии.

Инициализируйте конвейер для модели OpenJourney. Проследите за тем, чтобы у вас был доступ к файлам модели или была возможность их загрузить:

```
model_id = 'prompthero/openjourney-v4'
pipe = DiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float32
)
pipe.scheduler = DPMSolverMultistepScheduler.from_
config(pipe.scheduler\
.config)
pipe.vae = AutoencoderKL.from_pretrained("stabilityai-
vae-ft-mse", torch_dtype=torch.float32)
```

Эти строки выполняют следующие действия.

- Загружают предварительно обученную модель OpenJourney в переменную конвейера.
- Настраивают планировщик и VAE (вариационный автокодировщик) для конвейера.

Шаг 5. Настройка генерации изображений.

Придумайте текстовый промпт с описанием изображения, которое вы хотите сгенерировать, и вставьте его в строку промпта. Например, следующее. Промпт: «Улица Стамбула, старый город, лето, солнечно, гипердетализация, кинематографическое качество, атмосфера, шедевр, гипердетализация 8k», негативный промпт: «размытие дымкой, карандаши, ручки, пальцы»:

```
# Входные параметры генерации изображения
import random
```

```
prompt = 'Istanbul street, bosforas, old city, summer,
sunny, hyper-detailed, comprehensive cinematic,
Atmosphere, Masterpiece, hyperdetailed
8k '
```

```
negative_prompt = 'blur haze, pencils, pens, fingers'
```

```
num_steps = 20
# количество вариантов
num_variations = 1
prompt_guidance = 9
dimensions = (400, 600)
random_seeds = [random.randint(0, 65000) for _ in
range(num_variations)]
```

Этот фрагмент кода выполняет следующее.

- Импортирует модуль `random`.
- Задаёт промпт, количество шагов для генерации изображения, количество вариантов, масштаб управления, размеры и случайные «сиды» для генерации нескольких изображений. При желании сгенерировать больше изображений, измените параметр `num_variations`.

Шаг 6. Генерация изображений.

```
images = pipe(prompt= num_variations * [prompt],
num_inference_steps=num_steps,
guidance_scale=prompt_guidance,
height = dimensions[0],
width = dimensions[1],
Negative_Prompt = negative_prompt,
num_outputs = 1,
generator = [torch.Generator().manual_seed(i) for i in
random_seeds]
).images
print ('Images: ', len(images))
```

Этот фрагмент кода выполняет следующее.

- Генерирует изображения на основе заданного промпта и настроек.
- Использует случайные «сиды» для вариативности генерируемых изображений.



Иллюстрация 6.10

После выполнения этого кода на основе заданного промпта будет сгенерировано изображение улицы в Стамбуле, представляющее собой, по сути, художественное произведение, созданное LLM. Учтите, что конечный результат может отличаться в зависимости от аппаратного обеспечения компьютера и настроек конфигурации. Тем не менее на этом изображении должна быть показана стамбульская улица в солнечный летний день. Весь исходный код для блокнота Jupyter доступен на GitHub. Ниже приведены советы по генерации изображения.

Советы по улучшению процесса обработки изображений.

1. **Включайте больше деталей в описание.** Чем больше деталей вы предоставите в текстовом промпте, тем лучше модель сможет

понять и сгенерировать нужное вам изображение. Некоторые идеи по составлению промптов описаны на этом веб-сайте.

2. Экспериментируйте. Попробуйте разные промпты и их варианты, чтобы понять, как на них реагирует модель.

3. Рассмотрите возможность файн-тюнинга. Если у вас есть особые требования, рассмотрите возможность файн-тюнинга модели с помощью пользовательских наборов данных.

OpenJourney — это мощный инструмент, способный изменить различные отрасли благодаря своей способности генерировать высококачественные изображения на основе текстовых описаний (промптов). Выполнив описанные в данном разделе шаги, вы сможете создавать собственные изображения и применять описанную модель для самых разных задач.

Ссылки

1. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). Highresolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (Ромбах, Р., Блаттманн, А., Лоренц, Д., Эссер, П., и Оммер, Б. (2022). Синтез изображений высокого разрешения с помощью моделей скрытой диффузии. Статья опубликована в «Материалах конференции IEEE/CVF по компьютерному зрению и распознаванию образов») (страницы 10684–10695).

2. Zhang, L., & Agrawala, M. (2023). Adding conditional control to text-to-image diffusion models (Чжан, Л. и Агравала, М. «Добавление условного контроля к моделям диффузионного преобразования текста в изображение»). Архив препринтов arXiv:2302.05543.

Заключение

В этой главе была представлена модель LLaVA, способная отвечать на вопросы, связанные с изображениями, описывать диаграммы

и схемы, а также составлять подробные описания сложных систем. Показанные в главе фрагменты кода послужат практическим руководством по интеграции LLM LLaVA в практические приложения, связанные с анализом и обработкой изображений. С помощью этой модели разработчики откроют для себя новые возможности в сфере обработки изображений.

Также мы рассмотрели возможности модели OpenJourney, создающей высококачественные изображения на основе текстовых промптов, и описали процесс создания приложений с использованием API OpenJourney для генерации убедительно реалистичных изображений.

ГЛАВА 7

РАЗРАБОТКА И ИСПОЛЬЗОВАНИЕ ИИ-АГЕНТОВ

В данной книге мы рассматриваем широкий спектр тем, от фундаментальных концепций до реальных приложений, включая создание локальной среды для больших языковых моделей (LLM), расширение их возможностей с помощью RAG, тонкую настройку, генерацию SQL-запросов и обработку изображений. В этой главе мы опишем роль LLM в качестве персональных помощников, повышающих нашу продуктивность в повседневной жизни.

Разработка ИИ-агентов — одно из передовых направлений в области искусственного интеллекта. ИИ-агенты — это комплексные системы, использующие генеративные модели для выполнения сложных задач, связанных с обработкой естественного языка и со взаимодействием с пользователем на естественном языке. Они создаются для имитации взаимодействия с человеком, для автоматизации процесса принятия решений и для выполнения широкого спектра прикладных задач — от создания контента до персонализации пользовательского опыта.

По состоянию на 2024 год огромное внимание ИИ-агентам уделяют такие крупнейшие IT-гиганты, как Google, Microsoft и Salesforce, привлеченные возможностями автоматизации сложных рабочих процессов для своих клиентов. По мнению представителей компании Google, в ближайшем будущем ИИ-агенты станут основным средством управления сложными рабочими процессами и координации работы нескольких систем.

По мере того как бизнес-приложения становятся все более зависимыми от взаимосвязи различных систем, ИИ-агенты смогут пользоваться возможностями LLM для автономного или полуавтономного выполнения задач. Такие агенты оптимизируют рабочие процессы благодаря способностям языковых моделей понимать и генерировать текст на естественном языке и их сочетанию с внешними инструментами или средами, предоставляющими возможности дополнительной памяти, планирования и взаимодействия с другими инструментами.

Будущее ИИ-агентов

Предполагается, что в перспективе ИИ-агенты должны последовательно пройти ряд этапов развития по мере их интеграции в корпоративные проекты. Благодаря автоматизации простых и сложных задач ИИ-агенты освободят людей от рутинной работы, чтобы те смогли сосредоточиться на творческом решении проблем и на принятии стратегических решений.

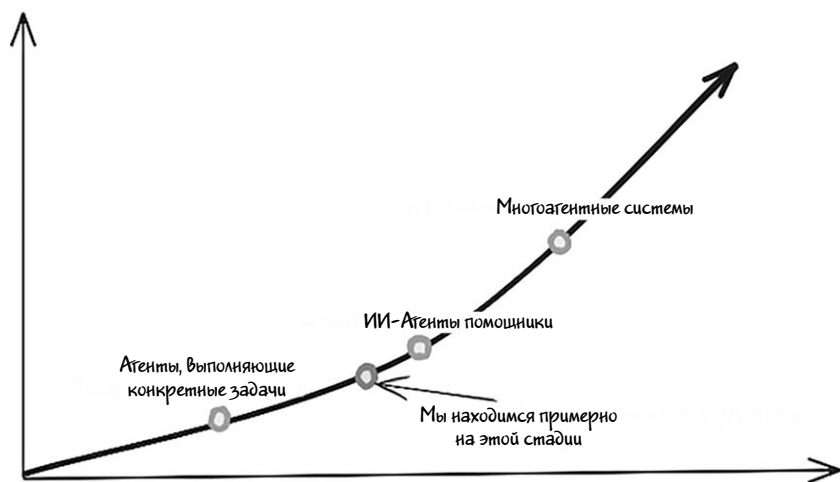


Иллюстрация 7.1

По наблюдениям Google Cloud, эта трансформация будет происходить в три ключевых этапа.

1. Агенты, выполняющие конкретные задачи. ИИ-агенты первой волны будут выполнять специализированные, четко сформулированные задачи. Такие агенты предназначены для выполнения повторяющихся или рутинных процессов, и благодаря им повысится эффективность в конкретных областях применения. Так, например, ИИ-агент может обрабатывать запросы клиентов в колл-центр, автоматизировать ввод данных или генерировать отчеты на основе структурированных данных. Эти первые ИИ-агенты будут отлично справляться с одним заданием, что позволит предприятиям автоматизировать мелкие, изолированные задачи.

2. ИИ-агенты помощники. Следующим шагом станет разработка более универсальных ИИ-помощников, способных работать и повышать производительность вместе с человеком. Такие агенты будут не просто выполнять задания самостоятельно, но и активно сотрудничать с людьми, внося свои предложения, предоставляя информацию в режиме реального времени и автоматизируя часть более сложных задач. Например, ИИ-помощники смогут помогать маркетологу составлять материалы для рекламной кампании, а разработчику программного обеспечения — писать код, что ускорит выполнение работы.

3. Многоагентные системы. На самом продвинутом этапе мы станем свидетелями появления многоагентных систем. Они будут состоять из нескольких ИИ-агентов, работающих совместно для поддержания сложных последовательных и взаимосвязанных процессов, охватывающих различные отделы или рабочие циклы. На этой стадии ИИ-агенты будут координировать свои действия между собой и выполнять такие масштабные операции, как управление всей цепочкой поставок или многоступенчатым процессом утверждения. Агенты смогут автономно делегировать друг другу задачи и адаптироваться к специфическим требованиям различных этапов рабочего цикла.



Инфо

Не удивляйтесь, если в ближайшем будущем в схемах стандартных рабочих процессов (BPMN) вы увидите такие обозначения, как *AI-agent* («ИИ-агент»).

В настоящее время несколько компаний и IT-гигантов уже серьезно работают над созданием ИИ-помощников и мультиагентных ИИ-систем. Несмотря на то что эти системы все еще находятся на ранних стадиях разработки, они уже способны решать конкретные задачи. На наш взгляд, мы находимся на переломе между этапом ИИ-агентов для решения конкретных задач и этапом ИИ-помощников. В ближайшем будущем ИИ-агенты, скорее всего, станут неотъемлемой частью бизнес-процессов, а мультиагентные системы будут играть ключевую роль в оптимизации рабочих процессов и стимулировании инноваций в различных отраслях.

Отличие ИИ-агентов от ИИ-инструментов

Вы наверняка слышали об ИИ-инструментах, интегрированных сегодня во многие онлайн-сервисы — например, Canva, Draw.io и Adobe. Разница между ИИ-агентами и ИИ-инструментами довольно тонкая, но весьма существенная: они отличаются функциональностью, интерактивностью и автономностью.

ИИ-инструменты — это программные приложения, использующие искусственный интеллект для выполнения определенных задач, имеющих, как правило, узкую сферу применения и требующих явных инструкций от пользователя. ИИ-инструменты предназначены для помощи пользователям в выполнении определенных функций, таких как анализ данных, обработка естественного языка, распознавание изображений или создание контента.

ИИ-агенты же представляют собой более совершенные и динамичные системы, обладающие определенной степенью автономности. Они предназначены для выполнения задач, принятия решений и адаптации к новой информации или среде без постоянного вмешательства человека. ИИ-агенты можно рассматривать как интеллектуальные сущности, взаимодействующие с окружающей средой и выполняющие действия для достижения определенных целей.

Ключевые различия.

Характеристика	ИИ-инструменты	ИИ-агенты
Сфера функциональности	Узкая, связанная с конкретной задачей.	Широкая, могут обрабатывать несколько связанных между собой задач.
Принятие решений	Ограниченное, по заранее определенным правилам.	Динамическое, могут принимать решения на основе меняющихся данных.
Взаимодействие с пользователем	Требуют явно заданных команд со стороны пользователя.	Действуют автономно, могут инициировать действия.
Адаптируемость	Без перепрограммирования небольшая или отсутствует.	Учатся и адаптируются на основе опыта и среды.
Примеры	Grammarly, Canva, Adobe.	Alexa, торговые боты.

Будучи неотъемлемой частью ИИ-экосистемы, ИИ-инструменты и ИИ-агенты выполняют разные функции. ИИ-инструменты повышают производительность пользователей, эффективно выполняя конкретные задачи, в то время как ИИ-агенты обеспечивают более высокий уровень автономности, выполняя сложные и многоэтапные задачи при минимальном контроле со стороны человека. Поэтому в последней главе нашей книги мы сосредоточимся исключительно на ИИ-агентах.

Примеры использования ИИ-агентов в сфере генеративного ИИ.

Теперь, получив представление о том, что же такое ИИ-агенты и каковы их возможности, рассмотрим основные варианты использования. Так как многие из читателей работают в IT-индустрии, будет полезно рассмотреть примеры использования этих агентов как с точки зрения разработчика, так и с точки зрения владельца или менеджера продукта.

Примеры использования с точки зрения разработчика.

Для разработчиков ИИ-агенты предлагают целый ряд мощных сценариев своего использования, позволяющих оптимизировать рабочие процессы, автоматизировать повторяющиеся задачи и даже находить творческие решения сложных проблем. Вот некоторые ключевые сценарии использования, в которых ключевую роль могут сыграть ИИ-агенты.

- **Автоматизированная генерация кода.** ИИ-агенты могут автоматически генерировать программный код на основе некоего задания или на основе предоставленных технических характеристик и требований. Такие агенты анализируют предоставленные разработчиком требования, понимают контекст задачи и генерируют фрагменты кода или даже целые функции. Многие компании, такие как JetBrains и Sbertech, в последнее время внедряют ИИ-ассистентов непосредственно в свои интегрированные среды разработки (IDE). Такая интеграция позволяет разработчикам автоматизировать повторяющиеся задачи, упростить процесс разработки и сосредоточиться на программировании более высокого уровня.
 - › Пример использования. Написание шаблонного кода, настройка структуры проекта или создание общих алгоритмов.
 - › Преимущество. Экономия времени на выполнение рутинных задач, благодаря чему разработчики сосредотачиваются на решении более сложных проблем.
- **Отладка и рефакторинг кода.** ИИ-агенты могут автономно обнаруживать ошибки в коде, предлагать исправления или выполнять рефакторинг с целью повышения производительности и читаемости кода. Такие агенты анализируют кодовую базу, выявляют неэффективные участки или фрагменты с ошибками и предлагают решения.
 - › Пример использования. Обнаружение неэффективных фрагментов кода, снижающих производительность, или утечек памяти в большой кодовой базе.

- › Преимущество. Повышение качества и сокращение времени, которое тратится на ручной поиск ошибок и недостатков.
- **Автоматизированное тестирование.** ИИ-агенты могут автоматизировать модульное, интеграционное и сквозное тестирование, генерируя тестовые примеры и анализируя поведение приложения. Они могут моделировать взаимодействие с пользователем, проводить стресс-тестирование приложений и выявлять уязвимости или фрагменты, требующие улучшения.
 - › Пример использования. Автоматическая генерация тестов для проверки новых функций или проведение регрессионного тестирования после обновлений.
 - › Преимущество. Ускорение процесса тестирования, повышение надежности кода, уменьшение количества ошибок.
- **Составление документации.** Одна из самых трудоемких задач, которые часто приходится выполнять программистам — создание подробной документации. Но эту работу теперь могут взять на себя ИИ-агенты, освободив разработчиков для более интересных задач. Анализируя кодовые базы, определения функций и параметры, ИИ-агенты автоматически генерируют высококачественную документацию, избавляя разработчиков от утомительной работы по ее написанию.
 - › Пример использования. Генерация документации по API или встроенных комментариев к коду для больших кодовых баз.
 - › Преимущество. Повышение удобства поддерживаемости кода, облегчение понимания особенностей проекта для других разработчиков.
- **Рекомендации по проектированию и архитектуре систем на основе ИИ.** ИИ-агенты могут помогать проектировать системы, анализируя существующие архитектуры и предлагая способы их улучшения. Например, они могут найти более эффективные структуры данных, методы проектирования микросервисов или способы оптимизации облачной архитектуры. Некоторые из существующих ИИ-агентов, вооруженных специфическими

инструментами и плагинами, уже легко справляются с этой задачей.

- › Пример использования. Выдача рекомендаций по структуре баз данных, по выбору монолитной архитектуры или архитектуры на основе микросервисов, по стратегии кэширования.
 - › Преимущество. Оптимизация производительности и масштабируемости системы с помощью удачных архитектурных решений, основанных на анализе, проведенном искусственным интеллектом.
- **Совместная работа и обмен знаниями.** ИИ-агенты могут выполнять роль базы знаний для команд разработчиков, предоставляя в режиме реального времени ответы на технические вопросы, предлагая соответствующую документацию или даже рекомендуя способы оптимизации конкретных сценариев.
- › Пример использования. Ответы на вопросы о стандартах кодирования или рекомендации с предложением воспользоваться решениями из прошлых проектов.
 - › Преимущество. Улучшение совместной работы, облегчение доступа к знаниям для всей команды.

Помимо этих конкретных примеров использования, ИИ-агенты могут предлагать отдельным разработчикам и коллективам другие повышающие производительность преимущества. Например, они экономят время для разработки и внедрения конвейеров CI/CD, повышая их эффективность и предоставляя ценные советы по автоматизации, что сводит к минимуму риск человеческих ошибок. По мере развития ИИ-агентов их роль в жизненном цикле разработки программного обеспечения будет только повышаться, благодаря чему разработчики смогут сосредотачиваться на более творческих задачах, оставляя агентам рутинные и сложные операции.

Примеры использования с точки зрения менеджера продукта.

С точки зрения менеджера или владельца продукта ИИ-агенты предлагают множество вариантов использования, улучшающих процесс разработки, повышающих качество обслуживания клиентов и оптимизирующих работу. Вот несколько основных сценариев использования, в которых ИИ-агенты играют решающую роль в управлении продуктами.

- **Автоматизация определения приоритетов в дорожной карте продукта.** ИИ-агенты могут автоматически определять приоритеты функций продукта или элементов бэклога (списка задач, которые нужно выполнить в последующих циклах разработки), анализируя такие факторы, как потребительский спрос, рыночные тенденции, ресурсы разработки и потенциальная окупаемость инвестиций. ИИ-агенты помогают менеджерам по продуктам принимать более взвешенные и объективные решения по выбору приоритетных направлений для последующей разработки.
 - › Пример использования. Сортировка запросов на новые функции по приоритету на основе спроса со стороны клиентов и прогнозируемого влияния на бизнес.
 - › Преимущество. Повышение концентрации усилий производственных команд на наиболее ценных функциях и снижение риска выбора неправильных приоритетов и нерационального использования ресурсов.
- **Анализ конкурентов и мониторинг рынка.** ИИ-агенты могут осуществлять постоянный мониторинг рынка, анализировать деятельность конкурентов и выявлять отраслевые тенденции в режиме реального времени. Они могут предупреждать менеджеров по продуктам об изменениях в конкурентной среде или сообщать им о новых возможностях на рынке, что позволяет более оперативно и стратегически грамотно планировать производство продукции.

- › Пример использования. ИИ-агент, отслеживающий появление новой продукции конкурентов, их ценовые стратегии или отзывы пользователей в режиме реального времени.
 - › Преимущество. Предоставление менеджерам по продуктам информации о конкурентной среде для успешного корректирования продуктовых стратегий.
- **Прогнозная аналитика для предсказания спроса.** ИИ-агенты могут анализировать исторические данные и рыночные тенденции, на основе которых предсказывают спрос на продукты или услуги в будущем. Тем самым они помогают менеджерам по продуктам планировать запасы, управлять цепочками поставок и более точно прогнозировать доходы.
 - › Пример использования. ИИ-агенты прогнозируют сезонный спрос на продукт, помогая планировать запасы и производство.
 - › Преимущество. Снижение риска перепроизводства или нехватки запасов, повышение операционной эффективности и рентабельности.
- **Проактивное управление рисками.** ИИ-агенты могут предсказывать потенциальные риски на протяжении жизненного цикла продукта, такие как задержки в разработке, сбои в цепочке поставок или изменения спроса со стороны клиентов. Такие агенты заранее предупреждают менеджеров по продуктам о возможных рисках и позволяют снизить их до того, как эти риски повлияют на успешность продукта.
 - › Пример использования. ИИ-агент, который предупреждает о возможных задержках в производстве и предлагает альтернативные варианты или способы перераспределения ресурсов.
 - › Преимущество. Снижение вероятности срыва сроков или превышения затрат, повышение плавности запуска продукта, увеличение срока его эксплуатации.

- **Автоматизированная поддержка клиентов.** ИИ-агенты могут выступать в роли виртуальных помощников, обрабатывая запросы клиентов и запросы на поддержку в режиме реального времени. Они могут вести диалог на естественном языке, отвечать на часто задаваемые вопросы, помогать решать типичные проблемы и даже передавать более сложные запросы людям в случае необходимости.

- › Пример использования. Автоматизация круглосуточной поддержки клиентов с помощью чат-ботов или голосовых агентов.
- › Преимущество. Сокращение потребности в человеческом персонале, снижение операционных расходов и уменьшение времени отклика, а также обеспечение постоянного обслуживания клиентов.

Оптимизация цен. ИИ-агенты могут динамически корректировать цены в зависимости от спроса, состояния рынка, поведения клиентов и цен конкурентов. Постоянно анализируя эти факторы, агенты ИИ оптимизируют цены на продукцию, обеспечивая тем самым максимальную рентабельность.

- › Пример использования. ИИ-агент, рекомендующий повышать цены в периоды пикового спроса или предлагающий скидки в периоды низких продаж.
- › Преимущество. Максимизация доходов и поддержание конкурентной ценовой стратегии в режиме реального времени.

С точки зрения управления продуктами ИИ-агенты представляют собой огромное достижение в области автоматизации и оптимизации процесса разработки продуктов. Примерно те же методы можно применить для анализа поведения пользователей и для персонализации их опыта. С помощью предлагаемых ИИ-агентами возможностей менеджеры по продуктам могут принимать более обоснованные решения, повышать привлекательность продукта для пользователей и держать позади конкурентов на современном, быстро меняющемся рынке.

Классификация ИИ-агентов в сфере генеративного ИИ

В контексте генеративного искусственного интеллекта ИИ-агентов можно классифицировать по их сложности, функциональности и способу взаимодействия. В этой сфере ИИ-агентов можно разделить на следующие категории.

1. По функциональности и по способу применения.

- **Агенты, генерирующие контент.** Это агенты, которые с помощью генеративных моделей генерируют какой-либо контент — например, статьи, рассказы, фрагменты программного кода или изображения. Их часто используют в творческих областях или для автоматизации создания контента.
 - › Пример. ИИ-фреймворк, генерирующий на основе промптов маркетинговые материалы, посты в блогах или контент для социальных сетей.
- **Разговорные агенты.** Эти агенты ведут диалог на естественном языке с пользователями. Они могут отвечать на вопросы (выполнять запросы), оказывать поддержку клиентам или имитировать человеческое общение.
 - › Пример. Чат-боты на базе Llama или виртуальные помощники, такие как Google Assistant и Alexa.
- **Агенты, ориентированные на выполнение задач.** Эти агенты предназначены для выполнения конкретных задач, таких как бронирование встреч, управление расписанием или выполнение инструкций в заранее определенных областях.
 - › Пример. ИИ-помощник, управляющий электронной почтой, устанавливающий напоминания и выполняющий базовые административные задачи на основе инструкций пользователя.

- **Информационно-поисковые агенты.** ИИ-агенты, специализирующиеся на получении доступа к информации и на поиске информации из различных источников, включая базы данных, API и базы знаний. Обработывая полученную информацию с помощью LLM, они могут давать сложные и развернутые ответы.
 - › Пример. Агент, отвечающий на технические запросы, извлекающий информацию из научных баз данных и предлагающий объяснения на естественном языке.
- **Мультимодальные агенты.** Агенты, которые могут обрабатывать входные и выходные данные в различных форматах, — например, текст, изображения и аудио. Такие агенты реализуют разные возможности искусственного интеллекта, в том числе обработку текста на естественном языке, компьютерное зрение и распознавание речи.
 - › Пример. ИИ-агент, анализирующий изображения, отвечающий на голосовые команды и генерирующий письменные отчеты.

2. По способам взаимодействия и интеграции.

- **Агенты с расширенной памятью.** Эти агенты хранят в памяти данные о прошлом взаимодействии с пользователями, сведения об их предпочтениях и контекстную информацию, что позволяет им со временем предоставлять более последовательные и контекстуально релевантные ответы.
 - › Пример. Виртуальный репетитор, запоминающий успехи ученика и соответствующим образом корректирующий программу обучения.
- **Автономные агенты.** Автономные агенты работают самостоятельно, принимая решения и совершая действия без постоянного участия пользователя. Они способны самостоятельно ставить цели и предпринимать действия по их достижению.

- › Пример. Автономный торговый бот, покупающий и продающий акции на основе анализа рынка без непосредственного вмешательства человека.

3. По способам обучения и адаптации.

- **Предварительно обученные агенты.** Эти агенты используют предварительно обученные LLM для выполнения задач без дополнительной настройки и без обучения на новых данных. Они полагаются на возможности базовой модели.
 - › Пример. Чат-бот, использующий модель Llama для ответов на вопросы общего характера на основе знаний этой модели.
- **Дообученные агенты.** Эти агенты настраиваются под конкретные задачи или сферу знаний с помощью дообучения на специализированных наборах данных. Благодаря этому повышается их производительность в целевых областях.
 - › Пример. ИИ-помощник, прошедший дополнительное обучение на медицинских данных и предоставляющий консультации по вопросам здравоохранения.
- **Адаптивные обучающиеся агенты.** Эти агенты могут обучаться и адаптироваться к новым данным в процессе развертывания. Со временем они улучшают свою работу, используя обратную связь и новую информацию.
 - › Пример. Бот для обслуживания клиентов, улучшающий свои ответы на основе взаимодействия с пользователями и на основе обратной связи.

Эта классификация далеко не полная, но она предлагает прочную основу для выделения различных типов ИИ-агентов в области генеративного искусственного интеллекта. По мере развития этой области, вероятнее всего, будут появляться новые категории помощников с различными сферами применения.

Архитектура ИИ-агентов

Архитектура ИИ-агентов отличается в зависимости от их типа или назначения. Например, дообученный агент может включать такие компоненты, как обучающиеся слои, помогающие ему со временем адаптироваться к новым условиям. В отличие от него, агенту для создания контента могут понадобиться дополнительные компоненты для выполнения таких задач, как токенизация или генерация текстовых эмбеддингов. В этом разделе мы рассмотрим базовую архитектуру ИИ-агента, используемого обычно для выполнения повседневных задач.

ИИ-агенты — это сложные системы, состоящие из разных модулей, которые выполняют различные задачи вроде обработки естественного языка, принятия решений и взаимодействия с окружением. Для объяснения архитектуры ИИ можно разделить компоненты по их функционалу. Далее приведена концептуальная схема типичного ИИ-агента с объяснением ее подробностей.

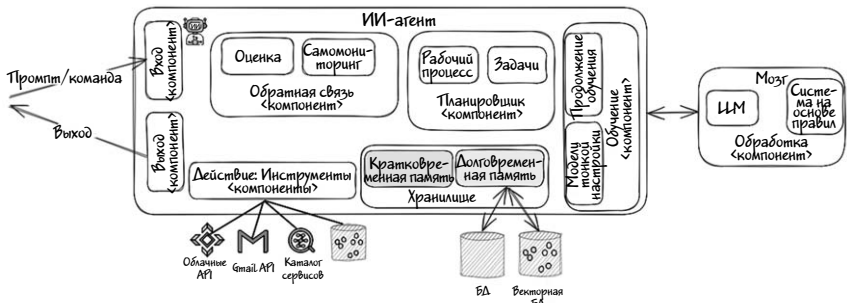


Иллюстрация 7.2

Архитектура ИИ-агента состоит из следующих компонентов.

1. **Вход.** Этот компонент получает входные данные из окружения (среды) или от пользователя — например, текст, изображения или данные датчиков. Для разговорного агента этот компонент может включать модули обработки естественного языка (NLP), преобразующие речь в текст и предварительно обрабатывающие его.

- Ключевые элементы.

1. Датчики (для роботизированных ИИ-агентов).
2. Устройства ввода текста/речи/видео (для виртуальных агентов).
3. Модули предварительной обработки (выполняющие токенизацию, нормализацию и т.д.).

2. «Мозг», или компонент обработки. Это основной логический компонент, обрабатывающий входные данные, анализирующий их и создающий выходные данные. Он включает в себя разные модули и алгоритмы.

- Модель естественного языка (NLP). Для обработки естественного языка в чат-ботах или виртуальных помощниках
- Модели восприятия. В агентах для работы с изображениями или видео (например, распознающих объекты и классифицирующих изображения).
- Модели принятия решений. Для логических рассуждений, планирования или прогнозирования действий.
- Ключевые элементы.

1. Большие языковые модели (LLM).
2. Модели восприятия (например, сверточные или рекуррентные нейронные сети).
3. Системы с логикой на основе правил.

3. Хранилище. В этом компоненте хранятся знания агента, данные о предыдущих взаимодействиях и любая дополнительная внешняя информация. Агенты могут обращаться к различным элементам хранилища.

- Кратковременная память. Модуль запоминания недавних взаимодействий
- Долгосрочная база знаний. Модуль, содержащий факты, правила или обученные модели.
- Ключевые элементы.

1. База данных в памяти.
2. Базы данных (структурированные и неструктурированные).
3. Хранилища эмбеддингов (векторные БД).
4. Обучающий компонент. Многие ИИ-агенты со временем адаптируются, и этот слой позволяет агентам обновлять свои модели или обучаться на новых данных.

- Ключевые элементы.
 - › Модуль контролируемого/неконтролируемого обучения.
 - › Модуль обучения с подкреплением (на основе обратной связи).
 - › Дообученные модели.
 - › Модуль непрерывного обучения.

5. **Компонент действия.** Иногда этот компонент называют «инструментами». Он определяет способы взаимодействия агента с окружением или пользователями. Это может быть, например, генерация ответов в виде текста, выполнение команд или вызов API сторонних разработчиков.

- Ключевые элементы.
 - › Модули генерации текста/аудио (например, модели GPT или TTS).
 - › Актуаторы (для роботизированных агентов).
 - › Модуль выполнения API/команд (внешних систем).

6. **Компонент обратной связи.** Этот слой часто используется в ИИ-агентах для оценки эффективности и внесения корректировок, передаваемых в обучающий слой. Компонент обратной связи позволяет агенту оценивать свою работу и учиться на своих ошибках.

- Ключевые элементы.
 - › Функции вознаграждения (в механизмах обучения с подкреплением).

- › Модули мониторинга ошибок (для проверки на наличие сбоев).
- › Модули обратной связи с пользователем (для улучшения работы).

7. Планировщик. Этот компонент разбивает сложные задачи на легко выполнимые отдельные шаги. В некоторых случаях он также содержит модуль рабочего процесса или модуль потока обработки данных.

- Ключевые элементы.

- › Модуль рабочего процесса.
- › Модуль потока обработки данных (конвейер задач).

В зависимости от фреймворка или способа реализации архитектура конкретного агента может отличаться от описанной выше. Однако большинство фреймворков ИИ-агентов включают в себя указанные основные компоненты, хотя те и могут называться по-разному. В своей совокупности эти компоненты образуют комплексный агент, способный выполнять сложные пользовательские задачи с помощью LLM.

Фреймворки для разработки ИИ-агентов

Разработка ИИ-агентов требует надежного и эффективного фреймворка, поддерживающего различные архитектуры, алгоритмы и языки программирования. На момент написания этой книги существует более 150 фреймворков и инструментов для создания ИИ-агентов, полный список которых можно найти по данной ссылке: <https://github.com/e2b-dev/awesome-ai-agents>. При этом многие из указанных инструментов специализированы для решения конкретных задач и не требуют предварительных знаний программирования, в то время как некоторые предназначены для решения более широкого круга задач и требуют развитых навыков программирования. Ниже перечислены несколько из наиболее распространенных

фреймворков с открытым исходным кодом для создания ИИ-агентов на основе больших языковых моделей (LLM).

1. Langchain и LangGraph

LangChain — это *полный* фреймворк, предназначенный для упрощения разработки приложений, подразумевающих интеграцию LLM с внешними источниками данных, рабочими процессами или API. Его основная цель — создать систему на основе цепочек, способную выполнять такие задачи, как ответы на вопросы, выполнение команд и взаимодействие с внешними системами с помощью моделей типа GPT или локальных моделей вроде LLaMA. Кроме того, в рамках проекта создан новый подпроект под названием LangGraph для разработки сложных агентных систем.



Инфо

LangChain и LangGraph часто путают между собой. В LangChain тоже имеются свои средства построения агентно-ориентированных систем. В главе 4 мы рассматривали SQL-агент LangChain, обрабатывающий ошибки при выполнении SQL-запросов. В отличие от LangChain, LangGraph — это библиотека, построенная на основе LangChain, и специально предназначенная для создания многоакторных приложений с отслеживанием состояния и сложных агентных систем на основе LLM.

- Ключевые особенности LangGraph.
 - › **Графовая структура.** Для построения мощных систем со сложной структурой в LangGraph используется концепция графа. Каждый узел графа в LangGraph представляет собой отдельный LLM-агент, а соединяющие эти узлы ребра — связи между агентами. Такая архитектура позволяет создавать простые и легко управляемые рабочие процессы, в которых каждый LLM-агент выполняет определенные задачи и при необходимости обменивается информацией с другими агентами. Такая структура также полезна для визуализации рабочих процессов и зависимостей.

- **Управление состоянием.** Одна из ключевых особенностей данного фреймворка — автоматическое отслеживание состояния и управление им. Каждый узел графа получает информацию о текущем состоянии, может менять его по мере необходимости, а затем передает информацию о состоянии следующему узлу. Такой подход позволяет системе отслеживать и сохранять информацию о взаимодействии компонентов. Кроме того, эта функция позволяет хранить информацию о состоянии системы во внешних базах данных, таких как SQLite или PostgreSQL, что повышает гибкость и масштабируемость модели.
- **Обработка ошибок.** LangGraph предлагает надежный механизм обработки ошибок, позволяющий узлам вызывать исключения при возникновении ошибок во время выполнения программы. Благодаря гибкому способу реализации логики повторных попыток, узлы эффективно обрабатывают исключения и восстанавливаются после ошибок. Эта функция LangGraph позволяет разработчикам писать более устойчивый код, допускающий восстановление системы после неожиданных проблем.
- Недостатки LangGraph. Несмотря на широкий набор функций, LangGraph имеет свои ограничения и проблемы, о которых должны знать разработчики.
 - **Кривая обучения.**
 - * Несмотря на свою эффективность, модульный характер LnanGraph усложняет процесс освоения этой системы и ее настройки, особенно для новичков. Для создания надежного агента часто требуется глубокое понимание особенностей взаимодействия между различными компонентами.
 - * На освоение промпт-инжиниринга, принципов проектирования графов и особенностей поведения агентов разработчикам требуется время, порой довольно продолжительное.
 - **Требовательность к ресурсам.**

* Сложные схемы LangGraph могут требовать для своей работы много ресурсов, особенно если граф содержит множество узлов или подразумевает обработку большого количества данных.

* Разработчикам приходится искать баланс между производительностью и затратами, особенно при использовании облачных LLM или больших объемов данных.

В целом в сочетании с LangGraph LangChain представляет собой мощный фреймворк для создания ИИ-агентов, использующих возможности LLM для взаимодействия с внешними данными и выполнения сложных задач. Это сочетание предоставляет много преимуществ, но у него имеются некоторые ограничения, о которых следует знать разработчикам, такие как крутая кривая обучения, потенциальные проблемы масштабируемости и повышенная сложность отладки. Поэтому тем, кто рассматривает для своих проектов LangGraph, нужно тщательно взвесить все его преимущества (например, гибкость), и недостатки (например, возможные затраты на его изучение и на развертывание в производственной среде).

2. CrewAI

CrewAI — это еще один широко используемый высокоуровневый фреймворк, предназначенный для ролевых мультиагентных взаимодействий. Он ориентирован на создание ИИ-агентов, работающих над сложными задачами сообща, и управление ими. Фреймворк позволяет координировать работу сразу нескольких ИИ-моделей или ИИ-агентов, что делает его идеальным для задач, требующих разносторонней экспертизы, таких как создание контента, анализ данных и решение проблем в различных областях.

Инфо

Термин «мультиагентный» подразумевает поддерживаемое фреймворком сотрудничество нескольких независимых агентов или участников с целью выполнения сложных задач.

- Ключевые особенности CrewAI.

- › **Совместная работа нескольких ИИ-агентов.** CrewAI облегчает формирование команд или групп ИИ-агентов, общающихся между собой и обменивающихся ресурсами для решения комплексных задач. При этом отдельные агенты могут выполнять специализированные роли, имитируя тем самым работу человеческих команд.
- › **Модульная интеграция.** Фреймворк поддерживает совместную интеграцию различных ИИ-моделей, специализирующихся на различных задачах, таких как обработка языка, распознавание изображений или принятие решений.
- › **Оркестровка задач.** Система предлагает сложные механизмы оркестровки, распределяющие задачи между наиболее подходящими агентами в команде на основе их возможностей. По мере изменения сложности задачи можно осуществлять динамическую настройку.
- › **Масштабируемость.** CrewAI создана с учетом требований масштабируемости, благодаря чему она может управлять крупномасштабными операциями в реальном времени, такими как поддержка клиентов или мониторинг.
- › **Контроль со стороны человека.** CrewAI позволяет осуществлять человеческий контроль, а люди-операторы могут вмешиваться в рабочий процесс, изменяя поведение агентов ИИ, особенно в сценариях, требующих этических суждений или принятия важных решений с высокой ставкой.
- › **Легкая кривая обучения.** По ряду причин кривая обучения CrewAI считается относительно легкой (пологой), что делает этот фреймворк популярным среди нетехнических пользователей.

* Высокоуровневые абстракции. Для управления ИИ-агентами и задачами в CrewAI имеются интуитивно понятные высокоуровневые абстрактные инструменты, позволяющие разработчикам не увязать в низкоуровневых деталях, а сосредоточиться на определении ролей и способов взаимодействия разных агентов.

* Удобные API. CrewAI предлагает понятные API и хорошо задокументированные функции, так что новичкам не обязательно обладать обширными познаниями в области архитектуры ИИ.

- Недостатки CrewAI. Несмотря на широкую популярность CrewAI среди нетехнических пользователей, особенно в сфере создания контента для социальных сетей, у этого фреймворка есть и свои недостатки.
 - ✧ **Плохо продуманные промпты.** Некоторые встроенные в код промпты содержат значительные грамматические и орфографические ошибки, снижающие их полезность.
 - ✧ **Навязчивая телеметрия.** Наличие жестко зашитой в коде телеметрии, которую нельзя отключить, вызывает опасения по поводу конфиденциальности и контроля со стороны пользователя.

В этой книге хватило места для описания только двух популярных фреймворков, но внимания заслуживают и некоторые другие. Например, AutoGen, разработанный компанией Microsoft, и Vertex AI Agent Builder, разработанный компанией Google. Оба имеют схожую архитектуру и представляют собой надежные платформы для создания генеративных ИИ-приложений и ИИ-агентов корпоративного уровня. Более того, эти инструменты позволяют разработчикам создавать сложные модели без особых знаний в сфере глубокого машинного обучения.

В следующем разделе мы рассмотрим ИИ-агентов с практической точки зрения. Создадим агента с нуля, интегрируем его с LLM и продемонстрируем, как с его помощью можно выполнять конкретные задачи.

Пошаговое руководство по созданию ИИ-агента на практике.

Итак, мы познакомились с теоретическими основами ИИ-агентов, рассмотрели сферы их применения, архитектуру и различные

фреймворки. Как это было и с другими темами, на данном этапе настало время привести реальный пример, иллюстрирующий функциональность таких ИИ-агентов и их возможности. Как обычно, было довольно сложно решить, какой вариант окажется идеальным для книги. Сначала мы рассматривали агентов для генерации документации из кодовых баз или для резюмирования ежедневного потока электронных писем.

Но затем, учтя широкое распространение социальных сетей в современном цифровом ландшафте, решили, что будет разумнее продемонстрировать на практике работу ИИ-агента, генерирующего контент для таких платформ, как блоги. В последнее время все большую роль играют умные персональные помощники, повышающие продуктивность и упрощающие выполнение повседневных задач. Создавать таких интеллектуальных и совместно работающих помощников позволяет мощный фреймворк CrewAI, не требующий глубоких знаний в области программирования. Так, например, с помощью CrewAI можно сконструировать несколько специализированных агентов, которые будут писать статьи для блога (хотя все равно большую часть контента придется переписывать и редактировать).

В этом разделе мы покажем процесс создания умного персонального помощника (агента) с помощью фреймворка CrewAI. Этот пример продемонстрирует универсальность и простоту использования фреймворка, а также покажет, как различные агенты действуют сообща, выполняя такие определенные функции, как управление задачами, поиск информации и общение, повышая тем самым эффективность взаимодействия с пользователем.

Пример использования.

Предположим, что мы планируем еженедельно публиковать в своем блоге статьи об установке и развертывании базы данных в памяти Apache Ignite на Docker и Kubernetes. Наша цель — предоставить подробное пошаговое руководство, с помощью которого пользователи блога смогут повторить описанный процесс. Чтобы сэкономить время, для создания набросков и черновиков, которые будем потом

дорабатывать, мы решили воспользоваться фреймворком CrewAI. Ниже показана схема работы с ним.

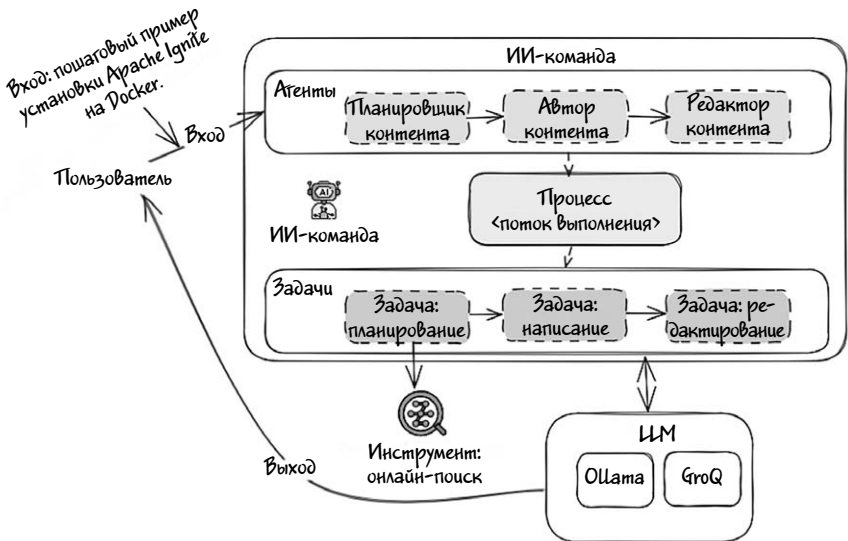


Иллюстрация 7.3

Перед тем как переходить к практике, рассмотрим основную структуру CrewAI.

Основные компоненты.

1. **Агенты.** Агенты — это автономные сущности CrewAI, выполняющие определенные задачи. Каждый агент, как правило, обладает собственным набором навыков и своей базой знаний.

- *Ключевые характеристики агента.*
 - › Роль. Агенты могут выполнять различные задачи, такие как поиск информации, логические рассуждения, принятие решений и даже взаимодействие с внешней средой. Они могут работать как самостоятельно, так и в составе команды.
 - › Цель. Конкретная задача, которую стремится выполнить агент.

- › Предыстория. Контекстная и фоновая информация, определяющая поведение агента и его взаимодействие с другими компонентами.
- › Инструменты. Пользовательские инструменты и утилиты, которые агент использует для выполнения своих задач.

2. Поток выполнения. Поток выполнения определяет последовательность и логику выполнения агентами задач и их взаимодействия с другими агентами или инструментами. Это организатор работы агентов, следящий за тем, чтобы задачи эффективно выполнялись в правильной последовательности. В зависимости от требований задачи поток бывает линейным, параллельным или условным.

- *Ключевые характеристики потока выполнения.*
 - › Инициализация. Настройка агентов и задач на основе конфигурации.
 - › Назначение задач. Назначение задач соответствующим агентам.
 - › Выполнение. Выполнение задач в соответствии с заданным процессом (последовательным или параллельным).

3. Задачи. Задачи — это назначаемые агентам CrewAI единицы (куски) работы.

- *Ключевые характеристики задач.*
 - › Описание. Подробное описание задачи.
 - › Контекст. Зависимости и условия, необходимые для выполнения задачи.
 - › Назначенный агент. Конкретный агент, ответственный за выполнение задачи.
 - › Режим выполнения. Выполняется ли задача синхронно или асинхронно.

4. Команда. Команда — это группа агентов, работающих вместе над достижением общей цели, подобно человеческой команде.

- *Ключевые характеристики команды.*
 - › **Агенты.** Список входящих в состав команды агентов.
 - › **Задачи.** Список задач, которые нужно выполнить.
 - › **Управление процессами.** Порядок выполнения задач (например, последовательный или параллельный).

5. **Инструменты.** Инструменты — это внешние системы, API или среды, которыми агенты пользуются для выполнения своих задач. Например, это могут быть поисковые системы или REST API сторонних сервисов.

6. **LLM.** Большие языковые модели, такие как LLaMa, GPT, Mistral, предоставляющие агентам расширенные возможности по обработке естественного языка. LLM можно назвать мозгом CrewAI.

Для достижения поставленной цели мы собираемся воспользоваться следующими агентами, выполняющими описанные ниже задачи.

- **Агенты.**
 - › **Планировщик.** Планировщик контента. Его задача — собирать информацию по темам и подготавливать наброски статей для блога. Планировщик использует поисковую систему DuckDuckGo.
 - › **Автор (писатель).** Автор технического контента. В его обязанности входит написание подробного пошагового руководства на основе конспекта, предоставленного контент-планировщиком.
 - › **Редактор.** Редактор публикации. Его задача — просмотреть запись в блоге и проверить ее на грамматику и ясность описания. Кроме того, он может постараться переписать контекст в более естественном и удобном для чтения виде с сохранением первоначального смысла.
- **Задачи.**

- Составление плана. Разработка подробного плана контента, включающего общую тему, основные ценности, главные пункты и пошаговые примеры.
 - Написание статьи. Создание подробной статьи для блога, написанной согласно составленному на предыдущем этапе плану.
 - Редактирование. Проверка грамматических ошибок в статье и ее переписывание для лучшей читаемости с сохранением смысла.
- Поток выполнения. Команда распределяет агентов в соответствии с их конкретными задачами. Процесс выполняется в определенной последовательности, а конечные результаты обобщаются и отправляются на выход.

Теперь, когда мы поняли, в чем заключается наша цель, и как с помощью ИИ-агентов будут выполняться задачи, приступим к разработке первого мультиагентного приложения.

Шаг 1. Убедитесь в том, что Ollama-сервер установлен и работает с какой-либо LLM, например, LLaMa или Mistral.

Шаг 2. Активируйте среду conda, запустив Jupyter Notebook. Если вы не уверены, как выполнить эти шаги, более подробные сведения содержатся в главе 1.

Шаг 3. Создайте в Jupyter новый блокнот и добавьте в него следующий фрагмент кода на языке Python.

```
!pip install crewai
!pip install duckduckgo-search
!pip install langchain_community
```

Эти строки с помощью pip устанавливают три пакета Python: crewai, duckduckgo-search и langchain_community, представляющие собой библиотеки для создания ИИ-агентов и поиска в проекте.

Инфо

Полный блокнот со всем кодом можно найти в каталоге проекта на GitHub. Постарайтесь не копировать и не вставлять в блокнот код из книги. Кроме того, можно набирать и выполнять каждую ячейку отдельно.

Шаг 4. Импортируйте нужные классы из библиотек.

```
from langchain_community.llms import Ollama
from langchain_community.tools import DuckDuckGoSearchRun
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI
from langchain_groq import ChatGroq import os
```

Шаг 5. Инициализация экземпляра языковой модели (LLM).

```
llm = ChatOpenAI(
    model = "llama3.1",
    base_url = "http://192.168.1.124:11434/v1",
    temperature=0.7,
    max_tokens=1024,
    timeout=None,
    max_retries=2
)
```

Этот фрагмент кода задает экземпляр LLM с помощью класса `ChatOpenAI`, импортированного из фреймворка `LangChain`. Вот объяснение некоторых ключевых параметров.

- `model=llama3.1`. Указание на версию используемой модели. Измените его на название модели, которую используете в своем Ollama-сервере.
- `base_url=»http://192.168.1.124:11434/v1»`. URL — адрес вашего локального Ollama-сервера.

- `max_retries=2`. Количество повторений запроса в случае неудачи. Если у вас ПК или ноутбук с низкой производительностью, увеличьте его более чем на 5 пунктов.

Обратите внимание, что в зависимости от типа и конфигурации экземпляра LLM процесс может потреблять значительное количество вычислительных ресурсов. Если вам не хватает аппаратной мощности, настоятельно рекомендую воспользоваться сторонним сервисом инференса LLM, таким как Groq. Чтобы воспользоваться сервисом Groq, введите следующий фрагмент псевдокода (или скопируйте его и уберите комментарии).

```
#api_key= os.environ.get("$GROQ_API_KEY")
#model_name="llama-3.1-70b-versatile"
#llm = ChatGroq(
#    model=model_name,
#    temperature=0.7,
#    max_tokens=1024,
#    timeout=None,
#    max_retries=2
#)
```

Инфо

Если вы забыли, что такое Groq, подробнее об этом сервисе написано разделе «Аппаратное ускорение» главы 1.

Шаг 6. Добавьте инструмент поисковой системы с нужным нам промптом.

```
search_tool = DuckDuckGoSearchRun()
inputs = {
    "topic": "Step by step example of installing
Apache Ignite on Docker\
.",
    "target_audience": "Programmer, Product owner,
Analyst"
}
```

Первая строка кода инициализирует поисковую систему, а следующая задает ИИ-агенту тему публикации для блога («Пошаговый пример установки Apache Ignite на Docker») и целевую аудиторию («Программист, владелец продукта, аналитик»).

Шаг 7. Добавляем нашего первого агента, а именно планировщика.

Агент планировщик контента

```
planner = Agent(
    role="Content Planner",
    goal="You are tasked with planning an engaging and
informative blogpost on {topic}",
    backstory="You're working on planning a blog article "
        "about the topic: {topic}. "
        "Your goal is to gather accurate, up-to-date
information and structure the content "
        "in a way that captures the reader's interest
while delivering valuable insights. "
        "The audience for this blog post is {target_
audience}, "
        "and they are looking for clear, practical
information that they can apply immediately. "
        "You must consider their needs, interests,
and potential challenges they might face related to the
topic."
        "Your job is to create an outline that will guide
the writer in crafting a piece that "
        "is both educational and easy to digest. "
        "Your work is the base for "
        "the Content Writer to write an article on this
topic.",
    llm=llm, allow_delegation=False, verbose=True
)
```

Этот фрагмент кода задает агента-планировщика контента с помощью класса Agent. Роль агента заключается в планировании контента статьи для блога посредством сбора информации и создания

черновика. Этот агент не имеет права делегировать задачи другим агентам.

Перевод промпта.

Роль: «Планировщик контента».

Цель: «Тебе поручено составить план увлекательной и информативной публикации в блоге на тему {topic}».

Контекст: «Ты работаешь над составлением плана статьи для блога на тему: {topic}. Твоя цель — собрать точную, актуальную информацию и структурировать содержание таким образом, чтобы заинтересовать читателя и в то же время поделиться ценными сведениями. Аудитория этой статьи для блога — {target_audience}, и они хотят получить четко выраженную, практическую информацию, которую можно было бы немедленно применить. Ты должен учитывать потребности аудитории, ее интересы и потенциальные проблемы, которые могут у нее возникнуть в связи с этой темой. Твоя задача — создать набросок, который поможет писателю создать легкую для усвоения и познавательную статью. Твоя работа — это основа для писателя контента, который будет писать статью на эту тему».

Шаг 8. Добавляем второго агента, а именно писателя (автора контента).

Агент писатель (автор) контента

```
writer = Agent(
    role='Technical content writer',
    goal="You are the technical content writer assigned
to create a detailed and factually accurate blog post on:
{topic}",
    backstory="You're working on a writing a new step by
step tutorial about the topic: {topic}."
    "Your job is to ensure that the content is
factually accurate, well-organized, and clearly explains
the topic at hand. You will need to break down complex
technical concepts into simple, digestible parts, while
```

```
still maintaining depth and technical rigor. You must also
incorporate relevant data, examples, and use cases to make
the article practical and valuable to the reader. You
base your writing on the work of the Content Planner, who
provides an outline and relevant context about the topic."
"You follow the main objectives and direction of the
outline, as provide by the Content Planner. Your audience
consists of {target_audience}, who are likely seeking
clear and actionable insights. Whether it's professionals
looking to apply the knowledge in their work, or beginners
trying to grasp the basics, your writing should meet
their needs. Make sure to maintain an approachable tone,
balancing technical depth with readability.",
verbose=True, allow_delegation=False, llm=llm,
)
```

Этот фрагмент кода с помощью класса Agent задает агента-писателя (автора контента), который отвечает за написание подробной и фактологически точной статьи. Этот агент также не имеет права делегировать задачи.

Перевод промпта.

Роль: «Автор технического контента (писатель)».

Цель: «Ты автор технического контента, которому поручено сочинить увлекательную и информативную публикацию в блоге на тему {topic}».

Контекст: «Ты работаешь над сочинением пошагового руководства на тему {topic}. Твоя задача — проследить за тем, чтобы контент был фактологически верным, хорошо организованным и четко объясняющим заданную тему. Тебе нужно будет разложить сложные технические концепции на простые, доступные для понимания части с сохранением глубины и технической строгости. Тебе также нужно будет включить в публикацию релевантные данные и примеры использования, чтобы она представляла практическую ценность для читателя. Ты строишь свою работу на плане, созданном

планировщиком контента, предоставившим ее набросок, и на относящемся к этой теме контенте. Ты руководствуешься основными целями и направлениями, заданными в наброске планировщиком контента. Твоя аудитория — {target_audience}, и они, скорее всего, хотят получить ясные и практически полезные примеры. Твоя статья должна удовлетворять потребностям как профессионалов, желающих расширить свои знания, так и новичков, желающих усвоить основы. Сохраняй доброжелательный тон, сочетая техническую глубину с доступностью».

Шаг 9. Добавляем последнего агента из списка: редактора.

```
editor = Agent(  
    role="Editor",  
    goal="You responsible for reviewing and refining  
a blog post to ensure it aligns with the clear, concise,  
and engaging writing style of platforms like. 'https://dz\  
one.com'. ,  
    backstory="You are the technical content editor who  
receives a blogpost from the Content Writer. Your goal  
is to make sure the post is grammatically correct, free  
of errors, and easy to read. You will need to carefully  
check sentence structure, punctuation, and overall  
coherence while ensuring the tone remains conversational,  
yet informative – similar to the approachable style used  
on Medium.com. The content should feel natural, guiding  
readers effortlessly from one section to the next, while  
maintaining technical accuracy.",  
    llm=llm, allow_delegation=False, verbose=True  
)
```

Этот код задает агента-редактора, основная обязанность которого — редактирование записей в блогах, чтобы они соответствовали стилю конкретной организации или платформы. Редактор проверяет содержание на соответствие журналистским стандартам, поддерживает сбалансированную точку зрения и избегает спорных тем.

Перевод промпта.

Роль: «Редактор».

Цель: «Ты отвечаешь за проверку и доработку статьи для блога; ты следишь за тем, чтобы она соответствовала четкому, лаконичному и увлекательному стилю таких платформ, как. <<https://dzone.com/>>».

Контекст: «Ты технический редактор контента, получивший статью для блога от автора контента (писателя). Твоя цель — проследить за тем, чтобы статья была грамматически правильной, не содержала ошибок и легко читалась. Тебе нужно тщательно проверить структуру предложений, пунктуацию и общую связность, а также убедиться в том, что тон остается разговорным, но в то же время информативным в соответствии со стилем, используемым на Medium.com. Контент должен казаться естественным и легко вести читателя от одного раздела к другому с сохранением технической точности».



Инфо

Как вы, вероятно, уже заметили, решающую роль в настройке агентов играет формулировка промптов. Чем лучше сформулированы контекст и инструкции, тем лучше будет результат.

Теперь мы готовы определить задачи для каждого агента.

Шаг 10. Добавьте задачу plan.

Задачи для агента планировщика

```
plan = Task description=(  
    "1. Step by step example, and noteworthy latest news  
on {topic}.\n"  
    "2. Identify the target audience, considering their  
interests and pain points.\n"  
    "3. Develop a detailed content outline including  
an introduction, values, key points, and step by step  
example.\n"  
    "4. Include SEO keywords and relevant data or  
sources."
```

```

),
    expected_output="A comprehensive content plan document
with an outline, audience analysis, SEO keywords, and
resources.",
    agent=planner,
    tools=[search_tool],
)

```

Приведенный выше фрагмент кода определяет задачу для агента-планировщика с указанием конкретных шагов по составлению плана контента. В ходе выполнения задачи будет использован поисковый инструмент DuckDuckGo для сбора информации, новостей и данных, нужных для составления плана контента.

Перевод задачи для планировщика.

Описание.

«1. Пример пошагового выполнения и важные последние новости по теме.

2. Определи целевую аудиторию с учетом ее интересов и болевых точек.

3. Разработай подробный план контента, включая введение, ценности, ключевые моменты и пошаговый пример.

4. Включи ключевые SEO-слова и релевантные данные или источники».

Ожидаемый результат: «Развернутый документ с планом контента, содержащий структуру, анализ аудитории, SEO-ключевые слова и ресурсы».

Шаг 11. Добавьте следующую задачу write.

```
write = Task description=(
```

```

    "1. Use the content plan to craft a compelling blog
    post on {topic}.\n"
    "2. Incorporate SEO keywords naturally.\n"
    "3. Sections/Subtitles are properly named in an
    engaging manner.\n"
    "4. Ensure the post is structured with an engaging
    introduction, insightful body, and a summarizing
    conclusion.\n"
    "5. Proofread for grammatical errors and alignment
    with the brand's voice.\n"
  ), expected_output="A well-written blog post in markdown
  format, ready for publication, each section should have 3
  or 5 paragraphs.", agent=writer,
  )

```

Здесь мы определяем задачу для агента-писателя с конкретными инструкциями по созданию статьи для блога. Задача подразумевает использование контент-плана, включение SEO-ключевых слов, создание привлекательных названий для разделов, эффективное структурирование публикации, а также проверку на предмет ошибок и соответствия бренду.

Перевод задачи для автора.

Описание.

«1. Используй план контента для создания увлекательной записи в блоге на тему {topic}.
2. Ключевые SEO-слова должны казаться естественными.
3. Разделы и подзаголовки должны иметь привлекательные и осмысленные названия.
4. Продумай структуру публикации с захватывающим вступлением, содержательной основной частью и заключением.
5. Проверь текст на грамматические ошибки и соответствие бренду».

Ожидаемый результат: «Хорошо написанная статья для блога в формате markdown, готовая к публикации, в которой каждый раздел содержит от 3 до 5 абзацев».

Шаг 12. Добавьте в новую ячейку код для последней задачи — `edit`.

```
edit = Task description=("Proofread the given blog post
for grammatical errors and alignment with human language
for better readability while maintaining its original
meaning."),
    expected_output="A well-written blog post in markdown
format, ready for publication, each section should have 3
or 5 paragraphs.",
    agent=editor
)
```

Этот фрагмент кода определяет для агента-редактора задачу по проверке и редактированию публикации в блоге. Агент должен проверить текст на грамматические ошибки и на соответствие естественному языку для улучшения читаемости с сохранением первоначального смысла. Ожидаемый результат — отредактированная статья для блога в формате `markdown`, готовая к публикации, в которой каждый раздел содержит указанное количество абзацев.

Перевод задачи для автора.

Описание: «Проверка статьи для блога на предмет грамматических ошибок и соответствия естественному языку для улучшения читаемости при сохранении исходного смысла».

Ожидаемый результат: «Хорошо написанный пост в формате Markdown, готовый к публикации, каждый раздел которого содержит от 3 до 5 абзацев».

Шаг 13. Под конец инициализируем экземпляр `crew`, на чем этап подготовки агентов будет закончен.

Инициализация crew с последовательным процессом

```

crew = Crewagents=[planner, writer, editor],
      tasks=[plan, edit, write],
      verbose=2, # 1 или 2 для степени вывода сообщений
)

```

```

# запуск crew
result = crew.kickoff(inputs=inputs)

```

```

print("**Final result**")
print(result)

```

Здесь мы инициализируем экземпляр `crew`, состоящий из планировщика, писателя и редактора, каждому из которых поручается определенная задача для последовательного выполнения. Метод `kickoff` запускает экземпляр `crew` с заданными исходными данными, а конечный результат выводится на консоль.

Запустите все ячейки, нажимая на кнопку в строке меню. Если все прошло гладко, в консоли начнут появляться сообщения, подобные показанным ниже.

```

[2024-09-27 13:55:54][DEBUG]: == Working Agent: Content
Planner
[2024-09-27 13:55:54][INFO]: == Starting Task: 1. Step by
step example, and noteworthy latest news on Step by step
example of installing Apache Ignite on Docker..
2.  Identify the Programmer, Product owner, Analyst,
considering their interests and pain points.
3.  Develop a detailed content outline including an
introduction, values, key points, and step by step
example.
4.  Include SEO keywords and relevant data or sources.

```

```

> Entering new CrewAgentExecutor chain...
Thought: To create a comprehensive content plan for
the blog post "Step by step example of installing
Apache Ignite on Docker", I need to start by gathering

```

information on the latest news and updates on Apache Ignite and Docker.

Action: duckduckgo_search

Action Input: {"query": "Apache Ignite Docker latest news"}

...

Thought: Based on the gathered information, I can create a comprehensive content plan for the blog post "Step by step example of installing Apache Ignite on Docker".

Final Answer:

****Content Plan for "Step by step example of installing Apache Ignite on Docker"****

****Target Audience:****

- * Programmer
- * Product owner
- * Analyst

****Interests and Pain Points:****

- * Programmers: looking for a step-by-step guide to install Apache Ignite on Docker, interested in learning about the latest versions and updates.
- * Product owners: interested in understanding the benefits and challenges of using Apache Ignite on Docker, looking for a practical guide to implementation.
- * Analysts: looking for information on the scalability and performance of Apache Ignite on Docker, interested in understanding the technical requirements.

... omitted for space saving purpose.

Выводимый журнал содержит подробное описание последовательности действий, выполняемых агентами «планировщик» (Content Planner), «писатель» (Writer) и «редактор» («Editor») в многоагент-

ной системе. Сначала планировщик собирает информацию, на основе которой создает набросок контента и определяет целевую аудиторию. Затем писатель на основе этого наброска планировщика создает статью, а редактор дорабатывает ее и проверяет на грамматическую правильность и читаемость. Последовательный процесс отражает ход сотрудничества этих агентов, создающих высококачественный контент.

Созданная нами публикация для блога получилась объемной, потому что мы попросили, чтобы каждый раздел содержал от 3 до 5 абзацев.

Step by Step Example of Installing Apache Ignite on Docker

=====

Introduction -----

Apache Ignite is a powerful, open-source distributed database for high-performance computing that can be used as a database, caching system, or for in-memory data processing. It is designed to work seamlessly with various environments, including Docker. In this blog post, we will provide a step-by-step guide on installing Apache Ignite on Docker, configuring it with Spring Boot, and troubleshooting common issues.

Apache Ignite offers several features that make it an attractive choice for developers, including:

- * In-memory computing for high-performance data processing
- * Distributed database capabilities for scalability and reliability
- * Support for ACID transactions and SQL queries
- * Integration with popular frameworks such as Spring Boot

Importance of Using Apache Ignite on Docker

Using Apache Ignite on Docker provides several benefits, including:

- * Easy deployment and management of Apache Ignite instances
- * Scalability and flexibility in deploying Apache Ignite on various environments
- * Improved performance and reliability in distributed computing

Installing Apache Ignite on Docker

... omitted for space saving purpose.

Примерный перевод полученного на выходе отрывка.

Пошаговый пример установки Apache Ignite на Docker

=====

Введение

Apache Ignite — это мощная распределенная база данных с открытым исходным кодом для высокопроизводительных вычислений, которая может использоваться как база данных, система кэширования или как инструмент обработки данных в памяти. Она создана для работы в различных средах, включая Docker. В этой статье мы опишем пошаговое руководство по установке Apache Ignite на Docker, его настройке с использованием Spring Boot, а также по устранению типичных проблем.

Apache Ignite обладает рядом особенностей, которые делают его привлекательным выбором для разработчиков, включая:

- * Вычисления в памяти для высокопроизводительной обработки данных.
- * Возможности распределенной базы данных для масштабируемости и надежности.
- * Поддержка ACID-транзакций и SQL-запросов.
- * Интеграция с популярными фреймворками, такими как Spring Boot.

Почему важно использовать Apache Ignite на Docker

Apache Ignite на Docker предоставляет ряд преимуществ, включая следующие:

- * Простота развертывания и управления экземплярами Apache Ignite.
- * Масштабируемость и гибкость при развертывании Apache Ignite в различных средах.
- * Повышение производительности и надежности распределенных вычислений.

Установка Apache Ignite на Docker

... сокращено для экономии места.

Созданный текст представляет собой структурированную статью со вступлением и основной частью — обзором установки Apache Ignite на Docker. Мы можем оценить этот текст по таким критериям, как ясность, структура, содержание и общее впечатление.

Критерии оценки.

- Ясность.
 - › Текст ясен и написан на понятном языке, доступном в том числе и для новичков в этой теме (Apache Ignite или Docker).
 - › В тексте объясняются технические термины, что облегчает понимание.
- Структура.
 - › Документ хорошо структурирован и имеет информативные заголовки (например, «Введение», «Почему важно использовать Apache Ignite в Docker»).
 - › Каждый раздел логично перетекает в следующий, что помогает читателю усваивать информацию.

- Содержание.
 - › Введение. Во введении четко объясняется, что такое Apache Ignite, и описаны сферы ее применения, поэтому читатель при желании может получить представление, что это такое. Список возможностей подчеркивает сильные стороны Apache Ignite, привлекая внимание потенциальных пользователей.
 - › Раздел о важности. Здесь описываются преимущества использования Apache Ignite с Docker, подчеркивается простота развертывания, упоминаются масштабируемость и производительность. Этот раздел можно улучшить, добавив примеры или конкретные сценарии, в которых реализуются эти преимущества.
 - › Следующие шаги. Далее следует инструкция по установке, но текст дальше не показывается. Предполагается, что руководство продолжается и после этого отрывка. Это хороший подход к пошаговому руководству.
- Увлекательность.
 - › Текст увлекает читателя, рассказывая о практическом применении Apache Ignite и Docker.
 - › Маркированные списки удачно резюмируют ключевые особенности и преимущества, повышая читаемость и запоминаемость.
- Техническая точность.
 - › Описания возможностей Apache Ignite на первый взгляд выглядят точными и соответствуют практике использования этой базы данных с Docker. Но этот раздел следует проверить на точность и при необходимости переписать.
 - › Читаемость. Маркированные списки, короткие абзацы и заголовки повышают удобочитаемость. Язык в достаточной степени технический, но не слишком сложный, что очень важно для руководства, рассчитанного на широкую аудиторию.

Созданный текст представляет собой основательное описание системы Apache Ignite и процесса ее установки на Docker. Он написан понятным языком, хорошо организован и увлекателен. Но некоторые разделы нужно отредактировать и переписать с добавлением технических подробностей, чтобы сделать руководство более информативным и удобным для пользователя, готовым к публикации.

На этом мы можем завершить работу. Внеся в представленный код некоторые изменения, вы сможете создать свой план туристической поездки или маркетинговую публикацию для социальных сетей. Просто добавьте подходящих агентов с конкретными ролями и контекстом, а также составьте инструкции по выполнению определенной задачи (или отредактируйте показанные выше). По большей части код фреймворка CrewAI носит декларативный характер, то есть представляет собой описание простых инструкций, а не сложных команд, поэтому он так популярен среди нетехнических пользователей.

Заключение

В целом можно сказать, что ИИ-агенты, относящиеся к сфере генеративного ИИ и больших языковых моделей LLM, находятся на передовом рубеже развития автономных систем. Они сочетают в себе механизмы обработки и генерации текстов на естественном языке с такими возможностями, как сохранение результатов в памяти, использование дополнительных инструментов и интеграция данных в реальном времени. ИИ-агентам можно делегировать многие рутинные задачи и тем самым высвобождать время для более важной работы. Такие агенты могут служить в качестве персональных цифровых помощников, полезных не только для решения индивидуальных задач, но и в бизнес-приложениях для решения сложных корпоративных проблем.

На протяжении всей главы мы рассматривали архитектуру ИИ-агентов и их основные компоненты. Также рассмотрели классификацию и наиболее популярные фреймворки для практического применения. Под конец мы рассказали о мультиагентном фреймворке

CrewAI, позволяющем нетехническим пользователям управлять ИИ-агентами и их задачами с помощью интуитивно понятных высокоуровневых абстракций.

Процесс разработки и реализации выполняющих определенные задачи агентов во фреймворке CrewAI мы разобрали на примере задачи по генерации текста. Этот пример показывает потенциал мультиагентных систем для автоматизации сложных задач и повышения производительности в различных областях.

ЗАКЛЮЧИТЕЛЬНЫЕ СЛОВА

Поздравляем!

Вы добрались до конца увлекательного путешествия в мир генеративного ИИ. Надеемся, получив массу положительных впечатлений.

Но не забывайте, что это только начало, своего рода ознакомительная прогулка. Мир ИИ стремительно развивается, а такие технологические гиганты, как Google и OpenAI, постоянно выпускают новые версии своих фундаментальных моделей. Будущее сулит огромные перемены во многих отраслях, и держать руку на пульсе передовых разработок — значит быть готовым к адаптации и развитию.

Эта книга помогла вам заложить прочный фундамент знаний в области генеративного искусственного интеллекта. Они позволят продолжить обучение и применять ИИ-технологии в личной жизни, на работе и даже в корпоративных решениях. Чтобы не отставать от стремительно развивающихся технологий, постарайтесь выработать у себя привычку интересоваться последними достижениями в данной сфере — читать блоги, слушать подкасты, принимать участие в дискуссиях, знакомиться с другими ресурсами. Пусть ИИ станет частью вашей повседневной жизни, расширяющей знания и развивающей творческий потенциал.

На прощание хотим напомнить мудрые слова Иоганна Вольфганга Гете: «Мы должны всегда меняться, обновляться, омолаживаться, иначе затвердеем».

Желаем удачи в дальнейшем путешествии в мир ИИ!

УДК 004.8
ББК 32.813
Б94

Shamim Bhuiyan and Timur Isachenko
Generative AI with local LLM
A comprehensive roadmap for building AI-Driven applications with local LLMs
© 2024 Shamim Bhuiyan

Бхуян, Шамим.
Б94 Генеративный ИИ с обучением больших языковых моделей (LLM) для джунов / Шамим Бхуян, Тимур Исаченко ; [перевод О. И. Перфильева]. — Москва : Эксмо, 2025. — 320 с. — (Путеводитель по GPT и AI).

ISBN 978-5-04-220583-5

Это практическое руководство по созданию приложений на основе генеративного искусственного интеллекта и больших языковых моделей (LLM). Особое внимание уделяется прикладным аспектам: промпт-инжинирингу, работе с локальными LLM, тонкой настройке моделей на частных данных и созданию автономных AI-агентов. Приводятся примеры реальных решений, таких как интеллектуальная обработка SQL-запросов и автоматизация работы с изображениями.

Подходит для разработчиков и аналитиков данных с базовыми знаниями Python, желающих освоить генеративный ИИ.

УДК 004.8
ББК 32.813

ISBN 978-5-04-220583-5

© Перфильев О.И., перевод на русский язык, 2025
© Оформление. ООО «Издательство «Эксмо», 2025

Все права защищены. Книга или любая ее часть не может быть скопирована, воспроизведена в электронной или механической форме, в виде фотокопии, записи в память ЭВМ, репродукции или каким-либо иным способом, а также использована в любой информационной системе без получения разрешения от издателя. Копирование, воспроизведение и иное использование книги или ее части без согласия издателя является незаконным и влечет уголовную, административную и гражданскую ответственность.

Научно-популярное издание

ПУТЕВОДИТЕЛЬ ПО GPT И AI

Бхуян Шамим
Исаченко Тимур

ГЕНЕРАТИВНЫЙ ИИ С ОБУЧЕНИЕМ БОЛЬШИХ ЯЗЫКОВЫХ МОДЕЛЕЙ (LLM) ДЛЯ ДЖУНОВ

Главный редактор *Р. Фасхутдинов*
Руководитель направления *В. Обручев*
Ответственный редактор *Л. Салихова*
Научный редактор *Е. Суворова*
Литературный редактор *Е. Ерошкина*
Младший редактор *М. Назаренко*
Компьютерная верстка *Р. Муртазин*
Корректоры *Н. Бабаева, Л. Макарова*

Страна происхождения: Российская Федерация
Шығарушы ел: Ресей Федерациясы

ООО «Издательство «Эксмо»

123308, Россия, г. Москва, ул. Зорге, д. 1, стр. 1, эт. 20, каб. 2013. Тел.: 8 (495) 411-68-86.
Home page: www.eksmo.ru E-mail: info@eksmo.ru

Өндіруші: «Издательство «Эксмо» ЖШҚ

123308, Ресей, Мәскеу қаласы, Зорге көшесі, 1-үй, 1-құрылыс, 20 қабат, 2013-каб.
Тел.: 8 (495) 411-68-86. Home page: www.eksmo.ru E-mail: info@eksmo.ru.

Tayyar belrici: «Эксмо»

Интернет-магазин : www.book24.ru

Интернет-магазин : www.book24.kz

Интернет-дүкен : www.book24.kz

Импортёр в Республику Казахстан ТОО «РДЦ-Алматы».
Қазақстан Республикасына импорттаушы «РДЦ-Алматы» ЖШС.

Дистрибьютор и представитель по приему претензий на продукцию
в Республике Казахстан: ТОО «РДЦ-Алматы»

ТОО РДЦ Алматы, Алматы, ул. Домбровского, 3-а», литер Б, офис 1.

Дистрибьютор және Қазақстан Республикасында өнімге шағымдар
қабылдау жөніндегі өкіл: «РДЦ-Алматы» ЖШС.

Алматы қ., Домбровский көш., 3 «а», литер Б, офис 1.
Тел.: 8 (727) 251-59-90/91/92. E-mail: RDC-Almaty@eksmo.kz

Сведения о подтверждении соответствия издания согласно законодательству РФ
о техническом регулировании можно получить на сайте Издательства «Эксмо»:
www.eksmo.ru/certification

Техникалық реттеу туралы РФ заңнамасына сай басылымның сәйкестігін растау
туралы мәліметтерді мына адрес бойынша алуға болады: <http://eksmo.ru/certification/>

Произведено в Российской Федерации
Ресей Федерациясында өндірілген

Сертификаттауға жатпайды

Дата изготовления / Подписано в печать 13.10.2025.
Формат 70x100¹/₁₆. Печать офсетная. Усл. печ. л. 25,93.
Тираж экз. Заказ



Москва. ООО «Торговый Дом «Эксмо»

Адрес: 123308, г. Москва, ул. Зорге, д. 1, строение 1.
Телефон: +7 (495) 411-50-74. **E-mail:** reception@eksmo-sale.ru

По вопросам приобретения книг «Эксмо» зарубежными оптовыми покупателями обращаться в отдел зарубежных продаж ТД «Эксмо»
E-mail: international@eksmo-sale.ru

International Sales: International wholesale customers should contact Foreign Sales Department of Trading House «Eksmo» for their orders.
international@eksmo-sale.ru

По вопросам заказа книг корпоративным клиентам, в том числе в специальном оформлении, обращаться по тел.: +7 (495) 411-68-59, доб. 2151.
E-mail: borodkin.da@eksmo.ru

Оптовая торговля бумажно-беловыми и канцелярскими товарами для школы и офиса «Канц-Эксмо»:
Компания «Канц-Эксмо»: 142702, Московская обл., Ленинский р-н, г. Видное-2, Белокаменное ш., д. 1, а/я 5. Тел./факс: +7 (495) 745-28-87 (многоканальный).
e-mail: kanc@eksmo-sale.ru, сайт: www.kanc-eksmo.ru

Филиал «Торгового Дома «Эксмо» в Нижнем Новгороде
Адрес: 603094, г. Нижний Новгород, улица Карпинского, д. 29, бизнес-парк «Грин Плаза»
Телефон: +7 (831) 216-15-91 (92, 93, 94). **E-mail:** reception@eksmonn.ru

Филиал ООО «Издательство «Эксмо» в г. Санкт-Петербурге
Адрес: 192029, г. Санкт-Петербург, пр. Обуховской обороны, д. 84, лит. «Е»
Телефон: +7 (812) 365-46-03 / 04. **E-mail:** server@szko.ru

Филиал ООО «Издательство «Эксмо» в г. Екатеринбурге
Адрес: 620024, г. Екатеринбург, ул. Новинская, д. 2щ
Телефон: +7 (343) 272-72-01 (02/03/04/05/06/08)

Филиал ООО «Издательство «Эксмо» в г. Самаре
Адрес: 443052, г. Самара, пр-т Кирова, д. 75/1, лит. «Е»
Телефон: +7 (846) 207-55-50. **E-mail:** RDC-samara@mail.ru

Филиал ООО «Издательство «Эксмо» в г. Ростове-на-Дону
Адрес: 344023, г. Ростов-на-Дону, ул. Страны Советов, 44А
Телефон: +7(863) 303-62-10. **E-mail:** info@rnd.eksmo.ru

Филиал ООО «Издательство «Эксмо» в г. Новосибирске
Адрес: 630015, г. Новосибирск, Комбинатский пер., д. 3
Телефон: +7(383) 289-91-42. **E-mail:** eksmo-nsk@yandex.ru

Обособленное подразделение в г. Хабаровске
Фактический адрес: 680000, г. Хабаровск, ул. Фрунзе, 22, оф. 703
Почтовый адрес: 680020, г. Хабаровск, А/Я 1006
Телефон: (4212) 910-120, 910-211. **E-mail:** eksmo-khv@mail.ru

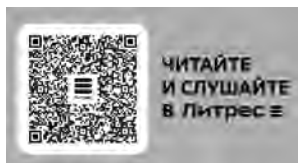
Республика Беларусь: ООО «ЭКСМО АСТ Си энд Си»
Центр оптово-розничных продаж Cash&Carry в г. Минске
Адрес: 220014, Республика Беларусь, г. Минск, проспект Жукова, 44, пом. 1-17, ТЦ «Outleto»
Телефон: +375 17 251-40-23; +375 44 581-81-92
Режим работы: с 10.00 до 22.00. **E-mail:** exmoast@yandex.by

Казахстан: «РДЦ Алматы»
Адрес: 050039, г. Алматы, ул. Домбровского, 3А
Телефон: +7 (727) 251-58-12, 251-59-90 (91,92,99). **E-mail:** RDC-Almaty@eksmo.kz

Полный ассортимент продукции ООО «Издательство «Эксмо» можно приобрести в книжных магазинах «Читай-город» и заказать в интернет-магазине: www.chitai-gorod.ru.
Телефон единой справочной службы: 8 (800) 444-8-444. Звонок по России бесплатный.

Интернет-магазин ООО «Издательство «Эксмо»
www.eksmo.ru

Розничная продажа книг с доставкой по всему миру.
Тел.: +7 (495) 745-89-14. **E-mail: imarket@eksmo-sale.ru**



ТЕРИТОРИЯ
КНИЖНЫЙ МАГАЗИН
Официальная франшиза
издательства «Эксмо»



Хочешь стать
автором «Эксмо»?



eksmo.ru

Официальный
интернет-магазин
издательства «Эксмо»



БОМБОРА – лидер на рынке полезных и вдохновляющих книг.
Мы любим книги и создаем их, чтобы вы могли творить, открывать
мир, пробовать новое, расти. Быть счастливыми. Быть на волне.

bombora.ru bomborabooks bombora

ISBN 978-5-04-220583-5



9 785042 205835 >

Если вы разработчик, стремящийся интегрировать ИИ в свои продукты, специалист по анализу данных, желающий расширить свой инструментарий, или просто увлеченный энтузиаст, стремящийся освоить передовые технологии, то эта книга станет незаменимым ресурсом. Она вооружит конкретными стратегиями и инструментами, необходимыми для эффективного, масштабируемого и безопасного внедрения ИИ в любые проекты.

Ключевые концепции:

1. Основы машинного обучения
2. Введение в глубокое обучение
3. Обработка естественного языка
4. Трансформер

Научитесь применять LLM для:

- обогащения моделей с помощью локальных данных
- интеллектуальной обработки SQL-запросов
- автоматизации работы с изображениями
- разработки умных, автономных ИИ-агентов

ОБ АВТОРАХ

ШАМИМ БХУЯН — корпоративный архитектор (Enterprise architect), разрабатывающий высокомасштабируемые ПО-решения. Имеет опыт работы в отрасли более 22 лет. Специализируется на аналитике данных и SOA-решениях, участвует в создании высоконагруженного ПО для ИТ, банков и телекома. Ведет блог frommyworkshop.

ТИМУР ИСАЧЕНКО — технический специалист и архитектор решений, обладающий экспертными знаниями в области разработки бэкенда и архитектуры микросервисов, с опытом работы более 14 лет. Участвовал в реализации сложных проектов в финансах, здравоохранении и онлайн-банкинге. Один из авторов книги «High Performance In-Memory Data Grid with Ignite», спикер отраслевых мероприятий.

«Отличный практический гид для тех, кто хочет начать работать с локальными LLM. Книга сочетает необходимую теоретическую базу с понятными пошаговыми примерами и реальными кейсами — всё, чтобы не просто понять, а сразу попробовать самостоятельно».

ДМИТРИЙ РУСИН, рецензент Read IT Club, технический менеджер, более 5 лет опыта разработки и внедрения комплексных ИТ-систем, в том числе с применением ML

ISBN 978-5-04-220583-5



9 785042 205835 >

БОМБОРА
ИЗДАТЕЛЬСТВО

БОМБОРА — лидер на рынке полезных и вдохновляющих книг. Мы любим книги и создаем их, чтобы вы могли творить, открывать мир, пробовать новое, расти. Быть счастливыми. Быть на волне.

ПРОВЕРЕНО

Read IT Club

Клуб рецензентов
на базе бизнес-лаборатории
цифрового развития КРОК